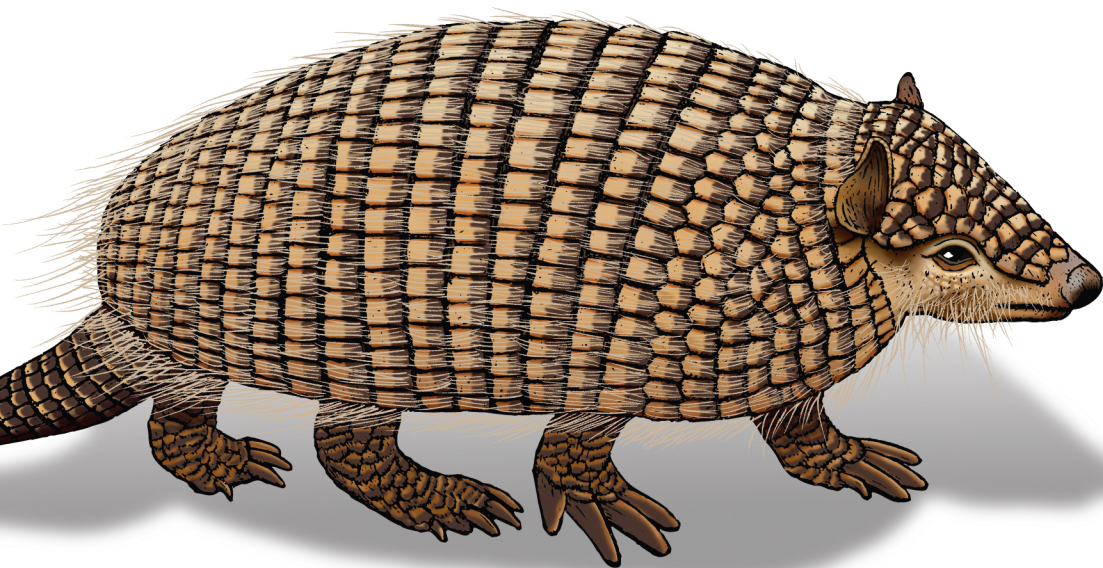


O'REILLY®

Промт- инжиниринг для GenAI

Паттерны надежных запросов
для качественных результатов



SPRINT
book

Джеймс Феникс
Майк Тейлор

Prompt Engineering for Generative AI

*Future-Proof Inputs for Reliable
AI Outputs at Scale*

James Phoenix and Mike Taylor

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

Промт-инжиниринг для GenAI

*Паттерны надежных запросов
для качественных результатов*

Джеймс Феникс, Майк Тейлор

SPRINT
book

2025

Выпущено
при поддержке

КРОК

ББК 32.818
УДК 004.8.032.26
Ф42

Феникс Джеймс, Тейлор Майк

**Ф42 Промт-инжиниринг для GenAI. Паттерны надежных запросов для качественных результатов. — Астана: «Спринт Бук», 2025. — 432 с.: ил.
ISBN 978-601-08-4722-4**

Стремительное развитие ИИ меняет нашу жизнь, а поспевать за ним становится все труднее. Каждый, кто профессионально использует ИИ, быстро приходит к пониманию, что качество результатов напрямую зависит от качества входных данных. Поэтому нам сейчас не обойтись без промт-инжиниринга (prompt engineering) — набора методов повышения надежности, эффективности и точности моделей ИИ.

Большие языковые (LLM) и диффузионные модели, такие как ChatGPT и DALL-E, постоянно обучаются на общедоступных текстах и изображениях из Интернета. Так что практически любой человек может использовать модели ИИ для решения задач, которые ранее не поддавались автоматизации.

Эта книга поможет вам осваивать и применять на практике модели генеративного ИИ. Необдуманная интеграция LLM и диффузионных моделей в рабочие процессы часто дает весьма спорные результаты. Джеймс Феникс и Майк Тейлор научат вас использовать промт-инжиниринг, чтобы эффективно внедрить ИИ в любой процесс.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.818
УДК 004.8.032.26

Права на издание получены по соглашению с O'Reilly. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими. В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др.

ISBN 978-1098153434 англ.

Authorized Russian translation of the English edition of Prompt Engineering for Generative AI ISBN 9781098153434 © 2024 Saxifrage, LLC and Just Understanding Data LTD.

This translation is published and sold by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same.

ISBN 978-601-08-4722-4

© Перевод на русский язык ТОО «Спринт Бук», 2025

© Издание на русском языке, оформление ТОО «Спринт Бук», 2025

Краткое содержание

Предисловие	14
Глава 1. Пять принципов промтинга	21
Глава 2. Введение в большие языковые модели	61
Глава 3. Стандартные приемы генерации текста с помощью ChatGPT.....	77
Глава 4. Продвинутое приемы генерации текста с помощью LangChain.....	150
Глава 5. Векторные базы данных с использованием FAISS и Pinecone.....	213
Глава 6. Автономные агенты с памятью и инструментами.....	253
Глава 7. Введение в диффузионные модели генерирования изображений.....	301
Глава 8. Стандартные практики создания изображений с помощью Midjourney	316
Глава 9. Продвинутое способы создания изображений с помощью Stable Diffusion.....	352
Глава 10. Создание приложений на базе ИИ	403
Об авторах	428
Иллюстрация на обложке.....	429

Оглавление

Отзывы о книге.....	12
Предисловие	14
Требования к программному обеспечению для работы с книгой	15
Условные обозначения.....	18
Использование примеров кода	19
Благодарности.....	19
От издательства	20
Глава 1. Пять принципов промтинга	21
Обзор пяти принципов промтинга	25
1. Задайте направление.....	28
2. Укажите формат.....	34
3. Приведите примеры	37
4. Оцените качество	41
5. Разделите задачу на подзадачи	53
Резюме	60
Глава 2. Введение в большие языковые модели	61
Что такое модели генерации текста	61
Векторные представления: числовая сущность языка	62
Архитектура трансформеров: организация контекстных отношений	63
Вероятностная генерация текста: механизм принятия решений	65
Исторические предпосылки: появление архитектуры трансформеров.....	66
Предварительно обученные генеративные трансформеры OpenAI.....	68
GPT-3.5-turbo и ChatGPT.....	69
GPT-4.....	71
Google Gemini	71
Meta Llama и открытый исходный код	72
Использование квантования и LoRA.....	73
Mistral.....	74
Anthropic: Claude	74
GPT-4V(ision).....	75
Сравнение моделей.....	75
Резюме	76

Глава 3. Стандартные приемы генерации текста с помощью ChatGPT.....	77
Генерирование списков.....	77
Генерирование иерархического списка	79
Когда следует избегать регулярных выражений.....	85
Генерирование данных в формате JSON.....	85
YAML.....	87
Фильтрация текста в формате YAML.....	89
Обработка недопустимых полезных данных в YAML	90
Генерирование разных форматов с помощью ChatGPT.....	93
Пример данных CSV.....	94
Объясни это пятилетнему ребенку.....	95
Универсальный перевод с помощью LLM	96
Возможность запросить контекст	98
Разделение текстовых стилей	101
Выявление текстовых характеристик.....	102
Создание нового контента с использованием извлеченных характеристик.....	103
Извлечение текстовых характеристик с помощью LLM.....	104
Обобщение.....	104
Обобщение с учетом ограниченного окна контекста.....	105
Разбиение текста на части	107
Преимущества разбиения текста на части.....	108
Сценарии разбиения текста на части.....	108
Пример плохого разбиения.....	109
Стратегии разбиения на части.....	110
Обнаружение предложений с использованием spaCy.....	112
Создание простого алгоритма разбиения на Python.....	113
Разбиение методом скользящего окна.....	115
Пакеты для разбиения текста на части	117
Разбиение текста на части с помощью tiktoken	117
Кодировки.....	118
Что такое токенизация строк	118
Оценка числа токенов, используемых в вызовах Chat API	119
Анализ настроений	122
Методы улучшения анализа настроений	123
Ограничения и сложности анализа настроений.....	124
От меньшего к большему	124
Планирование архитектуры.....	125
Реализация отдельных функций	125
Добавление тестов.....	126
Преимущества метода «от меньшего к большему».....	127
Сложности метода «от меньшего к большему»	128

Рольевые запросы	128
Преимущества рольевых запросов	130
Сложности рольевых запросов	130
Когда использовать рольевые запросы.....	130
Тактика запросов к GPT	131
Как избежать галлюцинаций с помощью референсов.....	131
Предоставление модели GPT «времени на размышления»	133
Тактика внутреннего монолога	134
Оценка моделью своих собственных ответов	136
Классификация с помощью LLM.....	138
Построение модели классификации	139
Классификация голосованием.....	140
Критерии оценки.....	141
Метапромтинг.....	145
Резюме	149
Глава 4. Продвинутые приемы генерации текста с помощью LangChain	150
Введение в LangChain	150
Настройка среды.....	152
Модели чатов.....	154
Потоковые модели чата	155
Создание нескольких ответов от LLM.....	156
Шаблоны запросов в LangChain.....	157
Язык выражений LangChain	157
Использование PromptTemplate с моделями чата	159
Парсеры вывода	160
Оценивание с помощью LangChain	164
Вызов функций OpenAI.....	172
Параллельный вызов функций	175
Вызов функций в LangChain	177
Извлечение данных с помощью LangChain	179
Планирование запросов.....	179
Создание шаблонов запросов с несколькими примерами	181
Включение нескольких примеров фиксированной длины.....	181
Форматирование примеров.....	182
Выбор нескольких примеров по длине.....	183
Ограничения обучения с несколькими примерами.....	185
Сохранение и загрузка подсказок LLM	186
Включение данных	187
Загрузчики документов.....	189

Инструменты для разбиения текста	192
Разбиение текста по количеству символов и токенов	193
Рекурсивное разбиение по символам	194
Декомпозиция задач	196
Объединение запросов в цепочки	198
Последовательная цепочка	199
itemgetter и извлечение ключей словаря	201
Структурирование цепочек LCEL	206
Цепочки документов	207
Наполнение	210
Уточнение	210
Отображение и свертка (Map Reduce)	210
Отображение с повторным ранжированием (Map Re-rank)	211
Резюме	212
Глава 5. Векторные базы данных с использованием FAISS и Pinecone	213
Генерация с дополненной выборкой	216
Введение в эмбединги	218
Загрузка документа	227
Извлечение данных из памяти с помощью FAISS	230
Реализация RAG с использованием LangChain	235
Векторные базы данных на хостинге от Pinecone	236
Собственные запросы	245
Альтернативные механизмы извлечения	250
Резюме	252
Глава 6. Автономные агенты с памятью и инструментами	253
Метод рассуждения с цепочкой мыслей	253
Агенты	255
Рассуждения и действие (Reason and Act, ReAct)	258
Реализация паттерна ReAct	260
Использование инструментов	266
Использование LLM в качестве API (функции OpenAI)	269
Сравнение типов агентов ReAct и функции OpenAI	273
Варианты использования функций OpenAI	274
ReAct	274
Варианты использования ReAct	275
Наборы инструментов для агентов	275
Настройка стандартных агентов	277
Пользовательские агенты в LCEL	279

Память и ее использование	281
Долгосрочная память	281
Краткосрочная память	282
Краткосрочная память в диалоговых агентах типа вопрос-ответ.....	283
Память в LangChain	283
Сохранение состояния	284
Запрос состояния.....	285
ConversationBufferMemory.....	285
Другие популярные типы памяти в LangChain.....	288
ConversationBufferWindowMemory	288
ConversationSummaryMemory	288
ConversationSummaryBufferMemory	289
ConversationTokenBufferMemory.....	289
Агент функций OpenAI с памятью.....	290
Продвинутые фреймворки для агентов	292
Агенты планирования и выполнения.....	292
Дерево рассуждений	293
Обратные вызовы	295
Глобальные обратные вызовы (в вызове конструктора).....	297
Обратные вызовы, ограниченные запросами.....	297
Аргумент verbose	297
Когда и какой подход использовать	298
Подсчет токенов с помощью LangChain.....	298
Резюме	300
Глава 7. Введение в диффузионные модели генерирования изображений.....	301
OpenAI DALL-E.....	304
Midjourney	307
Stable Diffusion.....	310
Google Gemini	312
Преобразование текста в видео	313
Сравнение моделей	313
Резюме	314
Глава 8. Стандартные практики создания изображений с помощью Midjourney	316
Управление форматом.....	316
Управление художественным стилем	320
Реконструирование запросов.....	322
Усилители качества	323
Отрицательные запросы	324
Взвешенные понятия.....	327

Запросы с изображениями	330
Изменение фрагментов изображений	333
Дорисовывание обрамления изображений	336
Получение разных изображений одного персонажа	338
Переделка запросов	340
Выделение мемов	342
Картинирование мемов	347
Анализ запросов	350
Резюме	351
Глава 9. Продвинутое создание изображений	
с помощью Stable Diffusion	352
Запуск Stable Diffusion	352
Веб-интерфейс AUTOMATIC1111	360
Img2Img	367
Увеличение разрешения изображений	370
Реконструирование запроса по изображению	373
Изменение фрагментов и добавление обрамления	373
ControlNet	377
Segment Anything Model (SAM)	387
Тонкая настройка DreamBooth	390
Уточняющая модель в Stable Diffusion XL	397
Резюме	401
Глава 10. Создание приложений на базе ИИ	403
Разработка блога на основе ИИ	403
Исследуемая тема	405
Интервью с экспертом	407
Генерирование плана публикации	409
Генерирование текста	411
Стиль письма	414
Оптимизация заголовков	417
Генерирование иллюстраций для блога	418
Пользовательский интерфейс	424
Резюме	426
Об авторах	428
Иллюстрация на обложке	429

Отзывы о книге

Это, бесспорно, лучшая книга по промт-инжинирингу, которую я когда-либо читал. Майк и Джеймс — мастера своего дела.

*Дэн Шиппер (Dan Shipper), соучредитель
и генеральный директор Every*

Эта книга — отличное введение в основы промт-инжиниринга и генеративно-го ИИ. Авторы описывают широкий спектр полезных приемов от начального до экспертного уровня в простой и понятной форме. Если вы хотите повысить точность и надежность своих ИИ-систем, эта книга должна быть у вас на полке.

*Мэйо Ошин (Mayo Oshin), основатель
и генеральный директор Siennai Analytics,
один из первых участников LangChain*

Руководство Феникса и Тейлора — это маяк в огромном океане генеративного ИИ. Их книга стала важной опорой для моей команды в Phiture AI Labs, когда мы учились использовать большие языковые и диффузионные модели для создания маркетинговых материалов, точно передающих суть приложений и игр наших клиентов. С помощью промт-инжиниринга нам удалось генерировать уникальный и соответствующий бренду контент в больших объемах. Это не просто теория; это практический мастер-класс по превращению сырого потенциала ИИ в индивидуальные решения, что делает книгу обязательной для разработчиков, стремящихся поднять интеграцию ИИ на новые высоты креативности и эффективности.

*Мориц Даан (Moritz Daan),
основатель/партнер Phiture Mobile
Growth Consultancy*

Изучение книги «Промт-инжиниринг для GenAI. Паттерны надежных запросов для качественных результатов», пожалуй, наиболее удачный способ обрести уверенность в успешном развитии вашей карьеры. Это, без сомнения, лучший источник знаний для всех, кто занимается практическим применением ИИ. Изложенные здесь разнообразные и усовершенствованные принципы помогут как начинающим, так и опытным инженерам ИИ сохранить лидирующие позиции в этой очень конкурентной игре на долгие годы вперед.

*Эллис Кропси (Ellis Crosby), технический
директор и соучредитель Incremento*

Это важное руководство для агентств и специалистов в сфере обслуживания. Интеграция ИИ с клиентскими сервисами, автоматизация управления и ускорение принятия решений задают новые отраслевые стандарты. В книге вы найдете полезную практическую информацию и тактики, которые позволят вам понять и использовать весь потенциал ИИ.

*Байрон Тассони-Реш (Byron Tassoni-Resch),
генеральный директор и соучредитель
WeDiscover*

Весьма интересное и познавательное издание, в котором практические советы и рекомендации сочетаются с подробной фундаментальной информацией. Мир генеративного ИИ развивается с головокружительной скоростью, и инструменты, способные дать результат, независимо от используемой базовой модели, — на вес золота!

*Риан Дрейер (Riaan Dreyer), директор
по цифровым технологиям и данным,
Bank of Iceland*

Авторы умело переводят тонкости промт-инжиниринга в плоскость практических инструментов, помогающих генерировать текст и изображения. Это руководство, охватывающее как стандартные практики, так и передовые технологии, дает читателям ценные советы по максимальному использованию возможностей генеративных моделей ИИ.

*Адитья Гоэл (Aditya Goel),
консультант по генеративному ИИ*

Предисловие

Стремительное развитие инноваций в области генеративного ИИ обещает изменить нашу жизнь, но поспевать за ним становится все труднее. Число статей об искусственном интеллекте, публикуемых на сайте arXiv (<https://oreil.ly/EN5ay>), растет в геометрической прогрессии. Stable Diffusion (<https://oreil.ly/QX-yy>) — один из самых быстроразвивающихся проектов с открытым исходным кодом в истории, а инструмент для создания произведений искусства с помощью ИИ на сервере Discord Midjourney (<https://oreil.ly/ZVZ5o>) насчитывает десятки миллионов пользователей, превосходя даже крупнейшие игровые сообщества. Но больше всего внимание общественности привлек выпуск модели ChatGPT компанией OpenAI, которая за два месяца набрала 100 миллионов пользователей (<https://oreil.ly/FbYWk>) и оказалась самым быстрорастущим потребительским приложением в истории. Умение работать с ИИ быстро стало одним из самых востребованных навыков.

Каждый, кто профессионально использует ИИ, быстро приходит к пониманию, что качество результатов во многом зависит от качества входных данных. Дисциплина «*промт-инжиниринг*» (prompt engineering) возникла как набор методов повышения надежности, эффективности и точности моделей ИИ. «Через десять лет половина рабочих мест в мире будет связана с промт-инжинирингом», — утверждает Робин Ли (Robin Li; <https://oreil.ly/IdIfO>), соучредитель и генеральный директор китайского технологического гиганта Baidu. Однако мы ожидаем, что промтинг (prompting — составление инструкций и подсказок для ИИ, или промтов) станет необходимым техническим навыком для многих профессий, как это произошло с Microsoft Excel, а не просто новомодным названием. Эта новая волна цифрового развития меняет все, что мы знали о компьютерах. Мы привыкли писать алгоритмы, которые каждый раз выдают один и тот же результат, но с ИИ, где ответы не детерминированы, ситуация иная. Такие понятия, как стоимость и задержка, снова стали реальными факторами после десятилетий действия закона Мура, который заставил нас успокоиться в ожидании получения возможности вычислений в реальном времени с незначительными затратами. Самым большим препятствием является склонность моделей ИИ что-либо выдумывать, так называемые *галлюцинации*, заставляющие нас переосмыслить подходы к оценке точности нашей работы.

Мы работаем с генеративным ИИ с момента начала бета-тестирования GPT-3 в 2020 году и по мере развития моделей наблюдаем, как многие прежние приемы

работы и уловки становятся неактуальными. С течением временем был создан набор принципов, остающихся эффективными при работе с новыми моделями ИИ для генерирования не только текста, но и изображений. Мы написали эту книгу, основываясь на существующих базовых принципах, чтобы помочь вам овладеть теми навыками, которые будут полезны независимо от того, что произойдет с ИИ в течение следующих пяти лет. Ключ к работе с ИИ заключается не в том, чтобы «придумать особенный промт, добавив в конце волшебное слово, которое изменит все остальное», как утверждает соучредитель OpenAI Сэм Альтман (Sam Altman; <https://oreil.ly/oo262>), а в том, что всегда будет иметь значение, — «качественная передача идей и понимание того, чего вы хотите». Мы не знаем, будет ли эта дисциплина называться промт-инжинирингом через пять лет, но уверены, что эффективная работа с генеративным ИИ станет еще более важной.

Требования к программному обеспечению для работы с книгой

Весь код, представленный в книге, написан на Python и предназначен для запуска в Jupyter Notebook (<https://jupyter.org>) или Google Colab (<https://colab.research.google.com>). Изложенные здесь концепции можно при желании перенести в JavaScript или любой другой язык программирования, потому что основное внимание в книге уделено методам промтинга, составления подсказок и инструкций для ИИ, а не традиционным навыкам программирования. Весь код можно найти на GitHub (<https://oreil.ly/BrightPool>), и, конечно, мы указали ссылки на соответствующие блокноты для Jupyter Notebook. Настоятельно рекомендуем использовать репозиторий GitHub (<https://oreil.ly/BrightPool>) и запускать находящиеся в нем примеры во время чтения книги.

Примеры, не предназначенные для запуска в блокноте Jupyter Notebook, можно запускать в терминале как обычные скрипты командой `python content/chapter_x/script.py` в своем терминале, где `x` — номер главы, а `script.py` — имя скрипта. Иногда будет необходимо устанавливать ключи API в переменных окружения, и эти случаи мы подробно поясним. Пакеты для Python, используемые скриптами, часто обновляются, поэтому перед запуском примеров необходимо установить наш файл `requirements.txt` (<https://oreil.ly/BPreq>) в виртуальной среде.

Файл `requirements.txt` создан для Python 3.9. Если вы используете другую версию Python, то создайте новый документ `requirements.txt` из файла `requirements.in` (<https://oreil.ly/YRwP7>), находящегося в репозитории GitHub, для чего выполните следующие команды:

```
`pip install pip-tools`  
`pip-compile requirements.in`
```

Для пользователей Mac

1. Откройте приложение **Terminal** (Терминал), которое можно найти в папке **Applications** (Приложения) в разделе **Utilities** (Утилиты), или используйте **Spotlight** для его поиска.
2. Перейдите в папку проекта: выполните команду `cd`, указав в ней путь к папке проекта. Например: `cd путь/к/вашему/проекту`.
3. Создайте виртуальную среду: выполните следующую команду, чтобы создать виртуальную среду с именем `venv` (вы можете назвать ее как угодно): `python3 -m venv venv`.
4. Активируйте виртуальную среду: прежде чем устанавливать пакеты, вы должны активировать виртуальную среду. Сделайте это с помощью команды `source venv/bin/activate`.
5. Установите пакеты: после активации виртуальной среды можно установить пакеты с помощью `pip`. Чтобы установить пакеты, перечисленные в файле `requirements.txt`, выполните команду `pip install -r requirements.txt`.
6. По окончании сеанса работы деактивируйте виртуальную среду: чтобы выйти из виртуальной среды, выполните команду `deactivate`.

Для пользователей Windows

1. Откройте приложение `cmd` (Командная строка): для этого можно выполнить поиск по слову `cmd` в меню **Start** (Пуск).
2. Перейдите в папку проекта: выполните команду `cd`, указав в ней путь к папке проекта. Например: `cd путь/к/вашему/проекту`.
3. Создайте виртуальную среду: выполните следующую команду, чтобы создать виртуальную среду с именем `venv` (вы можете назвать ее как угодно): `python -m venv venv`.
4. Активируйте виртуальную среду: прежде чем устанавливать пакеты, вы должны активировать виртуальную среду. Сделайте это с помощью команды `.\venv\Scripts\activate`.
5. Установите пакеты: после активации виртуальной среды можно установить пакеты с помощью `pip`. Чтобы установить пакеты, перечисленные в файле `requirements.txt`, выполните команду `pip install -r requirements.txt`.
6. По окончании сеанса работы деактивируйте виртуальную среду: чтобы выйти из виртуальной среды, выполните команду `deactivate`.

Несколько дополнительных советов по настройке:

- всегда проверяйте актуальность установленной у вас версии Python, чтобы избежать проблем с совместимостью;

- не забывайте активировать виртуальную среду каждый раз, когда начинаете работать над проектом;
- файл `requirements.txt` должен находиться в том же каталоге, где создается виртуальная среда, в противном случае нужно указать путь к нему в команде `pip install -r.`

Примеры, импортирующие библиотеку OpenAI, предполагают наличие доступа к учетной записи разработчика OpenAI, следовательно, у вас должна быть определена переменная окружения `OPENAI_API_KEY` с ключом доступа к OpenAI API. Мы используем версию 1.0 библиотеки. Краткие инструкции по настройке среды разработки можно найти в документации OpenAI (<https://oreil.ly/YqbrY>).

Также убедитесь, что в вашей учетной записи OpenAI включено выставление счетов (параметр `billing is enabled`) и указан действительный способ оплаты. Это необходимо для запуска некоторых примеров из книги. Также в некоторых примерах (где это явно указано) используется GPT-4, однако мы кратко рассмотрим также конкурирующую модель Anthropic Claude 3 (<https://oreil.ly/jY8Ai>), а также Llama 3 с открытым исходным кодом от компании Meta (<https://oreil.ly/BbXZ3>) и Google Gemini (<https://oreil.ly/KYgij>).

Генерировать изображения мы будем с помощью проекта Midjourney (<https://www.midjourney.com>), для регистрации в котором необходима учетная запись Discord. По такому же принципу работают DALL-E 3 (доступна также при наличии подписки ChatGPT Plus или через API) и Stable Diffusion (доступна как API (<https://oreil.ly/cmTtW>) и может работать локально (<https://oreil.ly/Ha0T5>) на вашем компьютере, если у него есть графический процессор). В примерах создания изображений в этой книге будут использованы Midjourney v6, Stable Diffusion v1.5 (поскольку многие расширения по-прежнему совместимы только с этой версией) или Stable Diffusion XL (<https://oreil.ly/S0P4s>). При необходимости мы также будем указывать на имеющиеся различия.

В примерах мы по возможности используем библиотеки с открытым исходным кодом, хотя при необходимости указываем и коммерческие, если имеются. Например, в главе 5, где рассказывается о векторных базах данных, демонстрируется применение FAISS (библиотеки с открытым исходным кодом) и Pinecone (платной библиотеки). Примеры в книге составлены так, чтобы их легко было адаптировать для альтернативных моделей и поставщиков, а приобретенные навыки передавать другим пользователям. В главе 4, посвященной расширенной генерации текста, описывается LLM-фреймворк LangChain, а в главе 9, посвященной расширенной генерации изображений, основное внимание уделяется графическому интерфейсу Stable Diffusion Web UI от AUTOMATIC1111.

Условные обозначения

В книге используются следующие условные обозначения.

Курсив

Курсивом выделены новые термины.

Моноширинный шрифт

Используется для листингов программ, а также внутри абзацев для выделения элементов программы, таких как имена переменных, функций, баз данных, типов данных, переменных окружения, инструкций и ключевых слов, имен и расширений файлов.

Моноширинный жирный шрифт

Показывает команды или другой текст, который пользователь должен ввести самостоятельно.

Моноширинный курсивный шрифт

Показывает текст, который должен быть заменен значениями, введенными пользователем, или значениями, определяемыми контекстом.

Шрифт без засечек

Используется для обозначения URL, адресов электронной почты, названий кнопок, каталогов.



Этот рисунок указывает на совет или предложение.



Этот рисунок указывает на общее замечание.



Этот рисунок указывает на предупреждение.

В книге мы будем обращаться к пяти так называемым Пяти Принципам Промтинга, определяя, какой из них наиболее применим к рассматриваемому примеру. Подробное описание принципов вы найдете в главе 1.



Название принципа

Так мы объясняем, как тот или иной принцип применяется к текущему примеру или разделу текста.

Использование примеров кода

Дополнительный материал (примеры кода, упражнения и т. д.) можно скачать по ссылке <https://oreil.ly/prompt-engineering-for-generative-ai>.

В общем случае все примеры кода из книги вы можете использовать в своих программах и документации. Вам не нужно обращаться в издательство за разрешением, если вы не собираетесь воспроизводить существенные части программного кода. Если вы разрабатываете программу и используете в ней несколько фрагментов кода из книги, вам не нужно обращаться за разрешением. Но для продажи или распространения примеров из книги вам потребуется разрешение от издательства O'Reilly. Вы можете отвечать на вопросы, цитируя данную книгу или примеры из нее, но для включения существенных объемов программного кода из книги в документацию вашего продукта потребуется разрешение.

Мы рекомендуем, но не требуем добавлять ссылку на первоисточник при цитировании. Под ссылкой на первоисточник мы подразумеваем указание авторов, издательства и ISBN.

За получением разрешения на использование значительных объемов программного кода из книги обращайтесь по адресу permissions@oreilly.com.

Благодарности

Мы хотели бы поблагодарить следующих людей за их вклад в техническое рецензирование книги и терпение, проявленное при исправлении текста из-за необходимых доработок:

- Мэйо Ошина (Mayo Oshin), одного из первых участников LangChain и основателя SeinnAI Analytics (<https://www.siennaianalytics.com>);
- Эллиса Кросби (Ellis Crosby), основателя Scarlett Panda (<https://www.scarlett-panda.com>) и агентства Incremen.to (<https://incremen.to>);

- Дэйва Поусона (Dave Pawson), автора книги «XSL-FO» (<https://oreil.ly/XSL-FO>);
- Марка Феникса (Mark Phoenix), старшего инженера-программиста;
- Адитью Гоэл (Aditya Goel), консультанта GenAI.

Мы также благодарны нашим семьям за их терпение и понимание и хотели бы заверить их, что по-прежнему предпочитаем общаться с ними, а не с ChatGPT.

От издательства

Ваши замечания, предложения, вопросы отправляйте по адресу comp@sprintbook.kz (издательство «SprintBook», компьютерная редакция).

Мы выражаем огромную благодарность компании «КРОК» за помощь в работе над русскоязычным изданием книги и их вклад в повышение качества переводной литературы.

Мы будем рады узнать ваше мнение!

Уважаемые читатели, хотелось бы отметить, что мир нейронных сетей постоянно меняется, обучается и развивается, поэтому результаты, полученные вами, могут отличаться от тех, что вы увидите в книге.

О научном редакторе русскоязычного издания

Дмитрий Русин — технический менеджер в «КРОК», имеет более пяти лет опыта разработки и внедрения комплексных ИТ-систем, в том числе с применением ML.

Пять принципов промтинга

Промт-инжиниринг (prompt engineering) — это процесс составления запросов к нейросети, которые приведут к необходимым или желаемым результатам.

Промт (запрос, подсказка) — это данные (обычно текст), которые вы вводите при взаимодействии с моделью ИИ, такой как ChatGPT или Midjourney. Промт представляет собой набор инструкций, которые модель использует для формирования желаемого ответа: текст для *больших языковых моделей* (Large Language Model, LLM), таких как ChatGPT (<https://chat.openai.com>), или изображения для *диффузионных моделей*, таких как Midjourney (<https://www.midjourney.com>).

Вот простой пример запроса для генератора названий продуктов (на основе одного из примеров OpenAI (<https://oreil.ly/Fc8cq>)), а также результат, полученный моделью ChatGPT.

Ввод:

Можешь придумать список торговых названий для обуви,
которая подойдет на любой размер ноги?

Вывод:

Конечно! Вот несколько придуманных торговых названий для обуви,
которая подойдет на любой размер ноги:

```
UniFit SoleStrides  
FlexiSize All-Fit Shoes  
OmniFit FootFlex  
...  
AnyStep AdaptiWalkers  
FlexiRange UniversalSteps  
PerfectGlide MultiFit Shoes
```

Этот ответ на столь наивный запрос можно считать волшебством, потому что мы получили его, приложив минимум усилий. По мере совершенствования моделей

вероятность получения *достаточно хороших* результатов с первой попытки будет расти. Для любого однократного взаимодействия с ИИ, когда не планируется повторно решать ту же задачу, наивного подхода вполне достаточно.

Однако, если вы собираетесь использовать запрос на продукitive, потребуется приложить больше усилий, чтобы правильно его сформулировать. За свои ошибки вы платите деньги в виде комиссии, взимаемой OpenAI в зависимости от длины запроса и ответа, а также времени, затраченного на исправление ошибок. При создании генератора названий товаров с тысячами пользователей следует исправить ряд очевидных проблем.

Неопределенность направления

Запрос не сообщает модели ИИ, какой стиль названия вы хотите получить или какими атрибутами оно должно обладать. Вам нужно одно слово или словосочетание? Можно ли использовать выдуманные слова или важно, чтобы эти слова действительно существовали в языке? Название, придуманное ИИ, должно быть похоже на те, которыми вы восхищаетесь, или копировать известные удачные торговые марки?

Неформатированный вывод

Вы получаете произвольный по длине список названий, каждое из которых начинается с новой строки. Отправив один и тот же запрос несколько раз, можно заметить, что иногда приходит ответ в виде нумерованного списка с предшествующим вступительным текстом, что затрудняет программный анализ.

Отсутствие примеров

В запросе нет ни одного примера, показывающего, как должно выглядеть *хорошее* название, модель будет использовать усредненные значения из данных, которым она обучена, то есть всего Интернета (со всей присущей ему предвзятостью), но этого ли вы хотите? В идеале необходимо привести примеры названий, успешных в отрасли, или тех, которые вам нравятся.

Ограниченность оценки

Нет последовательного или масштабируемого способа, позволяющего определить, какие названия хорошие, а какие плохие, то придется самостоятельно просматривать каждый ответ. Установив систему ранжирования или форму оценки, вы сможете оптимизировать запрос, получить лучшие результаты и определить, сколько раз система не срабатывала.

Отсутствие разделения задач

В рамках одного запроса вы ставите слишком много задач: при выборе названия продукта учитывается множество факторов, однако ИИ должен решить поставленную задачу целиком, без разделения задач или понимания, как именно она выполняется для вас.

Исправление этих проблем лежит в основе пяти принципов, на которые мы будем ссылаться на протяжении всей книги. Попросить модель ИИ выполнить одну и ту же задачу можно множеством способов, и даже небольшие изменения в запросе могут вызвать существенные изменения в ответе. Большая языковая модель (LLM) поочередно предсказывает следующий токен (примерно три четверти слова), начиная с вашего запроса-подсказки. Каждый новый токен выбирается на основе вероятности его появления в последовательности предыдущих с некоторым элементом случайности (контролируется *температурным* параметром). Как показано на рис. 1.1, слово *shoes* (обувь) с меньшей вероятностью встретится в названии после слова *AnyFit* (0,88 %), тогда как более предсказуемым будет слово *Athletic* (72,35 %).

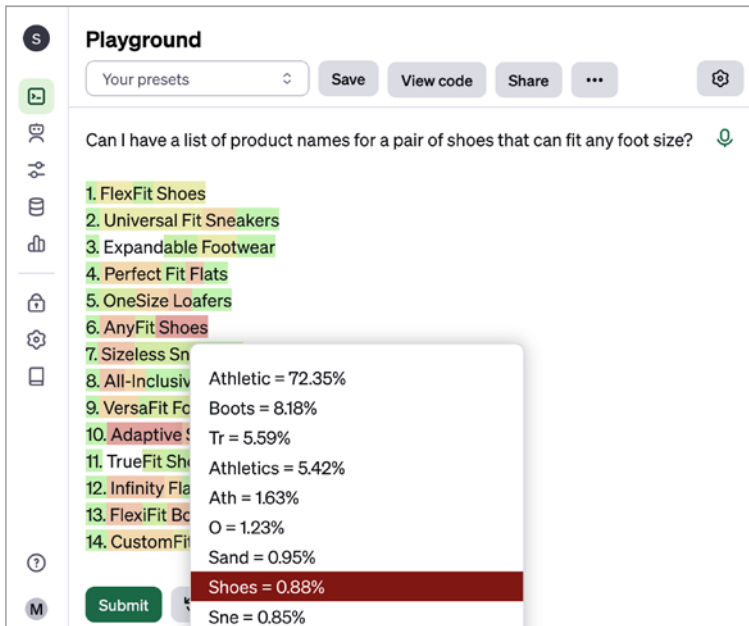


Рис. 1.1. Разбиение ответа на токены

Модели LLM обучаются практически на всем тексте, который существует в Интернете, а затем дополнительно настраиваются, чтобы давать полезные ответы. На усредненные запросы будут даны усредненные ответы, что может привести к разочарованию из-за результатов, не оправдавших ожидания. Все, что вы вводите в теле запроса, меняет вероятность каждого сгенерированного слова и влияет на итоговый результат. Эти модели видели лучшее и худшее из того, что создавали люди, и способны смоделировать практически все что угодно, если вы правильно сформулируете запрос. OpenAI взимает плату в зависимости от количества токенов в запросе и ответе (<https://openai.com/pricing>), следовательно, инженеры должны учитывать это и оптимизировать запросы по стоимости, качеству и надежности.

Вот еще один пример использования приемов промт-инжиниринга. Мы просим подобрать названия в стиле Стива Джобса, указывая, что нам нужен список названий, разделенных запятыми, и приводим примеры хороших названий.

Ввод:

Придумай список торговых названий для обуви, которая подойдет на любой размер ноги, в стиле Стива Джобса.

Представь результаты в виде списка названий, разделенных запятыми, в формате:

Описание товара: обувь, которая подойдет на любой размер ноги

Названия товаров: [список из 3 наименований]

Примеры

Описание товара: холодильник для пива

Названия товаров: iBarFridge, iFridgeBeer, iDrinkBeerFridge

Описание товара: часы, показывающие точное время в космосе

Названия товаров: iNaut, iSpace, iTime

Описание товара: машина для приготовления молочных коктейлей

Названия товаров: iShake, iSmoothie, iShake Mini

Вывод:

Описание товара: обувь, которая подойдет на любой размер ноги

Названия товаров: iFitFoot, iPerfectFit, iShoeSize

Невозможно создать идеальный запрос, но его можно оптимизировать для получения хороших результатов в правильном формате. Например, для генератора названий товаров можно выбрать кого-то еще, кроме Стива Джобса, чтобы получить результат, соответствующий вашим предпочтениям; изменить формат ответа, если необходимо; использовать вывод, полученный в ответ на запрос, в качестве ввода для следующей попытки. Наконец, вы можете периодически обновлять примеры, используемые в запросе, основываясь на отзывах пользователей, что со временем сделает вашу систему умнее.

Обзор пяти принципов промтинга

Процесс оптимизации описанного запроса следует пяти принципам промтинга, которые мы проанализируем на данном примере далее в этой главе и на которые будем ссылаться на протяжении всей книги. Они в точности соответствуют пяти задачам, выполненным при составлении этого простого текстового запроса. Далее в книге вы найдете ссылки на эти принципы, чтобы понять, как их применить на практике. Итак, пять принципов промтинга заключаются в следующем.

Задайте направление

Подробно опишите желаемый стиль или укажите соответствующую ему личность.

Укажите формат

Определите правила, которым должна следовать модель, и структуру ответа.

Приведите примеры

Добавьте несколько примеров правильного решения задачи.

Оцените качество

Найдите ошибки, оцените ответы и определите, что влияет на качество ответа.

Разделите задачу на подзадачи

Разбейте сложные задачи на связанные между собой этапы.

Эти принципы — не временные *советы* или *приемы*, а общепринятые соглашения, применимые при работе с интеллектом любого уровня, биологического или искусственного. Они не зависят от модели и должны способствовать улучшению ответов на ваш запрос, независимо от того, какую генеративную модель, создающую текст или изображение, вы используете. Мы впервые опубликовали эти принципы в июле 2022 года в статье *Prompt Engineering: From Words to Art and Copy* (<https://oreil.ly/RYYiV>), и они выдержали испытание временем, а также нашли свое отражение в соответствующих принципах OpenAI, изложенных в их руководстве по промт-инжинирингу (<https://oreil.ly/dF8q->), вышедшем годом позже. Любой, кто тесно работает с генеративными моделями ИИ, скорее всего, придет к похожему набору стратегий для решения общих проблем, и в этой книге вы увидите сотни наглядных примеров улучшения ваших запросов с их помощью.

Мы подготовили ряд загружаемых примеров создания текста и изображений, которые вы можете использовать в качестве контрольного списка для практики применения этих принципов. Они были созданы для нашего популярного курса *Udemy The Complete Prompt Engineering for AI Bootcamp* (<https://oreil.ly/>)

V40zg), который прослушали более 70 000 студентов, и основываются на тех же стратегиях.

- Пример создания текста (<https://oreil.ly/VCcgy>).
- Пример создания изображений (<https://oreil.ly/q7wQF>).

Теперь рассмотрим, как применить каждый из пяти принципов промтинга к сценарию создания изображения с помощью генеративных моделей. Скопируйте и вставьте предложенный ниже промт в поле ввода бота Midjourney в Discord, включая ссылку на изображение в начале, после ввода команды `/image`, чтобы появилось окно ввода запроса (требуется бесплатная учетная запись Discord (<https://discord.com>) и платный аккаунт Midjourney [<https://www.midjourney.com>]).

Ввод:

<https://s.mj.run/TKAsyhNikmc> стоковая фотография деловой встречи, где 4 человека смотрят видео на белом MacBook, стоящем на стеклянной столешнице, Panasonic, DC-GH5

Вывод показан на рис. 1.2.



Рис. 1.2. Стоковая фотография деловой встречи

В этом запросе используются возможности Midjourney взять базовое изображение в качестве примера, загрузить его в Discord, а затем вставить URL в запрос (<https://s.mj.run/TKAsyhNiKmc>). В данном случае взято изображение из Unsplash, доступное бесплатно (рис. 1.3). Если вы столкнулись с ошибкой при обработке запроса, попробуйте загрузить изображение самостоятельно и посмотреть документацию Midjourney (<https://oreil.ly/UTxpX>) на предмет изменений в форматировании.



Рис. 1.3. Фото Мими Тиан (Mimi Thian) на Unsplash (<https://oreil.ly/J4Hkr>)

Теперь сравним результат, полученный от Midjourney в ответ на этот хорошо продуманный запрос, с тем, который вы получите, игнорируя приемы промт-инжиниринга (рис. 1.4): стилистически изображение выглядит более темным, чем ожидается.

Ввод:

люди на деловой встрече

Вывод показан на рис. 1.4.

Эта проблема менее заметна в пятой и последующих версиях Midjourney, однако механизмы обратной связи (когда пользователи хотят изменить разрешение изображения на более высокое, их выбор может расцениваться как инструкция для обучения модели) смещают модель, как говорится, в сторону *фэнтезийной* эстетики, мало подходящей для работы со стоковыми фотографиями. Первые пользователи Midjourney пришли из мира цифрового искусства и, естественно, тяготели к фэнтезийному и научно-фантастическому стилям, что, вероятно, нашло отражение в результатах модели, которые она генерирует даже тогда, когда такая эстетика неуместна.



Рис. 1.4. Люди на деловой встрече

Примеры, используемые в этой книге, совместимы с текстовой моделью ChatGPT Plus (GPT-4) и моделями генерации изображений Midjourney v6 и Stable Diffusion XL. Мы также будем уточнять различия между ними там, где это важно. Эти базовые современные модели хорошо справляются с широким кругом задач. Описываемые принципы максимально ориентированы на будущее, поэтому если вы читаете эту книгу, когда уже вышли GPT-5, Midjourney v7 или Stable Diffusion XXL, или используете модель другого производителя, например Google, то все изложенное здесь также будет полезно для вас.

1. Задайте направление

Одна из проблем наивных текстовых запросов, обсуждавшихся выше, состоит в том, что они не сообщают ИИ желаемые виды названий товаров. Присваивание названия товару — это в некоторой степени субъективное занятие, и если не подсказать ИИ, какие названия вам нравятся, то маловероятно, что он сможет угадать правильно.

Кстати, человеку тоже будет трудно решить эту задачу без хорошего объяснения, поэтому креативные и брендинговые агентства требуют от клиентов подробных инструкций по любой задаче.



Всегда полезно представить, какая информация может понадобиться человеку для решения задачи, и попробовать включить ее в запрос.

В примере запроса для получения названий товара мы задали направление, используя элементы ролевой игры. В данном случае потребовали подражания стилю Стива Джобса, который был известен своими знаковыми названиями продуктов. Заменив имя Стива Джобса в запросе именем другого известного человека (и добавив примеры в нужном стиле), вы получите совершенно другие результаты.

Ввод:

Придумай список торговых названий для обуви, которая подойдет на любой размер ноги, в стиле Илона Маска.

Представь результаты в виде списка названий, разделенных запятыми, в формате:

Описание товара: обувь, которая подойдет на любой размер ноги

Названия товаров: [список из 3 наименований]

Примеры

Описание товара: холодильник для пива

Названия товаров: Teslacoool, FridgeX, Neuradrink

Описание товара: часы, показывающие точное время в космосе

Названия товаров: WatchX, iSpace, The Watch Company

Описание товара: машина для приготовления молочных коктейлей

Названия товаров: ShakeX, The Milkshake Company, HyperShake

Вывод:

Описание товара: обувь, которая подойдет на любой размер ноги

Названия товаров: SoleX, FootprintX, HyperFit

Есть также ряд правил и рекомендаций, определяющих, что еще можно включить в запрос в качестве подсказки, чтобы помочь ИИ подобрать подходящее название. Этот метод иногда называют *предварительным разогревом*, или *внутренним извлечением*. Он прост, но эффективен (Liu et al., 2021, <https://oreil.ly/1lqzK>). Если начать диалог с запроса совета, а затем попросить ИИ последовать его собственному совету, то это может очень помочь ему выбрать правильное направление. По сути, этим приемом вы заставляете ИИ самостоятельно разработать собственное направление.

Ввод:

Пожалуйста, дай мне 5 советов по выбору названий для товара, основанных на рекомендациях отраслевых экспертов.

Вывод:

Вот пять советов по выбору названий для товара, основанных на рекомендациях отраслевых экспертов:

1. ****Сделайте название запоминающимся****: Название должно быть простым и легко запоминающимся. Используйте короткие слова или фразы, которые легко произносятся и пишутся, чтобы потребители могли легко вспомнить ваш продукт.
2. ****Отразите суть продукта****: Название должно передавать основное назначение или уникальные характеристики товара. Это поможет потенциальным покупателям сразу понять, что они могут ожидать от вашего продукта.
3. ****Избегайте сложных терминов****: Используйте понятный язык, избегая жаргона и сложных терминов, которые могут запутать потребителей. Название должно быть доступным для широкой аудитории.
4. ****Проверьте уникальность****: Перед окончательным выбором названия убедитесь, что оно не зарегистрировано другими компаниями и не нарушает авторские права. Это поможет избежать юридических проблем в будущем.
5. ****Тестируйте на целевой аудитории****: Перед тем как принять окончательное решение, протестируйте несколько вариантов названий на вашей целевой аудитории. Соберите отзывы и выберите то, которое вызывает наибольший отклик и интерес.

Затем в том же окне чата, где модель имеет контекст предыдущих советов, которые она дала, можно повторно ввести первоначальный запрос.

Ввод:

Сможешь ли ты, используя эти советы, составить список торговых названий для обуви, которая подойдет на любой размер ноги?

Вывод:

Конечно! Вот список торговых названий для обуви, которая подойдет на любой размер ноги, с учетом вышеуказанных советов:

1. **FitAll**
Простое и запоминающееся название, подчеркивающее универсальность.
2. **OneSize Comfort**
Название, отражающее концепцию удобства для всех размеров.
3. **FlexFit Shoes**
Уникальное название, которое подчеркивает гибкость и комфорт обуви.
4. **Universal Steps**
Название, создающее ассоциацию с доступностью для всех.
5. **Happy Size**
Эмоциональное название, вызывающее позитивные ассоциации с комфортом.

6. Easy Walk
Простое название, которое передает идею легкости и удобства при ходьбе.
7. AllStep Comfort
Название, подчеркивающее, что обувь подходит для любого шага и размера.
8. Comfort Zone
Название, создающее ощущение уюта и расслабления.
9. EveryFit
Уникальное название, говорящее о том, что обувь подходит всем.
10. Freedom Feet
Название, которое вызывает ассоциации со свободой и комфортом.

Надеюсь, эти названия вам понравятся и будут полезны!

Еще одна эффективная, на наш взгляд, стратегия — выбрать лучший ответ ИИ, а затем вставить его в запрос. Например, можно взять пять золотых правил маркетинга, предлагаемых компанией Brandwatch (<https://oreil.ly/3bWjz>), или воспользоваться другим надежным внешним ресурсом и добавить этот контекст в запрос. Конечно, это увеличит длину запроса и стоимость его обработки (при использовании API разработчика), но может оказаться оправданным, если качество ответа улучшится.

Ввод:

Могу ли я получить список торговых названий для обуви, которая подойдет на любой размер ноги? Пожалуйста, следуй нижеперечисленным пяти правилам:

1. Название должно быть простым для чтения и записи. Если название товара будет труднопроизносимым, то люди не будут о нем говорить, и если они не смогут его записать (причем правильно!) с чужих слов, то как они будут его гуглить?

Поэтому не усложняй и не используй разные нелепые написания.

...

5. Название товара должно вызывать эмоцию, чувство или идею. Оно должно быть связано с тем, что представляет собой товар, отражать ощущение, которое, по вашему мнению, должны испытывать люди при его использовании, и/или идею, которую вы пытаетесь донести.

Оно должно быть эмоциональным и вдохновляющим.

Вывод:

С учетом этих правил я могу предложить следующие торговые названия для обуви, которая подойдет на любой размер ноги:

1. "ComfortFit" — обувь, которая обеспечивает комфортную посадку любой ноге.
2. "EasyStep" — обувь, которая позволяет ногам чувствовать себя свободно.

3. "UniversalShoes" – обувь, которая подойдет любой ноге, независимо от размера.
4. "AdaptiveWear" – обувь, которая адаптируется к любой ноге.
5. "FreedomFit" – обувь, которая дает ногам чувство свободы.
6. "OneSize" – обувь, которая подойдет любой ноге, независимо от размера.
7. "SmartShoes" – обувь, которая умна и комфортна.
8. "CozyFit" – обувь, которая обеспечивает уютную посадку любой ноге.
9. "PerfectFit" – обувь, которая обеспечивает идеальную посадку любой ноге.
10. "UniversalFit" – обувь, которая подойдет любой ноге, независимо от размера.

Есть множество других способов задать направление. В примере создания изображения в качестве уточнения направления было указано, что встреча проходит за стеклянным столом. Если изменить только эту деталь, то можно получить совершенно другое изображение, как показано на рис. 1.5.

Ввод:

<https://s.mj.run/TKAsyhNikмс> стоковая фотография деловой встречи в лесу, где 4 человека собрались у костра на открытом воздухе, Panasonic, DC-GH5

Вывод показан на рис. 1.5.



Рис. 1.5. Стоковая фотография деловой встречи в лесу

Ролевая игра также важна для генерации изображений, и к наиболее действенным способам уточнения направления для Midjourney относится указание имени художника или художественного стиля в целях подражания. Одним из художников, на которых часто ссылаются в мире ИИ, является Ван Гог, известный своими смелыми, драматичными мазками и яркими цветами. Посмотрите, что произойдет, если добавить его имя в запрос (рис. 1.6).

Ввод:

Люди на деловой встрече в стиле Ван Гога

Вывод показан на рис. 1.6.



Рис. 1.6. Люди на деловой встрече в стиле Ван Гога

В последнем запросе мы убрали большую часть уточнений направления. Например, отсутствие слов *stock photo* (стоковая фотография) и модели фотокамеры *Panasonic DC-GH5* помогло оформить изображение в стиле Ван Гога. Вы также можете столкнуться с проблемой, когда слишком большое количество уточнений может породить конфликтующую между собой комбинацию условий,

с которыми модель может не справиться. Если запрос слишком конкретный, в обучающих данных может оказаться недостаточно образцов для создания изображения, соответствующего всем вашим критериям. В подобных случаях следует выбрать наиболее важный элемент (в данном случае «в стиле Ван Гога») и положиться на него.

Задать направление — один из самых распространенных и часто используемых принципов. Он может заключаться в выборе правильных описательных слов, поясняющих ваши намерения, или изображений, так или иначе связанных со знаменитостями. Слишком большое количество уточнений может сузить творческие возможности модели, а слишком малое (более распространенная проблема) — привести к результатам, далеким от ожидаемых.

2. Укажите формат

Модели ИИ — универсальные переводчики. Они могут переводить не только текст с французского на английский или с урду на клингонский, но также преобразовывать структуры данных, например JSON в YAML, или текст на естественном языке в код на Python. Эти модели способны выводить ответ практически в любом формате, поэтому важной частью промт-инжиниринга является поиск способов представления ответа в нужном формате.

Иногда в ответ на один и тот же запрос вы можете получать данные в разных форматах, например нумерованный список вместо списка элементов, разделенных запятыми. Обычно это не имеет большого значения, поскольку большинство запросов являются однократными и вводятся непосредственно в форму взаимодействия с ChatGPT или Midjourney. Однако, если вы разрабатываете промышленные инструменты взаимодействия с ИИ, случайные изменения формата могут вызвать всевозможные ошибки.

Как и при работе с человеком, вы можете избежать пустой траты сил и времени, заранее указав формат, в котором ожидаете получить ответ. При работе с моделями генерации текста часто предпочтительнее получать ответы в формате JSON вместо простого упорядоченного списка, потому что это универсальный формат ответов API, который может упростить анализ и обнаружение ошибок, а также использоваться для отображения во внешней HTML-странице приложения. Формат YAML тоже популярен, поскольку имеет структуру, доступную для анализа машиной, оставаясь при этом простым и удобочитаемым.

В исходном запросе мы указали направление с помощью примеров, а двоеточие в конце запроса означало, что ответ должен быть представлен в форме простого списка. Чтобы получить ответ в формате JSON, нужно оформить запрос в том же формате и оставить блок JSON незавершенным, таким образом модель GPT-4 поймет, что его нужно завершить.

Ввод:

Выведи список названий товаров через запятую в формате JSON в ответ на запрос "Обувь, которая подойдет на любой размер ноги".
Представь ответ в формате JSON.

Примеры:

```
[{
  "Описание товара": "машина для приготовления молочных коктейлей.",
  "Название товара": ["HomeShaker", "Fit Shaker",
    "QuickShake", "Shake Maker"]
},
{
  "Описание товара": "Часы, показывающие точное время в космосе.",
  "Название товара": ["AstroTime", "SpaceGuard",
    "Orbit-Accurate", "EliptoTime"]}
]
```

Вывод:

```
[
  {
    "Описание товара": "Обувь, которая подойдет на любой размер ноги.",
    "Название товара": ["EasyFit", "ComfyWalk", "AllSize",
      "Joyful Steps", "FlexiFit", "Happy Feet", "SoftWalk",
      "TrueComfort", "OneStep", "Comfort Zone"]
  }
]
```

В результате мы получили список названий товаров в формате JSON. Его можно проанализировать и использовать программно — в приложении или локальном сценарии. Его также легко проверить на наличие ошибок в форматировании, используя анализатор JSON, такой как стандартная библиотека *json* в Python, поскольку неправильно сформированный код JSON приведет к ошибке анализа, которая может послужить триггером для повторной попытки отправления запроса или дополнительного исследования проблемы. Если вы продолжаете получать неправильный формат, то можно указать в начале или в конце запроса или в системном сообщении (если вы работаете с моделью в ее чате): «Ты хороший помощник, который отвечает только в формате JSON» — либо указать, что результаты должны выводиться в формате JSON (<https://oreil.ly/E7wua>), в параметрах модели, если они доступны (в настройках моделей Llama они называются *грамматиками* — *grammars* (<https://oreil.ly/yU27T>)).



Для тех, кто не знаком с форматом JSON, в W3Schools есть хорошее введение (<https://oreil.ly/Xakgc>).

Для моделей генерации изображений формат очень важен, поскольку возможности представления изображений практически безграничны, от очевидных, таких как стоковые фотографии, иллюстрации и картины маслом, до более необычных, таких как ролики с видеорегистраторов, ледяные скульптуры или сцены из Minecraft (рис. 1.7).

Ввод:

Деловая встреча четырех человек, смотрящих на экран ноутбука, стоящего на столе, в стиле Minecraft

Вывод показан на рис. 1.7.

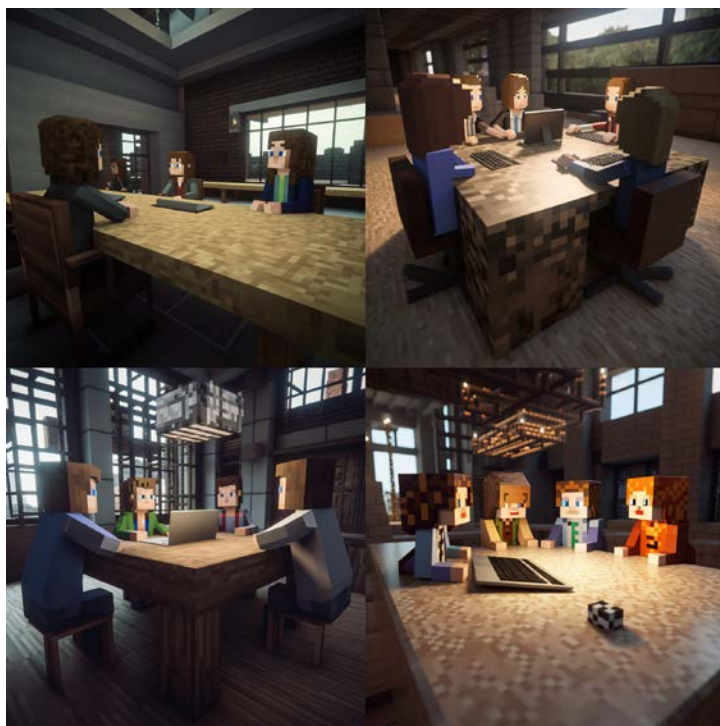


Рис. 1.7. Деловая встреча в стиле Minecraft

При настройке формата обычно лучше убирать из запроса другие аспекты, которые могут конфликтовать с указанным форматом. Например, если указать, что изображение должно быть в формате стоковой фотографии, то результатом

будет комбинация фотографии и желаемого формата. Модели генерации изображений могут до некоторой степени обобщать новые варианты и комбинации, которые они не видели в обучающем наборе данных, но, как показывает опыт, чем больше не связанных между собой элементов, тем выше вероятность получить неподходящее изображение.

Обычно первый и второй принципы — «Задайте направление» и «Укажите формат» — дополняют друг друга. Последний определяет, какой тип вывода вы хотите получить: в формате JSON или в формате стоковой фотографии. Первый задает желаемый стиль ответа, независимого от формата, например названия товаров в стиле Стива Джобса или изображение деловой встречи в стиле Ван Гога. Когда возникают противоречия между стилем и форматом, часто лучше всего устранить их, исключив тот элемент, который менее важен для конечного результата.

3. Приведите примеры

Первоначальный вариант нашего запроса не содержал примеров хороших названий товаров. Поэтому ответ оказался приближенным к усредненным значениям, которые можно найти в Интернете. Но вы можете добиться лучших результатов. Исследователи назвали бы запрос без примеров *пробным выстрелом (zero-shot)*, и всегда, когда ИИ оказывается под силу решить задачу с первого же пробного раза, возникает чувство приятного удивления: это признак широты возможностей модели. Задавая вопрос и не приводя примеров, вы просите слишком много, почти ничего не давая взамен. Даже наличие одного примера (*one-shot*) существенно улучшает результат, поэтому среди исследователей принято проверять эффективность моделей на нескольких примерах (*few-shot*). Как показано в одном из исследований (в знаменитой статье *Language Models are Few-Shot Learners* (<https://oreil.ly/KW5PS>)), добавлением в запрос даже одного примера можно повысить точность выполнения некоторых задач с 10 до почти 50 % (рис. 1.8)!

Инструктируя коллегу или обучая молодого сотрудника, демонстрировать примеры хорошего выполнения поставленной задачи естественно. Работа с ИИ мало чем отличается от подобной ситуации, и сила запроса часто зависит от используемых примеров. Иногда намного проще привести примеры, чем пытаться объяснить, что конкретно требуется, поэтому метод на основе примеров особенно эффективен, когда вы не являетесь экспертом в предметной области той задачи, которую пытаетесь выполнить. Объем текста, который можно поместить в запрос, ограничен (на момент написания книги — около 6000 символов в Midjourney и примерно 32 000 символов в бесплатной версии

ChatGPT), поэтому основная работа при составлении запросов заключается в выборе и вставке необходимых примеров.

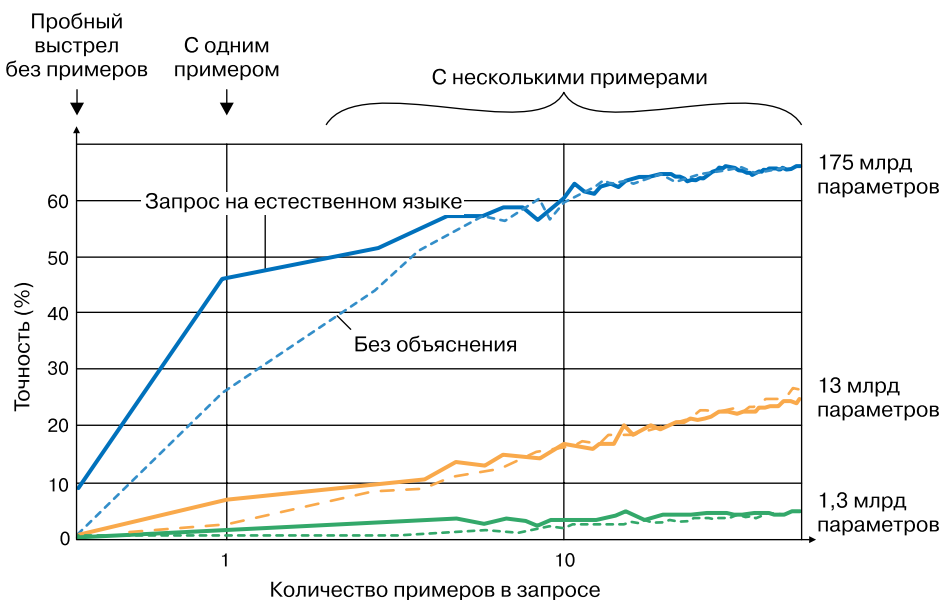


Рис. 1.8. Количество примеров в запросе

Однако между надежностью и креативностью есть компромисс: приведите три-пять примеров, и ваши результаты станут более надежными, но эта надежность будет достигнута за счет креативности. Чем больше примеров вы предоставите и чем меньше будет различий между ними, тем более ограниченным получится ответ, так как модель будет стремиться учитывать все, что было введено. Исправив все примеры в предыдущем запросе на названия животных, вы сильно измените ответ, который будет исправно выводить только названия животных.

Ввод:

Придумай список торговых названий обуви, которая подойдет на любой размер ноги.

Представь результаты в виде списка названий, разделенных запятыми, в формате:
 Описание товара: обувь, которая подойдет на любой размер ноги
 Названия товаров: [список из 3 наименований]

Примеры

Описание товара: машина для приготовления молочных коктейлей.

Названия товаров: Fast Panda, Healthy Bear, Compact Koala

Описание товара: часы, показывающие точное время в космосе.

Названия товаров: AstroLamb, Space Bear, Eagle Orbit

Описание товара: холодильник для пива.

Названия товаров: BearFridge, Cool Cat, PenguinBox

Вывод:

Описание товара: обувь, которая подойдет на любой размер ноги

Названия товаров: EasyFit, ComfyWalk, AllSize

Конечно, при этом возникает риск пропустить гораздо более удачное название, выходящее за рамки ограниченного пространства, отведенного для творчества ИИ. Отсутствие разнообразия и вариаций в примерах — еще одна проблема при обработке крайних случаев или необычных сценариев. Добавление одного-трех примеров почти всегда дает положительный эффект, но если есть желание увеличить это число, то лучше поэкспериментировать с их количеством, а также со сходством между ними. Есть некоторые свидетельства (Hsieh et al., 2023, <https://oreil.ly/6Ixcw>), что уточнение направления более эффективно, чем предоставление примеров. Часто бывает непросто собрать хорошие примеры, поэтому разумнее сначала последовать принципу «Задайте направление».

Для генерирования изображений обычно в запросе представляются примеры в форме, которая в сообществе Stable Diffusion называется *img2img* (<https://oreil.ly/huVRu>), в качестве примера используется изображение. В зависимости от модели примеры могут сильно влиять на результат, и их можно использовать в качестве отправной точки. Вы можете оставить текст запроса без изменений, но заменить базовое изображение, чтобы получить совершенно другой эффект (рис. 1.9).

Ввод:

Стоковая фотография деловой встречи, где 4 человека смотрят видео на белом MacBook, стоящем на стеклянной столешнице, Panasonic, DC-GH5

Вывод показан на рис. 1.9.

В этом случае, когда мы подставили в качестве примера изображение, показанное на рис. 1.10 (также взято из Unsplash), модель изменила результат в другом направлении и на сгенерированном изображении появились доска и стикеры.



Эти примеры демонстрируют возможности моделей, но будьте осторожны, загружая базовые изображения для использования в запросах. Проверьте лицензию изображения, которое вы планируете загрузить, и избегайте использования тех, которые защищены авторским правом, иначе у вас могут возникнуть юридические проблемы. Несоблюдение авторских прав также противоречит условиям обслуживания всех основных поставщиков моделей, генерирующих изображения.



Рис. 1.9. Стоковая фотография деловой встречи четырех человек



Рис. 1.10. Фотография Джейсона Гудмана (Jason Goodman) на Unsplash (<https://oreil.ly/ZbzZy>)

4. Оцените качество

В настоящее время не существует инструментов, позволяющих оценить качество получаемых вами ответов, кроме элементарного метода проб и ошибок и самостоятельного просмотра результатов, который называется «слепым запросом» (<https://oreil.ly/42rSz>). Это нормально, когда запросы используются только один раз для решения одной задачи и редко пересматриваются. Однако, если вы используете один и тот же запрос несколько раз или создаете приложение, использующее запрос в промышленных масштабах, необходим более строгий подход к оценке результатов.

Существует несколько способов оценки качества, и выбор того или иного способа во многом зависит от решаемых задач. Когда выпускается новая модель ИИ, основное внимание, как правило, уделяется тому, насколько хорошо она справляется с *evals* (системой оценки), дает ответы на стандартный набор вопросов с заранее определенными ответами или насколько хорошо она соответствует критериям, которые используются для проверки качества разных моделей. Разные модели ИИ работают по-разному, и нет никакой гарантии, что подсказка в запросе, которая раньше давала хороший результат, будет так же хорошо работать в новой версии. В OpenAI создали свою систему оценки (<https://oreil.ly/wolEL>) для сравнительного анализа эффективности LLM с открытым исходным кодом, и все желающие могут предоставлять дополнительные шаблоны оценки.

Помимо стандартных академических оценок, существуют и более достойные внимания тесты, такие как сдача экзамена на адвоката с помощью GPT-4 (<https://oreil.ly/txhSZ>). Оценка более субъективных задач сложнее и может занять много времени или оказаться непомерно дорогостоящей для небольших команд. В некоторых случаях исследователи обращались к применению более продвинутых моделей, таких как GPT-4, для оценки ответов менее сложных моделей, как это было сделано в случае с Vicuna-13B (<https://oreil.ly/NW3WX>), основанной на модели с открытым исходным кодом Llama от компании Meta и дообученной на специализированном наборе данных (рис. 1.11).

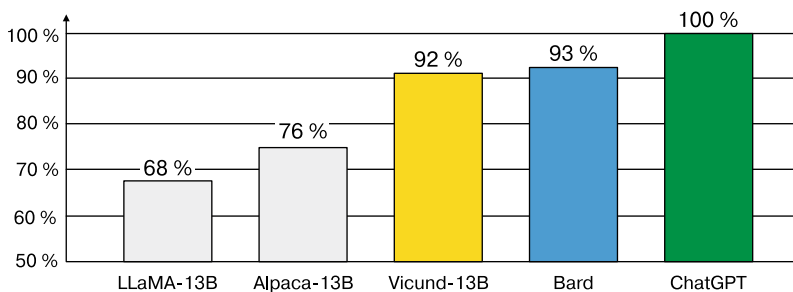


Рис. 1.11. Оценка модели Vicuna с помощью GPT-4

При написании научных статей или оценке новой версии базовой модели нужны более строгие методы оценки, но чаще необходимо сделать всего один шаг за пределы простого метода проб и ошибок. Например, вы можете обнаружить, что простой системы рейтингов «Нравится/Не нравится», реализованной в Jupyter Notebook, более чем достаточно, чтобы увеличить строгость оценки без лишних затрат. Один из распространенных подходов — выяснить, стоит ли предоставлять примеры, увеличивая затраты на обработку более длинных запросов, или можно обойтись без примеров. Первый шаг — выполнить каждый запрос несколько раз и сохранить полученные ответы в электронной таблице, что мы и сделаем после настройки нашей среды.

Вы можете установить пакет OpenAI для Python с помощью команды `pip install openai`. Если у вас возникли проблемы с совместимостью этого пакета, создайте виртуальную среду и установите в нее наш файл `requirements.txt` (<https://oreil.ly/2KDV6>), как описано в предисловии.

Чтобы использовать API, вы должны создать учетную запись OpenAI (<https://oreil.ly/oGv4j>), а затем получить ключ API (<https://oreil.ly/oHID1>).



Из соображений безопасности не рекомендуется явно указывать ключи API в коде сценария. Вместо этого используйте переменные окружения или файлы конфигурации.

Получив ключ API, присвойте его переменной окружения, выполнив следующую команду (замените строку `"api_key"` фактическим значением ключа):

```
export OPENAI_API_KEY="api_key"
```

Или то же самое в Windows:

```
set OPENAI_API_KEY=api_key
```

Если вы предпочитаете не задавать вручную ключ API в переменной окружения, то можете вручную установить его при инициализации модели или загрузить его из файла `.env` с помощью `python-dotenv` (<https://oreil.ly/IaQjS>). Сначала установите библиотеку командой `pip install python-dotenv`, а затем загрузите переменные окружения, добавив следующий код в начало вашего сценария или блокнота:

```
from dotenv import load_dotenv
```

```
load_dotenv() # инициализировать переменные окружения из .env
```

Первый шаг — получение ответов на несколько попыток запустить каждый запрос и сохранение их в электронной таблице.

Ввод:

```
# Определение двух вариантов запросов для сравнения методов
# с одной и с несколькими подсказками
prompt_A = """Описание товара: обувь, которая подойдет на любой размер ноги.
Исходные слова: adaptable, fit, omni-fit.
Название товара: """
```

```
prompt_B = """Описание товара: машина для приготовления молочных коктейлей.
Исходные слова: fast, healthy, compact.
Название товара: HomeShaker, Fit Shaker, QuickShake, Shake Maker
```

```
Описание товара: часы, показывающие точное время в космосе.
Исходные слова: astronaut, space-hardened, elliptical orbit
Название товара: AstroTime, SpaceGuard, Orbit-Accurate, EliptoTime
```

```
Описание товара: Обувь, которая подойдет на любой размер ноги.
Исходные слова: adaptable, fit, omni-fit.
Название товара: """
```

```
test_prompts = [prompt_A, prompt_B]
import pandas as pd
from openai import OpenAI
import os

# Сохраните ключ OpenAI в переменной окружения
# https://platform.openai.com/api-keys
client = OpenAI(
    api_key=os.environ['OPENAI_API_KEY'], # По умолчанию
)
```

```
def get_response(prompt):
    response = client.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=[
            {
                "role": "system",
                "content": "Ты хороший помощник."
            },
            {
                "role": "user",
                "content": prompt
            }
        ]
    )
    return response.choices[0].message.content
```

```
# Опробовать все запросы и получить ответы
responses = []
num_tests = 5

for idx, prompt in enumerate(test_prompts):
    # Номер запроса в виде буквы
    var_name = chr(ord('A') + idx)

    for i in range(num_tests):
        # Получить ответ от модели
        response = get_response(prompt)
        data = {
            "variant": var_name,
            "prompt": prompt,
            "response": response
        }
        responses.append(data)

# Преобразовать ответы в объекты DataFrame
df = pd.DataFrame(responses)

# Сохранить объект в файле CSV
df.to_csv("responses.csv", index=False)

print(df)
```

Вывод:

	variant	prompt	response
0	A	Описание товара: обувь, которая подойдет...	1. Adapt-a-Fit Shoes \n2. Omni-Fit Footwear \n...
1	A	Описание товара: обувь, которая подойдет...	1. OmniFit Shoes\n2. Adapt-a-Sneaks \n3. OneFi...
2	A	Описание товара: обувь, которая подойдет...	1. Adapt-a-fit\n2. Flexi-fit shoes\n3. Omni-fe...
3	A	Описание товара: обувь, которая подойдет...	1. Adapt-A-Sole\n2. FitFlex\n3. Omni-FitX\n4. ...
4	A	Описание товара: обувь, которая подойдет...	1. Omni-Fit Shoes\n2. Adapt-a-Fit Shoes\n3. An...
5	B	Описание товара: машина для приготовления молочных коктейлей.\n...	Adapt-a-Fit, Perfect Fit Shoes, OmniShoe, OneS...
6	B	Описание товара: машина для приготовления молочных коктейлей.\n...	FitAll, OmniFit Shoes, SizeLess, AdaptaShoes
7	B	Описание товара: машина для приготовления молочных коктейлей.\n...	AdaptaFit, OmniShoe, PerfectFit, AllSizeFit.
8	B	Описание товара: машина для приготовления молочных коктейлей.\n...	FitMaster, AdaptoShoe, OmniFit, AnySize Footwe...
9	B	Описание товара: машина для приготовления молочных коктейлей.\n...	Adapt-a-Shoe, PerfectFit, OmniSize, FitForm

Здесь с помощью OpenAI API мы генерируем ответы на множество запросов и сохраняем результаты в объекте `DataFrame`, который затем записываем в файл CSV. Вот как работает этот сценарий.

1. Определяет два варианта запроса, каждый из которых включает описание товара, исходные слова и потенциальные названия, но в запросе `Prompt_B` предоставляются два примера.
2. Операторы импорта подключают библиотеки `Pandas`, `OpenAI` и `os`.
3. Функция `get_response` принимает запрос и возвращает ответ модели `gpt-3.5-turbo`. Запрос пересылается в модель как пользовательское сообщение вместе с системным сообщением для настройки поведения модели.
4. В списке `test_prompts` сохраняются два варианта запроса.
5. Для хранения сгенерированных ответов создается пустой список `responses`, а переменной `num_tests` присваивается значение 5.
6. Для генерации ответов используется вложенный цикл. Внешний цикл перебирает запросы, а внутренний цикл на каждый запрос генерирует ответы, количество которых задано в `num_tests` (в данном случае пять):
 - 1) функция `enumerate` используется для получения индекса и значения каждого запроса в `test_prompts`. Затем индекс преобразуется в соответствующую букву верхнего регистра (например, 0 превращается в *A*, 1 — в *B*) для использования в качестве имени варианта;
 - 2) в каждой итерации вызывается функция `get_response` с текущим запросом, чтобы получить ответ модели;
 - 3) затем создается словарь с именем варианта, запросом и ответом модели, и этот словарь добавляется в список `responses`.
7. Когда будут получены все ответы, список `responses` (который теперь представляет собой список словарей) преобразуется в объект `DataFrame` библиотеки `Pandas`.
8. Затем этот объект сохраняется в CSV-файл с помощью функции `to_csv` из библиотеки `Pandas`, создающей файл `responses.csv` с аргументом `index=False`, который указывает, что индексы строк не должны записываться в файл.
9. В заключение содержимое `DataFrame` выводится в консоль.

Сохранение ответов в электронной таблице полезно само по себе, потому что, уже просто распечатав ответы, можно заметить, что `prompt_A` (zero-shot, без примеров) в первых пяти строках дает нумерованный список, тогда как `prompt_B` (few-shot, с несколькими примерами) в последних пяти строках выводит результаты в желаемом формате списка, перечисляющего названия через запятую. Следующий шаг — оценить каждый ответ, что лучше всего делать вслепую и рандомизированно, чтобы не отдавать предпочтение какому-то одному запросу.

Ввод:

```
import ipywidgets as widgets
from IPython.display import display
import pandas as pd

# загрузить файл responses.csv
df = pd.read_csv("responses.csv")

# Перемешать данные
df = df.sample(frac=1).reset_index(drop=True)

# df — это набор данных, а 'response' - столбец
# с оцениваемым текстом
response_index = 0
# добавить новый столбец для сохранения обратной связи
df['feedback'] = pd.Series(dtype='str')

def on_button_clicked(b):
    global response_index
    # преобразовать голос хорошо («палец вверх»)/плохо («палец вниз») в 1/0
    user_feedback = 1 if b.description == "\U0001F44D" else 0

    # обновить содержимое столбца обратной связи
    df.at[response_index, 'feedback'] = user_feedback

    response_index += 1
    if response_index < len(df):
        update_response()
    else:
        # сохранить оценку в файл CSV
        df.to_csv("results.csv", index=False)

        print("A/B testing completed. Here's the results:")
        # Вычислить оценку и количество записей для каждого варианта
        summary_df = df.groupby('variant').agg(
            count=('feedback', 'count'),
            score=('feedback', 'mean')).reset_index()
        print(summary_df)

def update_response():
    new_response = df.iloc[response_index]['response']
    if pd.notna(new_response):
        new_response = "<p>" + new_response + "</p>"
    else:
        new_response = "<p>No response</p>"
    response.value = new_response
    count_label.value = f"Response: {response_index + 1}"
    count_label.value += f"/{len(df)}"

response = widgets.HTML()
count_label = widgets.Label()
```

```

update_response()

thumbs_up_button = widgets.Button(description='\U0001F44D')
thumbs_up_button.on_click(on_button_clicked)

thumbs_down_button = widgets.Button(
    description='\U0001F44E')
thumbs_down_button.on_click(on_button_clicked)

button_box = widgets.HBox([thumbs_down_button,
thumbs_up_button])

display(response, button_box, count_label)

```

Вывод показан на рис. 1.12.



Рис. 1.12. Система ранжирования по оценкам «хорошо/плохо»

Если запустить этот код в Jupyter Notebook, то каждый ответ ИИ будет отображаться с кнопкой «палец вверх» или «палец вниз» (см. рис. 1.12). Такой простой интерфейс позволяет быстро промаркировать ответы с минимальными затратами. Чтобы проделать то же самое без использования Jupyter Notebook, можно заменить значки «палец вверх» и «палец вниз» на Y (Да) и N (Нет) и реализовать цикл с использованием встроенной функции `input()` взамен `ipywidgets`.

По окончании маркировки ответов вы получите оценку качества каждого запроса.

Вывод:

```

A/B testing completed. Here's the results:
  variant  count  score
0        A      5    0.2
1        B      5    0.6

```

Набор данных был перемешан случайным образом, и каждый ответ маркировался вслепую (без просмотра подсказки), поэтому мы получили довольно точную оценку качества каждой подсказки. Объясним, как работает код, более подробно.

1. Импортируются три модуля: `ipywidgets`, `IPython.display` и `pandas`. Модуль `ipywidgets` содержит интерактивные HTML-виджеты для Jupyter Notebooks и ядро IPython. `IPython.display` предоставляет классы для вывода различных

элементов, таких как изображения, звук, разметка HTML и т. д. Библиотека `pandas` — мощный инструмент для работы с данными.

2. Библиотека `pandas` используется для чтения CSV-файла `responses.csv` с ответами, которые требуется оценить. С ее помощью создается экземпляр `DataFrame` с именем `df`.
3. Содержимое `df` перемешивается вызовом функции `sample()` с параметром `frac=1`, означающим, что в перемешивании участвуют все записи. Вызов `reset_index(drop=True)` используется для перехода к стандартному индексу `0, 1, 2... n`.
4. Сценарий задает индекс первого ответа для оценки `response_index = 0`. Он используется для извлечения и отображения перед пользователем ответа из набора данных.
5. В набор данных `df` добавляется новая оценка содержимого столбца со строковым типом данных `str`.
6. Далее определяется функция `on_button_clicked(b)`, которая будет выполняться при каждом нажатии одной из двух кнопок в интерфейсе:
 - 1) если была нажата кнопка «палец вверх» (`\U0001F44D`; 👍), то функция записывает в `user_feedback` значение `1`, если была нажата кнопка «палец вниз» (`\U0001F44E`; 👎) — значение `0`;
 - 2) после этого полученное значение `user_feedback` записывается в столбец `feedback` текущей записи с индексом `response_index` в наборе данных;
 - 3) затем `response_index` увеличивается для перехода к следующему ответу;
 - 4) если `response_index` все еще меньше общего количества ответов (то есть длины набора данных), то вызывается функция `update_response()`;
 - 5) если ответы закончились, то набор данных сохраняется в новый файл CSV `results.csv` и выводится сообщение с полученными результатами по вариантам: количество полученных оценок и средний балл для каждого варианта.
7. Функция `update_response()` извлекает следующий ответ из набора данных, заключает его в HTML-теги абзаца (если он не пустой) и отображает ответ в виджете `response` и номер текущего ответа с общим количеством ответов в виджете `count_label`.
8. Создаются экземпляры виджетов `response` (виджет HTML) и `count_label` (виджет `Label`). Затем вызывается функция `update_response()` для инициализации виджетов первым ответом и соответствующими значениями счетчиков.
9. Далее создаются еще два виджета, `thumbs_up_button` и `thumbs_down_button`, типа `Button` с использованием значков «палец вверх» и «палец вниз» в качестве описания соответственно. Обе кнопки настраиваются на вызов функции `on_button_clicked()` при нажатии.

10. Две кнопки объединяются в горизонтальный блок (`button_box`) с помощью функции `HBox`.
11. Наконец, виджеты `response`, `button_box` и `count_label` отображаются с помощью функции `display()` из модуля `IPython.display`.

Простая система ранжирования, подобная этой, может пригодиться для оперативной оценки качества и выявления граничных случаев. Обычно менее чем за десять попыток выполнения запроса можно заметить отклонение, которое в противном случае осталось бы незамеченным до момента начала использования в продуктивной среде. Обратной стороной является трудоемкость оценки множества ответов вручную, а кроме того, ваши оценки могут не отражать предпочтений целевой аудитории. Однако даже небольшое количество тестов может помочь заметить различия между двумя стратегиями запросов и выявить неочевидные проблемы еще до того, как они дойдут до продуктивной среды.

Многократное тестирование промтов может привести к радикальному уменьшению их длины и, следовательно, накладных расходов и задержек вашей системы. Если вам удастся найти другой промт, работающий не хуже (а то и лучше), но более короткий, то вы сможете позволить себе значительно расширить круг своей деятельности. Часто в процессе оценки можно обнаружить, что многие элементы сложного промта совершенно излишни или даже контрпродуктивны.

«Палец вверх» и другие индикаторы качества, устанавливаемые вручную, необязательно должны быть единственным критерием оценки. Оценка человеком часто считается наиболее точной формой обратной связи. Однако оценивать множество образцов вручную — довольно утомительное и дорогостоящее занятие. Во многих случаях, например, в задачах по математике или классификации, можно определить *эталонные значения* (эталонные ответы на тестовые запросы) для оценки результатов программно и получить возможность значительно расширить применение тестирования и мониторинга. Ниже приводится список характеристик, которые можно оценивать программно, но имейте в виду, что он далеко не исчерпывающий, потому что таких характеристик можно найти множество.

Стоимость

Промты, использующие много токенов или показывающие хорошие результаты только с дорогими моделями, могут оказаться непрактичными для продуктивной среды.

Задержка

Аналогично чем больше токенов в промте или чем более сложная модель используется, тем больше времени требуется для выполнения задачи, что может отрицательно сказаться на пользовательском опыте.

Количество вызовов

Многим системам ИИ для решения задачи требуется несколько циклических вызовов, что может серьезно замедлить процесс.

Производительность

Реализуйте некоторую форму внешней системы обратной связи, например физический движок или другую модель для прогнозирования реальных результатов.

Классификация

Определите, как часто промт правильно маркирует данный текст, используя другую модель ИИ или маркировку на основе правил.

Рассуждение

Определите, в каких случаях ИИ оказывается неспособен рассуждать логически или допускает ошибки в математических расчетах по сравнению с эталонными примерами.

Галлюцинации

Оцените частоту встречающихся галлюцинаций, которую можно измерить появлением новых терминов, отсутствующих в контексте промта.

Безопасность

Отмечайте любые сценарии, когда система возвращает небезопасные или нежелательные результаты, используя фильтр безопасности или систему обнаружения.

Отказы

Определите, как часто система ошибочно отказывается выполнять обоснованные запросы пользователя, отмечая формулировки отказов.

Состязательность

Сделайте промт устойчивым к известным атакам типа prompt injection (<https://oreil.ly/KGAqe>), которые могут заставить модель выполнять нежелательные запросы вместо тех, что запрограммировали вы.

Сходство

Используйте общие слова и фразы (оценки BLEU и ROGUE (<https://oreil.ly/iEGZ9>)) или векторное расстояние (описывается в главе 5) для измерения сходства между сгенерированным и эталонным текстами.

С помощью оценки хороших результатов вы сможете легко менять примеры в промте и постепенно совершенствовать свою систему. Результаты, полученные

благодаря этой обратной связи, также можно использовать для тонкой настройки (fine-tuning), которая начнет превосходить запросы, созданные на скорую руку, как только вы накопите несколько тысяч примеров (<https://oreil.ly/DZ-br>), как показано на рис. 1.13.

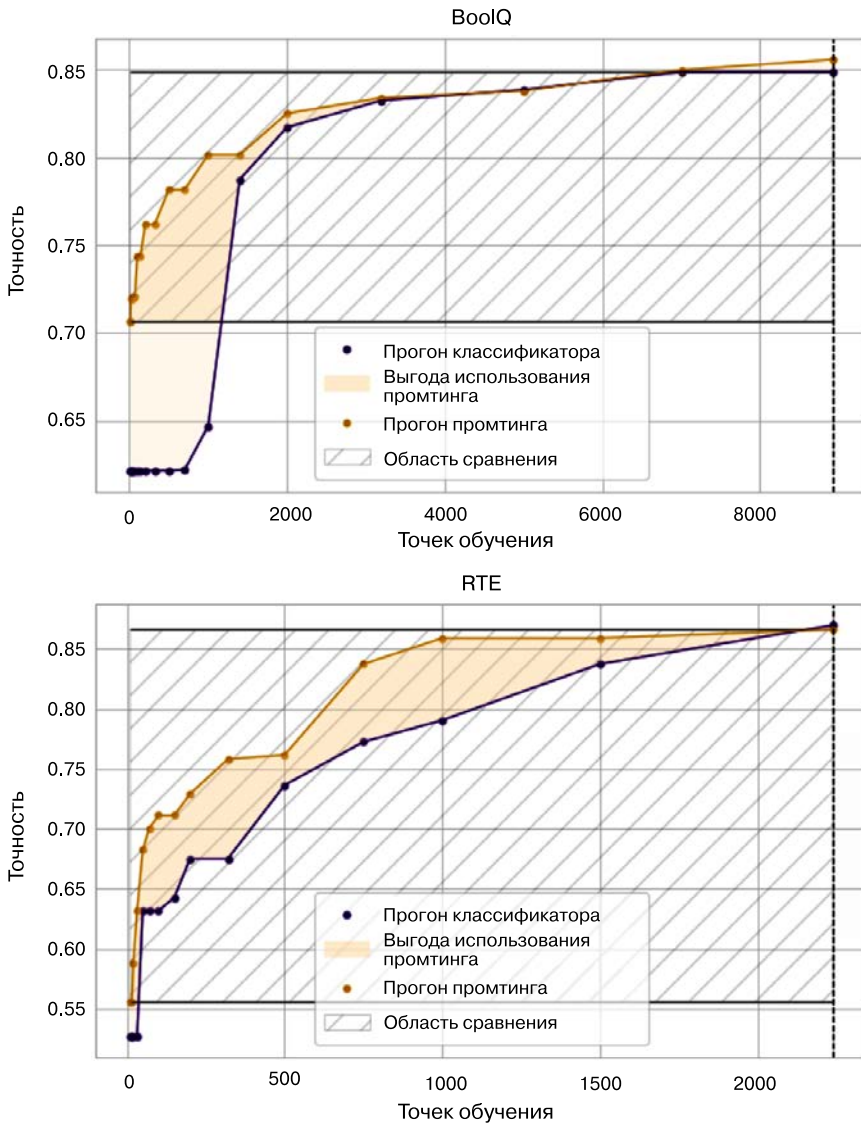


Рис. 1.13. Сколько точек данных стоит промт?

Закончив оценку по принципу «хорошо/плохо», можно внедрить 3-, 5- или 10-балльную систему оценок и получить более детальную обратную связь о качестве промтов. Также можно определить относительную эффективность, сравнивая ответы друг с другом, вместо рассмотрения их по одному. Далее можно провести объективное сравнение моделей с помощью рейтинга Эло (<https://oreil.ly/TlIdE>), популярного в шахматах и используемого в Chatbot Arena (<https://oreil.ly/P2IcU>) на lmsys.org.

Системы генерации изображений обычно оцениваются путем *перестановки в промте*, когда определяется несколько направлений или форматов и для каждой их комбинации генерируется изображение. Изображения можно затем расположить в сетке, чтобы наглядно увидеть, какой эффект оказывают различные элементы запроса.

Ввод:

{стоковая фотография, картина маслом, иллюстрация} деловой встречи, где {четыре, восемь} человек смотрят видео на белом MacBook, стоящем на стеклянной столешнице

При работе с Midjourney этот промт можно преобразовать в шесть разных запросов, по одному для каждой комбинации из трех форматов (стоковая фотография, картина маслом, иллюстрация) и двух человек (четыре, восемь).

Ввод:

1. Стоковая фотография деловой встречи, где четыре человека смотрят видео на белом MacBook, стоящем на стеклянной столешнице
2. Стоковая фотография деловой встречи, где восемь человек смотрят видео на белом MacBook, стоящем на стеклянной столешнице
3. Картина маслом деловой встречи, где четыре человека смотрят видео на белом MacBook, стоящем на стеклянной столешнице
4. Картина маслом деловой встречи, где восемь человек смотрят видео на белом MacBook, стоящем на стеклянной столешнице
5. Иллюстрация деловой встречи, где четыре человека смотрят видео на белом MacBook, стоящем на стеклянной столешнице
6. Иллюстрация деловой встречи, где восемь человек смотрят видео на белом MacBook, стоящем на стеклянной столешнице

В ответ на каждый запрос, как обычно, генерируются четыре изображения, что несколько усложняет просмотр результатов. Мы выбрали по одному изображению, сгенерированному для каждого запроса, и расположили их в виде сетки, как показано на рис. 1.14. Обратите внимание, что модель не всегда генерирует правильное количество человек (генеративные модели ИИ на

удивление плохо разбираются в математике), но она правильно определяет общее намерение и для запросов, где указано восемь человек, генерирует большее число людей.

Вывод показан на рис. 1.14.



Рис. 1.14. Сетка с результатами, полученными методом перестановки в промте

Модели, имеющие API, такие как Stable Diffusion, упрощают манипулирование фотографиями и отображение их в сетке для удобства оценки. Вы также можете изменять случайное зерно изображения, чтобы закрепить стиль для достижения максимальной воспроизводимости результатов. Кроме того, изображения можно оценивать программно с помощью классификаторов по безопасности их содержимого или наличию в них определенных элементов, сигнализирующих об успехе или неудаче.

5. Разделите задачу на подзадачи

По мере создания промта вы начинаете понимать, что запрашиваете слишком много за одно обращение к ИИ. Когда промты становятся длиннее и запутаннее, ответы могут стать менее детерминированными, а количество галлюцинаций или

аномалий — увеличиться. Даже если вам удастся получить надежный промт для решения вашей задачи, то это, скорее всего, означает, что задача является лишь одной из множества взаимосвязанных задач, необходимых вам для выполнения работы. Вполне естественно проанализировать, сколько еще подобных задач может решить ИИ и как их можно объединить.

Один из базовых принципов промт-инжиниринга заключается в том, чтобы с помощью приема декомпозиции разделить задачу на составные части и тем самым упростить решение каждой отдельной подзадачи, а затем объединить результаты. Разделение запроса к ИИ на несколько взаимосвязанных вызовов может помочь в решении очень сложных задач, а также обеспечит лучшую видимость недостатков в цепочке.

На выбор названия товара влияет множество факторов, и нередко решение такой важной задачи возлагается на ИИ, при этом вы не видите, как он оценивает важность этих факторов (если вообще оценивает). Наша текущая система работает так: мы получаем список названий, но все они выводятся без какого-либо дополнительного контекста, который помог бы оценить их важность и принять решение. К счастью, инструменты ИИ способны к самооценке; добавив в задачу второй шаг, мы сможем автоматически проверять наличие нежелательных результатов.

Ввод:

Оцени этот список торговых названий обуви, которая подойдет на любой размер ноги. Оценка должна быть числом от 1 до 10 и выводиться рядом с названием товара:

```
UniFit SoleStrides
FlexiSize All-Fit Shoes
OmniFit FootFlex
...
AnyStep AdaptiWalkers
FlexiRange UniversalSteps
PerfectGlide MultiFit Shoes
```

Вывод:

```
UniFit SoleStrides: 8/10
FlexiSize All-Fit Shoes: 7/10
OmniFit FootFlex: 8/10
...
AnyStep AdaptiWalkers: 9/10
FlexiRange UniversalSteps: 8/10
PerfectGlide MultiFit Shoes: 8/10
```

При многократных попытках ИИ неизменно оценивает название OneSize Glovewalkers как худшее, сообщая (если попросить), что это название может

приводить к недопониманию в контексте обуви. Возникает закономерный вопрос: почему, зная, что это название плохое, модель все равно предлагает его? Это объясняется особенностью работы больших языковых моделей (LLM), которые предсказывают следующий токен в последовательности и поэтому изо всех сил стараются узнать, каким получится общий ответ по завершении. Однако, имея все маркеры из предыдущего ответа, модели легче предсказать, будет ли этот ответ признан хорошим или плохим.

Мы можем выполнить декомпозицию этой задачи для улучшения результатов, например, разделить общий рейтинг на три отдельные оценки: ясность, запоминаемость и насколько хорошо название передает уникальное преимущество товара. А затем передать эти оценки человеку в качестве дополнительного контекста для окончательного решения или даже объединить их для выбора окончательного названия программным способом.

Главное отличие профессионального подхода к работе с ИИ по сравнению с простой игрой с промтами заключается в осознании, что каждую часть системы можно разбить на последовательность шагов. Этот ключ применим даже к простому запросу, поскольку простое добавление «Давай подумаем шаг за шагом» к промту может привести к заметным улучшениям результатов, а также обеспечить зацепки для дальнейшего улучшения качества и отладки. Если потратить время и токены на дополнительные размышления, то оценки изменятся и в большей степени будут соответствовать критериям.

Ввод:

Давай подумаем шаг за шагом. Оцени этот список торговых названий обуви, которая подойдет на любой размер ноги. Оценка должна быть числом от 1 до 10 и выводиться рядом с названием товара:

```
UniFit SoleStrides
FlexiSize All-Fit Shoes
OmniFit FootFlex
...
AnyStep AdaptiWalkers
FlexiRange UniversalSteps
PerfectGlide MultiFit Shoes
```

Вывод:

Вот оценки от 1 до 10 для каждого названия товара с учетом таких факторов, как описательность, запоминаемость и общая привлекательность:

1. ****UniFit SoleStrides****: 8/10 (простое и легко запоминаемое название, но не очень оригинальное).
2. ****FlexiSize All-Fit Shoes****: 7/10 (название отражает функцию обуви, но звучит немного длинно).

3. ****OmniFit FootFlex****: 8/10 (оригинальное название, но может быть трудно запомнить).
- ...
18. ****AnyStep AdaptiWalkers****: 9/10 (название отражает функцию обуви и легко запоминается).
19. ****FlexiRange UniversalSteps****: 8/10 (название отражает функцию обуви, но звучит немного длинно).
20. ****PerfectGlide MultiFit Shoes****: 8/10 (название отражает функцию обуви, но его трудно запомнить).

В OpenAI этот прием называют «предоставлением модели времени на размышление» (<https://oreil.ly/0MZ3->), и это ключевой принцип промт-инжиниринга. По сути, подобные приемы выстраивания *цепочек мышления*, когда модели предлагается перечислить свои шаги, подобны делению задачи в рамках одного запроса. Автоматизировав выбор названия товара с учетом его идеи, мы можем снова вызвать ChatGPT и попросить дать описание каждого товара, а затем передать полученные результаты в Midjourney, чтобы сгенерировать изображение каждого товара. Создание промта для модели ИИ с помощью ИИ называется *метапромтингом*, и это возможно, потому что LLM способны генерировать промты на уровне человека (Zhou, 2022, <https://oreil.ly/Dwszu>).

Ввод:

Опиши подробно новый товар OneFit UltraStride Shoes – обувь, которая подойдет на любой размер ноги:

Вывод:

OneFit UltraStride Shoes – это инновационная обувь, созданная с учетом потребностей каждого пользователя, независимо от размера ноги. Эти кроссовки идеально подходят для людей, которые ищут комфорт, стиль и универсальность в одном продукте.

Описание товара:

1. Адаптивный дизайн:

OneFit UltraStride Shoes оснащены уникальной технологией адаптивной подошвы, которая подстраивается под форму и размер вашей ноги. Эластичные материалы и продуманная конструкция обеспечивают идеальную посадку, что делает обувь комфортной для ношения в течение всего дня.

2. Комфорт и поддержка:

Обувь имеет амортизирующую стельку, которая обеспечивает отличную поддержку свода стопы и снижает нагрузку на суставы при ходьбе или беге. Специальные вентиляционные отверстия обеспечивают отличную воздухопроницаемость, предотвращая перегрев и потение ног.

3. Стильный и современный дизайн:

OneFit UltraStride Shoes представлены в различных цветовых вариантах и стилях, что позволяет легко сочетать их с любой одеждой – от спортивного до повседневного стиля. Современный внешний вид делает их подходящими для любого случая.

4. Устойчивые и долговечные материалы:

Обувь изготовлена из высококачественных, износостойких материалов, которые обеспечивают долговечность и защиту от внешних воздействий. Подошва выполнена из резины, что гарантирует отличное сцепление с поверхностью и предотвращает скольжение.

5. Легкость и гибкость:

OneFit UltraStride Shoes легкие и гибкие, что позволяет легко двигаться и не ощущать дискомфорта даже при длительном ношении. Это делает их идеальным выбором для активного образа жизни.

6. Подходит для всех:

Благодаря своей универсальности обувь подходит как для мужчин, так и для женщин, а также для подростков. OneFit UltraStride Shoes могут использоваться для различных видов активности, включая спорт, прогулки и повседневную носку.

Заключение:

OneFit UltraStride Shoes – это идеальное решение для тех, кто ищет обувь, способную адаптироваться под индивидуальные потребности. Сочетая в себе комфорт, стиль и универсальность, эта обувь станет незаменимым атрибутом в вашем гардеробе.

Модель DALL-E хорошо известна модели GPT-4, благодаря чему можно указать ее имя в ChatGPT и сгенерировать хороший запрос для передачи инструменту создания изображений. Если вы планируете использовать этот промт в продуктивной среде, то подумайте о возможности добавления примеров в подсказку, но для наших нужд она достаточно хороша и без примеров.

Ввод:

Преобразуй это описание в короткий запрос для модели ИИ, генерирующей изображения, такой как DALL-E:

Вывод:

Создайте изображение стильных кроссовок OneFit UltraStride Shoes с адаптивным дизайном, эластичными материалами, амортизирующей стелькой и вентиляционными отверстиями. Обувь должна быть представлена в различных цветах и стилях, подходящих для активного образа жизни, с акцентом на комфорт и универсальность. Подошва должна выглядеть износостойкой и резиновой, обеспечивающей отличное сцепление.

Этот сгенерированный запрос теперь можно передать модели, генерирующей изображения, например DALL-E или Midjourney, и получить хорошее начальное представление о том, как может выглядеть товар. Конечно, полученные изображения могут не быть окончательным дизайном, который вы выберете, но их просмотр вызовет эмоции и поможет быстрее сформировать мнение. С когнитивной точки зрения легче хвалить или критиковать существующее изображение, чем представлять себе внешний вид товара по фрагментам текста.

Вывод показан на рис. 1.15.

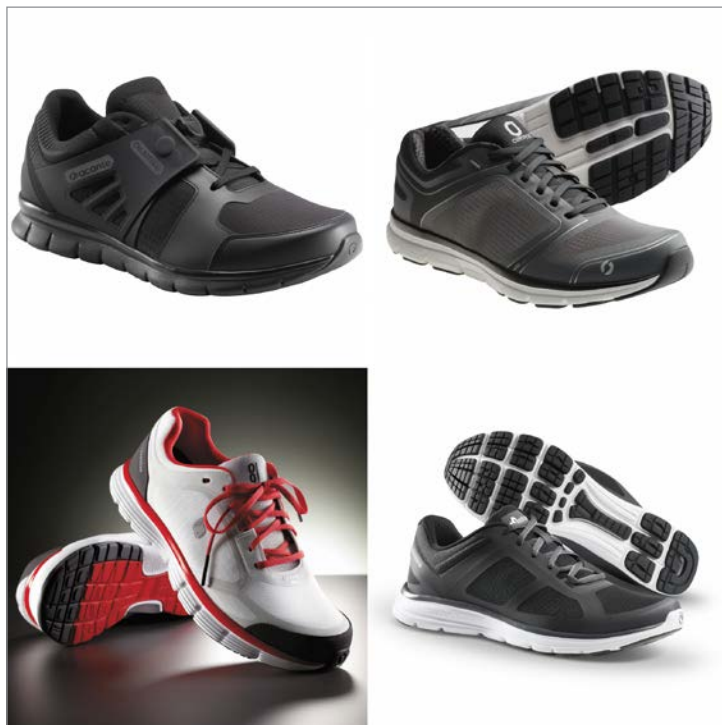


Рис. 1.15. OneFit UltraStride Shoes

При профессиональной работе с ИИ обычной практикой является объединение нескольких запросов и даже нескольких моделей для решения более сложных задач. Даже приложения с одним запросом часто создаются динамически на основе внешнего контекста, запрашиваемого из различных баз данных или получаемого из ответов моделей ИИ. Библиотека LangChain (<https://www.langchain.com>) предлагает инструменты для объединения нескольких шаблонов промтов и запросов, что делает весь процесс более прозрачным и структурированным. Основопологающим примером является прогрессивное обобщение, когда текст, не уместающийся в контекстное окно, делится на несколько фрагментов, после чего каждый из фрагментов обобщается и полученные результаты объединяются. Побеседовав с разработчиками ранних продуктов, взаимодействующих с искусственным интеллектом, вы обнаружите, что все они «за

кулисами» объединяют несколько промтов в цепочку для достижения лучших результатов.

Фреймворк Reason and Act (ReAct) (<https://oreil.ly/tPPW9>) был результатом одной из первых популярных попыток создания агентов ИИ, включая такие проекты с открытым исходным кодом, как BabyAGI (<https://oreil.ly/TEiQx>), AgentGPT (<https://oreil.ly/48lq6>) и Microsoft AutoGen (<https://oreil.ly/KG5XI>). По сути, все эти агенты являются результатом объединения нескольких запросов к ИИ для планирования, наблюдения, выполнения действий и последующей оценки результатов.

Подробнее об автономных агентах мы расскажем в главе 6, а пока отметим, что на момент написания книги они еще не получили широкого распространения. Такая практика саморассуждающих агентов все еще находится на ранней стадии развития и подвержена ошибкам, но есть многообещающие признаки того, что этот подход будет полезен при решении сложных задач и, вероятно, станет частью следующего этапа эволюции систем ИИ.

В настоящее время наблюдается жесткая конкуренция в области ИИ между технологическими гигантами, такими как Microsoft и Google, а также между многочисленными проектами с открытым исходным кодом на Hugging Face и стартапами, финансируемыми венчурным капиталом, такими как OpenAI и Anthropic. По мере появления новые модели диверсифицируются, чтобы конкурировать за разные сегменты растущего рынка. Например, модель Anthropic Claude 2 имеет контекстное окно на 100 000 токенов (<https://oreil.ly/NQcFW>), что намного больше стандартного окна на 8192 токена в GPT-4 (<https://oreil.ly/iZhMI>). Вскоре OpenAI ответила выпуском версии GPT-4 с окном на 128 000 токенов (<https://oreil.ly/3TTZ9>), а Google предлагает контекстное окно в 1 миллион токенов в своей модели Gemini 1.5 (<https://oreil.ly/cyhR4>). Для сравнения, одна из книг о Гарри Поттере содержит около 185 000 токенов, поэтому со временем может стать обычным явлением, когда в один запрос можно будет поместить целую книгу, однако обработка миллиона токенов в каждом вызове API может оказаться непомерно дорогим удовольствием для большинства сценариев.

В этой книге основное внимание уделяется методам генерирования текста с помощью GPT-4 и изображений с помощью Midjourney v6 и Stable Diffusion XL, но через несколько месяцев эти модели могут уже устареть. Это означает, что все более важным становится умение правильно выбирать модель и объединять несколько систем ИИ. Шаблоны промтов редко можно дословно переносить при переходе на новую модель, но пять принципов промтинга помогут приспособить любой ваш промт для любой модели и получить надежные результаты.

Резюме

В этой главе вы узнали о важности промт-инжиниринга в контексте генеративного ИИ. Мы определили промт-инжиниринг как процесс создания эффективных промтов, помогающих получать желаемые результаты при взаимодействии с моделями ИИ. Вы увидели, что задание четкого направления, указание формата результатов, включение примеров, создание системы оценки и разделение сложных задач на подзадачи являются ключевыми принципами промт-инжиниринга. Применяя эти принципы и используя общие методы создания промтов, можно повысить качество и надежность результатов, генерируемых ИИ.

Вы также познакомились с ролью промт-инжиниринга в создании названий и изображений товаров. Увидели, как указание желаемого формата и передача наглядных примеров могут повлиять на результаты работы ИИ. Познакомились с концепцией ролевой игры, в которой можно попросить ИИ сгенерировать результаты, как если бы он был известной личностью, такой как Стив Джобс. В главе подчеркивалось, что для достижения желаемых результатов при использовании генеративных моделей ИИ необходимо четко задать направление и контекст. Помимо этого, вы узнали, насколько важно оценивать качество результатов, генерируемых моделями ИИ, а также о компромиссе между качеством, количеством используемых токенов, стоимостью и задержкой.

В следующей главе мы подробнее поговорим о моделях ИИ для генерации текста. Расскажем о различных типах базовых моделей, их возможностях и ограничениях. Рассмотрим стандартные предложения OpenAI, их конкурентов и альтернативы с открытым исходным кодом. К концу главы вы получите достаточно полное представление об истории моделей генерации текста, их достоинствах и недостатках. В главах 7–9 мы вновь вернемся к вопросу создания промтов для моделей генерации изображений, поэтому вы можете смело пропустить эту часть книги, если генерация текста вас не интересует. Приготовьтесь глубже погрузиться в дисциплину промт-инжиниринга и начать обретать навыки работы с ИИ.

Введение в большие языковые модели

В сфере ИИ в последнее время основное внимание уделяется развитию больших языковых моделей (Large Language Models, LLM). В отличие от своих менее гибких предшественниц, эти модели способны обрабатывать и учиться на очень больших объемах данных, что способствует появлению новых возможностей генерации текста, очень похожего на текст, созданный человеком. Эти модели могут применяться для разных целей: от написания контента до автоматизации разработки программного обеспечения и предоставления интерактивного опыта взаимодействия с чат-ботами в реальном времени.

Что такое модели генерации текста

Модели генерации текста используют мощные алгоритмы для понимания смысла и создания текста, часто неотличимого от созданного человеком. Если вам приходилось общаться с моделью ChatGPT (<https://chat.openai.com>) и вас восхищала ее способность создавать связные и осмысленные предложения, то считайте, что вы были свидетелем возможностей LLM.

В обработке естественного языка (Natural Language Processing, NLP) и LLM фундаментальной лингвистической единицей является *token*. Токены (<https://oreil.ly/3fOsM>) могут представлять предложения, слова и даже части слов. Для взаимодействий с ИИ полезно понимать, как оценивается размер текстовых данных в токенах. Например, текст из 100 токенов соответствует примерно 75 словам. Это сравнение может стать важным для управления ограничениями LLM, поскольку разные модели могут иметь разную емкость в токенах.

Токенизация — это процесс разбиения текста на токены, который является важным шагом в подготовке данных для задач NLP. Существует множество

методов токенизации, включая кодирование пар байтов (Byte-Pair Encoding, BPE) (<https://oreil.ly/iSOp7>), WordPiece и SentencePiece. Каждый из этих методов имеет свои достоинства и подходит для конкретных случаев использования. Например, BPE пользуется большой популярностью из-за его эффективности в обработке широкого спектра словарного запаса и способности сохранить управляемым количество токенов.

Токенизация методом BPE начинается с представления текста как последовательности отдельных символов. После первичного анализа пары символов, часто встречающихся вместе, преобразуются в отдельные единицы, или токены. Чтобы лучше понять, как работает этот метод, рассмотрим слово *яблоко*. На первом этапе BPE может рассматривать это слово как последовательность символов *я*, *б*, *л*, *о*, *к* и *о*. Но, заметив, что *б* часто идет после *я* и перед *л*, он может объединить их и рассматривать *ябл* как один токен.

Этот подход помогает моделям LLM распознавать и генерировать слова или фразы, даже если они отсутствуют в обучающих данных, что делает модели более универсальными.

Понимание работы LLM требует знания основных математических принципов, лежащих в основе этих систем. Конкретные вычисления могут быть очень сложными, но мы можем упростить основные элементы, чтобы прийти к интуитивному пониманию основных принципов работы этих моделей. Точность и надежность LLM имеют первостепенное значение, особенно в контексте бизнеса.

Значительная часть этой надежности достигается на этапах предварительного обучения и тонкой настройки LLM. Первоначально модели обучаются на обширных наборах данных в ходе предварительного обучения и приобретают широкое понимание языка. Впоследствии, на этапе тонкой настройки, модели адаптируются для решения конкретных задач, оттачиваются их возможности по получению точных и надежных результатов для специализированных приложений.

Векторные представления: числовая сущность языка

В сфере NLP слова — это не просто буквенные символы. Их можно токенизировать, а затем представить в числовой форме, известной как *векторы*. Эти векторы представляют собой многомерные массивы чисел, фиксирующие семантические и синтаксические отношения:

$$w \rightarrow \mathbf{v} = [v_1, v_2 \dots v_n]$$

Создание *векторных представлений слов*, также известных как *эмбединги слов* (*word embeddings*), основывается на сложных шаблонах языка. На этапе интенсивного обучения модели разрабатываются для выявления и изучения этих закономерностей, обеспечивая, чтобы слова с похожими значениями располагались близко друг к другу в многомерном пространстве (рис. 2.1).

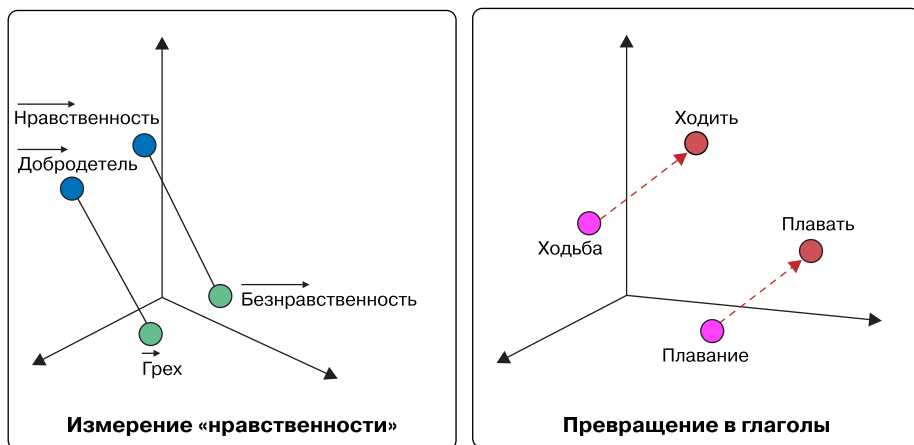


Рис. 2.1. Семантическая близость векторов слов в пространстве векторных представлений

Прелесть этого подхода в его способности улавливать нюансы отношений и рассчитывать расстояния между словами. По результатам исследования векторных представлений слов ясно видно, что слова со схожими или родственными значениями, такие как *добродетель* и *нравственность* или *ходьба* и *ходить*, располагаются рядом друг с другом. Эта близость в *пространстве векторных представлений* (*word embedding space*) становится мощным инструментом в различных задачах NLP, который позволяет моделям понимать контекст, семантику и сложную сеть отношений, формирующих язык.

Архитектура трансформеров: организация контекстных отношений

Прежде чем углубиться в механику работы архитектуры трансформеров, мы должны прийти к общему пониманию базовых деталей. Итак, получив некоторое предложение, например «Кот сидит на коврике», мы должны каждое слово в этом предложении преобразовать в числовое векторное представление. В результате «кот» может стать серией чисел, так же как «сидит», «на» и «коврике».

Как вы узнаете далее в этой главе, архитектура трансформера принимает эти векторы слов и выявляет их отношения — как структурные (синтаксис), так и смысловые (семантика). Существует много типов трансформеров. На рис. 2.2 показан пример архитектуры трансформера, включающей модели BERT и GPT. Кроме того, трансформер рассматривает слова не изолированно; он видит слово «кот» и понимает, что в этом предложении оно связано со словами «сидит» и «коврике».

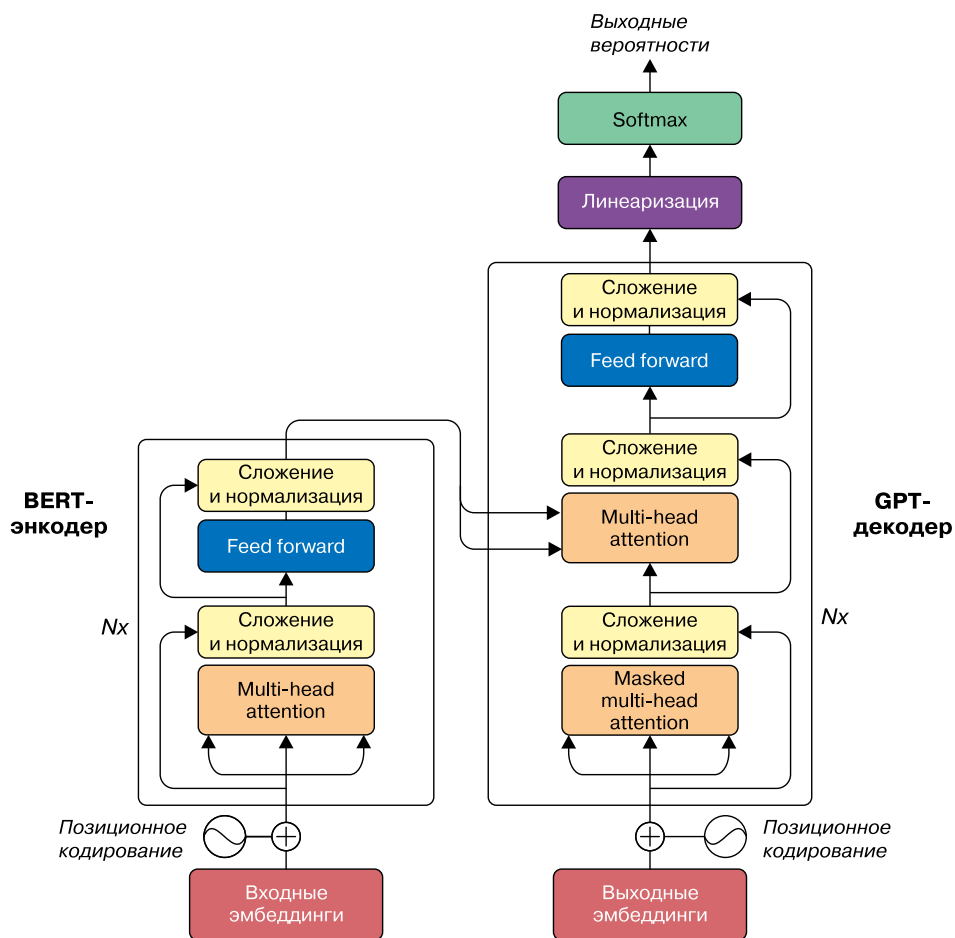


Рис. 2.2. BERT используется для кодирования входных данных, а GPT — для декодирования выходных

Обработывая эти векторы, трансформер с помощью математических операций пытается выявить взаимосвязи между словами и создать новые векторы, обогащенные контекстной информацией:

$$v_i' = \text{Transformer}(v_1, v_2 \dots v_m)$$

Одна из замечательных особенностей трансформеров — способность выявлять тонкие контекстуальные значения слов. Механизм *самовнимания* (*self-attention*) (<https://oreil.ly/xuovP>) в трансформерах позволяет рассматривать каждое слово в предложении с учетом всех остальных слов и лучше понимать его контекст. Это сродни оцениванию важности других слов для определения значения каждого

слова в предложении. Рассматривая предложение целиком, трансформеры могут более точно определить роль и значение каждого слова, делая свои *интерпретации более богатыми контекстуально*.

Вероятностная генерация текста: механизм принятия решений

Изучив контекст заданного текста, трансформер переходит к генерации нового текста, руководствуясь идеями правдоподобия и вероятности. Говоря математическим языком, модель вычисляет вероятность появления каждого следующего возможного слова в текущей последовательности и выбирает наиболее вероятное из них:

$$w_{\text{next}} = \operatorname{argmax} P(w | w_1, w_2, \dots, w_m)$$

Повторяя этот процесс, как показано на рис. 2.3, модель генерирует связную и контекстуально релевантную текстовую строку.

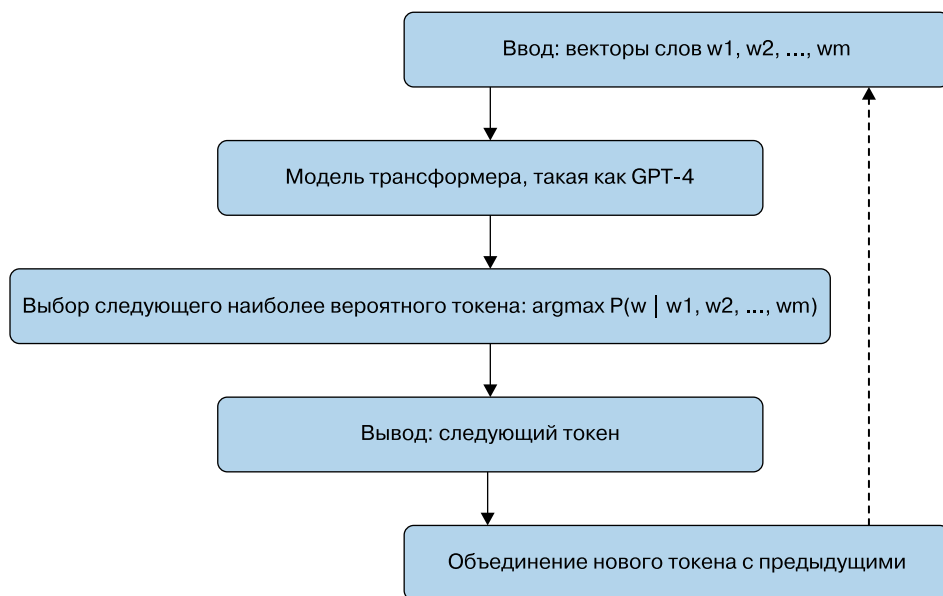


Рис. 2.3. Как генерируется текст с помощью моделей трансформеров, таких как GPT-4

Механизмы, лежащие в основе моделей LLM, основаны на векторной математике, линейных преобразованиях и теории вероятностей. Внутренние операции требуют значительных вычислительных ресурсов, но основные концепции построены на этих математических принципах, формирующих базовое понимание, которое устраняет разрыв между технической сложностью и бизнес-применимостью.

Исторические предпосылки: появление архитектуры трансформеров

Языковые модели, такие как ChatGPT (аббревиатура *GPT* расшифровывается как *generative pretrained transformer* — генеративный, предварительно обученный трансформер), не возникли как по волшебству. Они являются результатом многолетнего развития NLP, особенно ускорившегося в конце 2010-х годов. В основе этого достижения лежит внедрение архитектур трансформеров, которые были подробно описаны в новаторской статье *Attention Is All You Need*¹ (<https://oreil.ly/6NNbg>), написанной членами команды Google Brain.

Настоящим прорывом в архитектуре трансформеров стала концепция *внимания*. Традиционные модели обрабатывают текст последовательно, что ограничивает их возможности в понимании языковой структуры, особенно на больших расстояниях в тексте. Изобретение механизма внимания изменило положение дел и позволило моделям напрямую связывать слова друг с другом, независимо от их местоположения в тексте. Это стало важным нововведением, означающим, что словам и их контексту не нужно проходить через всю модель, чтобы влиять друг на друга. Это не только значительно улучшило понимание текста моделями, но и сделало их более эффективными.

Этот механизм внимания сыграл важную роль в расширении способности моделей обнаруживать зависимости в тексте, отстоящие далеко друг от друга. Это имело решающее значение для получения результатов, которые были не только контекстуально точными, но и согласованными на более длительных отрезках текста.

По словам пионера и преподавателя ИИ Эндрю Энга (Andrew Ng; <https://oreil.ly/JQd53>), большая часть ранних исследований в области NLP, включая фундаментальную работу по трансформерам, получила значительное финансирование от агентств военной разведки США. Их интерес к таким инструментам, как машинный перевод и распознавание речи, прежде всего для разведывательных целей, проложил путь к разработкам, выходящим за рамки простого перевода.

Обучение больших языковых моделей (LLM) требует значительных вычислительных ресурсов. Эти модели обучаются на огромных объемах данных (от терабайтов до петабайтов), включая интернет-контент, научные статьи, книги и другие специализированные наборы данных, предназначенные для конкретных целей. Однако важно отметить, что данные, используемые для обучения LLM, могут нести *предвзятость из своих источников*. Поэтому пользователи

¹ Статья переведена на русский язык: <https://habr.com/ru/companies/ruvds/articles/723538/>. — Примеч. пер.

должны проявлять осторожность и в идеале использовать человеческий контроль при использовании этих моделей, обеспечивая ответственное и этическое применение ИИ.

GPT-4 компании OpenAI, к примеру, может похвастаться примерно 1,7 триллиона параметров (<https://oreil.ly/pZvMo>), что эквивалентно электронной таблице Excel, занимающей 30 000 футбольных полей. *Параметры* в контексте нейронных сетей — это веса и смещения, корректируемые на протяжении всего процесса обучения, что позволяет модели представлять и генерировать сложные шаблоны на основе данных, на которых она обучалась. Стоимость обучения GPT-4 оценивается примерно в 63 миллиона долларов (https://oreil.ly/_NAq5), а данные обучения, если представить их в виде книг, могли бы заполнить около 650 километров книжных полок (<https://oreil.ly/D7jL5>).

Чтобы удовлетворить эти требования, крупные технологические компании, такие как Microsoft, Meta и Google, вложили значительные средства, превратив развитие LLM в задачу первостепенной важности.

Рост количества LLM привел к увеличению спроса на аппаратное обеспечение и особенно на графические процессоры (Graphics Processing Unit, GPU). NVIDIA, например, стала почти синонимом высокопроизводительных графических процессоров, необходимых для обучения LLM.

Спрос на мощные и эффективные графические процессоры резко вырос, потому что компании стремятся создавать все более крупные и сложные модели. Но им требуются не только чистые вычислительные мощности. Графические процессоры также необходимо точно настроить для решения задач, характерных для машинного обучения, таких как тензорные операции. *Тензорами* в машинном обучении называют многомерные массивы данных, и операции над ними лежат в основе нейронных сетей. Акцент на специализированных возможностях привел к появлению специализированного оборудования, такого как графические процессоры NVIDIA H100 с тензорными ядрами, специально созданного для ускорения машинного обучения.

Кроме того, огромный спрос на графические процессоры премиум-класса часто превышает предложение, что приводит к росту цен. Такое взаимодействие спроса и предложения превратило рынок графических процессоров в высококонкурентную и прибыльную сферу. Самые разные клиенты, начиная от технологических гигантов и заканчивая академическими исследователями, изо всех сил стараются закупить наиболее современное оборудование.

Этот всплеск спроса вызвал волну инноваций, затронувших не только графические процессоры. В настоящее время компании сосредоточены на создании специализированного оборудования для нужд ИИ, такого как тензорные процессоры (Tensor Processing Unit, TPU), способные удовлетворить растущие потребности моделей ИИ в вычислительных мощностях.

Такое развитие ситуации подчеркивает не только симбиотические связи между программным и аппаратным обеспечением в сфере ИИ, но также волновой эффект «золотой лихорадки» LLM. Он направляет инновации и инвестиции в различные сектора, особенно туда, где развиваются базовые компоненты для создания этих моделей.

Предварительно обученные генеративные трансформеры OpenAI

Компания OpenAI (<https://openai.com>), поставившая себе цель создать общий ИИ, который приносил бы пользу всему человечеству, в последнее время находится в авангарде революции ИИ. Одним из их самых новаторских достижений стала серия моделей GPT, существенно раздвинувших границы возможностей LLM.

Первоначальная модель GPT, разработанная в OpenAI, стала не просто результатом исследований, но убедительной демонстрацией потенциала архитектур трансформеров. Эта модель продемонстрировала способность машин понимать и генерировать текст на естественном языке и заложила основу для будущих достижений.

Публикация GPT-2 была встречена одновременно с надеждой и настороженностью. Признавая широкие возможности модели, OpenAI поначалу колебалась с ее выпуском из-за опасений по поводу потенциально неправильного ее использования. Мощность GPT-2 была настолько велика, что этические проблемы вышли на первый план, и это может показаться странным, учитывая мощь современных моделей. Однако, когда в OpenAI решили открыть исходный код проекта (<https://openai.com/research/gpt-2>), это означало не только обнародование кода, но также позволило предприятиям и исследователям использовать предварительно обученные модели в качестве строительных блоков в своих приложениях, не начиная все с нуля. Этот шаг демократизировал доступ к возможностям обработки естественного языка высокого уровня, стимулируя инновации в различных областях.

После выпуска GPT-2 в OpenAI решили сосредоточиться на создании коммерческих моделей с закрытым исходным кодом. Появление GPT-3 стало большим шагом в развитии LLM. Она привлекла значительное внимание средств массовой информации не только из-за своего технического совершенства, но и из-за социальных последствий. Эта модель могла создавать настолько убедительный текст, что его часто не могли отличить от текста, написанного человеком. GPT-3 продемонстрировала, казалось бы, безграничный потенциал ИИ — от создания сложных литературных произведений до написания фрагментов рабочего программного кода.

GPT-3.5-turbo и ChatGPT

Благодаря значительным инвестициям со стороны Microsoft OpenAI разработала GPT-3.5-turbo — оптимизированную версию ее и без того исключительной предшественницы. Получив 1 миллиард долларов (<https://oreil.ly/1C8qm>) от Microsoft в 2019 году, а потом еще 13 миллиардов за 49 % акций своего коммерческого подразделения, OpenAI использовала эти ресурсы для разработки модели GPT-3.5-turbo, отличающейся повышенной эффективностью и доступностью, что фактически сделало LLM еще более доступной для широкого спектра вариантов использования.

OpenAI хотела собрать как можно больше отзывов со всего мира для более тонкой настройки, и поэтому родилась ChatGPT (<https://chat.openai.com>). В отличие от своих сестер общего назначения, модель ChatGPT была настроена (<https://oreil.ly/6ib-Q>) для общения в диалоговом контексте, чтобы вести диалог с людьми, который казался бы им естественным и содержательным.

На рис. 2.4 показан процесс обучения ChatGPT, включающий три основных этапа.

Сбор демонстрационных данных

На этом этапе специалисты по разметке предоставляют примеры желаемых ответов модели на множество запросов. Разметчики, обученные в рамках проекта, следуют конкретным инструкциям и стараются давать максимально точные ответы.

Обучение контролируемой политики

Демонстрационные данные, собранные на предыдущем шаге, используются для тонкой настройки предварительно обученной модели GPT-3 с помощью метода контролируемого обучения. При данном подходе модели получают размеченный набор данных, содержащий правильные ответы. Этот шаг помогает модели научиться следовать заданным инструкциям и генерировать ответы, соответствующие желаемым.

Сбор сравнительных данных и обучение с подкреплением

На этом этапе собирается набор ответов модели, и специалисты по разметке ранжируют их, исходя из своих предпочтений. Затем обучается модель вознаграждения, которая предсказывает, какие выводы разметчики предпочли бы. Наконец, применяются методы обучения с подкреплением, в частности алгоритм оптимизации проксимальной политики (Proximal Policy Optimization, PPO), для оптимизации обучения с учителем и максимизации вознаграждения, получаемого от модели вознаграждения.

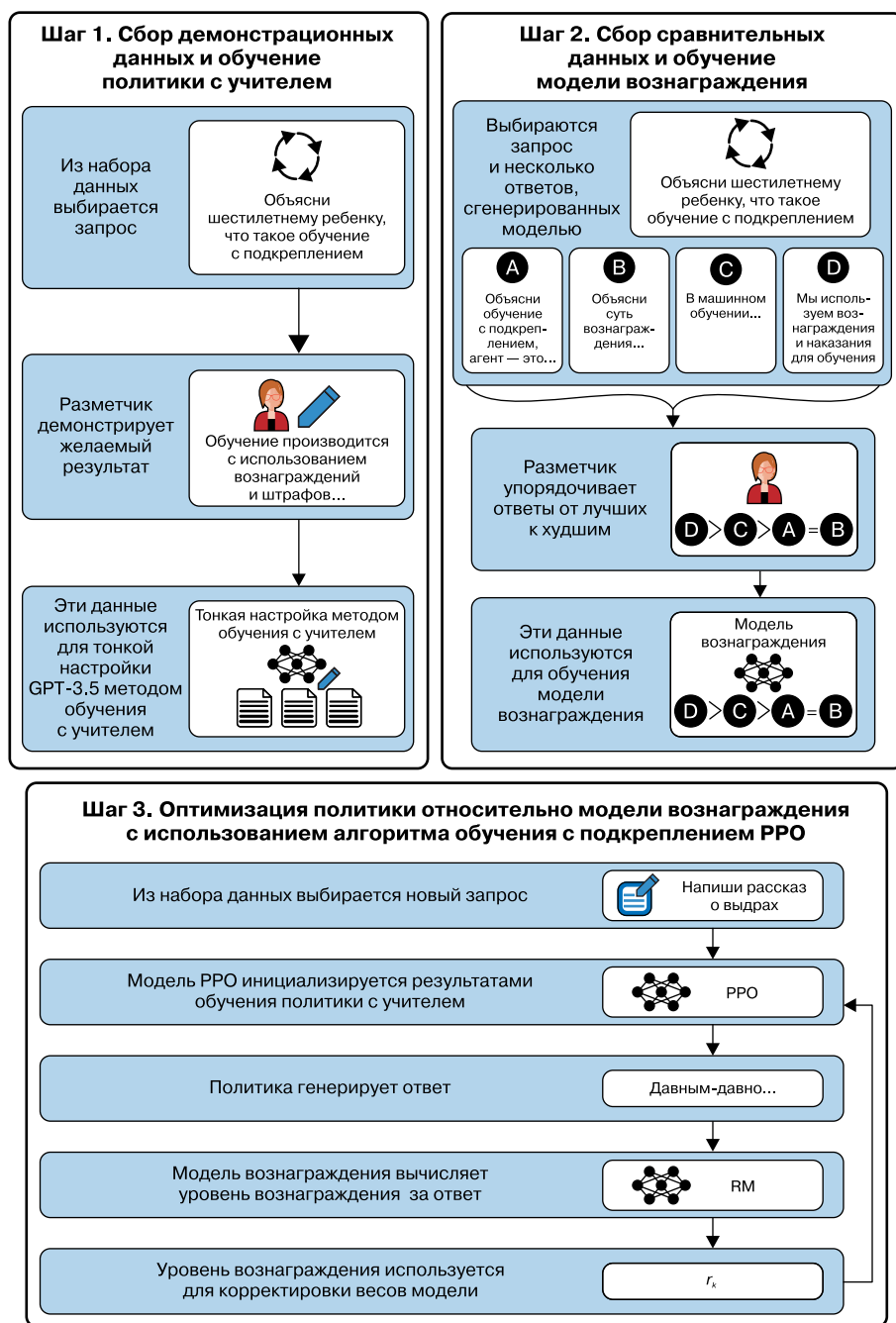


Рис. 2.4. Процесс тонкой настройки ChatGPT

Этот процесс обучения позволяет модели ChatGPT подстраивать свое поведение под намерения человека. Использование обучения с подкреплением и обратной связью от человека помогло создать модель, более полезную, честную и безопасную по сравнению с предварительно обученной моделью GPT-3.

Согласно исследованию UBS (<https://oreil.ly/2Ivq2>), к январю 2023 года ChatGPT установила новый рекорд, набрав 100 миллионов активных пользователей и став самым быстрорастущим потребительским приложением в истории Интернета. ChatGPT теперь является идеальным инструментом для обслуживания клиентов, виртуальной помощи и множества других применений, требующих изящества человеческого общения.

GPT-4

В 2024 году OpenAI выпустила модель GPT-4, превосходно понимающую сложные запросы и генерирующую контекстуально релевантный и связный текст. Например, GPT-4 достигла 90-го перцентиля на экзамене на адвоката с результатом 298 из 400. В настоящее время GPT-3.5-turbo можно использовать в ChatGPT бесплатно, но для пользования GPT-4 нужна подписка с ежемесячной оплатой (<https://oreil.ly/UOEBM>).

GPT-4 использует подход на основе модели коллектива экспертов (<https://oreil.ly/v45LZ>), то есть для получения еще более точных результатов она не полагается на выводы одной модели.

Тринадцатого мая 2024 года OpenAI представила GPT-4o (<https://oreil.ly/4ttmq>) — усовершенствованную модель, способную обрабатывать и анализировать текстовые, аудио- и визуальные данные в режиме реального времени. Эта модель обладает улучшенными характеристиками, особенно в области распознавания образов и звуков. Кроме того, она быстрее и экономичнее своих предшественниц, благодаря способности обрабатывать все три модальности в одной нейронной сети.

Google Gemini

После того как Google потеряла долю рынка поисковиков из-за использования ChatGPT, она выпустила модель Bard 21 марта 2023 года. Bard была далека от совершенства (<https://oreil.ly/Sj24h>) и изначально не отличалась высоким качеством ответов, как ChatGPT (рис. 2.5).

Со временем Google продолжила расширять возможности Bard, включив генерацию программного кода, распознавание изображений и голоса и поиск в реальном времени, и наконец приблизила ее по качеству к ChatGPT.

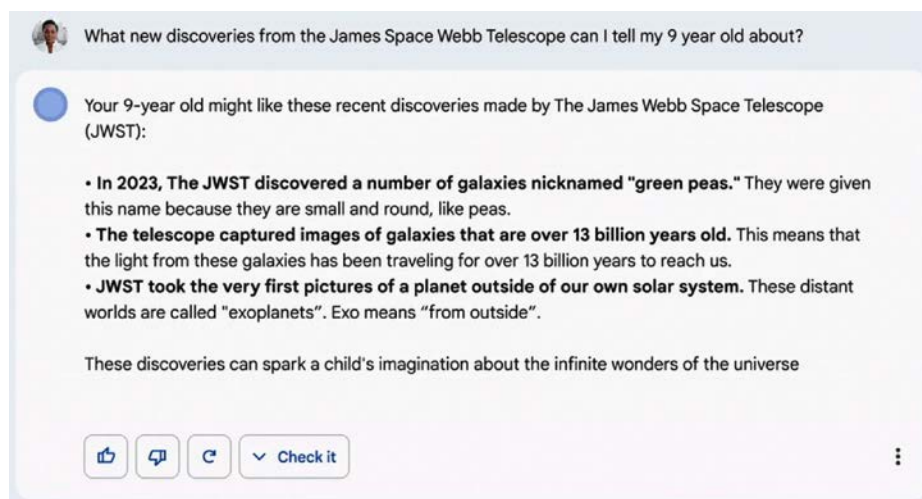


Рис. 2.5. Результаты галлюцинаций Bard о космическом телескопе Джеймса Уэбба

Четырнадцатого марта 2023 года Google выпустила PaLM API (<https://oreil.ly/EbI8->), доступный разработчикам через Google Cloud Platform. В апреле 2023 года Amazon Web Services (AWS) выпустила аналогичные сервисы: Amazon Bedrock (<https://oreil.ly/4fNqX>) и Titan FM (<https://oreil.ly/FJ-7D>). Google переименовала модель Bard в Gemini (<https://oreil.ly/EO42O>), выпустив версию 1.5 в феврале 2024 года, и начала получать результаты, аналогичные GPT-4.

Кроме того, Google выпустила две меньшие модели с открытым исходным кодом (<https://oreil.ly/LWIwv>), основанные на той же архитектуре, что и Gemini. Теперь OpenAI не является единственным очевидным вариантом для инженеров-программистов, ищущих возможность интеграции самых современных LLM в свои приложения.

Meta Llama и открытый исходный код

Подход, предпринятый в Meta к созданию языковых моделей, существенно отличается от подхода конкурентов. Последовательно выпуская модели с открытым исходным кодом: Llama (<https://oreil.ly/LroPn>), Llama 2 (<https://oreil.ly/NeZLw>) и Llama 3 (<https://oreil.ly/Vwlo->), Meta стремится способствовать созданию более инклюзивной и совместной экосистемы разработки ИИ.

Открытая природа Llama 2 и Llama 3 имеет серьезные последствия для всей технологической отрасли и особенно для крупных предприятий. Прозрачность и дух сотрудничества способствуют быстрым инновациям, позволяя

глобальному сообществу разработчиков быстро выявлять и устранять проблемы и уязвимости. Эти модели становятся все более надежными и безопасными, поэтому крупные корпорации могут применять их с большей уверенностью.

Стратегия Meta по развитию моделей с открытым исходным кодом не только открывает широкий доступ к новейшим технологиям ИИ, но и потенциально может оказать существенное влияние на всю отрасль. Благодаря созданию основы для совместного, прозрачного и децентрализованного процесса разработки появились новаторские модели Llama 2 и Llama 3, которые вполне могут определить будущее генеративного ИИ. Модели доступны в версиях с 7, 8 и 70 миллиардами параметров на AWS, Google Cloud, Hugging Face и других платформах.

Открытая природа этих моделей — палка о двух концах. С одной стороны, они выравнивают правила игры. Это означает, что даже мелкие разработчики могут внести свой вклад в инновации, улучшая и применяя модели с открытым исходным кодом в практических бизнес-приложениях. Такого рода децентрализованные инновации могут привести к прорывам, невозможным в пределах одной организации, расширяющим возможности моделей и круг их применения.

С другой стороны, открытость создает потенциальные риски, поскольку позволяет злоумышленникам использовать эту технологию в неблагоприятных целях. Это действительно вызывает беспокойство, которое разделяют такие организации, как OpenAI, предполагая, что некоторая степень контроля и ограничений может на самом деле смягчить последствия опасного применения этих мощных инструментов.

Использование квантования и LoRA

Один из революционных аспектов моделей с открытым исходным кодом — возможность использования методов квантования (<https://oreil.ly/bkWXk>) и низкоранговой аппроксимации (Low-Rank Approximation, LoRA (<https://oreil.ly/zORsB>)). Эти методы позволяют разработчикам размещать модели в меньшем объеме аппаратного обеспечения. Квантование помогает снизить числовую точность параметров модели и тем самым уменьшить ее общий размер без существенной потери качества. А LoRA помогает оптимизировать архитектуру сети, делая ее более эффективной для работы на устройствах потребительского уровня.

Такая оптимизация упрощает тонкую настройку моделей LLM на устройствах потребительского уровня. Это очень важно, потому что появляется возможность расширить границы экспериментов и практического применения. Отдельные разработчики, малые предприятия и стартапы больше не ограничиваются необходимостью прибегать к услугам мощных центров обработки данных. Теперь они могут работать над моделями в средах с более ограниченными ресурсами.

Mistral

Mistral 7B, детище французского стартапа Mistral AI (<https://mistral.ai>), стала локомотивом в области генеративного ИИ с ее 7,3 миллиарда параметров. Эта модель отличается не только размером, но также эффективностью и возможностями, обещающими блестящее будущее большим языковым моделям с открытым исходным кодом и их применимость во множестве сценариев. Ключом к эффективности является реализация скользящего окна внимания — метода, распространяемого на условиях лицензии Apache с открытым исходным кодом. Многие инженеры по ИИ взяли эту модель в качестве основы, в том числе создатели впечатляющей бета-версии Zephyr 7b (https://oreil.ly/Lg6_r). Существует также модель Mixtral 8x7b (<https://oreil.ly/itsJG>), моделирующая группу экспертов (по аналогии с архитектурой GPT-4), которая достигает результатов уровня GPT-3.5-turbo.

Более полное и актуальное сравнение моделей с открытым исходным кодом и их качества вы найдете в таблице Chatbot Arena Leaderboard (<https://oreil.ly/ttiji>) на площадке Hugging Face.

Anthropic: Claude

Вышедшая 11 июля 2023 года модель Claude 2 (<https://claude.ai/login>) выгодно отличается от других известных LLM, таких как ChatGPT и LLaMA, своим новаторским подходом к безопасности и обучению ИИ на основе конституции ИИ (<https://oreil.ly/Tim9W>), когда обучение модели производится с использованием списка правил и ценностей. Заметным улучшением в Claude 2 стало расширенное контекстное окно на 100 000 токенов, а также возможность загружать файлы. В сфере генеративного ИИ *контекстное окно* определяет объем текста или данных, которые модель может помнить и учитывать при генерации ответа. Чем больше контекстное окно, тем больший объем контекстной информации модель может использовать при генерировании данных.

Это достижение вызвало прилив энтузиазма у инженеров ИИ, открыв возможности для новых и более сложных вариантов использования. Например, способность Claude 2 обрабатывать большой объем информации одновременно позволяет ей обобщать обширные документы или поддерживать продолжительное общение. Но преимущество было недолгим, потому что OpenAI выпустила свою версию GPT-4 с контекстным окном 128 000 токенов всего шесть месяцев спустя (<https://oreil.ly/BWxrn>). Однако жесткая конкуренция между соперниками толкает отрасль вперед.

Следующее поколение Claude включало Opus (<https://oreil.ly/NH0jh>) — первую модель, конкурирующую с GPT-4 с точки зрения интеллекта, а также Haiku, быструю и небольшую модель, доступную по конкурентоспособной цене: 0,25 доллара за миллион токенов (в два раза дешевле, чем GPT-3.5-turbo на тот момент).

GPT-4V(ision)

Сделав значительный шаг вперед, 23 сентября 2023 года OpenAI расширила возможности GPT-4, представив версию Vision своей модели, позволяющую пользователям поручить ей анализировать изображения наряду с текстом. Это нововведение также повлекло изменение интерфейса ChatGPT, который теперь поддерживает включение как текста, так и изображений. Это означает сложившуюся тенденцию к созданию мультимодальных моделей, которые могут обрабатывать и понимать несколько типов данных, таких как изображения и текст, в одном контексте.

Сравнение моделей

На момент написания книги на рынке LLM доминировала OpenAI, а ее современная модель GPT-4, по общему мнению, имела значительное преимущество. Ближайшим конкурентом была Anthropic, и многие испытывали энтузиазм по поводу возможностей небольших моделей с открытым исходным кодом, таких как Llama и Mistral, особенно подвергшихся тонкой настройке. Специалисты ожидают, что OpenAI продолжит создавать лучшие в мире модели, но так как модели с открытым исходным кодом становятся *достаточно хорошими* для решения большего количества задач, приложения ИИ могут сместиться в сторону локальных, тонко настроенных моделей. Благодаря повышению производительности моделей и развитию методов квантования (помогающих найти компромисс между точностью, размером и стоимостью вычислений) однажды появится возможность запускать LLM на мобильных телефонах или других устройствах.

На данный момент прекрасный способ понять, какая модель лучше в той или иной сфере, — передать один и тот же запрос нескольким моделям и сравнить ответы. В нашей работе постоянно впереди всех оказывается модель GPT-4, которая намного лучше справляется с запросами, как показано в следующем примере, где эта модель оказалась единственной, давшей ответ в правильном формате и с названиями, соответствующими примерам (начинаются с буквы *i*).

Ввод:

Придумай список названий обуви как товара в стиле Стива Джобса, которая подойдет на любой размер ноги.

Верни результаты в виде списка названий, разделенных запятыми, в формате:
 Описание товара: обувь, которая подойдет на любой размер ноги
 Названия товаров: [список из трех наименований]

Примеры

Описание товара: холодильник для пива

Названия товаров: iBarFridge, iFridgeBeer, iDrinkBeerFridge

Описание товара: часы, показывающие точное время в космосе

Названия товаров: iNaut, iSpace, iTime

Описание товара: машина для приготовления молочных коктейлей

Названия товаров: iShake, iSmoothie, iShake Mini

Вывод (GPT-4):

Описание товара: обувь, которая подойдет на любой размер ноги

Названия товаров: iStride, iFit, iWalk

Вывод (Claude 3):

Описание товара: обувь, которая подойдет на любой размер ноги

Названия товаров: iFit, iComfort, iSole

Вывод (Llama 3 70b):

Вот список названий обуви, подходящей на любой размер ноги,
в стиле Стива Джобса:

Описание товара: обувь на любой размер ноги

Названия товаров: iFit, OneSize, iWalkFree

Резюме

Путь развития LLM от появления до нынешнего состояния — это путь непрерывных инноваций, сотрудничества и жесткой конкуренции. Поскольку эти модели продолжают развиваться, они, вероятно, еще плотнее войдут в нашу повседневную жизнь и изменят наши способы взаимодействия с технологиями и даже с другими людьми.

Понимая исторический контекст и возможности этих моделей, вы сможете лучше оценить имеющиеся в нашем распоряжении инструменты для разных приложений — от промт-инжиниринга до разработки виртуальных помощников. Однако важно отметить, что с расширением возможностей моделей все острее встает проблема конфиденциальности данных. Если для дообучения и тонкой настройки моделей вы используете свои данные, то соблюдайте осторожность и воздержитесь от ввода конфиденциальной информации.

В следующей главе вы познакомитесь со всеми основными приемами промт-инжиниринга, используемыми для работы с текстовыми LLM. Вы освоите базовые навыки, повышающие эффективность использования мощных языковых моделей, таких как GPT-4. Захватывающие идеи и практические методики ждут, когда вы раскроете истинный потенциал генеративного ИИ.

Стандартные приемы генерации текста с помощью ChatGPT

Простые методы промптинга помогают получать от LLM максимально точные результаты в желаемом формате. В этой главе мы рассмотрим все распространенные методы конструирования запросов, используемые для генерации текста.

Генерирование списков

Автоматическое генерирование списков — невероятно полезный прием, позволяющий сосредоточиться на задачах более высокого уровня и переложить создание, уточнение, переупорядочение и удаление дубликатов на GPT.

Ввод:

Сгенерируй список персонажей Диснея.

Вывод:

Конечно, вот список некоторых популярных персонажей Диснея:

1. Микки Маус
2. Минни Маус
- ...
30. Олаф (Холодное сердце)

GPT-4 способна перечислить известных персонажей. Однако такой подход имеет некоторые подводные камни:

- GPT решила представить 30 примеров в виде нумерованного списка, разделенного символами `\n`. Однако если этот список будет обрабатывать код на Python, ожидающий получить маркированный список, то, скорее всего, вы получите не те результаты, что ожидали, или даже ошибку во время выполнения;

- GPT добавила комментарий перед списком; удаление комментария в начале или в конце облегчит анализ вывода;
- размер списка не задавался и был выбран самой моделью;
- для одних персонажей в скобках указано название соответствующего фильма (например, *Олаф (Холодное сердце)*), а для других нет. Это усложняет извлечение имен, потому что вам придется удалить названия фильмов;
- при генерации списка модели ничего не сообщалось о предпочтительности выбора персонажей.

Ниже приводится оптимизированный запрос.

Ввод:

Составь маркированный список из 5 персонажей Диснея мужского пола.
В каждой строке указывай только имя персонажа.
Не включай названия фильмов, где встречается персонаж Диснея.
Верни только имена персонажей Диснея, без комментариев.

Пример списка:

- * Аладдин
- * Симба
- * Чудовище
- * Геркулес
- * Тарзан

Вывод:

- * Микки Маус
- * Дональд Дак
- * Гуфи
- * Аладдин
- * Капитан Крюк



Приведите примеры

Простое перефразирование запроса с включением примеров (*запрос с несколькими примерами*) может существенно повлиять на результат.

Оптимизировав запрос, мы добились следующего:

- ограничили список пятью именами;
- включили в список только персонажей мужского пола;
- придали списку требуемый формат;
- удалили все начальные комментарии.

Простые списки хорошо подходят для большинства задач; однако они менее структурированы, а, кроме того, иногда на выходе из GPT-4 желательно получать данные с вложенной структурой.

Вот три типичные структуры таких данных:

- вложенные текстовые данные (иерархические списки);
- JSON;
- YAML.

Генерирование иерархического списка

Иерархические списки удобны, когда результат должен быть представлен в виде вложенной структуры данных. Хорошим примером может служить описание структуры статьи.

Ввод:

Создай иерархический и очень подробный план статьи по следующей теме:

Каковы преимущества инженерии данных?

Пример иерархической структуры:

Название статьи: Каковы преимущества цифрового маркетинга?

- * Введение
 - а. Что такое цифровой маркетинг
 - б. Важность цифрового маркетинга в современном деловом мире
- * Повышение узнаваемости бренда
 - а. Определение узнаваемости бренда
 - б. Как цифровой маркетинг помогает повысить узнаваемость бренда

Вывод:

Название статьи: Каковы преимущества инженерии данных?

- * Введение
 - а. Определение инженерии данных
 - б. Роль инженерии данных в современном бизнесе
 - в. Цели и задачи статьи
- * Улучшение качества данных
 - а. Определение качества данных
 - б. Методы обеспечения качества данных
 - в. Влияние на принятие решений
- * Эффективность обработки данных
 - а. Оптимизация потоков данных
 - б. Упрощение ETL-процессов (извлечение, трансформация, загрузка)
 - в. Снижение времени на обработку данных

- * Поддержка аналитики и бизнес-анализа
 - а. Роль инженерии данных в аналитических процессах
 - б. Улучшение доступности данных для аналитиков
 - в. Примеры успешных бизнес-решений на основе данных
- * Масштабируемость и гибкость
 - а. Возможности масштабирования систем хранения данных
 - б. Адаптация к изменениям в объеме и типах данных
 - в. Примеры гибких архитектур данных
- * Повышение безопасности данных
 - а. Важность безопасности данных в инженерии данных
 - б. Методы защиты данных
 - в. Соответствие нормативным требованиям
- * Снижение затрат
 - а. Оптимизация ресурсов и затрат на хранение и обработку данных
 - б. Примеры снижения затрат в компаниях
 - в. Долгосрочные финансовые преимущества
- * Улучшение взаимодействия между командами
 - а. Роль инженерии данных в междисциплинарных командах
 - б. Улучшение коммуникации между инженерами данных и аналитиками
 - в. Примеры успешных проектов, основанных на сотрудничестве
- * Заключение
 - а. Резюме ключевых преимуществ инженерии данных
 - б. Значение инженерии данных для будущего бизнеса
 - в. Призыв к действию для организаций, не использующих инженерию данных

Чтобы создать такой подробный план статьи, мы добавили в запрос две ключевые фразы:

иерархический

чтобы сообщить, что план статьи должен иметь вложенную структуру;

очень подробный

чтобы вынудить языковую модель сгенерировать как можно более длинный результат. Похожий эффект может дать словосочетание «очень длинный» или явное указание количества подзаголовков, например: «включи не менее 10 заголовков верхнего уровня».



Запрос фиксированного количества элементов не гарантирует, что языковая модель выдаст именно такое их количество. Например, запросив 10 заголовков, вы можете получить только 8. Поэтому ваш код должен проверять количество заголовков в выводе либо быть достаточно гибким, чтобы обрабатывать ответы LLM различной длины.

Итак, мы получили желаемый иерархический план статьи, но как преобразовать этот текст в структурированные данные?

Рассмотрим пример 3.1 реализации такого преобразования на Python. Здесь используются два регулярных выражения, извлекающие заголовки и подзаголовки из строки в переменной `openai_result`. Модуль `re` в Python содержит функции для работы с регулярными выражениями.

Пример 3.1. Разбор иерархического списка (<https://oreil.ly/A0ot5>)

```
import re

# openai_result = generate_article_outline(prompt)
# Закомментировано, чтобы сосредоточиться на конкретном ответе LLM, см. ниже:

openai_result = '''
* Введение
    а. Определение инженерии данных
    б. Роль инженерии данных в современном бизнесе
    в. Цели и задачи статьи
* Улучшение качества данных
    а. Определение качества данных
    б. Методы обеспечения качества данных
    в. Влияние на принятие решений
* Заключение
    а. Резюме ключевых преимуществ инженерии данных
    б. Значение инженерии данных для будущего бизнеса
    в. Призыв к действию для организаций, не использующих инженерию данных
...

# Шаблоны регулярных выражений
heading_pattern = r'\* (.+)'
subheading_pattern = r'\s+[а-з,а-я]\. (.+)'

# Извлечение заголовков и подзаголовков
headings = re.findall(heading_pattern, openai_result)
subheadings = re.findall(subheading_pattern, openai_result)

# Вывод результатов
print("Заголовки:\n")
for heading in headings:
    print(f"* {heading}")

print("\nПодзаголовки:\n")
for subheading in subheadings:
    print(f"* {subheading}")
```

Этот код выведет:

```
Заголовки:
- Введение
- Улучшение качества данных
- Заключение
```

Подзаголовки:

- Определение инженерии данных
- Роль инженерии данных в современном бизнесе
- Цели и задачи статьи
- Определение качества данных
- Методы обеспечения качества данных
- Влияние на принятие решений
- Резюме ключевых преимуществ инженерии данных
- Значение инженерии данных для будущего бизнеса
- Призыв к действию для организаций, не использующих инженерию данных

Использование регулярных выражений позволяет эффективно отыскивать фрагменты текста, соответствующие шаблону, обрабатывать вариации во входном тексте, такие как наличие или отсутствие начальных пробелов или табуляций. Рассмотрим, как работают эти шаблоны:

```
heading_pattern = r'\* (.+)'
```

Шаблон предназначен для извлечения основных заголовков и включает следующие элементы:

- `\*` соответствует символу звездочки (*) в начале заголовка. Обратная косая черта используется для экранирования звездочки, потому что звездочка имеет особое значение в регулярных выражениях (ноль или более вхождений предшествующего ей символа);
- после звездочки должен следовать пробел;
- `(.+)` соответствует одному или нескольким символам, а круглые скобки создают сохраняющую группу. Символ точки (.) в шаблоне — это подстановочный знак, который соответствует любому символу, кроме символа новой строки, а знак плюс (+) — это квантификатор, означающий *одно или несколько* вхождений предыдущего элемента (в данном случае точки).

Применяя этот шаблон, можно извлечь в список все основные заголовки без звездочек.

```
subheading_pattern = r'\s+[a-z,a-я]\. (.+)'
```

Шаблон `subheading_pattern` соответствует всем подзаголовкам в строке `openai_result`:

- `\s+` соответствует одному или нескольким пробельным символам (пробелам, табуляциям и т. д.), а знак плюс (+) означает *одно или несколько* вхождений предшествующего элемента (в данном случае `\s`);
- `[a-z,a-я]` соответствует одной строчной букве от `a` до `z` или от `а` до `я`;

- \. соответствует символу точки. Обратная косая черта используется для экранирования точки, поскольку она имеет особое значение в регулярных выражениях (соответствует любому символу, кроме символа новой строки);
- *после точки должен следовать пробел*;
- (.+) соответствует одному или нескольким символам, а круглые скобки создают сохраняющую группу. Символ точки (.) в шаблоне — это подстановочный знак, который соответствует любому символу, кроме символа новой строки, а знак плюс (+) — это квантификатор, означающий *одно или несколько* вхождений предыдущего элемента (в данном случае точки).

Для поиска и получения списка всех непересекающихся совпадений во входной строке с шаблоном используется функция `re.findall()`. Затем полученные заголовки и подзаголовки выводятся инструкцией `print`.

Итак, теперь вы можете извлекать заголовки и подзаголовки из иерархических структур статей. Однако можно дополнительно уточнить регулярные выражения, чтобы каждый заголовок был связан с соответствующими подзаголовками.

В примере 3.2 используется немного измененное регулярное выражение, помогающее связать каждый заголовок с соответствующими ему подзаголовками.

Пример 3.2. Разбор иерархического списка в словарь Python (<https://oreil.ly/LcMtv>)

```
import re

openai_result = """
* Введение
    а. Определение инженерии данных
    б. Роль инженерии данных в современном бизнесе
    в. Цели и задачи статьи
* Улучшение качества данных
    а. Определение качества данных
    б. Методы обеспечения качества данных
    в. Влияние на принятие решений
* Заключение
    а. Резюме ключевых преимуществ инженерии данных
    б. Значение инженерии данных для будущего бизнеса
    в. Призыв к действию для организаций, не использующих инженерию данных
"""

section_regex = re.compile(r"\* (.+)"
subsection_regex = re.compile(r"\s*([a-z,а-я]\..+)"

result_dict = {}
current_section = None
```

```

for line in openai_result.split("\n"):
    section_match = section_regex.match(line)
    subsection_match = subsection_regex.match(line)
    if section_match:
        current_section = section_match.group(1)
        result_dict[current_section] = []
    elif subsection_match and current_section is not None:
        result_dict[current_section].append(subsection_match.group(1))

print(result_dict)

```

Этот код выведет:

```

{
  "Введение": [
    "а. Определение инженерии данных",
    "б. Роль инженерии данных в современном бизнесе",
    "в. Цели и задачи статьи"
  ],
  "Улучшение качества данных": [
    "а. Определение качества данных",
    "б. Методы обеспечения качества данных",
    "в. Влияние на принятие решений"
  ],
  "Заключение": [
    "а. Резюме ключевых преимуществ инженерии данных",
    "б. Значение инженерии данных для будущего бизнеса",
    "в. Призыв к действию для организаций, не использующих инженерию данных"
  ]
}

```

Регулярное выражение, извлекающее заголовки разделов, `r'\* (.+)'`, соответствует звездочке, за которой следует пробел, а затем один или несколько символов. Круглые скобки захватывают текст после звездочки и пробела, который используется в коде ниже.

Регулярное выражение подраздела `r'\s*([a-z,а-я]\. .+)'` начинается с элемента `\s*`, соответствующего нулю или более пробельным символам (пробелам или табуляциям). Это позволяет обнаруживать названия подразделов, с пробелами или табуляциями в начале или без них. Следующая часть `([a-z,а-я]\. .+)` соответствует строчной букве, за которой следует точка, а затем один или несколько символов. Круглые скобки захватывают весь текст названия подраздела для последующего использования в коде.

Цикл `for` перебирает каждую строку во входном тексте `openai_result`. Обнаружив строку, соответствующую регулярному выражению `section_regex`, цикл запоминает совпавший текст как заголовков текущего раздела в виде ключа словаря `result_dict` и присваивает ему пустой список в качестве значения. Если строка соответствует регулярному выражению `subsection_regex`, то совпавший текст подраздела добавляется в список текущего раздела.

Проще говоря, цикл обрабатывает *входной текст построчно*, классифицирует строки как заголовки разделов или подразделы и создает словарь, отражающий структуру заголовков.

Когда следует избегать регулярных выражений

Применение одних только регулярных выражений для извлечения еще более структурированных данных из ответов LLM *может сделать реализацию очень сложной*. Однако существуют другие форматы, которые могут облегчить анализ структурированных данных. Два из них — файлы `.json` и `.yaml`.

Генерирование данных в формате JSON

Давайте поэкспериментируем с организацией запроса и попробуем заставить LLM возвращать ответ в формате JSON.

Ввод:

Создай подробный план статьи по теме "Преимущества изучения программирования", который имеет формат JSON и выделяет ключевые моменты.

Верни только действительный JSON.

Пример структуры JSON:

```
{
  "Введение": [
    "а. Что такое инженерия данных",
    "б. Важность инженерии данных в современном мире, управляемом данными"
  ],
  ...
  "Заключение": [
    "а. Важность инженерии данных в современном деловом мире",
    "б. Будущее инженерии данных и ее влияние на экосистему данных"
  ]
}
```

Вывод:

```
{
  "Введение": [
    "а. Определение программирования",
    "б. Значение программирования в современном мире"
  ],
  ...
  "Заключение": [
    "а. Резюме ключевых преимуществ изучения программирования",
    "б. Призыв к действию для начинающих изучать программирование"
  ]
}
```



Задайте направление и приведите примеры

Обратите внимание, что в предыдущем запросе мы указали тип задачи, формат JSON и пример вывода в этом формате.

Распространенные ошибки, с которыми можно столкнуться при работе с JSON, связаны с недопустимыми данными или заключением кода JSON в тройные обратные кавычки (```), например:

Вывод:

Конечно, вот код JSON:

```
```json
{"Name": "John Smith"} # допустимые данные
{"Name": "John Smith", "some_key":} # недопустимые данные
```
```

В идеале желательно, чтобы модель реагировала на запрос следующим образом:

```
{"Name": "John Smith"}
```

Это важно, потому что в первом случае вам придется выделить текст после `json`, а затем проанализировать часть строки, содержащую действительный код JSON. Вот несколько подсказок, которые можно добавить в запрос, чтобы получить более качественный код JSON:

Ты должен следовать следующим принципам:

- * Возвращать только действительный код JSON
- * Никогда не использовать обратные кавычки, такие как: ```
- * Ответ будет анализироваться с помощью `json.loads()`, поэтому он должен содержать действительный код JSON.

Теперь посмотрим, как проанализировать вывод в формате JSON с помощью Python (<https://oreil.ly/MoJHn>):

```
import json

# openai_json_result = generate_article_outline(prompt)

openai_json_result = """
{
  "Введение": [
    "а. Определение программирования",
    "б. Значение программирования в современном мире"],
  "Заключение": [
    "а. Резюме ключевых преимуществ изучения программирования",
    "б. Призыв к действию для начинающих изучать программирование"]
}
```

```
"""
parsed_json_payload = json.loads(openai_json_result)

print(parsed_json_payload)

'''{'Введение': ['а. Определение программирования',
'б. Значение программирования в современном мире'],
'Заключение': ['а. Резюме ключевых преимуществ изучения программирования',
'б. Призыв к действию для начинающих изучать программирование]}'''
```

Мы успешно справились с анализом JSON.

Как видите, извлечение данных из ответа LLM в структурированном виде упрощается, если потребовать вернуть данные в допустимом формате JSON. По сравнению с применением регулярных выражений, продемонстрированным ранее, этот метод выглядит менее громоздким и более простым.

Но может ли что-то пойти не так?

- Языковая модель может случайно добавить в ответ дополнительный текст, например `json output:`, тогда как логика вашего приложения обрабатывает только действительный код JSON.
- Модель может сгенерировать недействительный код JSON, вызывающий ошибки при попытке проанализировать его (например, из-за размера или простого отсутствия экранирования служебных символов).

Позже мы рассмотрим стратегии корректной обработки таких крайних случаев.

YAML

Формат YAML способен представлять структурированные данные и имеет некоторые преимущества перед JSON.

Нет необходимости экранировать символы

Правила использования отступов в YAML избавляют от необходимости использовать для обозначения структуры фигурные и квадратные скобки и запятые. Это позволяет создавать более ясные данные с меньшим числом ошибок, поскольку снижается вероятность неправильного размещения знаков препинания.

Удобочитаемость

YAML удобочитаем и имеет более простой синтаксис и структуру по сравнению с JSON. Это упрощает создание, чтение и редактирование запросов, особенно при работе со сложными или вложенными структурами.

Комментарии

В отличие от JSON, YAML поддерживает комментарии, что позволяет добавлять пояснения к запросам непосредственно в код. Это может быть особенно полезно при работе в команде или при повторном изучении запросов через некоторое время.

Ввод:

- Ниже приводится текущая схема `yaml`.
- Количества можно менять на указанные в запросе пользователя.
- Запрос пользователя следует отфильтровать на основе приведенной схемы, и если он не соответствует схеме или товаров не осталось, то нужно вернуть сообщение ``"Нет товаров"``.
- Если есть частичное совпадение, то вернуть нужно только те товары, которые перечислены в схеме ниже:

схема:

- товар: ломтики яблока
количество: 5
единица измерения: штуки
- товар: молоко
количество: 1
единица измерения: литры
- товар: хлеб
количество: 2
единица измерения: буханки
- товар: яйца
количество: 1
единица измерения: десятки

Запрос пользователя: "5 ломтиков яблок и 2 десятка яиц".

Учитывая приведенную схему, верни только действительный код `yaml`, основываясь на запросе пользователя. В отсутствие совпадений верни "Нет элементов". Не предоставляй никаких комментариев или пояснений.

Вывод:

- товар: ломтики яблока
количество: 5
единица измерения: штуки
- товар: яйца
количество: 2
единица измерения: десятки

Обратите внимание, что в предыдущем примере LLM смогла сформировать правильный ответ в формате YAML, основываясь на строке запроса пользователя.

Кроме того, мы потребовали от LLM:

- вернуть ответ в действительном формате YAML;
- вернуть отфильтрованный ответ в формате YAML;
- если после фильтрации не осталось товаров, то вернуть текст «Нет товаров».

Фильтрация текста в формате YAML

Тот же запрос можно использовать для выделения полезных данных в формате YAML.

Для начала посмотрим, как выглядят полезные данные, имеющие допустимую и недопустимую схемы. Ломтики яблока соответствуют критериям, а бананы отсутствуют в схеме, и можно ожидать, что пользовательский запрос будет соответствующим образом отфильтрован.

Ввод:

```
# Запрос пользователя:
- товар: ломтики яблока
  количество: 5
  единица измерения: штуки
- товар: бананы
  количество: 3
  единица измерения: штуки
```

Вывод:

```
# Обновленный список yaml
- товар: ломтики яблока
  количество: 5
  единица измерения: штуки
```

В предыдущем примере мы успешно отфильтровали полезные данные из запроса пользователя по заданным критериям и использовали языковую модель в качестве *механизма рассуждения*.

Передавая модели набор инструкций в запросе, мы получаем ответ, близкий к тому, который дал бы человек, если бы он очищал данные вручную.

Подсказка в запросе помогает переложить на языковую модель LLM большую часть задач, которые обычно требуют написания кода на таком языке программирования, как Python или JavaScript.

На рис. 3.1 представлена логика, применяемая при обработке пользовательских запросов с помощью LLM.

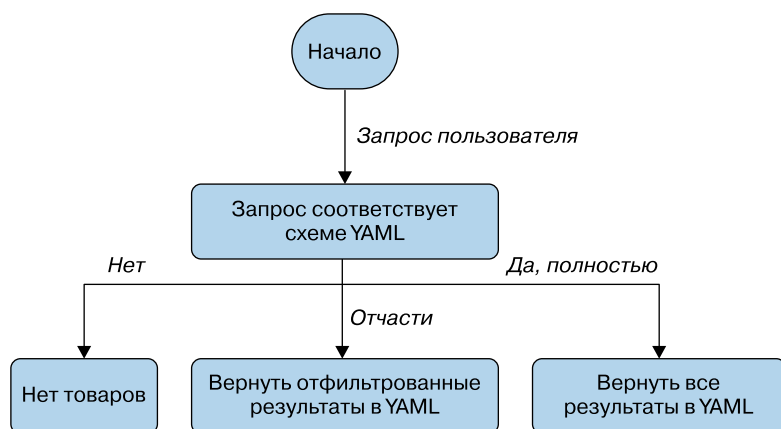


Рис. 3.1. Использование LLM вместо программного кода для решения прикладных задач

Обработка недопустимых полезных данных в YAML

Полностью недействительные полезные данные могут выглядеть так.

Ввод:

```
# Запрос пользователя:  
- товар: бананы  
  количество: 3  
  единица измерения: штуки
```

Вывод:

Нет товаров

Как и ожидалось, LLM вернула ответ «Нет товаров», потому что ни один из товаров в запросе пользователя не соответствует ранее определенной схеме.

Давайте напишем сценарий на Python, который корректно распознает разные ответы, возвращаемые моделью LLM. Основные части сценария будут сосредоточены на:

- создании пользовательских исключений для каждого типа ошибок, которые могут возникнуть в трех сценариях;
- анализе предложенной схемы;
- выполнении дополнительных проверок ответа, чтобы быть уверенными, что ответ YAML можно безопасно передать последующим программным приложениям/микросервисам.

Мы можем определить шесть конкретных ошибок, которые будут обрабатываться во всех крайних случаях:

```
class InvalidResponse(Exception):
    pass

class InvalidItemType(Exception):
    pass

class InvalidItemKeys(Exception):
    pass

class InvalidItemName(Exception):
    pass

class InvalidItemQuantity(Exception):
    pass

class InvalidItemUnit(Exception):
    pass
```

Затем предоставим ранее предложенную схему YAML в виде строки:

```
# Схема
schema = """
- товар: ломтики яблока
  количество: 5
  единица измерения: штуки
- товар: молоко
  количество: 1
  единица измерения: литры
- товар: хлеб
  количество: 2
  единица измерения: буханки
- товар: яйца
  количество: 1
  единица измерения: десятки
"""
```

Импортируем модуль `yaml` и создадим свою функцию анализа с именем `validate_response`, которая поможет нам определить, действительны ли выходные данные LLM:

```
import yaml

def validate_response(response, schema):
    # анализ схемы
    schema_parsed = yaml.safe_load(schema)
    maximum_quantity = 10

    # Проверка, является ли ответ response списком
    if not isinstance(response, list):
        raise InvalidResponse("Response is not a list")
```

```

# Проверка, является ли каждый элемент списка словарем
for item in response:
    if not isinstance(item, dict):
        raise InvalidItemType('Item is not a dictionary')

    # Проверка, содержит ли каждый словарь ключи "товар", "количество"
    # и "единица измерения"
    if not all(key in item for key in ("товар", "количество",
                                      "единица измерения")):
        raise InvalidItemKeys("Item does not have the correct keys")

    # Проверка, связано ли с каждым ключом значение корректного типа
    if not isinstance(item["товар"], str):
        raise InvalidItemName("Item name is not a string")
    if not isinstance(item["количество"], int):
        raise InvalidItemQuantity("Item quantity is not an integer")
    if not isinstance(item["единица измерения"], str):
        raise InvalidItemUnit("Item unit is not a string")

    # Проверка, связано ли с каждым ключом значение, имеющее
    # корректную величину
    if item["товар"] not in [x["товар"] for x in schema_parsed]:
        raise InvalidItemName("Item name is not in schema")
    if item["количество"] > maximum_quantity:
        raise InvalidItemQuantity(f'Item quantity is greater than
                                   {maximum_quantity}')
    if item["единица измерения"] not in ["штуки", "десятки"]:
        raise InvalidItemUnit("Item unit is not pieces or dozen")

```

Для тестирования этих крайних случаев используем несколько фиктивных ответов LLM:

```

# Фиктивные ответы
fake_response_1 = """
- товар: ломтики яблока
  количество: 5
  единица измерения: штуки
- товар: яйца
  количество: 2
  единица измерения: десятки
"""

fake_response_2 = """
# Обновленный список yam1
- товар: ломтики яблока
  количество: 5
  единица измерения: штуки
"""

fake_response_3 = """Не имеет совпадений"""

```


Теперь мы можем:

- использовать `yaml.safe_load(response)` для безопасного анализа схемы YAML;
- вызвать функцию `validate_response` для каждого ответа LLM, чтобы проверить его на соответствие пользовательской логике.

```
# Анализ фиктивных ответов
response_1_parsed = yaml.safe_load(fake_response_1)
response_2_parsed = yaml.safe_load(fake_response_2)
response_3_parsed = yaml.safe_load(fake_response_3)

# Проверка ответов на соответствие схеме
try:
    validate_response(response_1_parsed, schema)
    print("Response 1 is valid")
except Exception as e:
    print("Response 1 is invalid:", str(e))

try:
    validate_response(response_2_parsed, schema)
    print("Response 2 is valid")
except Exception as e:
    print("Response 2 is invalid:", str(e))

try:
    validate_response(response_3_parsed, schema)
    print("Response 3 is valid")
except Exception as e:
    print("Response 3 is invalid:", str(e))

Response 1 is valid
Response 2 is valid
Response 3 is invalid: Response is not a list
```

Генерирование разных форматов с помощью ChatGPT

ChatGPT способна генерировать ответы не только в JSON и YAML, но и во множестве других форматов. Например, она может помочь создать диаграмму на языке Mermaid или макет документа `.csv`. Она настолько универсальна, что может даже генерировать программный код, диалоги и сценарии.

Давайте подробно рассмотрим, как ChatGPT создает разнообразный форматированный контент. Mermaid — это язык сценариев, похожий на язык разметки markdown для создания диаграмм, графиков и других визуализаций.

Ввод:

Создай блок-схему на языке Mermaid, описывающую простой процесс заказа продуктов, который начинается с "Выбрать продукты" и заканчивается "Оплатить продукты". Включи шаги "Добавить в корзину" и "Подтвердить заказ".

Вывод:

```
graph TD
  A[Выбрать продукты] --> B[Добавить в корзину]
  B --> C[Подтвердить заказ]
  C --> D[Оплатить продукты]
```

Если ввести этот результат в инструмент визуализации диаграмм на языке Mermaid, то будет создана блок-схема, отражающая процесс заказа еды (рис. 3.2).

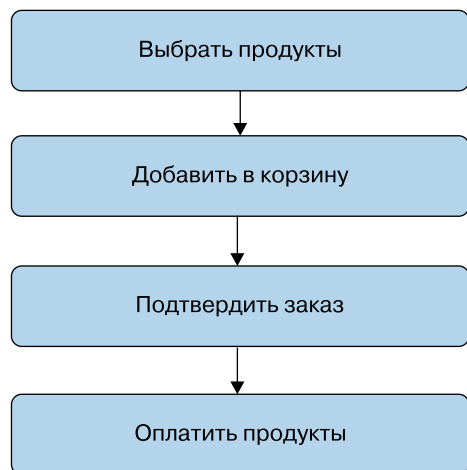


Рис. 3.2. Блок-схема, полученная на основе разметки на языке Mermaid, созданной моделью LLM

Пример данных CSV

ChatGPT также может генерировать данные в формате CSV, которые можно использовать для тестирования или других целей.

Ввод:

Сгенерируй образец данных CSV, представляющих пять учащихся, с полями: имя, возраст, оценка.

Вывод:

имя, возраст, оценка

Анна, 16, 4.5

Иван, 17, 3.8

Мария, 15, 4.9

Сергей, 16, 4.2

Елена, 17, 3.5

Полученный результат представляет собой блок данных в действительном формате CSV, который можно загрузить во многие инструменты, принимающие данные CSV.

Не забывайте, что вывод моделей GPT значительно зависит от указанного вами формата. Например, явное указание формата CSV подразумевает, что ИИ должен использовать запятые в качестве разделителей. Старайтесь четко определять желаемый формат, чтобы получить оптимальные результаты.

Мы исследовали разные способы извлечения структурированных данных из языковых моделей, включая регулярные выражения, JSON, YAML и др. Теперь вы знаете, что на LLM можно возложить некоторые функции вашего приложения.

Объясни это пятилетнему ребенку

Запрос «Объясни это пятилетнему ребенку» ориентирует на простоту объяснения, *которое будет понятно пятилетнему ребенку*.

Это очень эффективный стиль запросов, который является отличным способом преобразовать технические документы в простые описания, понятные каждому.

Применим этот подход для получения реферата статьи «Микроокружение опухоли: последние достижения в различных методах лечения рака» (<https://oreil.ly/MDOsz>).

Ввод:

Объясни следующий текст пятилетнему ребенку. Объясни это, как будто мне 5 лет.

Текст: Это обзор различных видов лечения рака.

Нашей целью было проанализировать микроокружение опухоли и последние тенденции и эффективность терапевтического лечения нескольких видов раковых заболеваний. Традиционно лечение рака основывалось на воздействии на неопластические клетки.

...(сокращенно для краткости)...

Вывод:

Хорошо, представь, что рак — это как плохие растения, которые растут в нашем теле, и мы хотим, чтобы они не росли больше. Врачи изучают, как эти плохие растения растут и как мы можем помочь людям, чтобы они стали здоровыми...

Текст, представленный моделью ChatGPT, гораздо проще, чем текст реферата статьи.

Универсальный перевод с помощью LLM

Кроме всего прочего, языковые модели могут выступать в качестве *универсальных переводчиков* благодаря их способности понимать и генерировать человеческий текст на нескольких языках.

Исследуем эту способность на простом примере. Для этого отдадим ИИ следующие команды.

1. Напиши простой текст.
2. Измени текст так, чтобы его было сложно читать.
3. Преобразуй этот же текст так, чтобы он легко читался на испанском языке.
4. Переведи упрощенный текст обратно на русский язык.

Ввод:

Напиши простой текст

Вывод:

Солнце поднимается над горизонтом, окрашивая небо в нежные оранжевые и розовые оттенки. Птицы весело щебечут, приветствуя новый день. Прохладный ветер шелестит в листьях деревьев и несет запах свежей травы и цветущих цветов. Жизнь просыпается с чувством спокойствия и надежды.

...(сокращенно для краткости)...

Ввод:

Измени текст так, чтобы его было сложно читать

Вывод:

В эфирном царстве небесного господства, где сияющий солнечный шар начинает постепенное восхождение, разворачивается изысканное явление, порождающее симфонию хроматического блеска на обширном небесном своде. Чудесные птицы, наделенные медоточивыми голосами, испускают свои ликующие приветствия, наполненные нотками счастья. Зефир, нежно струясь меж древесных стражей, несет в своих течениях благоухание зеленеющих листьев и цветущей флоры.

...(сокращенно для краткости)...

Ввод:

Преобразуй этот же текст так, чтобы он легко читался на испанском языке. Текст должен получиться максимально простым для понимания.

- Используй простые выражения.
- Старайся быть кратким.

Вывод:

En un lugar mágico y celestial, cuando el sol empieza a salir, ocurre algo hermoso en el cielo. Los pájaros cantan alegremente, llenando el aire con sus dulces melodías. Una brisa suave sopla entre los árboles, llevando consigo el olor fresco de la naturaleza. Todo cobra vida y promete momentos felices. Nos invita a disfrutar de las cosas simples y a encontrar la felicidad en los pequeños detalles. En este escenario encantador, encontramos paz y aprendemos que hasta los momentos más simples pueden ser mágicos.

Ввод:

Переведи упрощенный текст обратно на русский язык

Вывод:

В волшебных небесах, когда солнце начинает всходить, происходит что-то прекрасное. Птицы весело поют, наполняя воздух своими сладкими мелодиями. Легкий ветерок дует среди деревьев, неся свежий аромат природы. Все оживает в ожидании счастья. Наслаждайтесь простыми вещами и находите счастье в мелочах. В этой обстановке очарования мы находим покой и начинаем понимать, что даже самые простые моменты могут быть волшебными.

Как видите, при переводе часть смысла теряется, и это ожидаемо. Тем не менее значительная часть замысла, заложенного в исходный текст, сохраняется, что доказывает способность LLM выступать в качестве *хороших переводчиков* с одного языка на другой, если в Интернете имеется достаточно много ресурсов на этих языках. Если языки недостаточно хорошо представлены в Интернете, то результаты будут получаться неудовлетворительными.

Та же логика применима и к языкам программирования. Большие языковые модели (LLM) хорошо генерируют код на известных языках программирования, таких как Python и JavaScript, и хуже справляются с новыми языками программирования и пакетами.

Границы между разными формами информации становятся *все более размытыми*. Сама суть информации эволюционирует, что позволяет без особых усилий преобразовывать краткие изложения в рассказы, стихи или другие творческие выражения, что обогащает наше понимание информации и взаимодействие с ней.

Диффузионные модели — это уникальный класс генеративных моделей, используемых в машинном обучении, специально разработанных для создания новых изображений, имитирующих изображения, найденные в обучающем наборе.

Кроме того, объединение языковых моделей с диффузионными моделями обеспечивает плавные переходы между текстом, видео и другими модальностями. Это еще больше упрощает передачу сложных идей в разных форматах, обеспечивая более доступный и всеобъемлющий опыт.

Возможность запросить контекст

LLM способны не только генерировать текст, но могут также действовать как простые агенты с ограниченной возможностью *логических рассуждений*. Благодаря этому можно просить модель:

- вернуть действительный ответ на вопрос или утверждение;
- запрашивать дополнительный контекст, чтобы правильно ответить на вопрос.

В этом разделе мы поговорим о способности таких, как GPT-4, *запрашивать контекст*. Начнем с примера запроса, содержащего недостаточный объем контекста, из-за чего модель генерирует малополезный ответ.

Затем составим более качественный запрос, который побудит модель запрашивать дополнительный контекст при необходимости и давать более полезный ответ.

Ввод:

Какую БД я должен использовать в своем проекте, MongoDB или PostgreSQL?

Вывод:

Выбор между MongoDB и PostgreSQL зависит от специфики вашего проекта и требований к данным. Если возможно, протестируйте обе базы данных на небольших примерах, чтобы понять, какая из них лучше подходит для ваших нужд.

В предыдущем запросе недостаточно контекста, чтобы модель могла дать значимую рекомендацию. В подобных случаях можно запросить у ChatGPT список, что ей необходимо, чтобы она могла принять обоснованное решение.

Ввод:

Какую БД я должен использовать в своем проекте, MongoDB или PostgreSQL?

Напиши список требований, которые помогут тебе принять обоснованное решение о выборе базы данных.

Вывод:

Вот список требований, которые помогут вам принять обоснованное решение о выборе между MongoDB и PostgreSQL для вашего проекта:

1. Тип данных

Какой тип данных вы будете хранить (структурированные, неструктурированные, полуструктурированные)? Нужна ли поддержка сложных структур данных или вложенных объектов?

...

10. Долгосрочные планы

Каковы ваши долгосрочные планы по развитию проекта?

Как будут меняться структура и объем данных в будущем?

Собрав информацию по этим требованиям, вы сможете лучше понять, какая база данных – MongoDB или PostgreSQL – больше подходит для вашего проекта.

После подсказки со стороны ChatGPT о том, как принять обоснованное решение, мы можем включить в промт расширенный запрос, чтобы помочь модели принять решение.

Ввод:

Какую БД я должен использовать в своем проекте, MongoDB или PostgreSQL?

Контекст:

- Структура данных: данные не структурированы и имеют вид документов разных форматов без фиксированной схемы. Масштабируемость: проект должен обрабатывать огромные объемы данных и распределять их по нескольким серверам.
- Соответствие ACID: высокая согласованность и целостность данных имеют решающее значение для финансовых операций, выполняемых проектом.
- Возможности выполнения запросов: сложные геопространственные запросы и агрегирование данных на основе местоположения.
- Сообщество и экосистема: активное сообщество с многочисленными сторонними библиотеками и инструментами, специально разработанными для отрасли проекта.
- Целостность и ограничения данных: поддержка строгого внешнего ключа, сложных отношений и правил проверки данных.
- Транзакции и параллелизм: требуется поддержка высокого уровня параллелизма операций чтения и записи.
- Зрелость и стабильность: предпочтительнее хорошо зарекомендовавшие себя и проверенные базы данных.
- Простота разработки и эксплуатации: гибкий подход к разработке с возможностью быстрого прототипирования и итеративного проектирования схем.
- Стоимость: бюджетные ограничения требуют выбора решения с открытым исходным кодом без дополнительных затрат на лицензирование или поддержку.

Если понадобится дополнительный контекст, укажи, что поможет тебе принять более обоснованное решение.

Вывод:

С учетом всех перечисленных факторов PostgreSQL представляется более подходящим выбором для вашего проекта. Если у вас есть дополнительные вопросы или требуется уточнение по конкретным аспектам, пожалуйста, дайте знать!

В этом последнем примере модель использует дополнительный контекст и делает более обоснованный выбор в пользу PostgreSQL. Запрашивая контекст, когда это необходимо, языковые модели, такие как ChatGPT и GPT-4, могут давать более ценные и точные ответы.

На рис. 3.3 показано, как *запрос контекста* меняет процесс принятия решений в LLM. Получив ввод пользователя, модель сначала оценивает достаточность данного контекста. Если контекста недостаточно, то пользователю предлагается предоставить более подробную информацию и подчеркивается зависимость модели от богатства контекста во входных данных. Получив адекватный контекст, LLM генерирует обоснованный и релевантный ответ.

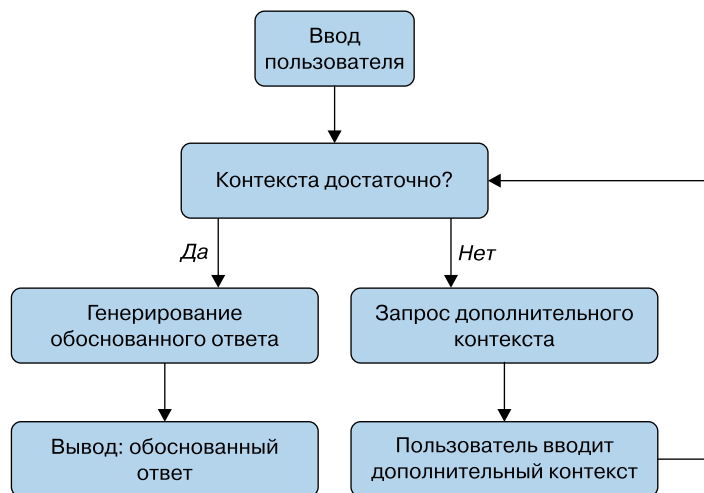


Рис. 3.3. Процесс принятия решений в LLM при наличии возможности запросить контекст

**Позвольте LLM запрашивать дополнительный контекст по умолчанию**

Вы можете позволить LLM запрашивать дополнительный контекст по умолчанию, добавив в запрос следующую ключевую фразу: «Если понадобится дополнительный контекст, укажи, что поможет тебе принять более обоснованное решение».

В этом разделе вы увидели, что LLM могут действовать как агенты и использовать дополнительный контекст для принятия решений. Итеративно уточняя запрос, следуя за рекомендациями модели, мы в конечном счете достигнем точки, когда модель получит *достаточный контекст для принятия обоснованного решения*.

Этот процесс подчеркивает, насколько важно предоставлять контекст в запросах и сообщать о готовности дать модели дополнительную информацию, если она ей понадобится. Следуя этим путем, вы сможете получать от LLM, таких как GPT-4, более точные и ценные рекомендации.

В системах на основе моделей, таких как GPT-4, возможность запрашивать дополнительный контекст и сообщать окончательный ответ имеет решающее значение для принятия обоснованных решений. Мультиагентная система AutoGPT (<https://oreil.ly/13Ihy>) имеет этап самооценки, который автоматически проверяет, можно ли решить задачу с учетом текущего контекста в запросе. В ней используются отношения «актер — критик» для анализа контекста, имеющегося в запросе, чтобы определить, нужно ли его дополнительно уточнить перед выполнением.

Разделение текстовых стилей

Разделение текстовых стилей — мощный метод промт-инжиниринга, позволяющий извлекать и изолировать определенные текстовые характеристики из данного документа, такие как эмоциональная окраска, длина, словарный запас и структура.

Он позволяет создавать новый контент, имеющий схожие характеристики с исходным документом, и тем самым обеспечивать единообразие стиля и эмоциональной окраски в разных формах коммуникации.

Такая согласованность может иметь решающее значение для предприятий и организаций, желающих передавать единообразную информацию по различным каналам. Этот прием обладает следующими преимуществами.

Улучшенная согласованность бренда

Оформляя весь контент в едином стиле, организации могут усилить индивидуальность своего бренда и сохранить целостность имиджа.

Оптимизация создания контента

При наличии набора четких рекомендаций авторам и создателям контента будет проще создавать материалы, соответствующие желаемому стилю.

Адаптивность

Разделение текстовых стилей позволяет легко адаптировать существующий контент к новым форматам или стилям, сохранив при этом сообщаемую информацию и эмоциональную окраску.

Процесс разделения текстовых стилей включает *выявление текстовых характеристик* или создание мета-запроса (запроса для создания запросов) для извлечения этих характеристик, а затем использование извлеченных характеристик для создания нового контента.

Выявление текстовых характеристик

Для успешного разделения текстовых стилей нужно сначала определить характеристики, которые необходимо извлечь из входного документа. К числу типичных характеристик, которые следует учитывать, относятся такие.

Эмоциональная окраска

Общее настроение или отношение, передаваемое текстом, например официальное, непринужденное, юмористическое или авторитетное.

Длина

Количество слов или общая длина контента.

Словарный запас и формулировка

Выбор слов и выражений, включая отраслевой жаргон, разговорные выражения и сложность.

Кроме того, можно даже создать мета-запрос, который динамически будет сообщать все эти характеристики.

Ввод:

Проанализируй следующий текст и определи основные характеристики, применение которых к другим документам поможет имитировать тот же стиль, согласованность и форматирование.

Опиши все характеристики в виде руководства по оформлению, которое будет использоваться при создании будущих документов.

Проанализируй следующий текст:

Искусственный интеллект стремительно меняет мир вокруг нас, открывая новые возможности и производя революцию в отраслях. От здравоохранения до развлечений, ИИ делает нашу жизнь лучше и удобнее во многих отношениях.

Однако большие возможности сопровождаются большой ответственностью. Мы должны гарантировать этическое и ответственное использование достижений технологии ИИ, защиту конфиденциальности и соблюдение наших ценностей.

Вывод:

Для имитации стиля, согласованности и форматирования приведенного текста следуйте приведенным ниже характеристикам:

1. Структура текста

Введение: Начинайте с общего утверждения о теме, которое привлекает внимание читателя. Используйте фразы, подчеркивающие значимость обсуждаемой темы.

Основная часть: Разделите текст на несколько абзацев, каждый из которых развивает отдельный аспект темы. Убедитесь, что абзацы логически связаны между собой.

Заключение: Завершите текст обобщением ключевых моментов и призывом к действию или размышлению.

2. Стиль написания

...

3. Тематика и содержание

...

4. Форматирование

...

5. Примеры и иллюстрации

...

6. Этика и ответственность

...

Применение

При создании будущих документов используйте это руководство как шаблон, чтобы обеспечить согласованность в стиле, тоне и формате. Это поможет создать качественные и профессиональные тексты, которые будут привлекать внимание и производить положительное впечатление на читателей.

Создание нового контента с использованием извлеченных характеристик

Получив нужные характеристики из исходного документа, их можно использовать для создания контента в будущем. Не забудьте сообщить модели ИИ желаемый стиль, перечислив извлеченные характеристики в запросе. Например:

Напиши новое сообщение в блоге на [тему], используя ту же интонацию, длину, словарный запас и структуру, что и в ранее проанализированном тексте.

Комбинируя этот метод с *образцовым текстом* (документами, которые служат образцами), можно создавать фирменный контент, вызывающий доверие и требующий минимальных доработок.

Извлечение текстовых характеристик с помощью LLM

При желании легко можно адаптировать запрос, чтобы помочь LLM извлечь из документа определенные текстовые характеристики. Этот прием можно применять не только для анализа текста с целью определения авторства, но и чтобы распознавать сущности или различать настроения.

Ввод:

Проанализируй следующий текст, определи и перечисли упомянутые в нем сущности:

Искусственный интеллект стремительно меняет мир вокруг нас, открывая новые возможности и производя революцию в отраслях. От здравоохранения до развлечений ИИ делает нашу жизнь лучше и удобнее во многих отношениях. Однако большие возможности сопровождаются большой ответственностью. Мы должны гарантировать этическое и ответственное использование достижений технологии ИИ, защиту конфиденциальности и соблюдение наших ценностей.

Вывод:

В приведенном тексте можно выделить следующие сущности:

- Искусственный интеллект (ИИ)
- Мир
- Революция
- Отрасли
 - Здравоохранение.
 - Развлечения
- Ответственность
- Этическое использование
- Конфиденциальность
- Ценности

Обобщение

В эпоху переизбытка информации способность сжимать большие объемы текста в краткие, удобоваримые резюме стала важным навыком. Поскольку объем производимых и потребляемых нами данных продолжает расти, спрос на эффективные методы обобщения существенно вырос.

ИИ добился значительных успехов в этой области, предоставив мощные инструменты для создания резюме, сохраняющих наиболее важную информацию и отбрасывающих менее важные детали.

Давайте посмотрим, как осуществить обобщение с помощью языковой модели.

Ввод:

Обобщи следующий текст:

Принимая во внимание, что первая сторона, именуемая в дальнейшем "Покупатель", и вторая сторона, именуемая в дальнейшем "Продавец", полностью осознавая свои юридические обязательства и ответственность, согласно действующему законодательству и положениям общего права, заключили юридически обязывающее соглашение, именуемое в дальнейшем "Соглашение", по передаче товаров, именуемых далее "Товары", от Продавца Покупателю на условиях, изложенных в настоящем Соглашении. Настоящим Стороны торжественно признают и подтверждают следующее:

...

Вывод:

Обобщение текста:

Покупатель и Продавец, осознавая свои юридические обязательства, заключили соглашение о передаче товаров от Продавца Покупателю на условиях, изложенных в данном документе. Стороны подтверждают свои намерения и обязательства в рамках этого соглашения.

Обобщение — это бесценная способность ИИ, позволяющая пользователям быстро извлекать ключевые идеи из длинных статей, отчетов или исследовательских работ. Этот процесс может помочь людям принимать обоснованные решения, экономить время и расставлять приоритеты при чтении длинных документов. Резюме, генерируемые искусственным интеллектом, также могут облегчить обмен информацией между командами и тем самым обеспечить более эффективное сотрудничество и общение.

Обобщение с учетом ограниченного окна контекста

При работе с документами, размер которых превышает размер одного запроса API, принято разбивать документы на части, обобщать каждый фрагмент отдельно, а затем объединять эти резюме, как показано на рис. 3.4.

Кроме того, по разным причинам людям могут потребоваться разные типы резюме, и именно здесь может пригодиться обобщение с помощью ИИ. Как показано на предыдущей диаграмме, большой PDF-документ можно с легкостью обобщить с помощью ИИ и получить несколько отдельных резюме, адаптированных к конкретным потребностям.

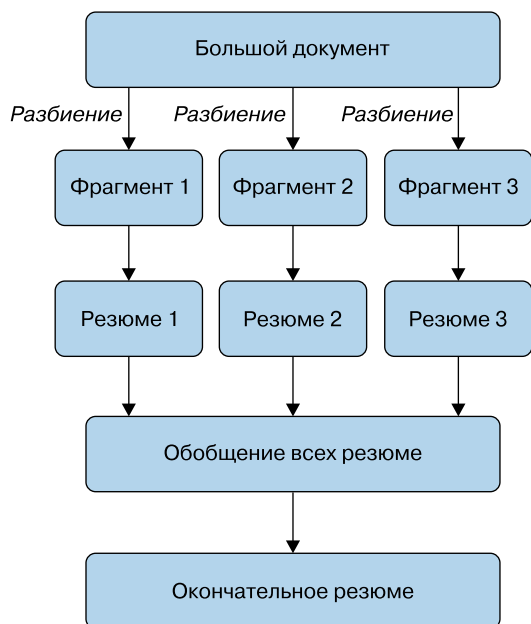


Рис. 3.4. Конвейер обобщения, использующий разделение процесса на несколько этапов

Резюме А

Содержит ключевые сведения для пользователей, стремящихся быстро понять содержание документа, и позволяет им сосредоточиться на наиболее важных моментах.

Резюме Б

Предлагает информацию для принятия обоснованных решений на основе последствий и рекомендаций.

Резюме В

Описывает способы эффективного и беспрепятственного сотрудничества и обмена информацией из документа.

Создание специализированных резюме с помощью ИИ упрощает поиск информации, нужной разным пользователям, и делает весь процесс более эффективным и целенаправленным.

Предположим, что вас интересует только поиск и обобщение информации о преимуществах цифрового маркетинга. Просто измените подсказку в запросе на "Дай краткое и обобщенное изложение приведенного выше текста. Включи в резюме только преимущества: ...".

Обобщение с помощью ИИ стало важным инструментом для быстрого преобразования огромных объемов информации в краткие, удобоваримые резюме, отвечающие различным потребностям пользователей. Используя передовые языковые модели, такие как GPT-4, приемы обобщения с привлечением ИИ позволяют эффективно извлекать ключевые идеи и информацию для принятия решений, а также облегчать сотрудничество и общение.

Учитывая продолжающийся рост объема данных, спрос на эффективное и специализированное обобщение будет только возрастать, что сделает ИИ важнейшим активом как для отдельных лиц, так и для организаций.

Разбиение текста на части

LLM продолжают развиваться и играют все более важную роль в различных сферах, поскольку способность обрабатывать большие объемы текста и управлять ими становится все более востребованной. Еще один важный метод обработки объемных документов известен как *разбиение на части* (чанкинг).

Разбиение на части — это процесс деления больших документов на более мелкие части (чанки). Разбиение может выполняться с учетом различных критериев, таких как границы предложений, абзацев или тем, а также по уровню сложности или длине. Разбив текст на более мелкие сегменты, модели ИИ могут эффективнее обрабатывать, анализировать и генерировать ответы.

На рис. 3.5 показан процесс разбиения большого текстового документа и последующего извлечения тем из отдельных фрагментов.

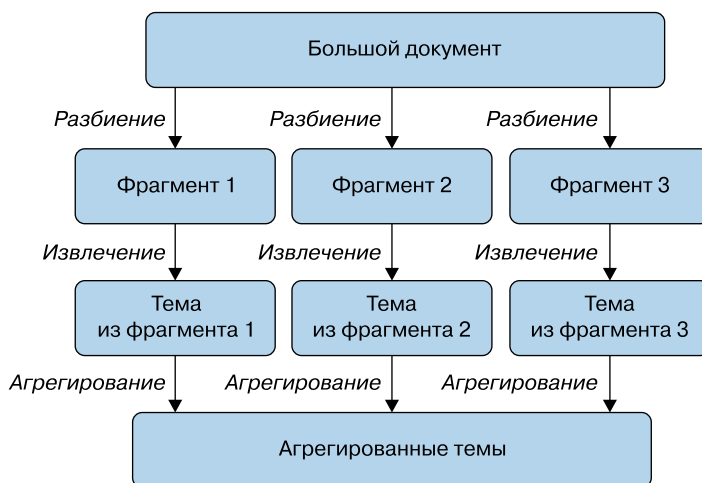


Рис. 3.5. Извлечение тем с помощью LLM после разбиения документа на части

Преимущества разбиения текста на части

Разбиение текста на части дает несколько преимуществ.

Подгонка под заданную длину контекста

LLM могут принимать и выводить не более определенного количества токенов, которое называется *длиной контекста*. Уменьшая количество токенов в запросе, можно гарантировать, что сам запрос не будет отклонен, а вывод не будет обрезан.

Снижение стоимости

Разбиение на части помогает извлечь из документов только самое важное и тем самым уменьшить количество использованных токенов и затраты на API.

Улучшение производительности

Разбиение на части снижает нагрузку на LLM, сокращает время отклика и способствует более эффективному использованию ресурсов моделью.

Увеличение гибкости

Разбиение на части позволяет разработчикам приспособить реакцию ИИ к конкретным потребностям задачи или приложения.

Сценарии разбиения текста на части

Разбиение текста на части может пригодиться в одних сценариях и не потребоваться в других. Понимание, когда применять этот метод, может помочь оптимизировать производительность и экономическую эффективность LLM.

Когда следует разбивать на части

При обработке больших документов

При работе с большими документами, размер которых превышает размер контекста LLM.

При проведении комплексного анализа

Когда требуется детальный анализ и документ необходимо разбить на части для лучшего понимания и эффективной обработки.

При обработке документов, охватывающих множество тем

Когда документ охватывает несколько тем и эти темы полезно обрабатывать по отдельности.

Когда не следует разбивать на части

При обработке коротких документов

Когда документ короткий и соответствует ограничениям LLM.

При проведении простого анализа

Когда требуется провести простой анализ или выполнить базовую обработку, дающее положительный эффект от разбиения документа на части.

При обработке документов, охватывающих одну тему

Когда документ охватывает одну тему и его разбиение на части не повысит ценность обработки.

Пример плохого разбиения

Неправильное разбиение текста на части может привести к ухудшению качества работы языковой модели. Давайте рассмотрим следующий абзац из новостной статьи:

Местный совет решил в этом году увеличить бюджет на образование на 10 %. Этот шаг приветствовали как родители, так и учителя. Дополнительные средства будут направлены на улучшение школьной инфраструктуры, наем большего числа учителей и обеспечение учащихся учебными материалами. Однако некоторые критики говорят, что этого увеличения недостаточно для удовлетворения растущих потребностей системы образования.

После разбиения текста на отдельные слова в получившемся списке пропал исходный контекст:

["Местный", "совет", "решил", "в", "этом", "году",
"увеличить", "бюджет", ...]

Вот основные проблемы этого примера плохого разбиения.

Потеря контекста

При разбиении текста на отдельные слова теряется первоначальный смысл и связи между словами. Из-за этого моделям ИИ трудно понять текст и найти эффективное решение.

Увеличение вычислительной нагрузки

Для обработки отдельных слов требуется больше вычислительных ресурсов, чем для обработки больших фрагментов текста.

В результате плохого разбиения на части, как в этом примере, LLM может столкнуться с несколькими сложностями:

- трудности с пониманием основных идей текста;
- трудности с созданием точного резюме или перевода;
- неспособность эффективно решать такие задачи, как анализ настроений или классификации текста.

Выяснив подводные камни неправильного разбиения текста на части, можно попробовать применить принципы промт-инженерии для достижения лучших результатов.

Рассмотрим улучшенный пример разбиения, используя тот же абзац из новостной статьи, но теперь разобьем текст по предложениям:

```
[ """"Местный совет решил в этом году увеличить бюджет на образование на 10 %.""",  
""""Этот шаг приветствовали как родители, так и учителя.""",  
""""Дополнительные средства будут направлены на улучшение школьной  
инфраструктуры, наем большего числа учителей и обеспечение учащихся  
учебными материалами.""",  
""""Однако некоторые критики говорят, что этого увеличения недостаточно  
для удовлетворения растущих потребностей системы образования."""]
```



Разделяйте задачу на подзадачи и оценивайте качество

Определите степень детальности разбиения текста, например по предложениям, абзацам или темам. Настройте такие модели, как количество токенов или температура, чтобы оптимизировать процесс разбиения.

Разбив текст таким способом, вы можете включать целые предложения в запросы к LLM.

Стратегии разбиения на части

Существует множество различных стратегий разбиения на части.

Разбиение по предложениям

Сохраняет контекст и структуру исходного контента, что упрощает для LLM понимание и обработку информации. Разбиение по предложениям особенно полезно для решения таких задач, как обобщение, перевод и анализ настроений.

Разбиение по абзацам

Этот способ особенно эффективен при работе с длинными документами, потому что позволяет модели LLM сосредоточиться на одном связном блоке за раз. Разбиение по абзацам идеально подходит для таких сценариев, как анализ документов, тематическое моделирование и извлечение информации.

Разбиение по темам или разделам

Этот метод может помочь моделям ИИ лучше идентифицировать и понимать основные темы и идеи в документе. Разбиение по темам хорошо подходит для таких задач, как классификация текста, составление рекомендаций по контенту и кластеризация.

Разделение по сложности

Для некоторых сценариев может быть полезно разбить текст по его сложности, например уровню удобочитаемости или технической сложности контента. Модели LLM способны эффективнее обрабатывать и анализировать фрагменты текста с одинаковым уровнем сложности. Этот подход может пригодиться для решения таких задач, как анализ удобочитаемости, адаптация контента и персонализированное обучение.

Разбиение по длине

Этот метод особенно полезен при работе с очень длинными или сложными документами, позволяя моделям LLM эффективнее обрабатывать их. Разбиение по длине подходит для таких сценариев, как крупномасштабный анализ текста, индексирование поисковыми системами и предварительная обработка текста.

Разбиение по токенам с помощью токенизатора

Использование токенизатора для разбиения текста на отдельные токены является важным шагом во многих задачах обработки естественного языка. Токенизаторы делят текст на такие мелкие единицы, как слова, фразы или символы, которые затем можно эффективно анализировать и обрабатывать моделями ИИ. Очень скоро в книге мы рассмотрим примеры использования пакета под названием `tiktoken`, осуществляющего разбиение текста на токены методом кодирования пар байтов (Byte-Pair Encoding, BPE).

В табл. 3.1 перечислены некоторые распространенные стратегии разбиения текста на части с их достоинствами и недостатками, знание которых поможет вам сделать правильный выбор.

Таблица 3.1. Шесть стратегий разбиения текста на части и их достоинства и недостатки

| Стратегия разбиения | Достоинства | Недостатки |
|--|--|--|
| Разбиение по предложениям | Сохраняет контекст, подходит для разных задач | Может быть не очень эффективной при обработке длинных документов |
| Разбиение по абзацам | Подходит для обработки длинных документов, фокусируется на связанных блоках | Меньшая детализация, могут быть упущены тонкие связи |
| Разбиение по темам | Выявляет основные темы, лучше подходит для классификации | Необходимо выделить темы, могут отсутствовать подробные сведения |
| Разбиение по сложности | Группирует фрагменты со схожим уровнем сложности, адаптивный | Требуется оценки уровня сложности, подходит не для всех задач |
| Разбиение по длине | Позволяет эффективно обрабатывать очень длинные документы | Потеря контекста, может потребоваться больше шагов предварительной обработки |
| Разбиение по токенам с использованием токенизатора | Подсчитывает точное количество токенов, что помогает избежать превышения ограничения на количество токенов | Требуется токенизация, может увеличить сложность вычислений |

Вас больше интересует сохранение смыслового контекста или достаточно простого разбиения по длине?

Выбирая стратегию разбиения, подходящую для вашего конкретного случая, можно оптимизировать производительность и точность языковых моделей ИИ.

Обнаружение предложений с использованием spaCy

Обнаружение предложений, также известное как определение границ предложений, — это процесс, используемый в NLP, который включает определение границ предложений в заданном тексте. Обнаружение предложений может пригодиться, например, в задачах, требующих сохранения контекста и структуры исходного документа. Разбивая текст на предложения, LLM может лучше справляться с такими задачами, как обобщение, перевод и анализ настроений.

Разбиение по предложениям можно организовать с использованием библиотек NLP, таких как spaCy (<https://spacy.io>). Установить эту библиотеку в свою

среду Python можно с помощью команды `pip install spacy`. Загрузите модель¹ `ru_core_news_sm` с помощью команды `python -m spacy download ru_core_news_sm`.

Пример 3.3 демонстрирует обнаружение предложений с использованием библиотеки `spaCy` для Python.

Пример 3.3. Обнаружение предложений с использованием библиотеки `spaCy` (<https://oreil.ly/GKDnc>)

```
import spacy

nlp = spacy.load("ru_core_news_sm")

text = "Это одно предложение. Это другое предложение."

doc = nlp(text)

for sent in doc.sents:
    print(sent.text)
```

Вывод:

```
Это одно предложение.
Это другое предложение
```

Код в примере сначала импортирует библиотеку `spaCy` и загружает модель для русского языка (`ru_core_news_sm`), чтобы инициализировать объект `nlp`. Далее определяется входной текст с двумя предложениями, после чего он обрабатывается с помощью инструкции `doc = nlp(text)`, которая создает объект `doc`. В заключение код перебирает обнаруженные предложения, используя атрибут `doc.sents`, и выводит их.

Создание простого алгоритма разбиения на Python

После знакомства со множеством стратегий разбиения важно получить практический опыт, написав простой алгоритм разбиения с нуля.

В примере 3.4 показано, как разбить по длине статью из блога *Hubspot* — *What Is Digital Marketing?*. Файл с текстом можно найти в репозитории Github под именем `content/chapter_3/hubspot_blog_post.txt` (<https://oreil.ly/30rlQ>).

Чтобы правильно прочитать файл `hubspot_blog_post.txt`, установите в качестве рабочего каталог `content/chapter_3` из Github (<https://oreil.ly/ONurh>).

¹ Список доступных моделей для разных языков можно найти на странице <https://spacy.io/models/>. — Примеч. пер.

Эта рекомендация применима и к запуску автономного кода на Python, и к запуску сервера Jupyter Notebook.

Пример 3.4. Разбиение по длине (<https://oreil.ly/n3sNy>)

```
with open("hubspot_blog_post.txt", "r") as f:
    text = f.read()

chunks = [text[i : i + 200] for i in range(0, len(text), 200)]

for chunk in chunks:
    print("-" * 20)
    print(chunk)
```

Вывод:

```
search engine optimization strategy for many local businesses is
an optimized Google My Business profile to appear in local search
results when people look for products or services related to what yo
```

```
-----
```

```
u offer.
```

```
For Keeps Bookstore, a local bookstore in Atlanta, GA, has optimized its
Google My Business profile for local SEO so it appears in queries for
"atlanta bookstore."
```

```
-----
```

```
...(обрезано для краткости)...
```

Код в примере сначала открывает текстовый файл `hubspot_blog_post.txt` с помощью функции `open` и читает его содержимое в переменную `text`. Затем, используя генератор списков, создает список фрагментов, по 200 символов в каждом.

Далее вызывается функция `range`, чтобы создать индекс для каждой 200-символьной подстроки, и используется нотация среза `[i:i+200]` для извлечения подстрок.

Наконец, код перебирает фрагменты в списке `chunks` и выводит их в консоль.

Как видите, разбиение по длине реализуется довольно просто, и в зависимости от выбранной длины предложения даже слова могут разрываться.

По этим причинам мы считаем, что хорошее разбиение на части должно обладать следующими свойствами:

- сохранять целыми слова и в идеале — предложения и контекстные фрагменты, выделяемые говорящим голосом;
- обрабатывать ситуации, когда предложения занимают несколько страниц;
- задавать адекватное количество токенов для каждого фрагмента, чтобы общее количество входных токенов вписывалось в размер контекста любой LLM.

Разбиение методом скользящего окна

Разбиение методом скользящего окна используется для разбиения текстовых данных на перекрывающиеся фрагменты, или *окна*, на основе заданного количества символов.

Но что такое скользящее окно?

Представьте себе, что вы просматриваете длинный фрагмент текста через маленькое окно. Это окно вмещает небольшое и фиксированное количество символов. Перемещая это окно от начала к концу текста, вы наблюдаете *перекрывающиеся фрагменты текста*. Этот механизм составляет суть подхода скользящего окна.

Размер окна определяется *фиксированным количеством символов*, а *размер шага* задает, как далеко сдвигается окно при каждом смещении.

На рис. 3.6 показано окно с размером пять символов, которое смещается на один символ на каждом шаге. В первый момент в окне отображаются первые пять символов текста. Затем окно сдвигается на один символ вправо и появляется второй фрагмент, содержащий со второго по шестой символы.

Этот процесс повторяется до тех пор, пока не будет достигнут конец текста, и каждый фрагмент перекрывается с предыдущим и следующим, сохраняя некоторый общий контекст.

Скользящее окно

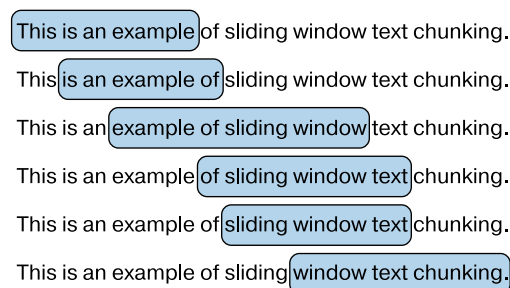


Рис. 3.6. Скользящее окно размером 5 и с величиной шага 1

Поскольку размер шага равен 1, фрагменты содержат множество повторяющейся информации, зато риск потери информации между фрагментами резко снижается.

Совершенно иначе выглядит ситуация на рис. 3.7, где размер окна равен 4, а размер шага — 2. Обратите внимание, что из-за увеличения размера шага

вдвое объем повторяющейся информации в разных фрагментах значительно уменьшился.

Скользящее окно

This is an example of sliding window text chunking.

This is an example of sliding window text chunking.

This is an example of sliding window text chunking.

Рис. 3.7. Скользящее окно размером 4 и с величиной шага 2

Если точность и сохранение смыслового контекста важнее минимизации числа токенов в запросе или числа запросов к LLM, то, вероятно, лучше использовать большее перекрытие.

В примере 3.5 показано, как на Python реализовать скользящее окно с помощью функции `len()`. Эта функция возвращает общее количество символов в заданной текстовой строке, которое помогает определить параметры скользящего окна.

Пример 3.5. Скользящее окно (<https://oreil.ly/aCkDo>)

```
def sliding_window(text, window_size, step_size):
    if window_size > len(text) or step_size < 1:
        return []
    return [text[i:i+window_size] for i
            in range(0, len(text) - window_size + 1, step_size)]

text = "This is an example of sliding window text chunking."
window_size = 20
step_size = 5
chunks = sliding_window(text, window_size, step_size)

for idx, chunk in enumerate(chunks):
    print(f"Chunk {idx + 1}: {chunk}")
```

Этот код выведет:

```
Chunk 1: This is an example o
Chunk 2: is an example of sli
Chunk 3: example of sliding
Chunk 4: ple of sliding windo
Chunk 5: f sliding window tex
Chunk 6: ding window text chu
Chunk 7: window text chunking
```

В контексте промт-инжиниринга метод скользящего окна предлагает несколько преимуществ по сравнению со стратегиями разбиения, описанными выше.

Он позволяет сохранить больше контекстной информации, поскольку фрагменты перекрываются, и предлагает альтернативный подход к сохранению контекста по сравнению с методом обнаружения предложений.

Пакеты для разбиения текста на части

При работе с LLM, такими как GPT-4, всегда обращайтесь внимание на максимальную длину контекста:

```
maximum_context_length = input_tokens + output_tokens
```

Существуют разные токенизаторы, позволяющие разбить текст на управляемые единицы, наибольшей популярностью из которых пользуются NLTK, spaCy и tiktoken.

Пакеты NLTK (<https://oreil.ly/wTmI7>) и spaCy (<https://oreil.ly/c4MvQ>) обеспечивают комплексную поддержку обработки текста, но далее мы сосредоточимся на tiktoken.

Разбиение текста на части с помощью tiktoken

Tiktoken (<https://oreil.ly/oSpVe>) — это токенизатор, осуществляющий разбиение текста на токены *методом кодирования пар байтов* (Byte-Pair Encoding, BPE). Он разбивает текст на под слова и предназначен для использования с моделями OpenAI. Tiktoken имеет более высокую производительность, чем аналогичные токенизаторы с открытым исходным кодом.

Нам, как разработчикам, пользующимся приложениями GPT-4, применение tiktoken дает несколько важных преимуществ.

Точная разбивка на токены

Разбивка на токены играет важную роль, потому что модели GPT интерпретируют текст как последовательности отдельных токенов. Определив количество токенов в тексте, можно сказать, не окажется ли он слишком длинным для передачи в модель.

Эффективное использование ресурсов

Возможность правильно определять количество токенов позволяет эффективно управлять ресурсами, особенно при работе с OpenAI API. Знание точного количества токенов помогает оптимизировать использование API и поддерживать баланс между затратами и потреблением ресурсов.

Кодировки

Кодировки определяют метод преобразования текста в токены, при этом разные модели используют разные кодировки. Tiktoken поддерживает три кодировки, обычно используемые моделями OpenAI:

| Название кодировки | Модели OpenAI |
|----------------------|--|
| cl100k_base | GPT-4, GPT-3.5-turbo, text-embedding-ada-002 |
| p50k_base | Модели серии codex, text-davinci-002, text-davinci-003 |
| r50k_base (или gpt2) | Модели на основе GPT-3, такие как davinci |

Что такое токенизация строк

В естественном языке токены (они же лексемы) могут иметь разную длину, от одного символа, например *и*, до целого слова, например *фильм*. Это связано с адаптируемой природой токенизации, согласно которой в языках со сложным письмом могут существовать токены короче одного символа, а в языках с письмом без пробелов или где фразы могут интерпретироваться как отдельные неделимые единицы — длиннее слова.

В токены нередко включаются пробелы, например, вместо "is" используется токен "is " или "is"+" ". Эта практика помогает сохранить исходное форматирование текста и уловить определенные лингвистические особенности.



Быстро посмотреть, как строка разбивается на токены, можно с помощью OpenAI Tokenizer (<https://oreil.ly/K6ZQK>).

Пакет tiktoken можно установить из PyPI (<https://oreil.ly/HA2QD>) командой `pip install tiktoken`. Следующий пример показывает, насколько просто преобразовать текст в токены и декодировать токены в текст:

```
# 1. Импортировать пакет:
import tiktoken

# 2. Загрузить таблицу кодирования вызовом tiktoken.get_encoding()
encoding = tiktoken.get_encoding("cl100k_base")

# 3. Преобразовать текст в токены вызовом encoding.encode()
# и затем преобразовать токены в текст вызовом encoding.decode()
print(encoding.encode("Learning how to use Tiktoken is fun!"))
```

```
print(encoding.decode([1061, 15009, 374, 264, 2294, 1648,
311, 4048, 922, 15592, 0]))
```

```
# [48567, 1268, 311, 1005, 73842, 5963, 374, 2523, 0]
# "Data engineering is a great way to learn about AI!"
```

Теперь напишем функцию, которая принимает текст в параметре `text_string`, токенизирует его, используя таблицу кодирования `coding_name`, и подсчитывает количество получившихся токенов.

```
def count_tokens(text_string: str, encoding_name: str) -> int:
    """
    Возвращает количество токенов в текстовой строке,
    используя указанную кодировку.

    Аргументы:
        text_string: текстовая строка для токенизации.
        encoding_name: название кодировки, используемой для токенизации.

    Возвращаемое значение:
        Количество токенов в текстовой строке text_string.

    Исключения:
        ValueError: если имя кодировки указано с ошибкой.
    """
    encoding = tiktoken.get_encoding(encoding_name)
    num_tokens = len(encoding.encode(text_string))
    return num_tokens
```

```
# 4. Использовать функцию для подсчета токенов в текстовой строке.
text_string = "Hello world! This is a test."
print(count_tokens(text_string, "cl100k_base"))
```

Этот код выведет число 8.

Оценка числа токенов, используемых в вызовах Chat API

Модели ChatGPT, такие как GPT-3.5-turbo и GPT-4, используют токены аналогично предыдущим моделям. Однако инфраструктура, основанная на сообщениях, использует довольно сложную арифметику при подсчете токенов в диалоге:

```
def num_tokens_from_messages(messages, model="gpt-3.5-turbo-0613"):
    """Возвращает количество токенов для списка сообщений."""
    try:
        encoding = tiktoken.encoding_for_model(model)
```

```

except KeyError:
    print("Warning: model not found. Using cl100k_base encoding.")
    encoding = tiktoken.get_encoding("cl100k_base")
if model in {
    "gpt-3.5-turbo-0613",
    "gpt-3.5-turbo-16k-0613",
    "gpt-4-0314",
    "gpt-4-32k-0314",
    "gpt-4-0613",
    "gpt-4-32k-0613",
    }:
    tokens_per_message = 3
    tokens_per_name = 1
elif model == "gpt-3.5-turbo-0301":
    tokens_per_message = 4 # токенов за каждое следующее сообщение
    # <|start|>{role/name}\n{content}<|end|>\n
    tokens_per_name = -1 # если имя роли опущено
elif "gpt-3.5-turbo" in model:
    print("'Warning: gpt-3.5-turbo may update over time. Returning
    num tokens assuming gpt-3.5-turbo-0613.'")
    return num_tokens_from_messages(messages, model="gpt-3.5-turbo-0613")
elif "gpt-4" in model:
    print("'Warning: gpt-4 may update over time.
    Returning num tokens assuming gpt-4-0613.'")
    return num_tokens_from_messages(messages, model="gpt-4-0613")
else:
    raise NotImplementedError(
        f"num_tokens_from_messages() is not implemented for model
        {model}."
    )
num_tokens = 0
for message in messages:
    num_tokens += tokens_per_message
    for key, value in message.items():
        num_tokens += len(encoding.encode(value))
        if key == "name":
            num_tokens += tokens_per_name
num_tokens += 3 # every reply is primed with
# <|start|>assistant<|message|>
return num_tokens

```

В примере 3.6 показана конкретная структура для выполнения запроса к любой из моделей — GPT-3х или GPT-4.

Обычно история чата предвращается системным сообщением, а затем идет поочередный обмен сообщениями между пользователем и помощником.

Пример 3.6. Данные для Chat Completions API на сайте OpenAI

```

example_messages = [
    {
        "role": "system",

```

```

        "content": '''You are a helpful, pattern-following assistant that
        translates corporate jargon into plain English.'''
    },
    {
        "role": "system",
        "name": "example_user",
        "content": "New synergies will help drive top-line growth.",
    },
    {
        "role": "system",
        "name": "example_assistant",
        "content": "Things working well together will increase revenue.",
    },
    {
        "role": "system",
        "name": "example_user",
        "content": '''Let's circle back when we have more bandwidth to touch
        base on opportunities for increased leverage.'''
    },
    {
        "role": "system",
        "name": "example_assistant",
        "content": '''Let's talk later when we're less busy about how to
        do better.'''
    },
    {
        "role": "user",
        "content": '''This late pivot means we don't have
        time to boil the ocean for the client deliverable.'''
    },
]

for model in ["gpt-3.5-turbo-0301", "gpt-4-0314"]:
    print(model)
    # подсчитать токены с помощью функции, определенной выше
    print(f'{{num_tokens_from_messages(example_messages, model)}}
    prompt tokens counted by num_tokens_from_messages().')

```

`"role": "system"` описывает системное сообщение, которое можно использовать для *передачи запросов*. Оно позволяет настроить характер помощника или дать указания относительно его подходов к диалогу. Однако важно понимать, что наличие системной команды не является обязательным условием и поведение модели по умолчанию без нее может очень напоминать поведение «полезного помощника».

Для использования доступны роли `["system", "user", "assistant"]`.

`"content": "некоторое содержимое"` — это место, где вы размещаете запрос, а модель — ответ в зависимости от роли сообщения (`"assistant"`, `"system"` или `"user"`).

Анализ настроений

Анализ настроений — широко известный метод NLP, помогающий идентифицировать эмоциональную окраску, мнения или чувства, выраженные во фрагменте текста. Благодаря возможностям больших языковых моделей (LLM), таких как GPT-4, анализ настроений стал важным инструментом для предприятий, исследователей и разработчиков в различных сферах.

Основная цель анализа настроений — определить эмоциональное отношение к чему-либо, передаваемое в тексте, — положительное, отрицательное или нейтральное. Эта информация может нести ценные сведения о мнениях клиентов и потребителей, помогать отслеживать репутацию бренда и даже способствовать прогнозированию рыночных тенденций.

Ниже показано несколько приемов промт-инжиниринга, направленных на получение эффективных запросов для анализа настроений:

Ввод:

Этот текст положительный или отрицательный?

Мне очень нравится дизайн этого телефона, но время его автономной работы разочаровывает.

Вывод:

Текст имеет смешанный характер. Он выражает положительное мнение о дизайне телефона, но в то же время содержит отрицательную оценку времени его автономной работы.

GPT-4 определяет эмоциональную окраску как «смешанную», но такой результат является следствием нескольких недостатков запроса.

Отсутствие ясности

В запросе отсутствует четкое определение желаемого формата вывода.

Отсутствие примеров

Запрос не содержит примеров положительной, отрицательной или нейтральной эмоциональной окраски, которые могли бы помочь LLM понять различия между ними.

Отсутствие указаний по обработке смешанных настроений

В запросе не указано, как действовать, когда текст содержит смесь положительных и отрицательных настроений.

Ввод:

Используй следующие примеры как руководство:

позитивный: "Мне очень нравится дизайн этого телефона!"

отрицательный: "Время автономной работы разочаровывает".

нейтральный: "Телефон мне понравился, но он имеет малое время автономной работы".

Верни только одно слово:

- положительный
- отрицательный
- нейтральный

А теперь определи эмоциональную окраску текста как положительную, отрицательную или нейтральную: мне очень нравится дизайн этого телефона, но время его автономной работы разочаровывает.

Вывод:

нейтральный

Этот запрос намного лучше предыдущего, потому что:

Содержит четкие инструкции

В запросе четко сформулирована задача: определить эмоциональную окраску данного текста как положительную, отрицательную или нейтральную.

Примеры

В запросе присутствуют примеры для каждой категории эмоциональной окраски, что помогает понять контекст и желаемый результат.

Указан формат вывода

В запросе указывается, что ответ должен состоять из одного слова, а это гарантирует краткость и простоту понимания ответа.

Методы улучшения анализа настроений

Для повышения точности анализа настроений совершенно необходимым шагом является предварительная обработка входного текста, включая следующее.

Удаление специальных символов

Некоторые символы, такие как смайлики, хештеги и знаки препинания, могут исказить суждение алгоритма оценки настроений. Кроме того, эти символы могут не распознаваться моделями машинного и глубокого обучения и привести к неправильной классификации.

Преобразование в нижний регистр

Преобразование всех символов в нижний регистр помогает добиться единообразия. Например, такие слова, как *Счастье* и *счастье*, рассматриваются моделями как разные слова, что может привести к неточностям.

Исправление орфографических ошибок

Орфографические ошибки могут привести к неверному толкованию и неправильной классификации. Создание конвейера проверки орфографии может значительно уменьшить количество таких ошибок и улучшить результаты.

Для текста, содержащего терминологию, характерную для данной предметной области, встраивание примеров со специфическими терминами в запрос поможет LLM понять структуру и эмоциональную окраску текста, повысит точность классификации и обеспечит лучшее понимание конкретного жаргона и выражений.

Ограничения и сложности анализа настроений

Несмотря на достижения в области LLM и применение методов промт-инжиниринга, анализ настроений по-прежнему сталкивается с некоторыми сложностями и ограничениями.

Сарказм и ирония

Выявление сарказма и иронии в тексте представляет большую трудность для LLM, потому что для этого часто необходимо понимать контекст и тонкие подсказки, которые легко распознаются людьми. Неправильная интерпретация саркастических или иронических высказываний может привести к неточной классификации настроений.

Определение настроения в зависимости от контекста

Анализ настроений может оказаться сложной задачей, когда эмоциональная окраска зависит от контекста, например от жаргона, специфического для конкретной области, или формы культурного самовыражения. В таких случаях языковым моделям может быть трудно точно классифицировать настроения без дополнительных инструкций или примеров, специфических для конкретной предметной области.

От меньшего к большему

Метод промт-инжиниринга «от меньшего к большему» — это мощный метод последовательного генерирования или извлечения все более подробных сведений по заданной теме. Этот метод особенно эффективен при работе со сложными темами или когда необходим высокий уровень детализации.

В методе «от меньшего к большому» используется цепочка запросов, в которой каждый новый запрос основан на предыдущем ответе. Такой пошаговый подход помогает постепенно получать все более подробную информацию и глубоко погрузиться в любую тему.

Этот метод также можно применить для генерации кода, как показано в примере приложения Hello World на Flask.

Планирование архитектуры

Прежде чем углубиться в архитектуру, выясним, что такое Flask. Flask (<https://oreil.ly/7N-bs>) — это легковесный фреймворк для быстрого создания веб-приложений на Python с минимальным количеством кода. (Flask используется здесь только в демонстрационных целях и не включен в файл `requirements.txt` (<https://oreil.ly/TRK0i>) книги.)

Теперь попросим LLM описать базовую архитектуру простого приложения «Hello World» на Flask.

Ввод:

Опиши базовую архитектуру простого приложения "Hello World" на Flask.

Вывод:

Базовая архитектура простого приложения "Hello World" на Flask состоит из следующих шагов:

Импорт Flask: Подключаем библиотеку Flask.

Создание приложения: Создаем экземпляр приложения.

Маршрут: Определяем, что делать, когда пользователь заходит на главную страницу (/).

Функция: Возвращаем текст "Hello, World!".

Запуск: Запускаем приложение в режиме отладки.

Реализация отдельных функций

Прежде чем писать код, давайте уточним, что в фреймворке Flask называется маршрутом: это функция, связанная с шаблоном URL, которая определяет, что увидят пользователи на конкретной веб-странице. Теперь предоставим фрагмент кода для простого маршрута «Hello World» в Flask.

Ввод:

Напиши фрагмент кода для простого маршрута "Hello World" в Flask.

Вывод:

Конечно! Вот фрагмент кода для маршрута "Hello World" в Flask:

```
from flask import Flask
```

```
app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello, World!'

if __name__ == '__main__':
    app.run()
```

Код в этом примере импортирует модуль Flask и создает экземпляр класса приложения Flask с именем `app`. Затем определяет маршрут с помощью декоратора `@app.route`, указывая корневой URL `'/'`. С этим маршрутом связана функция `hello_world`, которая возвращает строку "Hello, World!".

Приложение Flask запускается вызовом `app.run()`, когда этот сценарий на Python выполняется непосредственно из командной строки.

Добавление тестов

Теперь попросим модель сгенерировать тестовый пример для маршрута "Hello World" в приложении Flask.

Ввод:

Приведи тестовый пример для маршрута "Hello World" в приложении Flask.

Вывод:

Конечно! Вот тестовый пример для маршрута "Hello World" в приложении Flask

```
import unittest
from flask import Flask
from flask.testing import FlaskClient

# Создать экземпляр приложения Flask
app = Flask(__name__)

# Создать маршрут
@app.route('/')
def hello_world():
    return 'Hello, World!'

class FlaskAppTestCase(unittest.TestCase):
    def setUp(self):
        # Создать тестового клиента
        self.app = app.test_client()

    def test_hello_world_route(self):
        # Послать GET-запрос по маршруту
        response = self.app.get('/')

```

```
# Проверить код состояния в ответе
self.assertEqual(response.status_code, 200)

# Проверить данные, полученные в ответе
self.assertEqual(response.data, b'Hello, World!')

if __name__ == '__main__':
    unittest.main()
```

Используя подход «от меньшего к большому», можно постепенно накапливать знания и понимание, необходимые для выполнения поставленной задачи.

Преимущества метода «от меньшего к большому»

Этот метод особенно хорош для решения сложных задач, поскольку позволяет LLM генерировать соответствующие знания, которые затем используются как контекст для следующих задач.

Рассмотрим преимущества этого подхода в некоторых других сценариях.

Прогрессивное исследование

Разбиение сложной задачи на более мелкие задачи позволяет LLM представить более подробную и точную информацию на каждом этапе. Этот подход особенно полезен при работе с новым предметом или многогранной проблемой.

Гибкость

Методика «от меньшего к большому» обеспечивает гибкость и возможность учета различных аспектов задачи. Это позволяет менять условия, изучать альтернативные решения и углубляться в конкретные области по мере необходимости.

Улучшенное понимание

Разбивая задачу на более мелкие шаги, вы даете LLM возможность предоставлять информацию в более удобочитаемом формате, что облегчает ее понимание.

Совместное обучение

Этот метод способствует сотрудничеству между вами и LLM, поощряя итеративный процесс уточнения запросов и ответов для достижения желаемого результата.

Сложности метода «от меньшего к большему»

Чрезмерная зависимость от ранее сгенерированных знаний

Использование истории предыдущих запросов/ответов может привести к тому, что при решении крупных задач будут забыты предыдущие запросы/ответы из-за ограничений длины контекста.

Зависимость от предыдущих запросов

Поскольку каждый запрос основан на предыдущих результатах, крайне важно убедиться, что предыдущие запросы и ответы предоставляют достаточно информации для следующего шага.



Оцените качество

В процессе разработки запросов обязательно оценивайте качество каждого предыдущего ответа, полученного от LLM. Решение следующей задачи зависит от качества информации, полученной при решении предыдущей задачи.

Ролевые запросы

Ролевые запросы — это метод, когда искусственный интеллект ассоциируется с некоторой ролью или персонажем, которую он должен играть или которого он должен имитировать, генерируя ответы. Это помогает управлять стилем и содержанием ответов ИИ, чтобы они лучше соответствовали потребностям пользователя или желаемому результату.



Проект Awesome ChatGPT Prompts (<https://oreil.ly/8pf40>) предлагает обширный список ролевых запросов, которые вы можете использовать в своей практике.

Попросим ChatGPT взять на себя роль технического обозревателя MongoDB.

Ввод:

Я хочу, чтобы ты выступил в роли технического обозревателя. Я дам тебе название технологии, а ты представишь мне подробный обзор, в том числе плюсы, минусы, особенности и сравнение с другими технологиями, присутствующими на рынке.
Мой первый запрос: MongoDB.

Вывод:

MongoDB — это документно-ориентированная NoSQL база данных, которая хранит данные в формате BSON (Binary JSON).

Особенности:

1. Документная модель: Данные хранятся в виде документов, которые представляют собой структуры «ключ — значение», аналогичные JSON. Это позволяет легко хранить сложные данные и изменять их структуру без необходимости в миграциях.

...

Плюсы MongoDB:

1. Производительность: Высокая скорость записи и чтения данных благодаря использованию памяти и индексирования.

...

Минусы:

1. Отсутствие ACID-транзакций: Хотя MongoDB поддерживает транзакции на уровне документа, полная поддержка ACID-транзакций на уровне нескольких документов была добавлена только в более поздних версиях и может не быть столь надежной, как в реляционных базах данных.

...

Сравнение с другими технологиями:**1. MongoDB vs. MySQL:**

Структура данных: MySQL — реляционная база данных с жесткой схемой, в то время как MongoDB — документная база данных с гибкой схемой.

Запросы: MySQL использует SQL для запросов, что может быть более привычно для многих разработчиков. MongoDB использует собственные методы запросов, которые могут быть менее интуитивными.

Скорость: MongoDB может быть быстрее для операций записи и работы с неструктурированными данными.

...

Заключение:

MongoDB — это мощная и гибкая NoSQL база данных, которая отлично подходит для приложений с большими объемами данных и требованиями к производительности. Гибкая модель данных, богатый язык запросов и поддержка согласованности делают ее популярным выбором среди разработчиков.

Этот запрос — отличный пример ролевого запроса, поскольку он четко определяет роль, которую должен взять на себя ИИ (технический обозреватель), и устанавливает ожидания относительно типа желаемого ответа (углубленный обзор MongoDB).

**Задайте направление**

При создании запросов подумайте о возможности назначить некоторую роль искусственному интеллекту. Это поможет задать правильный контекст и получить более релевантный результат.

Преимущества ролевых запросов

Ролевые запросы помогают сузить контекст и получить более целенаправленные и релевантные ответы. Они также могут повысить креативность ИИ, заставляя его думать и реагировать с уникальной точки зрения.

Сложности ролевых запросов

Ролевые запросы могут создавать определенные сложности. Роли могут иметь свои потенциальные риски предвзятости или стереотипов. Назначение стереотипных ролей может привести к получению предвзятых ответов, способных нанести вред удобству использования или оскорбить людей.

Кроме того, удержание длительного диалога в рамках конкретной роли может оказаться затруднительным. Модель может отклониться от темы или ответить информацией, не имеющей отношения к назначенной роли.



Оцените качество

Постоянно проверяйте качество ответов LLM, особенно при использовании ролевых запросов. Старайтесь вовремя подметить, когда ИИ начнет отклоняться от назначенной роли или темы.

Когда использовать ролевые запросы

Ролевые запросы особенно полезны, когда нужно:

Привлечь экспертные знания

Когда нужен ответ, требующий знаний в предметной области или специализированных знаний, ролевые запросы могут помочь LLM генерировать более информированные и точные ответы.

Адаптировать стиль ответа

Назначение роли может помочь LLM сгенерировать ответ, соответствующий определенному тону, стилю или перспективе, например формальный, неприличный или юмористический ответ.

Получить творческий ответ

Ролевые запросы можно использовать для создания вымышленных сценариев или получения творческих ответов, назначая ИИ такие роли, как рассказчик, персонаж из романа или историческая личность.

Исследовать разные точки зрения

Если нужно узнать разные точки зрения на какую-то тему, то можно попробовать попросить ИИ ответить на один и тот же вопрос, приняв на себя разные роли или образы. Это позволит получить более полное понимание предмета.

Повысить вовлеченность пользователей

Ролевые запросы могут сделать общение более увлекательным и интересным, если позволить LLM выбирать персонажей, которые вызывают симпатию и пользуются доверием у пользователей.

Если вы используете OpenAI, то лучшее место для назначения роли — это системное сообщение.

Тактика запросов к GPT

К настоящему моменту мы довольно подробно рассмотрели несколько тактик промптинга, включая запрос контекста, объединение стилей текста, от меньшего к большему и ролевые запросы.

Теперь исследуем еще несколько тактик: от управления потенциальными галлюцинациями с использованием соответствующего референтного текста до предоставления LLM *времени на размышления* и понимания концепции *декомпозиции задач*.

Все эти методологии имеют целью повысить точность результатов, возвращаемых ИИ, и рекомендованы OpenAI (<https://oreil.ly/QZE8n>). Кроме того, каждая тактика использует один или несколько принципов промт-инжиниринга, обсуждавшихся в главе 1.

Как избежать галлюцинаций с помощью референсов

Первый метод избежать текстовых галлюцинаций — потребовать от модели формировать ответ *только на основе референтного текста*.

Предоставляя модели ИИ точную и актуальную информацию по заданному вопросу, можно заставить ее использовать эту информацию для генерации ответа.

Ввод:

Отвечая на вопросы, обращай к статьям, заключенным в тройные кавычки.

Следуй такому принципу:

- Если не удалось найти ответ в этих статьях, то просто верни "Я не нашел ответа".

""

B2B-клиенты, как правило, имеют более длительные процессы принятия решений и, следовательно, более длительные воронки продаж. Для таких клиентов лучший результат дают стратегии выстраивания отношений, тогда как для клиентов B2C, как правило, лучше подходят краткосрочные предложения.

""

Примеры ответов:

- Я не нашел ответа.
- Да, B2B-клиенты, как правило, имеют более длительный процесс принятия решений и, следовательно, более длинные воронки продаж.

Вывод:

Да, B2B-клиенты, как правило, имеют более длительный процесс принятия решений, что приводит к более длительным воронкам продаж.

Если включить в запрос тот же референсный текст, но задать вопрос, показанный ниже:

Ввод:

...Текст из предыдущего запроса...

Вопрос: являются ли продажи B2C более рентабельными?

Вывод:

Я не нашел ответа.



Задайте направление и укажите формат

Предыдущий запрос примечателен тем, что он не только инструктирует модель, как найти ответ, но и устанавливает формат ответа на вопросы, на которые нет ответа.

Учитывая ограниченное контекстное окно GPT, для применения этой стратегии может потребоваться метод динамического получения информации, относящейся к запросу.

Другой подход состоит в том, чтобы заставить модель *включать ссылки* в свой ответ. Когда входные данные дополняются релевантной информацией, модель может включать в свои ответы цитаты, ссылаясь на разделы предоставленных документов. Этот подход имеет дополнительное преимущество, заключающееся в том, что цитаты в выводе могут аутентифицироваться автоматически путем сопоставления строк в заданных документах.

Ввод:

Тебе предоставляется документ, заключенный в тройные кавычки, и вопрос. Твоя задача – ответить на вопрос, используя только предоставленный документ, и процитировать отрывок или отрывки документа, использованные для ответа на вопрос. Если документ не содержит информации, необходимой для ответа на этот вопрос, тогда просто сообщи: "Недостаточно информации". Если такая информация есть, то обозначь ее в ответе как цитату. Используй следующий формат для цитирования отрывков (`{"citation": ...}`).

"""

Ядерный реактор – это устройство, которое инициирует, управляет и поддерживает цепную ядерную реакцию. Ядерный реактор использует энергию, выделяемую при делении ядер для выработки тепла, которое затем часто используется для производства пара, приводящего в движение турбины, которые, в свою очередь, производят электроэнергию.

- Извлечено из https://en.wikipedia.org/wiki/Nuclear_reactor.

"""

Вопрос: что такое ядерный реактор?

Вывод:

Ядерный реактор – это устройство, которое инициирует, управляет и поддерживает цепную ядерную реакцию. Он использует энергию, выделяемую при делении ядер для выработки тепла, которое затем часто используется для производства пара, приводящего в движение турбины, которые, в свою очередь, производят электроэнергию. `{"citation": "Ядерный реактор – это устройство, которое инициирует, управляет и поддерживает цепную ядерную реакцию. Ядерный реактор использует энергию, выделяемую при делении ядер для выработки тепла, которое затем часто используется для производства пара, приводящего в движение турбины, которые, в свою очередь, производят электроэнергию."}`

Предоставление модели GPT «времени на размышления»

Часто, явно требуя от LLM *показать ход решения задачи*, можно получить более точные ответы. Предоставление модели LLM *времени на размышление* может способствовать улучшению результатов.

Ввод:

Для начала попробуй решить задачу самостоятельно. Затем посмотри, будет ли твой ответ соответствовать предложенному решению. Не учитывая данный ответ, пока не попробуешь решить задачу самостоятельно.

Задача:

У Джейн 5 яблок, а у Мэри – 15 яблок. Джон берет 2 яблока у Мэри и отдает их Джейн.

Сколько яблок у Джейн?

Предложенное решение:

1. Изначально у Джейн было 5 яблок.
2. Джон берет 2 яблока у Мэри, но это не влияет на количество яблок у Джейн.
3. Затем Джон отдает свои яблоки Джейн.

Итак, окончательное количество яблок у Джейн равно имевшимся у нее первоначально 5 яблокам плюс 2 яблока, полученные от Джона, то есть всего 7 яблок.

Вывод:

Давайте решим задачу самостоятельно.

- Изначально у Джейн 5 яблок.
- У Мэри 15 яблок.
- Джон берет 2 яблока у Мэри, у нее остается $15 - 2 = 13$ яблок.
- Джон отдает 2 яблока Джейн.

Теперь посчитаем, сколько яблок у Джейн:

- У Джейн было 5 яблок, и она получает 2 яблока от Джона.
- $5 + 2 = 7$ яблок.

Таким образом, у Джейн теперь 7 яблок.

Теперь сравним с предложенным решением: оно также пришло к выводу, что у Джейн 7 яблок.

Ответ: у Джейн 7 яблок.

В некоторых приложениях, использующих GPT-4 или другие генеративные модели ИИ, бывает нежелательно, чтобы процесс рассуждений модели был виден пользователю.

Скрыть эти рассуждения можно, потребовав от LLM *сгенерировать внутренний монолог*. Это особенно полезно в обучающих приложениях, где вывод аргументов модели может привести к преждевременному раскрытию решения.

Тактика внутреннего монолога

Тактика внутреннего монолога заключается в том, чтобы потребовать от модели структурировать части вывода, которые должны быть скрыты от пользователя, и представить их в определенном формате. Это позволяет легко удалять такие части перед представлением окончательного результата пользователю.

Вот как можно использовать эту тактику для эффективного сокрытия рассуждений в ответах на запросы пользователей.

Ввод:

Шаг 1: реши задачу

Сначала найди решение задачи. Не полагайся на решение пользователя, поскольку оно может быть неверным. Заклучи свои рассуждения в тройные кавычки (``````), чтобы указать, что этот раздел должен быть скрыт.

Шаг 2: сравни решения

Затем сравни свое решение с решением пользователя. Оцени правильность решения пользователя. И снова заклучи этот раздел в тройные кавычки (``````).

Шаг 3: подготовь подсказку

Если пользователь допустил ошибку, определи соответствующую подсказку, которая может помочь пользователю найти правильный ответ, не выдавая его сразу. Этот раздел тоже должен быть заключен в тройные кавычки (``````).

Шаг 4: выведи подсказку

Если была допущена ошибка, верни пользователю подсказку, созданную на предыдущем шаге. Не забудь использовать тройные кавычки. Вместо заголовка, например "Шаг 4", используй заголовок "Подсказка:"

Постановка задачи: В колоде 52 карты: 13 червей, 13 бубен, 13 треф и 13 пик. Если из колоды взять две карты без замены, то какова вероятность, что одна из карт окажется червовой масти, а другая – трефовой?

Решение пользователя: $(13/52) * (13/51) = 169/2652 = 13/204$

Вывод:

````

Шаг 1: Решение задачи

Для нахождения вероятности того, что одна карта будет червовой, а другая – трефовой, мы можем использовать комбинаторный подход.

1. Всего в колоде 52 карты.
2. Вероятность того, что первая карта будет червовой:  $P(\text{черви}) = 13/52$ .
3. Если первая карта – червь, осталась 51 карта, и вероятность того, что вторая карта будет трефой:  $P(\text{трефа} \mid \text{черви}) = 13/51$ .
4. Вероятность того, что первая карта будет трефой:  $P(\text{трефа}) = 13/52$ .
5. Если первая карта трефа, осталась 51 карта, и вероятность того, что вторая карта будет червовой:  $P(\text{червь} \mid \text{трефа}) = 13/51$ .

Теперь мы можем сложить вероятности двух событий:

$P(\text{червь, трефа}) + P(\text{трефа, червь}) = (13/52) * (13/51) + (13/52) * (13/51) = 2 * (13/52) * (13/51)$ .

Теперь вычислим:

$2 * (13/52) * (13/51) = 2 * 169 / 2652 = 338 / 2652 = 13 / 102.$

Таким образом, вероятность того, что одна карта окажется червовой, а другая — трефовой, составляет 13/102.

""  
""

Шаг 2: Сравнение решений

Решение пользователя:  $(13/52)*(13/51) = 169/2652 = 13/204.$

Решение пользователя неверно. Пользователь не учел, что порядок карт имеет значение, и не сложил вероятности двух возможных случаев (червь первой, трефа второй и наоборот). Правильный ответ составляет 13/102.

""  
""

Шаг 3: Подсказка

Обратите внимание на то, что порядок, в котором вы берете карты, может повлиять на общую вероятность. Попробуйте учесть оба сценария: когда первая карта червовая, а вторая трефовая, и наоборот. После этого сложите вероятности этих двух случаев.

""

Подсказка:

""Обратите внимание на то, что порядок, в котором вы берете карты, может повлиять на общую вероятность. Попробуйте учесть оба сценария: когда первая карта червовая, а вторая трефовая, и наоборот. После этого сложите вероятности этих двух случаев.""

## Оценка моделью своих собственных ответов

Еще одна возможная тактика — попросить LLM *проанализировать сгенерированные ею результаты* и сообщить, не пропустила ли она какую-либо информацию или важные факты. Тем самым вы попросите LLM *оценить себя* по своим предыдущим результатам.

Для начала создадим запрос для получения кода функции "Hello World".

Ввод:

Напиши функцию "Hello World" на Python.

Вывод:

Вот простая функция "Hello World" на Python:

```
def hello_world():
 print("Hello, World!")
```

```
Вызов функции
hello_world()
```

Когда вы выполните этот код, он выведет на экран "Hello, World!".

LLM возвращает простую функцию `print_hello_world()` на Python, которая выводит традиционное приветствие «Hello, World!».

Затем отправим второй запрос с предыдущей историей чата, попросив улучшить первоначальный результат.

Ввод:

...Предыдущая история чата...

Можно ли еще как-то улучшить предыдущую функцию? Например, добавив значения по умолчанию или подсказки типов?

Вывод:

Да, функцию можно улучшить, добавив значения по умолчанию и подсказки типов. Вот пример, как это можно сделать:

```
def hello_world(name: str = "World") -> None:
 """Выводит приветствие для указанного имени или 'World' по умолчанию."""
 print(f"Hello, {name}!")
```

```
Вызов функции без аргумента
hello_world()
```

```
Вызов функции с аргументом
hello_world("Alice")
```

Объяснение улучшений:

1. Подсказки типов: Указание типа `str` для параметра `name` и `None` для возвращаемого значения делает код более понятным и помогает инструментам статического анализа (например, `mypy`) проверять типы.
2. Значение по умолчанию: Параметр `name` имеет значение по умолчанию "World", что позволяет вызывать функцию без аргументов.
3. Докстринг: Добавлен комментарий, объясняющий, что делает функция. Это полезно для документирования кода и понимания его назначения другими разработчиками.

В ответ GPT-4 вернула обновленную версию функции на Python, теперь снабженную подсказками типов и аргументом со значением по умолчанию. Аргумент делает функцию более гибкой, позволяя ей выводить не просто "Hello, World!" но любое другое сообщение, указанное пользователем.

Эта последовательность запросов-ответов показывает, что можно постепенно уточнять результаты, сгенерированные моделью LLM, пока вы не будете удовлетворены ими.



Ответ LLM можно критиковать снова и снова, она не предоставит желаемых уточнений.

## Классификация с помощью LLM

Под классификацией в контексте ИИ понимается процесс прогнозирования класса или категории заданной точки данных или образца. Это обычная задача в машинном обучении, когда модели обучаются присваивать предопределенные метки немаркированным данным на основе изученных закономерностей.

LLM обладают мощными возможностями в сфере классификации, даже при отсутствии или небольшом количестве примеров в запросе. Почему? Да потому, что LLM, такие как GPT-4, прошли обучение на обширном наборе данных и теперь способны на рассуждения до определенной степени.

Существуют две основные стратегии решения задач классификации с помощью LLM: *обучение без примеров (zero-shot learning)* и *обучение с несколькими примерами (few-shot learning)*.

### *Обучение без примеров*

В этом процессе LLM классифицирует данные с исключительной точностью без предварительной передачи ей каких-либо конкретных примеров. Это похоже на реализацию проекта без всякой подготовки — впечатляет, правда?

### *Обучение с несколькими примерами*

В этом случае модели LLM передается небольшое количество примеров. Эта стратегия может существенно повлиять на структуру результата и повысить общую точность классификации.

Почему для нас это необычно?

Использование моделей LLM позволяет избежать длительных процедур обучения, используемых в традиционных процессах машинного обучения, быстро создать прототип модели классификации, определить базовый уровень точности и немедленно создать ценность для бизнеса.



Несмотря на способность LLM выполнять классификацию, иногда, в зависимости от специфики задачи и особенностей обучающих данных, может обнаружиться, что традиционный процесс машинного обучения позволяет получить лучшие результаты.

## Построение модели классификации

Рассмотрим обучение с несколькими примерами модели для определения эмоциональной окраски текста: «Комплимент», «Жалоба» или «Нейтральный».

Следующие высказывания классифицируются как "Комплимент", "Жалоба" или "Нейтральное":

1. "Солнце светит." - Нейтральное
2. "Ваша команда поддержки фантастически хороша!" - Комплимент
3. "У меня был ужасный опыт работы с вашим программным обеспечением". – Жалоба

Следуй перечисленным ниже принципам:

- возвращай только одно классифицирующее слово. Ответом должно быть только одно слово из: "Комплимент", "Жалоба" или "Нейтральный";
- классифицируй текст, заключенный в тройные кавычки """".

""""Пользовательский интерфейс интуитивно понятен.""""

Вывод:

Классификация:

Комплимент

Далее перечислено несколько вариантов, где может пригодиться умение LLM классифицировать текст.

### *Отзывы клиентов*

Классификация отзывов клиентов по таким категориям, как «Положительные», «Отрицательные» или «Нейтральные». Погрузившись глубже, можно определить такие подтемы, как «Удобство использования», «Поддержка клиентов» или «Цена».

### *Фильтрация электронной почты*

Определение намерения или цели электронных писем и их классификация по таким категориям, как «Запросы», «Жалобы», «Отзывы» или «Спам». Эта классификация может помочь компаниям расставить приоритеты в реагировании и эффективно управлять коммуникациями.

### *Анализ настроений в социальных сетях*

Отслеживание упоминаний брендов и отношения к ним в социальных сетях. Сообщения или комментарии можно классифицировать как «Похвала», «Критика», «Запрос» или «Нейтральный». Это поможет получить представление о восприятии бренда в обществе и соответствующим образом адаптировать маркетинговые или PR-стратегии.

### Классификация новостных статей

Учитывая огромное количество новостей, генерируемых ежедневно, LLM могут классифицировать статьи по таким темам, как «Политика», «Технологии», «Окружающая среда» или «Развлечения».

### Проверка резюме

Если отделы кадров завалены резюме, то им можно помочь и с помощью LLM классифицировать резюме по заранее определенным критериям на такие категории, как «Квалифицированный», «Высококвалифицированный», «Недостаточно квалифицированный», или разделить их по областям специализации, таким как «Разработка программного обеспечения», «Маркетинг» или «Продажи».



Имейте в виду, что использование электронных писем, резюме или конфиденциальных данных для обучения сопряжено с риском утечки этих данных в будущие модели OpenAI.

## Классификация голосованием

Использование нескольких запросов к LLM может помочь уменьшить вариативность классификации. Этот процесс, известный как *голосование*, чем-то похож на выбор самого распространенного фрукта из кучи. Например, если у вас есть 10 фруктов и 6 из них — яблоки, то яблок будет большинство. Тот же принцип применим и к выбору результатов классификации.

Выполнив классификацию несколько раз и выбрав *наиболее часто встречающийся результат*, можно уменьшить влияние потенциальных выбросов или необычных интерпретаций, возвращаемых одной моделью. Но имейте в виду, что этот подход может иметь существенные недостатки, в том числе увеличение времени и затрат на несколько вызовов API.

Давайте классифицируем один и тот же фрагмент текста три раза, а затем возьмем результат, набравший большинство голосов:

```
from openai import OpenAI
import os

client = OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))

base_template = """
Следующие высказывания классифицируются как "Комплимент", "Жалоба"
или "Нейтральное":
1. "Солнце светит." — Нейтральное
2. "Ваша команда поддержки фантастически хороша!" — Комплимент
```



3. "У меня был ужасный опыт работы с вашим программным обеспечением". – Жалоба

Следуй перечисленным ниже принципам:

- возвращай только одно классифицирующее слово. Ответом должно быть только одно слово из: "Комплимент", "Жалоба" или "Нейтральный";
- классифицируй текст, заключенный в тройные кавычки '''.

```
'''{content}'''
```

```
Классификация:
"""
```

```
responses = []

for i in range(0, 3):
 response = client.chat.completions.create(
 model="gpt-4",
 messages=[{"role": "system",
 "content": base_template.format(content='''На улице идет дождь,
но у меня отличное настроение. Я не понимаю, как люди могут
грустить, это так досадно!'''),}],)
 responses.append(response.choices[0].message.content.strip())

def most_frequent_classification(responses):
 # Использовать словарь для подсчета голосов
 count_dict = {}
 for classification in responses:
 count_dict[classification] = count_dict.get(classification, 0) + 1

 # Вернуть результат, набравший большинство голосов
 return max(count_dict, key=count_dict.get)

print(most_frequent_classification(responses)) # Ожидаемый вывод: Нейтральный
```

Вызов функции `most_frequent_classification(responses)` должен вернуть результат 'Нейтральный' как доминирующий. Теперь вы знаете, как использовать пакет OpenAI для классификации голосованием.

## Критерии оценки

В главе 1 использовалась система оценки, основанная на участии человека, с простой системой оценок «палец вверх»/«палец вниз», чтобы определить, насколько часто ответ соответствует нашим ожиданиям. Выставлять оценки вручную дорого и утомительно, потому что для оценки качества или выявления ошибок требуется квалифицированный человек. Теоретически эту работу можно поручить недорогим сервисам, таким как Mechanical Turk (<https://www.mturk.com>), однако

разработка задачи для получения достоверных результатов сама по себе может занять много времени и привести к ошибкам. Одним из подходов решения этой проблемы, набирающих популярность, является использование более сложной модели LLM для оценки ответов более простой модели.

Примеры использования LLM в качестве классификаторов несколько неоднозначны: в некоторых исследованиях утверждается, что LLM способны осуществлять классификацию на уровне человека (<https://oreil.ly/nfc3f>), другие демонстрируют несоответствия (<https://oreil.ly/ykkzY>). Как показывает наш опыт, GPT-4 — полезный инструмент классификации, показывающий стабильные результаты на широком круге задач. В частности, GPT-4 эффективно и надежно справляется с оценкой ответов небольших и более простых моделей, таких как GPT-3.5-turbo. Следующий пример генерирует краткие и подробные ответы на вопрос с использованием GPT-3.5-turbo, готовые для оценки GPT-4.

```
from openai import OpenAI
import os

client = OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))

responses = []
for i in range(10):
 # потребовать краткий ответ, если i — четное число,
 # и подробный, если нечетное
 style = "краткий" if i % 2 == 0 else "подробный"

 if style == "краткий":
 prompt = f"""Верни {style} ответ на следующий вопрос:
 В чем смысл жизни?"""
 else:
 prompt = f"""Верни ответ на следующий вопрос:
 В чем смысл жизни?"""

 response = client.chat.completions.create(
 # использовать модель GPT-3.5 Turbo
 model="gpt-3.5-turbo",
 messages=[{"role": "user",
 "content": prompt}]
)
 responses.append(
 response.choices[0].message.content.strip()
)

system_prompt = """Оцени краткость ответа чат-бота.
Поставь оценку 1, если ответ краткий, и 0 - если нет.
"""

ratings = []
```

```
for idx, response in enumerate(responses):
 rating = client.chat.completions.create(
 model="gpt-4",
 messages=[{"role": "system",
 "content": system_prompt},
 {"role": "system",
 "content": response}])
 ratings.append(
 rating.choices[0].message.content.strip())

for idx, rating in enumerate(ratings):
 style = "краткий" if idx % 2 == 0 else "подробный"
 print(f"Стиль: {style}, ", f"Оценка: {rating}")
```

Вывод:

```
Стиль: краткий, Оценка: 1
Стиль: подробный, Оценка: 0
Стиль: краткий, Оценка: 1
Стиль: подробный, Оценка: 0
Стиль: краткий, Оценка: 1
Стиль: подробный, Оценка: 0
Стиль: краткий, Оценка: 1
Стиль: подробный, Оценка: 0
Стиль: краткий, Оценка: 1
Стиль: подробный, Оценка: 0
```

Это программа на Python, которая взаимодействует с OpenAI API, чтобы сгенерировать и оценить краткость ответов. Вот детальное описание, как она работает.

1. `responses = []` создает пустой список с именем `responses` для хранения ответов, сгенерированных OpenAI API.
2. Цикл `for` выполняет десять итераций, генерируя ответ в каждой из них.
3. Внутри цикла определяется значение `style` — "краткий" или "подробный" — на основе текущего номера итерации (`i`), четного или нечетного соответственно.
4. В зависимости от стиля `style` формируется строка запроса `prompt` с вопросом «В чем смысл жизни?» и требованием дать ответ в краткой или подробной форме.
5. `response = client.chat.completions.create(...)` отправляет запрос в OpenAI API для генерации ответа на запрос. Здесь используется модель `gpt-3.5-turbo`.
6. Далее из сгенерированного ответа удаляются начальные и конечные пробелы, и затем он добавляется в список `responses`.
7. `system_prompt = """Оцени краткость ответа..."""` определяет запрос для оценки краткости сгенерированных ответов.

8. `ratings = []` инициализирует пустой список для хранения оценок.
9. Другой цикл `for` перебирает ответы в `responses`.
10. Каждый сгенерированный ответ отправляет вместе с `system_prompt` в OpenAI API, чтобы получить оценку краткости. На этот раз используется модель GPT-4.
11. Из ответа с оценкой (1 — для кратких ответов и 0 — для подробных) удаляются пробелы, и затем он добавляется в список `ratings`.
12. Последний цикл `for` перебирает список `ratings`. Для каждой оценки выводятся стиль ответа («краткий» или «подробный») и соответствующая ему оценка краткости.

В простых случаях, как оценка краткости, GPT-4 работает с почти 100%-ной точностью, но в более сложных ситуациях может потребоваться потратить некоторое время на оценку качества классификации. Например, можно подобрать тестовые примеры с правильными и неправильными ответами и по ним определить точность классификации, подсчитав количество ложноположительных результатов (когда LLM ошибочно видит проблему в тестовом случае, в котором ее нет), а также количество ложноотрицательных результатов (когда LLM пропускает проблему в тестовом случае, который, как известно, ее содержит). В примере выше мы сгенерировали примеры кратких и подробных ответов, поэтому можем легко проверить точность оценок, но в более сложных случаях для такой проверки может понадобиться участие человека.



#### Оцените качество

Использование GPT-4 для оценки ответов менее сложных моделей постепенно становится стандартной практикой, но необходимо позаботиться о том, чтобы результаты были надежными и последовательными.

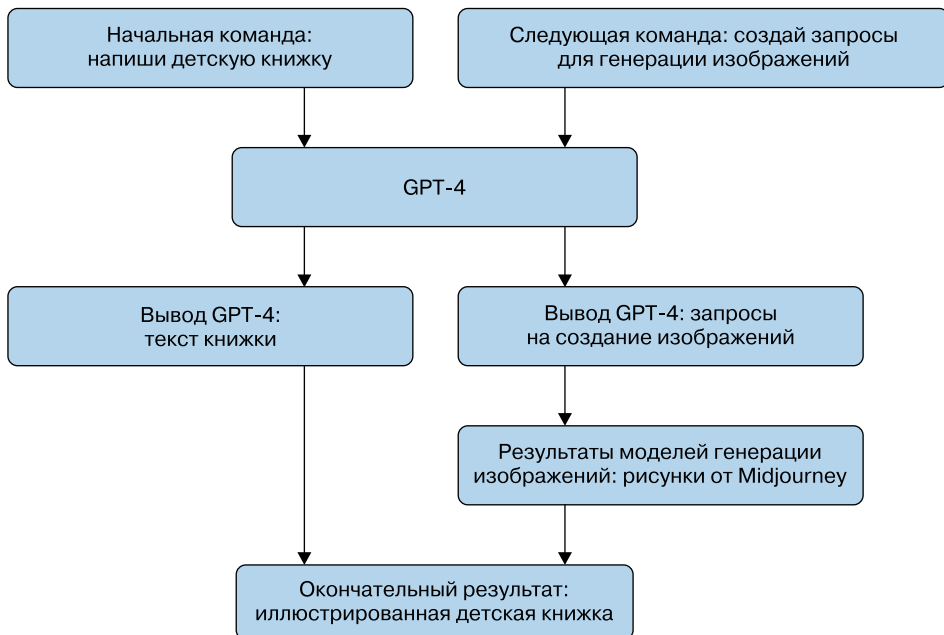
По сравнению с классификацией, проводимой человеком, классификация с помощью LLM, или синтетическая классификация, обычно обходится на порядок дешевле и выполняется за считанные минуты, а не дни или недели. Даже в важных или деликатных случаях, когда требуется заключительная проверка человеком, быстрый перебор и А/В-тестирование запроса с помощью ИИ могут сэкономить массу времени и значительно улучшить результаты. Однако в случае масштабного тестирования стоимость и ограниченная скорость работы GPT-4 могут стать труднопреодолимыми препятствиями. По мере возможности промт-инженер должен сначала провести тестирование с применением программных подходов без вызова LLM, например, просто измерить длину каждого ответа, что делается практически мгновенно и почти с нулевыми затратами.

# Метапромтинг

*Метапромтинг* — это прием создания текстовых запросов (метазапросов), предполагающих создание других текстовых запросов, которые, в свою очередь, используются для создания новых материалов разных видов, таких как изображения, видеоролики и другой текст.

Чтобы лучше понять суть метапромтинга, возьмем в качестве примера написание детской книжки с помощью GPT-4. Сначала вы поручаете LLM создать текст детской книжки. Затем передаете модели метазапросы, требуя от нее создать запросы для моделей, генерирующих изображения. Это могут быть описания ситуаций или конкретных сцен на основе сюжетной линии книги, которые затем можно будет передать моделям ИИ, таким как Midjourney или Stable Diffusion, а те, в свою очередь, сгенерируют изображения, гармонирующие с детской историей, созданной искусственным интеллектом.

Процесс метапромтинга в контексте создания детской книжки наглядно показан на рис. 3.8.



**Рис. 3.8.** Генерирование запросов к Midjourney на создание изображений для детской книжки с помощью LLM

Метазапросы дают множество преимуществ в разных сферах.

#### *Генерирование изображений на основе описаний товаров*

С помощью метазапросов можно получать запросы для таких моделей создания изображений, как Midjourney (<https://www.midjourney.com>), способных эффективно генерировать визуальные представления по описаниям товаров.

#### *Генерирование запросов по стилю/оформлению*

Предположим, что вы — копирайтер и вам нужно разработать уникальное руководство по стилю на основе нескольких сообщений в блоге. Учитывая, что каждый клиент имеет свой особый стиль изложения, будет полезно использовать метазапрос, инкапсулирующий различные стили, вместо создания единого запроса.

#### *Оптимизация запросов для достижения конкретных целей*

В практике часто используется прием, суть которого состоит в том, чтобы попросить ChatGPT или другую языковую модель уточнить или улучшить запрос для достижения конкретной цели. Этот метод помогает идентифицировать упущенные из виду особенности, добавление которых может помочь языковой модели сгенерировать более подробный и желательный ответ.

Поэкспериментируем с созданием метазапросов для примера с копирайтером.

Ввод:

Выступая в роли промт-инженера для GPT, ты несешь полную ответственность за создание эффективных текстовых промтов для крупной языковой модели в рамках поставленного задания.

Следуй перечисленным далее принципам:

- Возвращай только один запрос, никогда не возвращай ответ на запрос.
- Правильно выполняй задание.
- Доработай запрос или создай новый запрос в зависимости от задания.
- Отметь все важные особенности, чтобы другой

промт-инженер мог скопировать входные данные в запрос.

Задание: создай руководство по копирайтингу для ChatGPT на основе нескольких записей в блоге. Включи в запрос к ChatGPT важные особенности, которые позволят модели подражать стилю автора. Реши самостоятельно, какие особенности являются наиболее важными для подражания стилю автора и т. д. Ты будешь применять этот стиль только к новым записям в блоге.

Текст записи в блоге: чтобы преуспеть в технологическом секторе, помимо технических, вам понадобятся также навыки межличностного общения. Навыки межличностного общения используются профессионалами в области программного обеспечения для сотрудничества со своими коллегами...

Вывод:

Запрос:

На основе предоставленного текста создай серию записей в блоге, подражая стилю автора. В качестве руководства учитывай следующие важные рекомендации:

1. Используй простой и понятный язык: текст должен быть простым и понятным, а описания сложных понятий должны быть разбиты на простые концепции.
2. Используй подзаголовки: структурируй сообщения в блоге, разбивая их соответствующими подзаголовками. Это поможет направить читателя и сделать ваш текст более читабельным.
- ...
9. Включай технические термины: при обсуждении технических тем используй соответствующую терминологию для достижения точности и достоверности. Но не забудь предоставить пояснения или определения для особенно сложных терминов.

Как упоминалось выше, часто полезно *критиковать вывод модели* и просить ее уточнить запрос либо через пользовательский интерфейс, либо через API.

ChatGPT может выполнять веб-поиск и использовать содержимое веб-страниц вместе с ранее созданным руководством по стилю копирайтинга. Давайте предоставим два URL, которые ChatGPT будет читать для улучшения результата.

Ввод:

Опираясь на следующее руководство по стилю и две публикации в блоге, улучши запрос о стиле копирайтинга, чтобы LLM или копирайтер могли писать качественные тексты для моего клиента.

...Ранее созданное руководство по копирайтингу...

URL 1: <https://understandingdata.com/posts/how-to-clean-and-process-data>

URL 2: <https://understandingdata.com/posts/how-to-scrape-twitter-data>

Вывод показан на рис. 3.9.

Вот улучшенный запрос на создание руководства по копирайтингу, полученный на основе руководства по стилю и двух статей в блоге:

Чтобы создать запись в блоге, придерживайтесь следующих рекомендаций:

1. Используй простой и понятный язык для объяснения сложных тем. Представляй информацию в простой для понимания форме. Объясняй концепции так, чтобы они были понятны даже неподготовленной аудитории.
- ...
8. Используй технические термины: при обсуждении сложных тем используй соответствующую терминологию. Но не забудь добавить пояснения или определения терминов, чтобы текст оставался доступным.

Создай свой текст по образцу этих примеров записей в блоге:

"Что такое API? И как они связаны с инженерией данных?": в этой статье для объяснения концепции API используется простой и понятный язык. В ней даются подробные объяснения и приводятся практические примеры часто используемых API. Она связывает техническую концепцию API с ростом и эффективностью бизнеса. Информация хорошо организована с помощью подзаголовков, таких как "Типы API".

"Как легко настроить проект Google Cloud с помощью API": эта статья представляет собой практическое руководство по настройке проекта Google Cloud. Она разбита на пронумерованные шаги, каждый из которых имеет понятный подзаголовок. В статье используются подробные объяснения, практические примеры и скриншоты. В ней тоже используется простой и понятный язык, чтобы провести читателя через процесс.



#### ChatGPT

После просмотра и анализа структуры и содержания двух статей, доступных по указанному URL, было создано расширенное руководство по оформлению записей в блоге, которое лучше соответствует вашим потребностям:

##### **Обновленное руководство по стилю для эффективного написания блогов, ориентированных на данные**

1. **Ясность и точность:** используйте простой, лаконичный и понятный язык. Объясняйте сложные технические процессы как можно проще, чтобы все читатели, независимо от уровня подготовки, могли их понять.
2. **Структурированный формат с подзаголовками:** используйте информативные подзаголовки, которые логически организуют информацию. Это поможет читателям перемещаться по контенту без задержек и сделает сложную информацию доступной.
3. **Подробные пошаговые инструкции:** при описании таких процедур, как скрапинг или очистка данных, предоставляйте подробные пошаговые инструкции. Это поможет читателям понять и воспроизвести процессы самостоятельно.
4. **Варианты и примеры использования:** включайте варианты применения и практические примеры для иллюстрации. Это поможет преодолеть разрыв между теорией и практикой.
5. **Визуальные элементы:** включите диаграммы, фрагменты кода и снимки экрана, где это уместно, чтобы поддержать текст визуальными средствами, способными упростить понимание абстрактных концепций.
6. **Выделение распространенных ошибок:** укажите распространенные ошибки и подводные камни, которых следует избегать в описываемых процессах. Это поможет читателям сэкономить время и избавит их от разочарований.
7. **Этические и юридические соображения:** когда это применимо, обсудите этические и юридические соображения, которые следует учитывать, особенно при обработке конфиденциальных данных.
8. **Вовлекающие и интерактивные элементы:** поощряйте вовлеченность читателей, задавайте вопросы, приглашая их комментировать, или предлагайте упражнения, которые они могут попробовать выполнить самостоятельно.
9. **Краткие резюме:** завершайте разделы краткими резюме, в которых излагаются основные обсуждаемые моменты и тем самым закрепляются вновь приобретенные знания.
10. **Актуальность и релевантность:** контент должен отражать последние стандарты и практики в этой области, чтобы он оставался актуальным и полезным для читателей.

Рис. 3.9. ChatGPT вернула метазапрос, улучшенный по двум веб-страницам



Метазапросы предлагают динамичный и инновационный способ использования возможностей генеративных моделей ИИ, способствуют созданию сложных, многогранных запросов и даже запросов, которые генерируют другие запросы. Они расширяют область применения ИИ: от создания текста и изображений до создания запросов по стилю и оформлению, а также оптимизации для достижения конкретных целей. Продолжая совершенствоваться и исследовать потенциал метазапросов, вы сможете рассчитывать на значительные изменения, касающиеся использования и взаимодействия с LLM, и научитесь извлекать дополнительную выгоду.

## Резюме

Теперь, прочитав эту главу, вы наверняка поняли, насколько важно давать четкие указания и примеры для достижения желаемых результатов. Кроме того, здесь вы получили практический опыт извлечения структурированных данных из иерархических списков с помощью регулярных выражений в Python, а также научились использовать вложенные структуры данных, такие как JSON и YAML, для получения результатов, пригодных для анализа.

Вы исследовали несколько передовых практик и эффективных методов разработки запросов, в том числе знаменитую методику «Объясни это пятилетнему ребенку», методы ролевых запросов и метазапросов. В следующей главе мы расскажем, как использовать популярный пакет LangChain для работы с LLM, который поможет вам решать еще более сложные задачи промт-инжиниринга.

## ГЛАВА 4

---

# Продвинутые приемы генерации текста с помощью LangChain

Простые методы промтинга помогают в решении большинства задач, но иногда бывает нужно прибегнуть к более мощным инструментам для решения сложных задач и преодоления типичных проблем генеративного ИИ. К таким задачам и проблемам относятся следующие.

### *Длина контекста*

Обобщение всей книги в удобоваримое резюме.

### *Объединение последовательных вводов/выводов LLM*

Создание повествования для книги, включая персонажей, сюжет и построение мира.

### *Выполнение сложных логических задач*

Использование LLM в роли агента. Например, можно создать агента LLM, помогающего достичь ваших личных целей в фитнесе.

Для решения таких сложных задач генеративного ИИ можно с успехом использовать LangChain — фреймворк с открытым исходным кодом, который поможет существенно упростить и улучшить рабочие процессы LLM.

## Введение в LangChain

LangChain — универсальный фреймворк для создания приложений с использованием LLM, доступный в виде пакета как для Python (<https://oreil.ly/YPid->), так и для TypeScript (<https://oreil.ly/5VI0W>). Его главный принцип заключается в том,

что самые эффективные приложения не просто взаимодействуют с языковой моделью через API, но также:

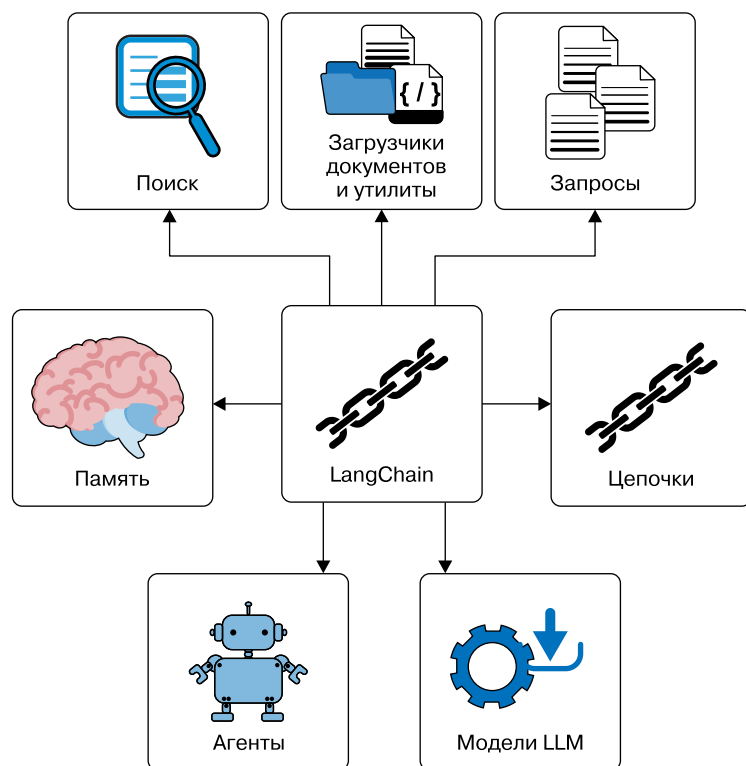
*Расширяют осведомленность о данных*

Создание бесшовной связи между языковой моделью и внешними источниками данных.

*Усиливают самостоятельность*

Стремятся наделить языковые модели способностью взаимодействовать с окружающей средой и влиять на нее.

Как показано на рис. 4.1, фреймворк LangChain состоит из набора абстракций, необходимых для работы с LLM, а также широкого выбора реализаций этих абстракций.



**Рис. 4.1.** Основные модули фреймворка LangChain LLM

Все модули проектировались с прицелом на максимальное удобство и эффективность использования как по отдельности, так и в различных комбинациях. В настоящее время в LangChain существуют шесть основных модулей.

#### *Модель ввода/вывода*

Реализует операции ввода/вывода для работы с моделью.

#### *Поиск*

Ориентирован на поиск релевантного текста для LLM.

#### *Цепочки*

Также известные как *выполняемые последовательности LangChain*, позволяют создавать последовательности операций с LLM или вызовов функций.

#### *Агенты*

Позволяет цепочкам принимать решения о выборе инструментов для использования на основе директив или инструкций высокого уровня.

#### *Память*

Сохраняет состояние приложения между различными участками цепочки.

#### *Обратные вызовы*

Используются для запуска дополнительного кода при определенных событиях, например при создании каждого нового токена.

## Настройка среды

Установить LangChain можно с помощью любой из этих команд:

- `pip install langchain langchain-openai;`
- `conda install -c conda-forge langchain langchain-openai.`

Если вы предпочтете вариант с одновременной установкой всех пакетов, необходимых для опробования примеров из этой книги, то используйте файл `requirements.txt` (<https://oreil.ly/WKOma>) из репозитория GitHub.

Мы советуем устанавливать пакеты в виртуальной среде:

#### *Создайте виртуальную среду*

```
python -m venv venv
```

*Активируйте виртуальную среду*

```
source venv/bin/activate
```

*Установите зависимости*

```
pip install -r requirements.txt
```

LangChain требует интеграции с одной или несколькими моделями. Например, чтобы использовать API модели OpenAI, необходимо установить их пакет для Python с помощью команды `pip install openai`.

Как обсуждалось в главе 1, лучший вариант настройки доступа к модели — определить переменную окружения с именем `OPENAI_API_KEY` в терминале или загрузить ее из файла `.env` с помощью `python-dotenv` (<https://oreil.ly/wvuO7>). Однако при работе над прототипом этот шаг можно пропустить, передав свой ключ API непосредственно функции загрузки модели:

```
from langchain_openai.chat_models import ChatOpenAI
chat = ChatOpenAI(api_key="api_key")
```



Из соображений безопасности не рекомендуется явно указывать ключи API в коде сценария. Вместо этого используйте переменные окружения или файлы конфигурации.

В постоянно меняющемся ландшафте LLM можно столкнуться с проблемой различий API разных моделей. Отсутствие стандартизации интерфейсов может усложнить работы по проектированию запросов и затруднить интеграцию различных моделей в проекты.

Для решения этой проблемы предназначен фреймворк LangChain, который благодаря своей комплексной организации позволяет использовать интерфейсы разных моделей.

LangChain гарантирует, что вам не придется заново разрабатывать запросы или код при переключении между моделями. Его платформенно-независимый подход помогает экспериментировать с широким спектром моделей, таких как Anthropic (<https://www.anthropic.com>), Vertex AI (<https://cloud.google.com/vertex-ai>), OpenAI (<https://openai.com>) и BedrockChat (<https://oreil.ly/bedrock>), чем не только ускоряет процесс опробования модели, но также экономит время и ресурсы за счет упрощения интеграции моделей.

В следующих разделах мы покажем, как использовать пакет OpenAI и их API в LangChain.

## Модели чатов

Модели чатов, такие как GPT-4, стали основным способом взаимодействия с OpenAI API. Они не просто реализуют интерфейс для ввода/вывода текста, но предлагают метод взаимодействия, в котором *сообщения чата* являются элементами ввода и вывода.

Генерация ответов LLM с использованием моделей чата включает ввод одного или нескольких сообщений. В настоящее время LangChain принимает следующие типы сообщений: `AIMessage`, `HumanMessage` и `SystemMessage`. А в качестве результата всегда возвращает `AIMessage`.

### *SystemMessage*

Представляет информацию, которая должна служить инструкцией для системы ИИ. Сообщения этого типа используются для управления поведением или действиями ИИ.

### *HumanMessage*

Представляет информацию, поступающую от человека, взаимодействующего с системой ИИ. Это может быть вопрос, команда или любой другой ввод от пользователя-человека, который ИИ должен обработать и отреагировать.

### *AIMessage*

Представляет информацию, поступающую от самой системы ИИ. Обычно это ответ ИИ на `HumanMessage` или результат выполнения инструкции `SystemMessage`.



Всегда используйте `SystemMessage` для передачи явных указаний. OpenAI усовершенствовала GPT-4, и будущие модели LLM будут уделять особое внимание инструкциям в сообщениях этого типа.

Создадим генератор шуток в LangChain.

Ввод:

```
from langchain_openai.chat_models import ChatOpenAI
from langchain.schema import AIMessage, HumanMessage, SystemMessage

chat = ChatOpenAI(temperature=0.5)
messages = [SystemMessage(content='''Представь, что ты - старший
инженер-программист в начинающей компании.'''),
HumanMessage(content='''Расскажи какую-нибудь смешную шутку
об инженерах-программистах.'''')]
response = chat.invoke(input=messages)
print(response.content)
```

Вывод:

Конечно, вот вам веселая шутка:  
Почему программисты такие плохие водители?  
Потому что они всегда думают, что могут отладить аварию!

Этот код сначала импортирует ChatOpenAI, AIMessage, HumanMessage и SystemMessage. Затем создает экземпляр класса ChatOpenAI с параметром температуры 0,5 (определяет степень случайности).

После создания модели в список messages добавляется объект SystemMessage, определяющий роль LLM, и объект HumanMessage, который запрашивает шутку об инженерах-программистах.

Вызов метода .invoke(input=messages) передает LLM список сообщений, после чего инструкция response.content получает ответ.

Существует более старый метод, позволяющий напрямую вызывать объект чата как chat(messages=messages):

```
response = chat(messages=messages)
```

## Потоковые модели чата

Возможно, используя ChatGPT, вы наблюдали, как модель возвращает слово за словом по одному за раз. Этот особый способ генерации ответов называется *поточковой передачей* и играет важную роль в повышении производительности приложений чата:

```
for chunk in chat.stream(messages):
 print(chunk.content, end="", flush=True)
```

Когда вызывается метод chat.stream(messages), он возвращает ответ частями, то есть каждый сегмент ответа возвращается индивидуально. По мере поступления каждого фрагмента он мгновенно выводится в терминал и сбрасывается. Таким образом, *поточковая передача* обеспечивает минимальную задержку ответов LLM.

Потоковая передача имеет ряд преимуществ с точки зрения конечного пользователя. Во-первых, это резко сокращает время ожидания для пользователей. Как только текст начинает генерироваться, пользователи получают возможность интерпретировать его. Нет необходимости конструировать сообщение целиком до его вывода. Это, в свою очередь, значительно повышает интерактивность и минимизирует задержки.

Тем не менее данный прием имеет свои проблемы. Одна из них — анализ вывода во время потоковой передачи. Понимание и адекватная реакция на сообщение по мере его формирования может оказаться непростой задачей, особенно если ответ сложный и подробный.

## Создание нескольких ответов от LLM

Иногда бывает полезно сгенерировать несколько ответов от LLM. Это особенно актуально при создании динамического контента, такого как публикации в социальных сетях. В таком случае вместо списка сообщений нужно передать модели список списков сообщений.

Ввод:

```
список с 2 списками сообщений, то есть [messages, messages]
synchronous_llm_result = chat.batch([messages]*2)
print(synchronous_llm_result)
```

Вывод:

```
[AIMessage(content='''Конечно, вот вам веселая шутка:\n\nПочему программисты любят кофе?\n\nПотому что он помогает им компилировать мысли!'''),
AIMessage(content='''Конечно, вот вам веселая шутка:\n\nПочему программисты любят темное оформление?\n\nПотому что на темном фоне байты заметнее!''')]
```

Преимущество использования `.batch()` перед `.invoke()` заключается в возможности послать в OpenAI API сразу несколько запросов.

Функции `batch` любого выполняемого объекта из LangChain можно передать аргумент `RunnableConfig`, включающий множество настраиваемых параметров, в том числе и `max_concurrency`:

```
from langchain_core.runnables.config import RunnableConfig

Создать экземпляр RunnableConfig с желаемым ограничением параллелизма:
config = RunnableConfig(max_concurrency=5)

Вызвать метод .batch() и передать ему аргумент config:
results = chat.batch([messages, messages], config=config)
```



Функции, работающие независимо от других процессов и позволяющие одновременно выполнять несколько запросов API, в информатике называют *асинхронными*. В LangChain асинхронные функции дают возможность выполнить множество запросов API параллельно, а не один за другим. Это очень полезно в сложных приложениях, так как уменьшает общую задержку.

Большинство асинхронных функций в LangChain имеют имена, начинающиеся с префикса `a`, например `.ainvoke()` и `.abatch()`. Если вы хотите добиться более эффективного выполнения задач, используйте эти функции.



## Шаблоны запросов в LangChain

До этого момента мы определяли строки запросов прямо в коде, при создании объектов ChatOpenAI. Но в реальных приложениях важно иметь возможность использовать *шаблоны запросов*.

*Шаблоны запросов* помогают создавать воспроизводимые запросы к языковым моделям ИИ. Они состоят из *шаблона*, текстовой строки с параметрами, и создают текстовый запрос для языковой модели.

Вместо шаблонов запросов можно было бы использовать f-строки Python:

```
language = "Python"
prompt = f"Подскажи лучший способ научиться программировать на {language}."
print(prompt) # Подскажи лучший способ научиться программировать на Python.
```

Так почему бы просто не использовать f-строки для быстрого создания шаблонов? А потому, что шаблоны запросов в LangChain позволяют легко и просто:

- проверить входные данные;
- скомпоновать несколько запросов в один;
- определить пользовательские селекторы, которые будут добавлять примеры в запрос;
- сохранять и загружать запросы из файлов .yaml и .json;
- создавать свои шаблоны запросов, которые при создании выполняют дополнительный код или инструкции.

## Язык выражений LangChain

Ключевым компонентом языка выражений LangChain (LangChain Expression Language, LCEL) является оператор конвейера (`|`), который позволяет объединять компоненты или *выполняемые объекты* в пайплайн обработки данных.

Оператор `|` подобен аналогичному оператору в командной оболочке Unix. Он принимает выходные данные одного компонента в цепочке и передает их в качестве входных данных следующему. Благодаря этому можно комбинировать различные компоненты и создавать сложные цепочки операций:

```
chain = prompt | model
```

Оператор `|` используется для объединения компонентов запроса (`prompt`) и модели (`model`). Выходные данные компонента запроса передаются в качестве

входных данных компоненту модели. Этот механизм дает возможность составлять сложные цепочки из базовых компонентов и обеспечивает передачу данных между разными этапами обработки пайплайна.

Кроме того, имейте в виду, что *порядок имеет значение*. Технически можно создать такую цепочку:

```
bad_order_chain = model | prompt
```

Но в таком случае вызов функции `invoke` может сгенерировать ошибку из-за несовместимости значений, возвращаемых моделью, с данными, которые ожидает получить запрос.

Создадим генератор названий компаний, используя шаблоны запросов, который будет возвращать пять-семь подходящих названий компаний:

```
from langchain_openai.chat_models import ChatOpenAI
from langchain_core.prompts import (SystemMessagePromptTemplate,
ChatPromptTemplate)
```

```
template = """
```

```
Представь, что ты творчески мыслящий консультант, придумывающий
названия компаний.
```

```
Ты должен руководствоваться следующими принципами:
```

```
{principles}
```

```
Составь нумерованный список из пяти запоминающихся названий для стартапа
в отрасли {industry}, которая занимается {context}.
```

```
Вот пример формата вывода:
```

```
1. Название1
2. Название2
3. Название3
4. Название4
5. Название5
"""
```

```
model = ChatOpenAI()
system_prompt = SystemMessagePromptTemplate.from_template(template)
chat_prompt = ChatPromptTemplate.from_messages([system_prompt])
```

```
chain = chat_prompt | model
```

```
result = chain.invoke({
 "industry": "медицина",
 "context": "'разработка решений на основе ИИ для автоматического
обобщения историй болезни пациентов'",
 "principles": "'1. Название должно быть коротким и простым для запоминания."
```

```
2. Название должно быть простым для произношения.
3. Название должно быть уникальным и не использоваться другими
компаниями. '''
})

print(result.content)
```

Вывод:

1. SummarAI
2. MediSummar
3. AutoDocs
4. RecordAI
5. SmartSummarize

Код в этом примере сначала импортирует `ChatOpenAI`, `SystemMessagePromptTemplate` и `ChatPromptTemplate`. Затем определяет шаблон запроса с конкретными рекомендациями, который поручает LLM сгенерировать названия компаний. `ChatOpenAI()` инициализирует чат, а `SystemMessagePromptTemplate.from_template(template)` и `ChatPromptTemplate.from_messages([system_prompt])` создают шаблон запроса.

Далее создается цепочка LCEL, соединяющая запрос `chat_prompt` и модель `model`, которая затем *вызывается* обращением к методу `invoke`. Этот метод подставляет значения из словаря вместо заполнителей `{industries}`, `{context}` и `{principles}` в запросе.

В заключение код извлекает ответ LLM в виде строки, обращаясь к свойству `.content` переменной `result`.



### Задайте направление и укажите формат

Тщательно составленные инструкции могут включать такие уточнения, как «Представь, что ты творчески мыслящий консультант, придумывающий названия компаний» и «Составь нумерованный список из пяти-семи запоминающихся названий для стартапа». Подобные сигналы помогут модели LLM решить именно ту задачу, которую вы от нее требуете.

## Использование PromptTemplate с моделями чата

LangChain предоставляет более традиционный класс шаблонов с именем `PromptTemplate`, конструктор которого принимает аргументы `input_variables` и `template`.

Ввод:

```
from langchain_core.prompts import PromptTemplate
from langchain.prompts.chat import SystemMessagePromptTemplate
from langchain_openai.chat_models import ChatOpenAI

prompt=PromptTemplate(
 template='''Представь, что ты переводчик с {input_language} на
 {output_language}.''' ,
 input_variables=["input_language", "output_language"],
)
system_message_prompt = SystemMessagePromptTemplate(prompt=prompt)
chat = ChatOpenAI()
chat.invoke(system_message_prompt.format_messages(
 input_language="английского", output_language="французский"))
```

Вывод:

```
AIMessage(content="Imagine que tu es un traducteur du russe vers le français.",
additional_kwargs={}, example=False)
```

## Парсеры вывода

В главе 3 для извлечения структурированных данных из ответа (в частности, нумерованного списка) мы использовали регулярные выражения, но в LangChain это можно делать автоматически с помощью *парсеров вывода*.

*Парсеры вывода* — это высокоуровневая абстракция, предлагаемая фреймворком LangChain для извлечения структурированных данных из тестовых ответов LLM. В настоящее время доступны следующие парсеры вывода.

### *Парсер списка*

Возвращает список элементов, разделенных запятыми.

### *Парсер даты и времени*

Преобразует вывод LLM в формат даты и времени.

### *Парсер перечислений*

Преобразует строки (string) в значения перечисления (enum).

### *Универсальный парсер*

Обертывает другой парсер вывода, и если тот другой парсер терпит неудачу, то вызывает другую LLM для исправления ошибок.

### *Парсер Pydantic (JSON)*

Преобразует ответы LLM в формат JSON, соответствующий схеме Pydantic.

*Повторяющий парсер*

Выполняет повторный вызов предыдущего парсера в случае неудачи.

*Парсер структурированного вывода*

Может использоваться, когда нужно вернуть несколько полей.

*Парсер XML*

Преобразует ответы LLM в формат на основе XML.

Парсеры вывода LangChain имеют две важные функции:

`.get_format_instructions()`

Эта функция подставляет в запрос необходимые инструкции для получения вывода в структурированном формате, который можно проанализировать.

`.parse(llm_output: str)`

Эта функция преобразует ответы LLM в заранее определенный формат.

Наибольшую гибкость при работе с `ChatOpenAI()`, как правило, обеспечивает парсер Pydantic (JSON).

Парсер Pydantic (JSON) использует библиотеку Pydantic (<https://oreil.ly/QIMih>) для Python, предназначенную для проверки данных с помощью аннотаций типов Python. То есть Pydantic позволяет создавать схемы данных и автоматически проверять и преобразовывать входные данные в соответствии с этими схемами.

Ввод:

```
from langchain_core.prompts.chat import (
 ChatPromptTemplate,
 SystemMessagePromptTemplate,
)
from langchain_openai.chat_models import ChatOpenAI
from langchain.output_parsers import PydanticOutputParser
from pydantic.v1 import BaseModel, Field
from typing import List

temperature = 0.0

class BusinessName(BaseModel):
 name: str = Field(description="Название компании")
 rating_score: float = Field(description="Оценка названия компании.
 0 - худшее, 10 - лучшее.")
```

```

class BusinessNames(BaseModel):
 names: List[BusinessName] = Field(description='''Список
 названий компаний''')

Настройка парсера + внедрение инструкций в шаблон запроса:
parser = PydanticOutputParser(pydantic_object=BusinessNames)

principles = """
- Название должно быть легко запоминаемым.
- При создании названия учитывай индустрию {industry} и сферу
 работы компании.
- Название должно быть простым для произношения.
- Возвращай только названия без любого другого текста.
- Не используй точки, переводы строки и любые другие символы.
- Название не должно содержать больше 10 символов.
"""

Парсер вывода для модели чата:
model = ChatOpenAI()
template = """Придумай пять названий для компании-стартапа в индустрии
{industry}.
Ты должен руководствоваться следующими принципами: {principles}
{format_instructions}
"""

system_message_prompt = SystemMessagePromptTemplate.from_template(template)
chat_prompt = ChatPromptTemplate.from_messages([system_message_prompt])

Создать цепочку LCEL:
prompt_and_model = chat_prompt | model

result = prompt_and_model.invoke(
 {
 "principles": principles,
 "industry": "Наука о данных",
 "format_instructions": parser.get_format_instructions(),
 }
)

Парсер вывода преобразует ответ LLM в объект Pydantic:
print(parser.parse(result.content))

```

Вывод:

```

names=[BusinessName(name='DataWiz', rating_score=8.5),
 BusinessName(name='InsightIQ', rating_score=9.2),
 BusinessName(name='AnalytiQ', rating_score=7.8),
 BusinessName(name='SciData', rating_score=8.1),
 BusinessName(name='InfoMax', rating_score=9.5)]

```

После импортирования необходимых библиотек этот код настраивает модель ChatOpenAI, затем создает SystemMessagePromptTemplate из текстового шаблона и формирует с его помощью шаблон ChatPromptTemplate. Модели Pydantic

(`BusinessName` и `BusinessNames`) используются для структурирования желаемого результата — списка уникальных названий компаний. Парсер Pydantic, созданный для анализа этих моделей и форматирования запроса с использованием введенных пользователем переменных, передается в вызов функции `invoke`. Передав созданный таким способом запрос в модель, вы позволяете ей сгенерировать уникальные названия компаний, которые затем извлекаются с помощью парсера.

Парсеры вывода можно также использовать внутри LCEL:

```
chain = prompt | model | output_parser
```

Добавим парсер вывода непосредственно в цепочку.

Ввод:

```
parser = PydanticOutputParser(pydantic_object=BusinessNames)
chain = chat_prompt | model | parser

result = chain.invoke(
 {
 "principles": principles,
 "industry": "Наука о данных",
 "format_instructions": parser.get_format_instructions(),
 }
)
print(result)
```

Вывод:

```
names=[BusinessName(name='DataTech', rating_score=9.5),...]
```

Теперь цепочка отвечает и за форматирование запроса, и за вызов LLM, и за преобразование ответа LLM в объект Pydantic.



### Укажите формат

В предыдущих запросах используются модели Pydantic и парсеры вывода, что позволяет явно сообщить модели LLM желаемый формат ответа.

Стоит отметить, что, потребовав от LLM вернуть результаты в структурированном формате JSON, на основе ответа LLM можно создать гибкий и универсальный API. У этого подхода есть свои ограничения, такие как размер создаваемого кода JSON и надежность ваших запросов, и все же он является весьма многообещающим для приложений LLM.



Не забывайте о крайних случаях и предусматривайте обработку ошибок, поскольку модели LLM не всегда возвращают результат в желаемом формате.

Парсеры вывода избавляют от сложности и запутанности регулярных выражений, предоставляя простые в использовании функции для различных вариантов использования. Теперь, после знакомства с примерами практического применения, вы сможете использовать парсеры вывода для структурирования и извлечения нужных вам данных из вывода LLM и в полной мере задействовать потенциал ИИ для решения своих задач.

Кроме того, использование парсеров для структурирования данных, извлеченных из LLM, позволяет выбирать, как организовать выходные данные для максимально эффективного использования. Эта гибкость может пригодиться при работе с обширными списками, когда их нужно отсортировать по определенным критериям, например по названиям компаний.

## Оценивание с помощью LangChain

Помимо парсеров вывода, проверяющих ошибки форматирования, большинство систем ИИ также используют *оценки* или метрики для измерения качества каждого ответа на запрос. В LangChain есть ряд готовых инструментов оценки, которые можно напрямую зарегистрировать на их платформе LangSmith (<https://oreil.ly/0Fn94>) для дальнейшей отладки, мониторинга и тестирования. Аналогичную функциональность предлагает альтернативная платформа машинного обучения Weights and Biases (<https://wandb.ai/site>).

Метрики оценки, позволяющие выявлять положительные и отрицательные примеры, полезны не только для тестирования запросов, но также для создания наборов данных и точной настройки пользовательских моделей.

Большинство метрик основано на наборе тестовых примеров в форме пар входных и правильных выходных данных. Обычно эти эталонные ответы создаются или курируются человеком, но также нередко для генерации правильных ответов используются высококачественные модели, такие как GPT-4, как мы сделали это в следующем примере. Имеющийся список описаний финансовых транзакций мы передали GPT-4 для классификации в форме пар значений `transaction_category` и `transaction_type`. Этот процесс можно найти в блокноте Jupyter Notebook `langchain-evals.ipynb` в репозитории книги на GitHub (<https://oreil.ly/a4Hut>).

Считая ответы GPT-4 правильными, мы можем оценить точность других, менее мощных моделей, таких как GPT-3.5-turbo и Mixtral 8x7b (в API она называется `mistral-small`). Добившись достаточно хорошей точности с помощью менее мощной модели, вы сможете сэкономить деньги или уменьшить задержку. Кроме того, если такая модель доступна с открытым исходным кодом, как модель Mistral (<https://oreil.ly/Ec578>), то вы сможете перенести эту задачу на свои серверы и исключить отправку потенциально конфиденциальных данных за



пределы своей организации. Но прежде, чем приступить к размещению модели в своей ОС, мы рекомендуем провести тестирование с помощью внешнего API.

Не забудьте зарегистрироваться (<https://mistral.ai>) и подписаться, чтобы получить ключ API; затем сохраните его в переменной окружения:

```
export MISTRAL_API_KEY=api-key
```

Следующий сценарий является частью блокнота (<https://oreil.ly/DqDOF>), в начале которого определен набор данных `df`. Для краткости рассмотрим только часть сценария, которая выполняет оценку, предположив, что набор данных уже определен.

Ввод:

```
import os
from langchain_mistralai.chat_models import ChatMistralAI
from langchain.output_parsers import PydanticOutputParser
from langchain_core.prompts import ChatPromptTemplate
from pydantic.v1 import BaseModel
from typing import Literal, Union
from langchain_core.output_parsers import StrOutputParser

1. Определение модели:
mistral_api_key = os.environ["MISTRAL_API_KEY"]

model = ChatMistralAI(model="mistral-small", mistral_api_key=mistral_api_key)

2. Определение запроса:
system_prompt = """Представь, что ты — эксперт в анализе банковских
операций, ты должен классифицировать одну операцию.
Всегда возвращай тип и категорию операции:
никогда не возвращай None.
Инструкции по форматированию:
{format_instructions}"""

user_prompt = """Описание операции:
{transaction}"""

prompt = ChatPromptTemplate.from_messages(
 [
 (
 "system",
 system_prompt,
),
 (
 "user",
 user_prompt,
),
]
)
```

```

3. Определение модели Pydantic:
class EnrichedTransactionInformation(BaseModel):
 transaction_type: Union[
 Literal["Покупка", "Списание", "Зачисление",
 "Оплата счета", "Возврат"], None
]
 transaction_category: Union[
 Literal["Еда", "Развлечения", "Транспорт",
 "Коммунальные услуги", "Аренда", "Прочее"],
 None,
]
]

4. Определение парсера вывода:
output_parser = PydanticOutputParser(
 pydantic_object=EnrichedTransactionInformation)

5. Определение функции для удаления лишних обратных слешей:
def remove_back_slashes(string):
 # двойной обратный слеш – это экранированный символ слеша
 cleaned_string = string.replace("\\", "")
 return cleaned_string

6. Создание цепочки LCEL с включенным удалением
обратных слешей и форматированием:
chain = prompt | model | StrOutputParser() \
 | remove_back_slashes | output_parser

transaction = df.iloc[0]["Описание операции"]
result = chain.invoke(
 {
 "transaction": transaction,
 "format_instructions": \
 output_parser.get_format_instructions(),
 }
)

7. Применение цепочки к целому набору данных:
results = []
for i, row in tqdm(df.iterrows(), total=len(df)):
 transaction = row["Описание операции"]
 try:
 result = chain.invoke(
 {
 "transaction": transaction,
 "format_instructions": \
 output_parser.get_format_instructions(),
 }
)
 except:
 result = EnrichedTransactionInformation(
 transaction_type=None,
 transaction_category=None
)

 results.append(result)

```

```
8. Добавление результатов в набор данных в виде значений столбцов
с типом и категорией операций:
transaction_types = []
transaction_categories = []

for result in results:
 transaction_types.append(result.transaction_type)
 transaction_categories.append(result.transaction_category)

df["mistral_transaction_type"] = transaction_types
df["mistral_transaction_category"] = transaction_categories
df.head()
```

Вывод:

Описание операции	transaction_type	transaction_category	mistral_transaction_type	mistral_transaction_category
0 вклад наличными в местном отделении	Зачисление	Прочее	Зачисление	Прочее
1 вклад наличными в местном отделении	Зачисление	Прочее	Зачисление	Прочее
2 списание денег для оплаты аренды	Списание		Аренда	Списание
3 списание денег для оплаты расходов на выходные	Списание		Прочее	Списание
4 покупка книг в книжном магазине	Покупка		Прочее	Покупка
Покупка Развлечения				

Вот подробное описание работы этого сценария.

1. `from langchain_mistralai.chat_models import ChatMistralAI`: импортирует реализацию Mistral из фреймворка LangChain.
2. `from langchain.output_parsers import PydanticOutputParser`: импортирует класс `PydanticOutputParser`, который используется для анализа выходных данных с применением моделей Pydantic, а также парсер выходных строк для удаления обратных слешей из ключей JSON (типичная проблема для моделей Mistral).
3. `mistral_api_key = os.environ["MISTRAL_API_KEY"]`: извлекает ключ API Mistral из переменной окружения. Эту переменную нужно создать до запуска блокнота.
4. `model = ChatMistralAI(model="mistral-small", mistral_api_key=mistral_api_key)`: инициализирует экземпляр `ChatMistralAI` с указанной моделью и ключом API. "mistral-small" — это название модели Mixtral 8x7b (также доступной с открытым исходным кодом) в ее API.
5. `system_prompt` и `user_prompt`: эти строки определяют шаблоны системных и пользовательских запросов, используемых в чате для классификации транзакций.

6. `class EnrichedTransactionInformation(BaseModel)`: определяет Pydantic-модель `EnrichedTransactionInformation` с двумя полями: `transaction_type` и `transaction_category`, каждое из которых имеет определенные разрешенные значения и может принимать значение `None`, если вывод имеет некорректный формат.
7. `def remove_back_slashes(string)`: определяет функцию для удаления лишних обратных слешей из строки.
8. `chain = prompt | model | StrOutputParser() | remove_back_slashes | output_parser`: определяет цепочку выражений и включает в нее парсер вывода и функцию `remove_back_slashes` перед стандартным парсером вывода.
9. `transaction = df.iloc[0]["Описание операции"]`: извлекает описание первой операции из набора данных `df`. Этот набор данных загружается в Jupyter Notebook ранее (<https://oreil.ly/-koAO>) (опущен для краткости).
10. `for i, row in tqdm(df.iterrows(), total=len(df))`: перебирает записи в наборе данных `df` и показывает индикатор выполнения.
11. `result = chain.invoke(...)`: в теле цикла производится вызов цепочки для каждой транзакции.
12. `except`: в случае появления исключения создается объект `EnrichedTransactionInformation` по умолчанию со значениями `None`. Они будут интерпретироваться как ошибки при оценке, но не нарушат цикл обработки.
13. `df["mistral_transaction_type"] = transaction_types, df["mistral_transaction_category"] = transaction_categories`: добавляет типы и категории операций в виде новых столбцов в набор данных, которые затем отображаются с помощью `df.head()`.

Сохранив ответы Mistral в наборе данных, их можно сравнить с категориями и типами операций, определенными ранее, и тем самым проверить точность Mistral. Основная метрика оценки в LangChain — это точность совпадения ответа модели с эталоном. Если ответ верен, для него возвращается 1 балл, если неверен — 0 баллов. В блокноте приводится пример реализации такой оценки (<https://oreil.ly/vPUfI>), которая показывает, что точность Mistral составляет 77,5 %. Однако если вы просто сравниваете строки, то вам, вероятно, не нужно привлекать для этого LangChain.

Главная ценность LangChain заключается в предоставлении проверенных подходов к внедрению сложных способов оценки с использованием LLM. Инструмент оценки `labeled_pairwise_string` сравнивает два вывода и обосновывает выбор между ними, задействуя GPT-4. Один из распространенных сценариев использования этого инструмента оценки — сравнение результатов двух разных запросов или моделей, особенно если тестируемые модели не такие мощные, как GPT-4. Этот инструмент оценки, использующий GPT-4, также можно

использовать для оценки ответов модели GPT-4, но при этом вам следует самому просмотреть аргументы и оценки: если GPT-4 плохо справляется с оценкой результатов решения задачи, то, значит, она так же плохо может решать саму задачу. В блокноте Jupyter (<https://oreil.ly/9O7Mb>) та же классификация операций была повторно запущена с использованием другой модели: `model = ChatOpenAI(model="gpt-3.5-turbo-1106", model_kwargs={"response_format": {"type": "json_object"}})`. В результате появилась возможность выполнить попарное сравнение ответов Mistral и GPT-3.5, как показано ниже. В выводе можно увидеть обоснование оценки.

Ввод:

```
Оценка ответов с помощью инструментов оценки в LangChain:
from langchain.evaluation import load_evaluator
evaluator = load_evaluator("labeled_pairwise_string")

row = df.iloc[0]
transaction = row["Transaction Description"]
gpt3pt5_category = row["gpt3.5_transaction_category"]
gpt3pt5_type = row["gpt3.5_transaction_type"]
mistral_category = row["mistral_transaction_category"]
mistral_type = row["mistral_transaction_type"]
reference_category = row["transaction_category"]
reference_type = row["transaction_type"]

Преобразовать данные в формат JSON для передачи инструменту оценки:
gpt3pt5_data = f"""{{
 "transaction_category": "{gpt3pt5_category}",
 "transaction_type": "{gpt3pt5_type}"
}}"""

mistral_data = f"""{{
 "transaction_category": "{mistral_category}",
 "transaction_type": "{mistral_type}"
}}"""

reference_data = f"""{{
 "transaction_category": "{reference_category}",
 "transaction_type": "{reference_type}"
}}"""

Подготовить запрос, который послужит контекстом для инструмента оценки:
input_prompt = """Представь, что ты - эксперт в анализе банковских операций,
ты должен классифицировать одну операцию.
Всегда возвращай тип и категорию операции:
никогда не возвращай None.
Инструкции по форматированию:
{format_instructions}
Описание операции:
{transaction}
"""
```

```

transaction_types.append(transaction_type_score)
transaction_categories.append(
 transaction_category_score)

accuracy_score = 0
for transaction_type_score, transaction_category_score \
 in zip(
 transaction_types, transaction_categories
):
 accuracy_score += transaction_type_score['score'] + \
 transaction_category_score['score']

accuracy_score = accuracy_score / (len(transaction_types) * 2)
print(f"Accuracy score: {accuracy_score}")

evaluator.evaluate_string_pairs(
 prediction=gpt3pt5_data,
 prediction_b=mistral_data,
 input=input_prompt.format(
 format_instructions=output_parser.get_format_instructions(),
 transaction=transaction),
 reference=reference_data,
)

```

Вывод:

```

{'reasoning': "'Оба эксперта, А и Б, вернули один и тот же ответ на вопрос пользователя. Их ответы полезны, актуальны и корректны и демонстрируют наличие глубоких познаний. Оба корректно определили тип операции как \"Зачисление\" и категорию как \"Прочее\", основываясь на описании операции, предоставленном пользователем. Оба ответа корректно отформатированы, согласно схеме JSON, предоставленной пользователем. Поэтому экспертам присуждается ничья.\\n\\nОкончательный вердикт: [[C]]'",
'value': None,
'score': 0.5}

```

Этот код демонстрирует применение простого инструмента оценки из LangChain, основанного на проверке совпадения строк.

1. `evaluator = load_evaluator("labeled_pairwise_string")`: вызов вспомогательной функции для загрузки инструмента оценки из LangChain по имени. В данном случае используется инструмент `labeled_pairwise_string`.
2. `row = df.iloc[0]`: эта и семь последующих инструкций получают первую запись из набора данных и извлекают значения необходимых столбцов, в том числе описание операции, а также категорию и тип операции, присвоенные моделями Mistral и GPT-3.5. Здесь демонстрируется оценка одной операции, но этот код можно запустить в цикле, заменив эту строку вызовом функции `iterrows` в цикле `for i, row in tqdm(df.iterrows()), total=len(df))` :, как это сделано ниже в блокноте (<https://oreil.ly/dcCOO>).

3. `gpt3pt5_data = f"""{{: чтобы использовать инструмент оценки, основанный на попарном сравнении, ему нужно передать результаты в правильном формате. Форматирование необходимо для моделей GPT-3.5, Mistral и создания эталонных данных.`
4. `input_prompt = """Представь, что ты - эксперт в анализе ...: формирование текста запроса, ответы на которые нужно оценить. Чтобы максимально точно оценить результаты, полученные от других моделей, инструмент оценки должен видеть переданные им инструкции.`
5. `evaluator.evaluate_string_pairs(...: передает инструменту оценки prediction и prediction_b (результат, полученный от GPT-3.5 и Mistral соответственно), а также исходный запрос и эталонный результат и запускает процедуру оценивания.`
6. Основываясь на этом коде, в блокноте (<https://oreil.ly/hW8Wr>) приводится пример оценивания каждой записи в наборе данных в цикле и сохранения результатов и обоснований обратно в набор данных.

Этот пример демонстрирует применение инструмента оценки на основе попарного сравнения, но существует множество других типов инструментов оценки. Инструменты на основе расстояния между строками (расстояния Левенштейна (<https://oreil.ly/AI5G3>)) или расстояния между векторными представлениями ([https://oreil.ly/0p\\_nE](https://oreil.ly/0p_nE)) часто используются в сценариях, где ответы не точно соответствуют эталону, а лишь близки семантически. Расстояние Левенштейна позволяет находить нечеткие совпадения в зависимости от того, сколько одно-символьных изменений потребуется для преобразования оцениваемого текста в эталонный, а расстояние между векторными представлениями (описаны в главе 5) оценивает сходство между ответом и эталоном.

Другой тип инструмента оценки, который мы часто используем в нашей работе, основан на попарном сравнении. Его удобно использовать для сравнения двух разных запросов или моделей с привлечением более мощной модели, такой как GPT-4. Уникальная особенность этого инструмента оценки — он предоставляет обоснование, почитав которое можно понять, почему один подход предпочтительнее другого. В блокноте для этого раздела (<https://oreil.ly/iahTJ>) показан пример использования инструмента попарного сравнения для оценки точности GPT-3.5-turbo и Mixtral 8x7b.



### Оцените качество

Не имея подходящего набора оценок, трудно определить, улучшают или ухудшают качество ответов изменения в запросе или в более широкой системе. Сумев автоматизировать оценивание с помощью интеллектуальных моделей, таких как GPT-4, вы сможете выполнять итерации быстрее и улучшать результаты без дорогостоящих и трудоемких проверок человеком.

## Вызов функций OpenAI

*Вызов функций* — это альтернативный метод парсинга вывода с использованием точно настроенных моделей OpenAI. Эти модели определяют, когда функция должна быть выполнена, и генерируют ответ JSON с *именем и аргументами* предопределенной функции. Некоторые варианты использования этой возможности перечислены ниже.

### *Проектирование сложных чат-ботов*

Возможность планировать встречи. Например, вы можете определить функцию для планирования встреч: `schedule_meeting(date: str, time: str, attendees: List[str])`.

### *Преобразование описания на естественном языке в вызовы API*

Команду типа «Включить свет в коридоре» можно преобразовать в вызов функции `control_device(device: str, action: 'on' | 'off')` домашней системы автоматизации.

### *Извлечение структурированных данных*

Для этого можно определить такую функцию, как `extract_contextual_data(context: str, data_points: List[str])` или `search_database(query: str)`.

Для каждой функции, вызываемой таким способом, требуется соответствующая *схема JSON*. Давайте рассмотрим пример с пакетом OpenAI:

```
from openai import OpenAI
import json
from os import getenv

def schedule_meeting(date, time, attendees):
 # Подключиться к сервису календаря:
 return { "event_id": "1234", "status": "Встреча успешно запланирована!",
 "date": date, "time": time, "attendees": attendees }

OPENAI_FUNCTIONS = {
 "schedule_meeting": schedule_meeting
}
```

После импорта OpenAI и json определяется функция с именем `schedule_meeting`. В этом примере она имитирует процесс планирования встречи и возвращает такие сведения, как идентификатор встречи, дата, время и состав участников. Далее создается вспомогательный словарь OPENAI\_FUNCTIONS, отображающий имя функции и ссылку на нее.



После этого нужно определить список `functions`, связывающий функции с соответствующими схемами JSON. Схема должна включать имя функции, краткое описание и перечень параметров, посредством которых LLM сможет взаимодействовать с ней:

# Схема JSON для нашей предопределенной функции:

```
functions = [
 {
 "type": "function",
 "function": {
 "type": "object",
 "name": "schedule_meeting",
 "description": '''Планирует встречу в указанные дату и время
с указанным составом участников''',
 "parameters": {
 "type": "object",
 "properties": {
 "date": {"type": "string", "format": "date"},
 "time": {"type": "string", "format": "time"},
 "attendees": {"type": "array", "items": {"type": "string"}},
 },
 "required": ["date", "time", "attendees"],
 },
 },
 },
]
```



### Укажите формат

При использовании возможности вызова функций из моделей OpenAI всегда определяйте подробную схему JSON (включая имя и описание). Она поможет модели понять, когда и как правильно вызывать эту функцию.

Определив функцию, сделаем запрос к OpenAI API. Для этого добавим в список `messages` запрос пользователя и с помощью объекта `client` — экземпляра класса `OpenAI` — отправим запрос и схему функции в модель. Получив запрос, LLM проанализирует диалог, определит необходимость вызова функции и вернет имя функции с аргументами. Из ответа LLM извлечем значения `function` и `function_args` и вызовем функцию, а ее результаты добавим обратно в диалог. Затем снова вызовем модель, чтобы получить обобщенное описание процесса для пользователя.

Ввод:

```
client = OpenAI(api_key=getenv("OPENAI_API_KEY"))
```

# Начало диалога:

```
messages = [
 {
```

```

 "role": "user",
 "content": '''Запланируй встречу 2023-11-01 в 14:00
 с Алисой и Бобом.'''
 }
]

Послать запрос и схему функции в модель:
response = client.chat.completions.create(
 model="gpt-3.5-turbo-1106",
 messages=messages,
 tools=functions,
)

response = response.choices[0].message

Проверить, решила ли модель вызвать нашу функцию:
if response.tool_calls:
 # Получить первый вызов функции:
 first_tool_call = response.tool_calls[0]

 # Найти имя функции и ее аргументы для вызова:
 function_name = first_tool_call.function.name
 function_args = json.loads(first_tool_call.function.arguments)
 print("Имя функции: ", function_name)
 print("Аргументы функции: ", function_args)

 function = OPENAI_FUNCTIONS.get(function_name)

 if not function:
 raise Exception(f"Функция {function_name} не найдена.")

 # Вызов функции:
 function_response = function(**function_args)

 # Вернуть результат функции модели:
 messages.append(
 {
 "role": "function",
 "name": "schedule_meeting",
 "content": json.dumps(function_response),
 }
)

 # Получить ответ модели:
 second_response = client.chat.completions.create(
 model="gpt-3.5-turbo-0613", messages=messages
)

 print(second_response.choices[0].message.content)

```

Вывод:

Имя функции: `schedule_meeting`  
 Аргументы функции: `{'date': '2023-11-01', 'time': '14:00', 'attendees': ['Алиса', 'Боб']}`  
 Я запланировал встречу 2023-11-01 в 14:00 с Алисой и Бобом.  
 Идентификатор встречи 1234.

Вот несколько важных замечаний, о которых следует помнить, планируя вызывать функции:

- LLM может вызывать множество функций;
- при нехватке информации OpenAI может «выдумывать» недопустимые параметры функции, поэтому явно указывайте свои намерения в системном сообщении;
- параметр `function_call` можно задать разными способами:
  - чтобы задать вызов конкретной функции: `tool_choice: {"type": "function", "function": {"name": "my_function"}}`;
  - если в ответ на сообщение пользователя не должна вызываться никакая функция: `tool_choice: "none"`;
  - по умолчанию (`tool_choice: "auto"`) модель сама решает, какую функцию вызывать.

## Параллельный вызов функций

При желании в сообщениях в чате можно выразить намерения, требующие одновременного вызова нескольких инструментов. Эта стратегия известна как *параллельный вызов функций*.

Для примера изменим список `messages` в коде, представленном выше, так, чтобы запланировать сразу две встречи:

```
Начало диалога:
messages = [
 {
 "role": "user",
 "content": '''Запланируй встречу 2023-11-01 в 14:00 с Алисой
и Бобом. Затем запланируй еще одну встречу 2023-11-02
в 15:00 с Чарли и Дейвом.'''
 }
]
```

Затем изменим предыдущий код, включив в него цикл `for`.

Ввод:

```
Послать запрос и схему функции в модель:
response = client.chat.completions.create(
 model="gpt-3.5-turbo-1106",
 messages=messages,
 tools=functions,
)
response = response.choices[0].message

Проверить, решила ли модель вызвать нашу функцию:
if response.tool_calls:
 for tool_call in response.tool_calls:
 # Получить имя функции и ее аргументы для вызова:
 function_name = tool_call.function.name
 function_args = json.loads(tool_call.function.arguments)
 print("Имя функции: ", function_name)
 print("Аргументы функции: ", function_args)

 function = OPENAI_FUNCTIONS.get(function_name)

 if not function:
 raise Exception(f"Function {function_name} not found.")

 # Вызов функции:
 function_response = function(**function_args)

 # Вернуть результат функции модели:
 messages.append(
 {
 "role": "function",
 "name": function_name,
 "content": json.dumps(function_response),
 }
)

Получить ответ модели:
second_response = client.chat.completions.create(
 model="gpt-3.5-turbo-0613", messages=messages
)

print(second_response.choices[0].message.content)
```

Вывод:

```
Имя функции: schedule_meeting
Аргументы функции: {'date': '2023-11-01', 'time': '14:00',
'attendees': ['Alice', 'Bob']}
```

Имя функции: `schedule_meeting`  
 Аргументы функции: `{'date': '2023-11-02', 'time': '15:00', 'attendees': ['Charlie', 'Dave']}`  
 Запланировано две встречи:  
 1. Встреча с Алисой и Бобом 2023-11-01 в 14:00.  
 2. Встреча с Чарли и Дейвом 2023-11-02 в 15:00.

Этот пример демонстрирует возможность эффективно управлять вызовами нескольких функций. В нем дважды была вызвана функция `schedule_meeting` для планирования двух встреч. Как видите, с помощью инструментов на базе ИИ легко можно обрабатывать разнообразные и сложные запросы.

## Вызов функций в LangChain

Если вы предпочитаете не писать схемы JSON, а просто извлекать структурированные данные из ответа LLM, то LangChain поможет вам в этом. Для этого вам нужно лишь воспользоваться возможностью вызова функций с помощью Pydantic.

Ввод:

```
from langchain.output_parsers.openai_tools import PydanticToolsParser
from langchain_core.utils.function_calling import convert_to_openai_tool
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai.chat_models import ChatOpenAI
from langchain_core.pydantic_v1 import BaseModel, Field
from typing import Optional

class Article(BaseModel):
 """Определяет ключевые моменты и противоположные мнения в статье."""
 points: str = Field(..., description="Ключевые моменты в статье")
 contrarian_points: Optional[str] = Field(
 None, description="Противоположные точки зрения в статье"
)
 author: Optional[str] = Field(None, description="Автор статьи")

_EXTRACTION_TEMPLATE = """Извлеки из следующего отрывка упомянутые объекты
вместе с их свойствами и сохрани.

Если свойство отсутствует и его не нужно передавать в параметрах функции,
то не включай его в выходные данные."""

Создать запрос, требующий от LLM извлечь информацию:
prompt = ChatPromptTemplate.from_messages(
 [{"system", _EXTRACTION_TEMPLATE}, {"user", "{input}"}]
)
```

```

model = ChatOpenAI()

pydantic_schemas = [Article]

Преобразовать объекты Pydantic в соответствующую схему:
tools = [convert_to_openai_tool(p) for p in pydantic_schemas]

Дать модели доступ к этим инструментам:
model = model.bind_tools(tools=tools)

Создать сквозную цепочку:
chain = prompt | model | PydanticToolsParser(tools=pydantic_schemas)

result = chain.invoke(
 {
 "input": """"В недавней статье с названием 'Внедрение ИИ
 в промышленности' рассматриваются такие ключевые моменты,
 как растущий интерес... Однако автор, доктор Джейн Смит, ...""""
 }
)
print(result)

```

Вывод:

```

[Article(points='Растущий интерес к ИИ в разных секторах, ...',
contrarian_points='Без строгих правил, ...',
author='Dr. Jane Smith')]

```

Этот код сначала импортирует различные модули, включая `PydanticToolsParser` и `ChatPromptTemplate`, необходимые для анализа и создания шаблонов запросов. Затем определяется `Pydantic`-модель `Article`, сообщающая структуру информации, которую нужно извлечь из заданного текста. Используя настраиваемый шаблон запроса и модель `ChatOpenAI`, код поручает ИИ извлечь из статьи ключевые моменты и противоположные мнения. Наконец, извлеченные данные аккуратно преобразуются в предопределенную `Pydantic`-модель и выводятся на экран.

Отметим несколько важных моментов в этом примере.

### *Преобразование Pydantic-схемы в инструменты OpenAI*

```
инструменты = [convert_to_openai_tool(p) для p в pydantic_schemas]
```

### *Привязка инструментов непосредственно к LLM*

```
model = model.bind_tools(tools=tools)
```

### *Создание цепочки LCEL, содержащей парсер инструментов*

```
chain = prompt | model | PydanticToolsParser(tools=pydantic_schemas)
```

## Извлечение данных с помощью LangChain

Функция `create_extraction_chain_pydantic` предоставляет более компактную версию предыдущей реализации. Просто передав ей Pydantic-модель и LLM, поддерживающую вызов функций, можно организовать параллельный вызов функций.

Ввод:

```
from langchain.chains.openai_tools import create_extraction_chain_pydantic
from langchain_openai.chat_models import ChatOpenAI
from langchain_core.pydantic_v1 import BaseModel, Field

Использовать новейшую модель, поддерживающую инструменты:
model = ChatOpenAI(model="gpt-3.5-turbo-1106")

class Person(BaseModel):
 """Имя и возраст человека."""

 name: str = Field(..., description="Имя человека")
 age: int = Field(..., description="Возраст человека")

chain = create_extraction_chain_pydantic(Person, model)
chain.invoke({'input': ''Бобу 25 лет. Он живет в Нью-Йорке.
Он любит играть в баскетбол. Саре 30 лет. Она живет в
Сан-Франциско. Она любит играть в теннис.''})
```

Вывод:

```
[Person(name='Боб', age=25), Person(name='Сара', age=30)]
```

Pydantic-модель `Person` имеет два свойства: `name` (имя) и `age` (возраст); вызовом функции `create_extraction_chain_pydantic` с входным текстом мы инициировали вызов одной и той же функции со стороны и создание двух объектов `Person`.

## Планирование запросов

У вас могут возникнуть проблемы, если пользовательские запросы выражают несколько намерений со сложными зависимостями. *Планирование запросов* — это эффективный способ разбить пользовательский запрос на ряд шагов, которые можно выполнить в виде графа запросов с соответствующими зависимостями:

```
from langchain_openai.chat_models import ChatOpenAI
from langchain.output_parsers.pydantic import PydanticOutputParser
from langchain_core.prompts.chat import (
 ChatPromptTemplate,
 SystemMessagePromptTemplate,
```

```

)
from pydantic.v1 import BaseModel, Field
from typing import List

class Query(BaseModel):
 id: int
 question: str
 dependencies: List[int] = Field(
 default_factory=list,
 description="""Список подзапросов, которые необходимо выполнить
 для завершения этой задачи.
 Используй подзапросы, если что-то неизвестно и может понадобиться
 задать много вопросов, чтобы получить ответ.
 Зависимости должны представлять собой только другие запросы."""
)

class QueryPlan(BaseModel):
 query_graph: List[Query]

```

Определение `QueryPlan` и `Query` позволяет сначала попросить LLM разобрать запрос пользователя на несколько этапов. Давайте выясним, как создать план запроса.

Ввод:

```

Подготовка модели чата:
model = ChatOpenAI()

Настройка парсера:
parser = PydanticOutputParser(pydantic_object=QueryPlan)

template = """Сгенерируй план запроса для выполнения задачи.

Ответь на следующий запрос: {query}

Верни последующие запросы в виде графа:
{format_instructions}
"""

system_message_prompt = SystemMessagePromptTemplate.from_template(template)
chat_prompt = ChatPromptTemplate.from_messages([system_message_prompt])

Создать цепочку LCEL с запросом, моделью и парсером:
chain = chat_prompt | model | parser

result = chain.invoke({
 "query": """Я хочу получить информацию из моей базы данных. Вычислить
 средний возраст десяти лучших клиентов. Отправить электронное письмо
 Джону. Кроме того, независимо от других задач, я хочу отправить
 приветственное письмо Саре.""",
 "format_instructions": parser.get_format_instructions()})

print(result.query_graph)

```



Вывод:

```
[Query(id=1, question='Получить 10 лучших клиентов', dependencies=[]),
Query(id=2, question='Вычислить средний возраст клиентов', dependencies=[1]),
Query(id=3, question='Послать письмо Джону', dependencies=[2]),
Query(id=4, question='Послать приветственное письмо Cape', dependencies=[])]
```

Этот код создает экземпляры ChatOpenAI и PydanticOutputParser для структуры QueryPlan. Затем вызывается модель LLM и по результатам анализа ее ответа создается структурированный граф запроса, определяющий порядок решения задач, указанных в запросе, с их уникальными зависимостями.

## Создание шаблонов запросов с несколькими примерами

Работа с генеративными возможностями LLM часто предполагает выбор между обучением *без примеров* или с *несколькими примерами*. Хотя обучение без примеров адаптируется к задачам исключительно на основе запросов, его зависимость от этапа предварительного обучения означает, что оно не всегда может давать точные результаты.

С другой стороны, при обучении на нескольких примерах, которое предполагает передачу нескольких вариантов решения задачи в запросе, есть возможность оптимизировать поведение модели и получить более желательные результаты.

Из-за ограниченной длины контекста LLM вам часто придется выбирать качественные примеры из множества для добавления в запросы, стремясь получить от LLM эффективный и детерминированный результат.



Несмотря на имеющиеся ограничения, окна контекста в моделях LLM продолжают увеличиваться, что позволяет передать некоторое количество примеров и минимизировать затраты на использование API.

Рассмотрим два метода добавления примеров в шаблоны запросов: включением *фиксированного числа примеров* и *селекторов примеров*.

### Включение нескольких примеров фиксированной длины

Сначала посмотрим, как создать шаблон запроса из нескольких фрагментов, используя фиксированное количество примеров. В основе этого метода лежит создание надежного набора из нескольких примеров:

```
from langchain_openai.chat_models import ChatOpenAI
from langchain_core.prompts import (
 FewShotChatMessagePromptTemplate,
```

```

 ChatPromptTemplate,
)

examples = [
 {
 "question": "Как называется столица Франции?",
 "answer": "Париж",
 },
 {
 "question": "Как называется столица Испании?",
 "answer": "Мадрид",
 } # ...еще примеры...
]

```

Каждый пример представляет собой словарь, содержащий ключи `question` и `answer`. Эти словари будут использоваться для создания пар сообщений `HumanMessage` и `AIMessage`.

## Форматирование примеров

Далее настроим `ChatPromptTemplate` для форматирования отдельных примеров, которые затем будут добавлены в `FewShotChatMessagePromptTemplate`.

Ввод:

```

example_prompt = ChatPromptTemplate.from_messages(
 [
 ("human", "{question}"),
 ("ai", "{answer}"),
]
)

few_shot_prompt = FewShotChatMessagePromptTemplate(
 example_prompt=example_prompt,
 examples=examples,
)

print(few_shot_prompt.format())

```

Вывод:

```

Human: Как называется столица Франции?
AI: Париж
Human: Как называется столица Испании?
AI: Мадрид
...еще примеры...

```

Обратите внимание, что `example_prompt` создает пары `HumanMessage` и `AIMessage` с запросами `{question}` и `{answer}`.

После вызова `few_shot_prompt.format()` примеры выводятся в виде строк. Поскольку нам нужно использовать их в запросе к LLM, давайте создадим новый `ChatPromptTemplate`.

Ввод:

```
from langchain_core.output_parsers import StrOutputParser

final_prompt = ChatPromptTemplate.from_messages(
 [("system", '''Ответь на несколько вопросов
 о странах. Возвращай только названия
 столиц.'''),
 few_shot_prompt, ("human", "{question}")],
)

model = ChatOpenAI()

Создать цепочку LCEL с запросом, моделью и StrOutputParser():
chain = final_prompt | model | StrOutputParser()

result = chain.invoke({"question": "Как называется столица Америки?"})

print(result)
```

Вывод:

Вашингтон

После вызова цепочки LCEL в `final_prompt` вслед за `SystemMessage` добавляется несколько примеров.

Обратите внимание, что LLM возвращает только 'Вашингтон'. Это связано с тем, что ответ LLM анализируется парсером вывода `StrOutputParser()`. Добавление `StrOutputParser()` в цепочку — это распространенный способ гарантировать получение ответов LLM в виде *строковых значений*. Подробнее об этом мы поговорим, когда начнем знакомиться с последовательными цепочками в LCEL.

## Выбор нескольких примеров по длине

Прежде чем углубиться в код, определим нашу задачу. Представьте, что вы создаете приложение на базе GPT-4, генерирующее рассказы. Пользователь вводит список имен персонажей с ранее созданными историями. Однако разные пользователи могут вводить списки персонажей разной длины. Включение слишком большого количества персонажей может привести к созданию рассказа, размер которого превысит ограничение контекстного окна LLM. В таких случаях можно

использовать `LongBasedExampleSelector`, чтобы адаптировать запрос к длине пользовательского ввода:

```
from langchain_core.prompts import FewShotPromptTemplate, PromptTemplate
from langchain.prompts.example_selector import LengthBasedExampleSelector
from langchain_openai.chat_models import ChatOpenAI
from langchain_core.messages import SystemMessage
import tiktoken

examples = [
 {"input": "Голлум", "output": "<Рассказ с участием Голлума>"},
 {"input": "Гэндальф", "output": "<Рассказ с участием Гэндальфа>"},
 {"input": "Бильбо", "output": "<Рассказ с участием Бильбо>"},
]

story_prompt = PromptTemplate(
 input_variables=["input", "output"],
 template="Персонаж: {input}\nРассказ: {output}",
)

def num_tokens_from_string(string: str) -> int:
 """Возвращает количество токенов в текстовой строке."""
 encoding = tiktoken.get_encoding("cl100k_base")
 num_tokens = len(encoding.encode(string))
 return num_tokens

example_selector = LengthBasedExampleSelector(
 examples=examples,
 example_prompt=story_prompt,
 max_length=1000, # в запрос нужно включить 1000 токенов из примеров
 # get_text_length: Callable[[str], int] = lambda x: len(re.split("\n| ", x))
 # Определим функцию get_text_length как псевдоним другой функции,
 # подсчитывающей токены с помощью библиотеки TikToken:
 get_text_length=num_tokens_from_string,
)
```

Здесь сначала настраивается экземпляр `PromptTemplate`, который принимает две входные переменные для каждого примера. Затем `LengthBasedExampleSelector` подбирает количество примеров с учетом *длины входных примеров*, чтобы LLM не сгенерировала рассказ длиннее контекстного окна.

На роль функции `get_text_length` также выбирается функция `num_tokens_from_string`, которая подсчитывает общее количество токенов с помощью `tiktoken`. Инструкция `max_length=1000` определяет *количество токенов*, которое должно использоваться вместо применения следующей функции по умолчанию:

```
get_text_length: Callable[[str], int] = lambda x: len(re.split("\n| ", x))
```

Теперь свяжем все эти элементы вместе:

```
dynamic_prompt = FewShotPromptTemplate(
 example_selector=example_selector,
```

```
example_prompt=story_prompt,
prefix="''Сгенерируй рассказ с участием {character}, используя
текущие пары персонажей/рассказов как контекст.'''",
suffix="Персонаж: {character}\nРассказ:",
input_variables=["character"],
)

Определить нового персонажа из романа "Властелин колец":
formatted_prompt = dynamic_prompt.format(character="Фродо")

Создать модель чата:
chat = ChatOpenAI()

response = chat.invoke([SystemMessage(content=formatted_prompt)])
print(response.content)
```

Вывод:

Фродо — молодой хоббит, живущий мирной жизнью в Шире. Однако его жизнь...



### Приведите примеры и укажите формат

При добавлении нескольких примеров важно учитывать их суммарную длину, чтобы модель ИИ могла принять во внимание их все. Настройте длину запроса и добавьте подходящие примеры, чтобы LLM не сгенерировала ответ, размер которого может превысить ограничение контекстного окна.

После форматирования запроса код создает модель чата с помощью `ChatOpenAI()` и загружает отформатированный запрос в `SystemMessage`, после чего генерирует небольшой рассказ о Фродо из «Властелина колец».

Вместо создания и форматирования `ChatPromptTemplate` зачастую проще просто вызвать `SystemMessage` с форматированным запросом:

```
result = model.invoke([SystemMessage(content=formatted_prompt)])
```

## Ограничения обучения с несколькими примерами

Обучение с несколькими примерами имеет ограничения. Иногда это может оказаться полезным, но такое обучение не всегда дает результаты высокого качества. Это обусловлено прежде всего двумя причинами:

- иногда при попытке представить обученным моделям, таким как GPT-4, может возникнуть эффект переобучения, заставляющий модели отдавать приоритет примерам в ущерб фактическому запросу;
- модели LLM имеют ограниченное окно контекста, из-за чего всегда будет существовать компромисс между количеством примеров и длиной ответа: слишком большое количество примеров может ограничить длину ответа и наоборот.

Преодолеть эти ограничения можно несколькими способами. Во-первых, если запросы, включающие несколько примеров, не приносят желаемых результатов, то перефразируйте их или поэкспериментируйте с языком самих запросов. Вариации в формулировке запроса могут приводить к разным ответам, что подчеркивает эмпирический характер промт-инжиниринга.

Во-вторых, попробуйте включить в запрос явные инструкции, требующие от модели игнорировать примеры после того, как она поймет задачу, или использовать примеры только для руководства по форматированию. Это может предотвратить переобучение модели на примерах.

Если задача сложная и качество результатов, получаемых моделью с обучением на нескольких примерах, оказывается неудовлетворительным, то, возможно, вам придется рассмотреть возможность тонкой настройки (<https://oreil.ly/S40bZ>) вашей модели. Тонкая настройка может обеспечить более точное понимание конкретной задачи моделью и значительно повысить качество ответов.

## Сохранение и загрузка подсказок LLM

Чтобы эффективно использовать генеративные модели ИИ, такие как GPT-4, запросы лучше хранить в отдельных файлах, а не в коде на Python. Это позволит расширить возможности совместного использования, хранения и управления версиями запросов.

LangChain поддерживает сохранение и загрузку подсказок в форматах JSON и YAML. Еще одна ключевая особенность LangChain — поддержка подробных спецификаций в одном или в нескольких файлах. Она позволяет хранить разные компоненты, такие как шаблоны, примеры и т. д., в отдельных файлах и ссылаться на них по мере необходимости.

Посмотрим, как сохранять и загружать подсказки:

```
from langchain_core.prompts import PromptTemplate, load_prompt

prompt = PromptTemplate(
 template='''Translate this sentence from English to Spanish.
 \nSentence: {sentence}\nTranslation:''',
 input_variables=["sentence"],
)

prompt.save("translation_prompt.json")

Загрузить шаблон запроса:
load_prompt("translation_prompt.json")
Возвращает PromptTemplate()
```

Здесь после импорта `PromptTemplate` и `load_prompt` из модуля `langchain.prompts` создается шаблон запроса `PromptTemplate` для задач перевода с английского на испанский и сохраняется в файл `translation_prompt.json`. Далее сохраненный шаблон запроса загружается вызовом функции `load_prompt`, которая возвращает экземпляр `PromptTemplate`.



Имейте в виду, что функция сохранения в `LangChain` может работать не со всеми типами шаблонов запросов. Чтобы избежать проблем, для сохранения и чтения файлов с запросами, которые `LangChain` не поддерживает, можно использовать библиотеку *pickle* или файлы `.txt`.

Теперь вы знаете два метода создания шаблонов запросов с помощью `LangChain`: с фиксированным количеством примеров и использованием селектора примеров.

Первый заключается в создании набора из нескольких примеров и использовании объекта `ChatPromptTemplate` для конструирования сообщения и формирования основы для создания объекта `FewShotChatMessagePromptTemplate`.

Второй, основанный на использовании селектора примеров, хорошо подходит для случаев, когда имеется множество пользовательских примеров, значительно различающихся по длине. В таких сценариях можно использовать `LongBasedExampleSelector`, чтобы адаптировать запрос к длине пользовательского ввода. Этот метод помогает не превысить ограничение контекстного окна в LLM.

Кроме того, вы увидели, что запросы можно сохранять и загружать из файлов. Эта возможность позволяет хранить и совместно использовать запросы, а также управлять их версиями.

## Подключение данных

Использование приложения LLM в сочетании с вашими данными открывает массу возможностей для повышения эффективности процессов принятия решений.

Данные вашей организации могут храниться в разных формах.

### *Неструктурированные данные*

Это могут быть Документы Google, потоки информации из коммуникационных платформ, таких как Slack или Microsoft Teams, веб-страницы, внутренняя документация или репозитории кода на GitHub.

### Структурированные данные

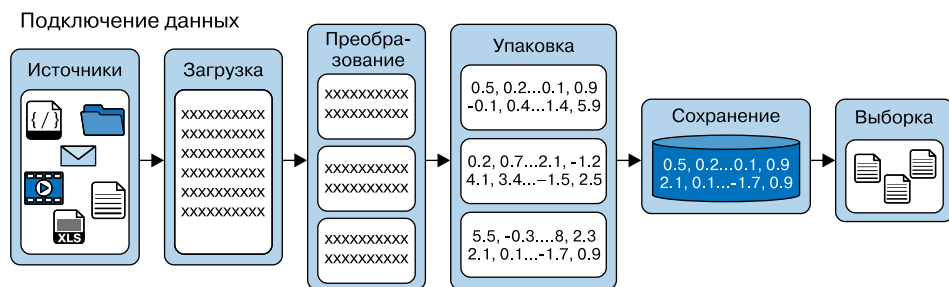
Информация, аккуратно размещенная в базах данных: SQL, NoSQL или графовых.

Для получения неструктурированных данных необходим процесс загрузки, преобразования, упаковки и последующего сохранения в векторной базе данных, если требуется. *Векторная база данных* — это специализированный тип баз данных, предназначенный для эффективного хранения данных в форме векторов, представляющих сложные данные, такие как текст или изображения, в формате, подходящем для машинного обучения и поиска по сходству.

Для получения структурированных данных, учитывая, что они уже проиндексированы, можно использовать агента LangChain и с его помощью выполнять промежуточные запросы к базе данных. Это позволит извлекать определенные признаки, которые затем можно использовать в запросах к LLM.

Для Python есть несколько пакетов, которые могут помочь при приеме данных, включая Unstructured (<https://oreil.ly/n0hDD>), LlamaIndex (<https://www.llamaindex.ai>) и LangChain (<https://oreil.ly/PjV9o>).

На рис. 4.2 показан стандартный подход к приему данных. Он начинается с определения источников данных, которые затем загружаются в документы. После этого документы разбиваются на части и сохраняются в векторной базе данных для последующего поиска.



**Рис. 4.2.** Включение данных в пайплайн выборки

LangChain, в частности, предоставляет все необходимое для загрузки, преобразования, сохранения и извлечения данных.

### Загрузчики документов

Упрощают загрузку информационных ресурсов, или *документов*, из различных источников, таких как документы Word, файлы PDF, текстовые файлы и даже веб-страницы.



### *Преобразователи документов*

Эти инструменты позволяют сегментировать документы, преобразовывать их в формат вопросов и ответов, удалять лишние документы и многое другое.

### *Модели векторизации текста*

Могут преобразовывать неструктурированный текст в последовательности чисел с плавающей точкой, используемые для поиска по сходству в векторных хранилищах.

### *Векторные базы данных (векторные хранилища)*

Эти хранилища могут сохранять и выполнять поиск по данным в векторном представлении.

### *Компоненты извлечения данных*

Эти инструменты позволяют запрашивать и извлекать данные.

Стоит также отметить, что другие платформы LLM, такие как LlamaIndex (<https://oreil.ly/9NcTB>), прекрасно работают с LangChain. LlamaHub (<https://llama-hub.ai>) — еще одна библиотека с открытым исходным кодом, предназначенная для создания загрузчиков документов и способная создавать объекты `Document`, характерные для LangChain.

## Загрузчики документов

Представим, что нам поручено создать пайплайн сбора данных для LLM в компании NutriFusion Foods. Информация, которую необходимо собрать для LLM, содержится:

- в PDF-файле книги *Principles of Marketing*;
- двух маркетинговых отчетов в формате `.docx` в общедоступном хранилище Google Cloud Storage;
- трех файлах `.csv` с данными о маркетинговой эффективности за 2021, 2022 и 2023 годы.

Создадим новый блокнот Jupyter или файл Python в папке `content/chapter_4` общедоступного репозитория (<https://oreil.ly/cVTyI>), а затем запустим команду `pip install pdf2image docx2txt pypdf`, которая установит три пакета.

Все данные, кроме файлов `.docx`, можно найти в папке `content/chapter_4/data` (<https://oreil.ly/u9gMx>). Начнем с того, что импортируем все необходимые загрузчики данных и создадим пустой список `all_documents` для хранения всех объектов `Document`, представляющих источники данных.

Ввод:

```
from langchain_community.document_loaders import Docx2txtLoader
from langchain_community.document_loaders import PyPDFLoader
from langchain_community.document_loaders.csv_loader import CSVLoader
import glob
from langchain.text_splitter import CharacterTextSplitter

Для хранения документов из всех источников данных:
all_documents = []

Загрузить PDF:
loader = PyPDFLoader("data/principles_of_marketing_book.pdf")
pages = loader.load_and_split()
print(pages[0])

Добавить дополнительные метаданные для каждой страницы:
for page in pages:
 page.metadata["description"] = "Principles of Marketing Book"

Убедиться, что метаданные успешно добавлены:
for page in pages[0:2]:
 print(page.metadata)

Сохранить страницы из книги о маркетинге:
all_documents.extend(pages)

csv_files = glob.glob("data/*.csv")

Составить список файлов, в именах которых присутствует слово Marketing:
csv_files = [f for f in csv_files if "Marketing" in f]

Данные из каждого файла .csv:
for csv_file in csv_files:
 loader = CSVLoader(file_path=csv_file)
 data = loader.load()
 # Сохранить в списке all_documents:
 all_documents.extend(data)

text_splitter = CharacterTextSplitter.from_tiktoken_encoder(
 chunk_size=200, chunk_overlap=0
)

urls = [

 '''https://storage.googleapis.com/oreilly-content/NutriFusion%20Foods%
 20Marketing%20Plan%202022.docx''',
```

```
'''https://storage.googleapis.com/oreilly-content/NutriFusion%20Foods%
20Marketing%20Plan%202023.docx'''
]

docs = []
for url in urls:
 loader = Docx2txtLoader(url.replace('\n', ''))
 pages = loader.load()
 chunks = text_splitter.split_documents(pages)

 # Добавить метаданные для каждого фрагмента:
 for chunk in chunks:
 chunk.metadata["source"] = "NutriFusion Foods Marketing Plan - 2022/2023"
 docs.extend(chunks)

Сохранить страницы из книги о маркетинге:
all_documents.extend(docs)
```

Вывод:

```
page_content='Principles of Marketing'
metadata={'source': 'data/principles_of_marketing_book.pdf', 'page': 0}
{'source': 'data/principles_of_marketing_book.pdf', 'page': 0,
'description': 'Principles of Marketing Book'}
{'source': 'data/principles_of_marketing_book.pdf', 'page': 1,
'description': 'Principles of Marketing Book'}
```

Этот код импортирует файл `.pdf` с помощью `PyPDFLoader` и разбивает его на несколько страниц с помощью функции `.load_and_split()`.

К каждой странице добавляются дополнительные метаданные, имеющие форму словаря Python в объекте `Document`. Обратите также внимание, как в предыдущем выводе выглядит свойство `source` в метаданных объектов `Document`.

Далее с помощью пакета `glob` ищутся все файлы `.csv` и загружаются в объекты `Document` с помощью `CSVLoader`.

Наконец, код загружает два маркетинговых отчета из общедоступной корзины в Google Cloud Storage, а затем с помощью `text_splitter` разбивает их на фрагменты по 200 токенов.

В этом разделе мы рассказали, как создать комплексный пайплайн загрузки документов компании NutriFusion Foods для LLM. Мы извлекли данные из PDF-файла, нескольких файлов CSV и двух файлов `.docx`, добавили в каждый документ дополнительные метаданные с информацией о контексте.

Теперь вы знаете, как интегрировать данные из различных источников в единый пайплайн данных.

## Инструменты для разбиения текста

Балансировка длины каждого документа играет важную роль. Если документ слишком длинный, он может превысить длину контекста LLM (максимальное количество токенов, которые LLM способна обработать в рамках одного запроса), а если документ слишком фрагментирован, то есть риск потерять важный контекст, что столь же нежелательно.

При разбиении текста можно столкнуться с некоторыми проблемами, такими как:

- особые фрагменты текста, такие как хештеги, символы @ или ссылки, могут быть разбиты не так, как надо бы, что повлияет на общую структуру документов;
- если документ имеет сложное форматирование — таблицы, списки или многоуровневые заголовки, — то у инструмента, осуществляющего разбиение текста, может не получиться сохранить исходное форматирование.

Эти проблемы имеют решения, которые мы рассмотрим позже.

В этом разделе вы познакомитесь с инструментами для разбиения текста, имеющимися в LangChain и используемыми для разбиения больших фрагментов текста, чтобы адаптировать их к размеру контекстного окна вашей модели.



Не существует идеального размера документа. Для начала выберите хорошую эвристику, а затем создайте обучающий/тестовый набор, который сможете использовать для оценки LLM.

LangChain предоставляет несколько инструментов, позволяющих разбивать текст любым из следующих способов:

- по количеству токенов;
- рекурсивно по символам;
- по количеству символов;
- по коду;
- по разметке Markdown заголовков.

Рассмотрим три наиболее популярных инструмента: `CharacterTextSplitter`, `TokenTextSplitter` и `RecursiveCharacterTextSplitter`.

## Разбиение текста по количеству символов и токенов

В главе 3 вы узнали, как подсчитать количество токенов в запросе с помощью библиотеки `tiktoken` (<https://oreil.ly/uz050>). Эту библиотеку также можно использовать для разбиения строк на фрагменты соответствующего размера.

Не забудьте установить библиотеки `tiktoken` и `langchain-text-splitters` командой `pip install tiktoken langchain-text-splitters`.

Попробуем разбить текст по количеству токенов с помощью объекта `CharacterTextSplitter` и его функции `.from_tiktoken_encoder()`.

Сначала создадим экземпляр `CharacterTextSplitter` с размером фрагмента в 50 символов и без перекрытия. Затем вызовом метода `split_text` разобьем текст на части и выведем получившееся количество фрагментов.

После этого сделаем то же самое, но уже с *перекрытием фрагментов* в 48 символов. Это поможет нам увидеть, как меняется количество фрагментов при разбиении с перекрытием, и понять влияние этих настроек на разбиение текста:

```
from langchain_text_splitters import CharacterTextSplitter

text = """
Biology is a fascinating and diverse field of science that explores the
living world and its intricacies \n\n. It encompasses the study of life, its
origins, diversity, structure, function, and interactions at various levels
from molecules and cells to organisms and ecosystems \n\n. In this 1000-word
essay, we will delve into the core concepts of biology, its history, key
areas of study, and its significance in shaping our understanding of the
natural world. \n\n ...(truncated to save space)...
"""

Без перекрытия фрагментов:
text_splitter = CharacterTextSplitter.from_tiktoken_encoder(
 chunk_size=50, chunk_overlap=0, separator="\n",
)
texts = text_splitter.split_text(text)
print(f"Number of texts with no chunk overlap: {len(texts)}")

С перекрытием фрагментов:
text_splitter = CharacterTextSplitter.from_tiktoken_encoder(
 chunk_size=50, chunk_overlap=48, separator="\n",
)
texts = text_splitter.split_text(text)
print(f"Number of texts with chunk overlap: {len(texts)}")
```

Вывод:

```
Number of texts with no chunk overlap: 3
Number of texts with chunk overlap: 6
```

В предыдущем разделе мы загружали и разбивали файл `.pdf` на документы LangChain, используя такую инструкцию:

```
pages = loader.load_and_split()
```

При желании можно более точно управлять размером каждого документа, создав `TextSplitter` и добавив его в пайплайн загрузки документов:

```
def load_and_split(text_splitter: TextSplitter | None = None) -> List[Document]
```

Давайте просто создадим `TokenTextSplitter` с `chunk_size=500` и `chunk_overlap=50`:

```
from langchain.text_splitter import TokenTextSplitter
from langchain_community.document_loaders import PyPDFLoader

text_splitter = TokenTextSplitter(chunk_size=500, chunk_overlap=50)
loader = PyPDFLoader("data/principles_of_marketing_book.pdf")
pages = loader.load_and_split(text_splitter=text_splitter)

print(len(pages)) #737
```

Книга *Principles of Marketing* содержит 497 страниц, но после использования `TokenTextSplitter` с `chunk_size=500` токенов у нас получилось 776 объектов `Document` меньшего размера.

## Рекурсивное разбиение по символам

Анализ больших блоков текста может создать уникальные проблемы. В таких случаях можно рекомендовать *рекурсивное разбиение по символам*. Данный метод предполагает разделение больших фрагментов текста на меньшие части, более доступные для дальнейшего анализа.

Этот подход особенно эффективен при работе с общим текстом. Он основан на использовании списка символов в качестве параметров и последовательно разбивает текст по этим символам. Полученные разделы продолжают делить до тех пор, пока они не достигнут приемлемого размера. По умолчанию список символов включает `"\n\n"`, `"\n"`, `" "` и `"."`. Такой порядок выбран с целью сохранения целостности абзацев, предложений, слов и в конечном счете семантического контекста.

Процесс зависит от заданного списка символов и определяет размер получающихся фрагментов по заданному количеству символов.

Прежде чем углубиться в код, вы должны понять, что делает `RecursiveCharacterTextSplitter`. Он принимает текст и список разделителей (символов, определяющих границы фрагментов текста). Затем пытается разбить текст по первому разделителю в списке. Если полученные фрагменты слишком велики, то выбирается следующий разделитель из списка и процесс повторяется. Так продолжается *рекурсивно* до тех пор, пока фрагменты не станут достаточно маленькими или пока не будут исчерпаны все разделители.

Возьмем для примера предыдущую переменную `text`, импортируем `RecursiveCharacterTextSplitter` и попробуем использовать экземпляр этого класса для разбиения текста. При инициализации объекта передадим параметры `chunk_size=100`, `chunk_overlap=20` и `length_function=len`.

Функция, переданная в параметре `length_function`, используется для определения размера фрагментов. При желании в `length_function` вместо `len`, подсчитывающей символы, можно передать функцию подсчета токенов:

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
 chunk_size=100,
 chunk_overlap=20,
 length_function=len,
)
```

После создания экземпляра `text_splitter` вызовем его метод `.split_text`, чтобы разбить переменную `text` на более мелкие фрагменты. Эти фрагменты возвращаются в виде списка Python:

```
Разбить текст на фрагменты:
texts = text_splitter.split_text(text)
```

Помимо простого разбиения текста на список строк с перекрытием, можно создать объекты `Document`, воспользовавшись функцией `.create_documents`. Создание объектов `Document` позволяет:

- сохранить полученные документы в векторной базе данных для семантического поиска;
- добавить метаданные к фрагментам текста;
- выполнить обход нескольких документов, чтобы создать обобщающую сводку.

Чтобы добавить метаданные, нужно передать функции аргумент `metadatas`:

```
Создать документы из фрагментов:
metadatas = {"title": "Biology", "author": "John Doe"}
docs = text_splitter.create_documents(texts, metadatas=[metadatas] * len(texts))
```

Но что, если получившиеся объекты `Document` окажутся слишком длинными?

Это легко исправить с помощью функции `.split_documents` объекта `TextSplitter`. Она принимает список объектов `Document` и возвращает новый список объектов `Document`, созданных с учетом значений аргументов, указанных при создании экземпляра `TextSplitter`:

```
text_splitter = RecursiveCharacterTextSplitter(chunk_size=300)
splitted_docs = text_splitter.split_documents(docs)
```

Теперь вы знаете, как создать эффективный пайплайн загрузки данных, используя такие источники, как PDF-файлы, CSV-файлы и ссылки на облачное хранилище Google Cloud Storage. Вы также узнали, как дополнить полученные документы метаданными с содержательным контекстом, который может пригодиться для анализа и разработки запросов.

Имея на вооружении инструменты разбиения текста, вы можете управлять размерами документов, оптимизировать контекстное окно LLM и обеспечивать сохранность контекстной информации. Вы узнали, как обрабатывать большие фрагменты текста, используя рекурсивное разбиение по символам. Эти новые знания позволят вам использовать самые разные источники документов и интегрировать их в надежный пайплайн данных.

## Декомпозиция задач

*Декомпозиция задач* — это стратегический процесс разделения сложных задач на набор более простых подзадач. Этот подход органично согласуется с естественным стремлением разработчиков ПО делить задачи на взаимосвязанные подкомпоненты.

Используя декомпозицию задач в разработке ПО, можно снизить когнитивную нагрузку и использовать преимущества принципа изоляции проблем и единственной ответственности.

Интересно отметить, что LLM тоже могут выиграть от декомпозиции задач. Этот подход помогает максимизировать полезность и эффективность LLM, позволяя им решать сложные задачи, которые было бы сложно решить, как единое целое (рис. 4.3).

Вот несколько примеров использования декомпозиции при работе с LLM.

### *Комплексное решение задач*

Когда задача многогранна и не может быть решена с помощью одного запроса, ее декомпозиция может очень пригодиться. Например, решение сложного юридического дела можно разбить на изучение контекста дела, определение



соответствующих законов, выявление юридических прецедентов и выработку доказательств. Каждую подзадачу можно решить с помощью LLM независимо и в конце получить комплексное решение.

### Генерирование контента

Задачу создания длинного контента, такого как статья или запись в блоге, можно разложить на создание плана, написание отдельных разделов и последующую компиляцию и доработку окончательного варианта. GPT-4 способна справиться с каждым шагом отдельно и помочь достигнуть хорошего результата.

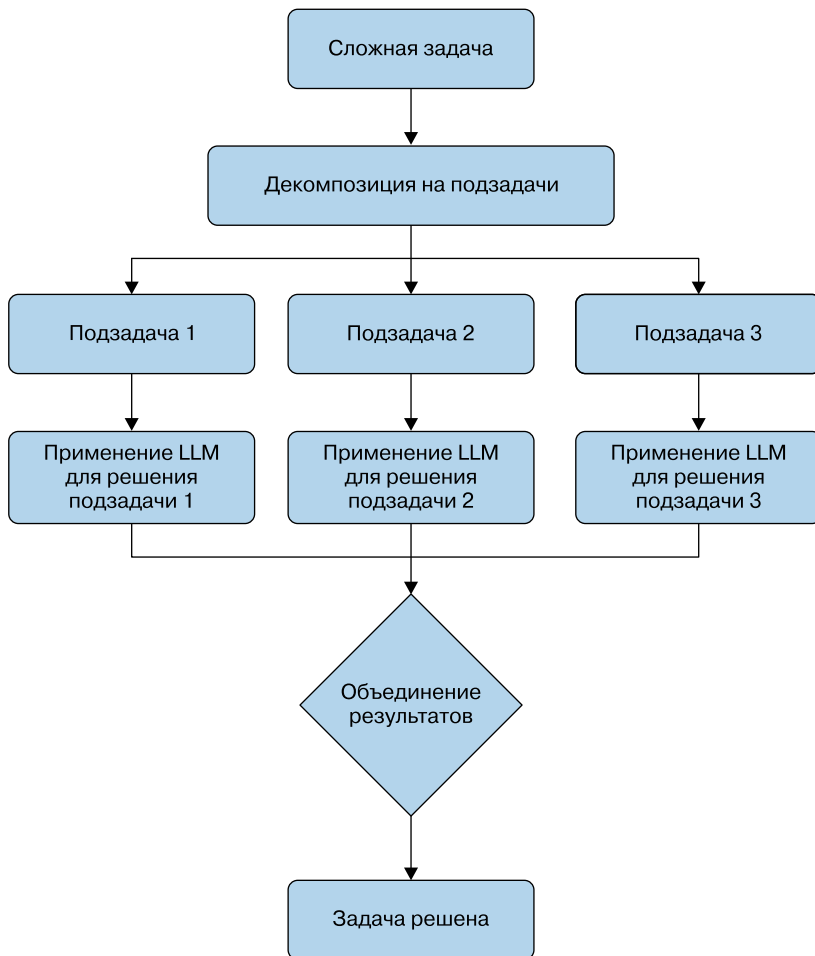


Рис. 4.3. Декомпозиция задачи при работе с LLM

### *Резюме большого документа*

Обобщение длинных документов, таких как исследовательские работы или отчеты, можно осуществить более эффективно, разделив задачу на несколько более мелких задач, таких как изучение отдельных разделов, их обобщение независимо друг от друга и последующее составление окончательного резюме.

### *Интерактивные диалоговые агенты*

При создании продвинутых чат-ботов декомпозиция задач может упростить управление различными аспектами диалога, такими как понимание ввода пользователя, поддержание контекста, генерирование релевантных ответов и управление потоком диалога.

### *Системы обучения и репетиторства*

В цифровых системах обучения декомпозиция задачи преподавания на понимание текущего уровня знаний учащегося, выявление пробелов, выбор учебных материалов и оценку прогресса может сделать систему более эффективной. Для каждой подзадачи можно использовать генеративные способности GPT-4.



#### **Разделите задачу на подзадачи**

Декомпозиция задач — важнейшая стратегия, позволяющая полностью раскрыть потенциал LLM. Разбивая сложные задачи на более простые подзадачи, можно эффективнее использовать возможности моделей ИИ для решения задач.

В последующих разделах вы узнаете, как создать и интегрировать несколько цепочек LLM для организации сложных рабочих процессов.

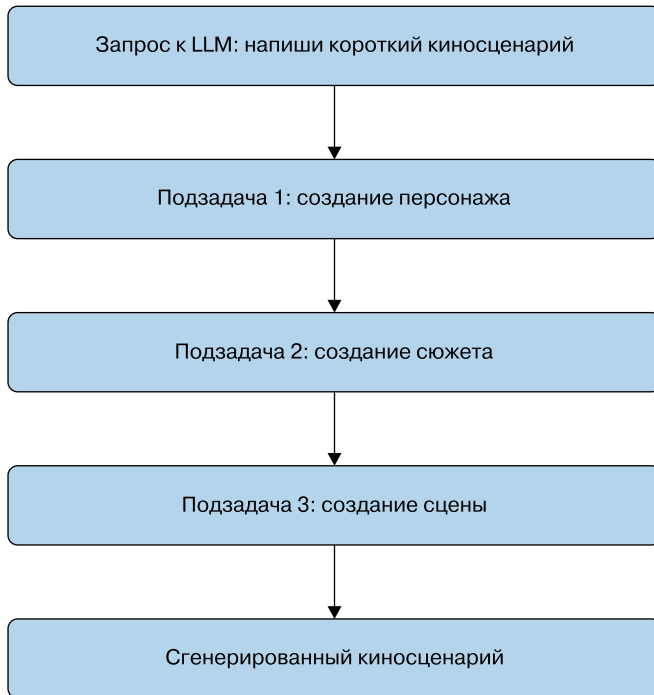
## **Объединение запросов в цепочки**

Нередко бывает невозможно решить задачу в рамках одного запроса. В таких случаях можно использовать *комбинацию запросов*, включающую последовательность запросов и ответов, которая помогает модели LLM понять идею.

Представим, что некая кинокомпания решила частично автоматизировать создание своих фильмов. Эту задачу можно разбить на несколько ключевых компонентов, таких как:

- создание персонажа;
- создание сюжета;
- построение сцены/мира.

На рис. 4.4 показано, как может выглядеть рабочий процесс.



**Рис. 4.4.** Процесс последовательного создания киносценария

## Последовательная цепочка

Разложим задачу на *несколько цепочек* и затем вновь объединим их в одну цепочку.

`character_generation_chain`

Цепочка, ответственная за создание нескольких персонажей в заданном жанре.

`plot_generation_chain`

Цепочка, ответственная за создание сюжета с учетом персонажей и жанра.

`scene_generation_chain`

Эта цепочка сгенерирует все недостающие сцены, которые изначально не были созданы цепочкой `plot_generation_chain`.

Начнем с создания трех отдельных переменных `ChatPromptTemplate`, по одной для каждой цепочки:

```
from langchain_core.prompts.chat import ChatPromptTemplate

character_generation_prompt = ChatPromptTemplate.from_template(
 """Придумай от трех до пяти персонажей для короткого киносценария
 в жанре {genre}. Каждый персонаж должен иметь имя и биографию.
 Придумай имя и биографию для каждого персонажа, это очень важно!

 Пример ответа:
 Имя: Маг, биография: волшебник, мастер магии.
 Имя: Воин, биография: воин, мастер меча.

 Персонажи: """
)

plot_generation_prompt = ChatPromptTemplate.from_template(
 """На основе имеющихся персонажей и с учетом жанра придумай сюжет
 для короткого киносценария:
 Персонажи:
 {characters}

 Жанр: {genre}

 Сюжет: """
)

scene_generation_prompt = ChatPromptTemplate.from_template(
 """Представь, что ты – опытный постановщик.
 С учетом придуманных персонажей и сюжета придумай
 разные сцены для каждого акта.

 Разложи сюжет на несколько сцен:

 Персонажи:
 {characters}

 Жанр: {genre}

 Сюжет: {plot}

 Пример ответа:
 Сцены:
 Сцена 1: описание сцены.
 Сцена 2: описание сцены.
 Сцена 3: описание сцены.

 Сцены:
 """
)
```

Обратите внимание, что, переходя от персонажей к сюжету и созданию сцен, мы добавляем в запросы все больше переменных-заполнителей с информацией, полученной на предыдущих шагах.

Остается вопрос: как сохранить эту дополнительную информацию в переменных для извлечения в последующих шаблонах `ChatPromptTemplate`?

## itemgetter и извлечение ключей словаря

В цепочках LCEL можно использовать функцию `itemgetter` из пакета `operator` и с ее помощью извлекать ключи из словаря, созданного на предыдущем шаге:

```
from operator import itemgetter
from langchain_core.runnables import RunnablePassthrough

chain = RunnablePassthrough() | {
 "genre": itemgetter("genre"),
}

chain.invoke({"genre": "фэнтези"})
{'genre': 'фэнтези'}
```

Функция `RunnablePassThrough` просто передает свои входные данные непосредственно в следующий шаг. Затем создается новый словарь с использованием того же ключа в функции `invoke`; этот ключ извлекается с помощью `itemgetter("genre")`.

Очень важно использовать функцию `itemgetter` на каждом шаге в цепочке LCEL, чтобы любые последующие переменные-заполнители `ChatPromptTemplate` всегда получали допустимые значения.

Кроме того, для управления предыдущими значениями словаря в цепочке LCEL можно использовать лямбда-функции и функции `RunnableLambda`. В Python лямбда-функция — это анонимная функция:

```
from langchain_core.runnables import RunnableLambda

chain = RunnablePassthrough() | {
 "genre": itemgetter("genre"),
 "upper_case_genre": lambda x: x["genre"].upper(),
 "lower_case_genre": RunnableLambda(lambda x: x["genre"].lower()),
}

chain.invoke({"genre": "фэнтези"})
{'genre': 'фэнтези', 'upper_case_genre': 'ФЭНТЕЗИ',
'lower_case_genre': 'фэнтези'}
```

Теперь, познакомившись с функциями `RunnablePassthrough`, `itemgetter` и лямбда-функциями, представим последний элемент: `RunnableParallel`:

```
from langchain_core.runnables import RunnableParallel

master_chain = RunnablePassthrough() | {
 "genre": itemgetter("genre"),
 "upper_case_genre": lambda x: x["genre"].upper(),
 "lower_case_genre": RunnableLambda(lambda x: x["genre"].lower()),
}

master_chain_two = RunnablePassthrough() | RunnableParallel(
 genre=itemgetter("genre"),
 upper_case_genre=lambda x: x["genre"].upper(),
 lower_case_genre=RunnableLambda(lambda x: x["genre"].lower()),
)

story_result = master_chain.invoke({"genre": "Фэнтези"})
print(story_result)

story_result = master_chain_two.invoke({"genre": "Фэнтези"})
print(story_result)

master chain: {'genre': 'Фэнтези', 'upper_case_genre': 'ФЭНТЕЗИ',
'lower_case_genre': 'фэнтези'}
master chain two: {'genre': 'Фэнтези', 'upper_case_genre': 'ФЭНТЕЗИ',
'lower_case_genre': 'фэнтези'}
```

Этот код импортирует `RunnableParallel` и создает две цепочки LCEL: `master_chain` и `master_chain_two`. Затем вызывает их с одинаковыми аргументами. После этого `RunnablePassthrough` передает словарь во вторую часть цепочки.

Вторая часть `master_chain` и `master_chain_two` вернет *один тот же результат*.

Поэтому вместо прямого использования словаря можно вызвать функцию `RunnableParallel`. Эти две цепочки *взаимозаменяемы*, поэтому выбирайте тот синтаксис, который покажется вам более удобным.

Теперь создадим три цепочки LCEL, используя шаблоны запросов:

```
from langchain_openai.chat_models import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

Создать модель чата:
model = ChatOpenAI()

Создать отдельные цепочки:
character_generation_chain = (character_generation_prompt
| model
| StrOutputParser())
```

```

plot_generation_chain = (plot_generation_prompt
| model
| StrOutputParser())

scene_generation_plot_chain = (scene_generation_plot_prompt
| model
| StrOutputParser())

```

После создания отдельных цепочек можете присоединить их к основной цепочке LCEL.

Ввод:

```

from langchain_core.runnables import RunnableParallel
from operator import itemgetter
from langchain_core.runnables import RunnablePassthrough

master_chain = (
 {"characters": character_generation_chain,
 "genre": RunnablePassthrough()}
 | RunnableParallel(
 characters=itemgetter("characters"),
 genre=itemgetter("genre"),
 plot=plot_generation_chain,
)
 | RunnableParallel(
 characters=itemgetter("characters"),
 genre=itemgetter("genre"),
 plot=itemgetter("plot"),
 scenes=scene_generation_plot_chain,
)
)

story_result = master_chain.invoke({"genre": "Фэнтези"})

```

Многоточия ... в следующем выводе сообщают, что в данном месте текст был обрезан. Это сделано для экономии места. Однако отметим, что всего было создано пять персонажей и девять сцен.

Вывод:

```

{'Персонажи': "'Имя: Лира, биография: Лира – молодая эльфийка, обладающая ...\n\nИмя: Орион, биография: Орион - ...'", 'жанр: {'genre': 'Фэнтези'} 'сюжет': "'На заколдованные леса волшебного царства надвигалась великая тьма, угрожавшая поглотить землю и ее жителей. Лира, молодая эльфийка, тесно связанная с природой, ...'", 'сцены': "'Сцена 1: Лира чувствует опасность, надвигающуюся на лес...\n\nСцена 2: Орион выступает на расследование беспорядков в лесу...\n\nСцена 9: После битвы Лира, Орион, Серафина, Финнеган ...'"}

```

Разобьем сцены на отдельные элементы в списке Python. Затем создадим два новых запроса для добавления сценариев действий персонажей и обобщенного описания:

```
Извлечь сцены с помощью .split('\n') и удалить пустые строки:
scenes = [scene for scene in story_result["сцены"].split("\n") if scene]
generated_scenes = []
previous_scene_summary = ""

character_script_prompt = ChatPromptTemplate.from_template(
 template="""На основе имеющихся персонажей: {characters} и с учетом жанра:
 {genre} придумай сценарий действий персонажа для сцены.

 Следуй принципам, перечисленным далее:
 - Используй обобщенное описание предыдущей сцены: {previous_scene_summary},
 чтобы избежать повторов.
 - Используй сюжет: {plot}, чтобы создать сценарий.
 - В данный момент ты придумываешь диалог с персонажем для сцены: {scene}

 Пример ответа:
 СЦЕНА 1: КВАРТИРА АННЫ

 (АННА перебирает старые книги, когда раздается стук в дверь.
 Она открывает дверь и видит ДЖОНА.)
 АННА: Могу ли я вам чем-то помочь, сэр?
 ДЖОН: Нет, это скорее я могу вам помочь. Я слышал, что вы
 занимаетесь исследованием путешествий во времени.
 (Анна выглядит заинтригованной, но все же осторожничает.)
 АННА: Да, верно, но как вы узнали?
 ДЖОН: Можно сказать, что... это я подтолкнул вас к этим исследованиям.

 СЦЕНА {index}:

 """,
)

summarize_prompt = ChatPromptTemplate.from_template(
 template="""На основе сценария действий персонажа дай
 обобщенное описание сцены.
 Сценарий действий персонажа: {character_script}""",
)
```

Технически сцены можно генерировать асинхронно. Однако в каждой следующей сцене полезно знать, что делали персонажи *в предыдущих сценах, чтобы избежать повтора каких-то моментов*.



Соответственно, можно создать две цепочки LCEL: одну для создания сценариев действий персонажей для каждой сцены, а другую для обобщения предыдущих сцен:

```
Загрузить модель чата:
model = ChatOpenAI(model='gpt-3.5-turbo-16k')

Создать цепочки LCEL:
character_script_generation_chain = (
 {
 "characters": RunnablePassthrough(),
 "genre": RunnablePassthrough(),
 "previous_scene_summary": RunnablePassthrough(),
 "plot": RunnablePassthrough(),
 "scene": RunnablePassthrough(),
 "index": RunnablePassthrough(),
 }
 | character_script_prompt
 | model
 | StrOutputParser()
)

summarize_chain = summarize_prompt | model | StrOutputParser()

Здесь можно использовать tqdm для отслеживания прогресса,
если сцен много:
for index, scene in enumerate(scenes[0:3]):

 # Создать сцены:
 scene_result = character_script_generation_chain.invoke(
 {
 "characters": story_result["characters"],
 "genre": "Фэнтези",
 "previous_scene_summary": previous_scene_summary,
 "index": index,
 }
)

 # Сохранить сгенерированные сцены:
 generated_scenes.append(
 {"character_script": scene_result, "scene": scenes[index]}
)

 # Если это первая сцена, то для нее
 # нет обобщенного описания предыдущей сцены:
 if index == 0:
 previous_scene_summary = scene_result
```

```

else:
 # Если это вторая сцена, третья... и т. д., то можно
 # сгенерировать и использовать обобщенное описание предыдущей сцены:
 summary_result = summarize_chain.invoke(
 {"character_script": scene_result}
)
 previous_scene_summary = summary_result

```

Здесь мы сначала создаем цепочку `character_script_generation_chain`, используя различные выполняемые объекты, такие как `RunnablePassthrough`. Важно отметить, что эта цепочка интегрирует мощную модель `model = ChatOpenAI(model='gpt-3.5-turbo-16k')` с большим контекстным окном размером 16K, идеально подходящую для задач по созданию контента. При вызове эта цепочка генерирует сценарии действий персонажей, опираясь на такие входные данные, как описание персонажей, жанр и описание сцены.

В каждую сцену динамически добавляется краткое описание предыдущей сцены, благодаря чему создается простая, но эффективная буферная память. Этот метод обеспечивает непрерывность и контекст повествования, а также улучшает способность LLM генерировать связные сценарии действий персонажей.

В коде выше можно видеть, как `StrOutputParser` элегантно преобразует выходные данные модели в структурированные строки, упрощая использование сгенерированного содержимого.



### Разделите задачу на подзадачи

Помните, что планирование задач в виде последовательности мелких цепочек может повысить общее качество вывода. Каждая цепочка в общей большой последовательности вносит свой вклад в достижение общей цели задачи.

Использование цепочек дает возможность использовать разные модели. Например, большой и дорогой модели — для формирования идей и меньшей и дешевой модели — для генерации контента. Такое разделение труда обычно дает очень неплохие результаты. Это также означает, что вы можете точно настраивать модели на каждом этапе.

## Структурирование цепочек LCEL

Создавая цепочки LCEL, вы должны гарантировать, что первая часть цепочки является *выполняемым* типом. Например, следующий код выдаст ошибку:

```

from langchain_core.prompts.chat import ChatPromptTemplate
from operator import itemgetter
from langchain_core.runnables import RunnablePassthrough, RunnableLambda

bad_first_input = {
 "film_required_age": 18,
}

```

```
prompt = ChatPromptTemplate.from_template(
 "Придумай название фильма для зрителей старше {film_required_age}"
)
```

```
Эта инструкция сгенерирует ошибку:
bad_chain = bad_first_input | prompt
```

Словарь Python со значением 18 не является выполняемым типом, который можно поместить в цепочку LCEL. Однако все следующие реализации будут работать:

```
Все следующие цепочки реализуют интерфейс выполняемых объектов:
first_good_input = {"film_required_age": itemgetter("film_required_age")}

Создать словарь внутри RunnableLambda:
second_good_input = RunnableLambda(lambda x: { "film_required_age":
x["film_required_age"] })

third_good_input = RunnablePassthrough()
fourth_good_input = {"film_required_age": RunnablePassthrough()}
Также можно создать цепочку, начинающуюся с объекта RunnableParallel(...)

first_good_chain = first_good_input | prompt
second_good_chain = second_good_input | prompt
third_good_chain = third_good_input | prompt
fourth_good_chain = fourth_good_input | prompt

first_good_chain.invoke({
 "film_required_age": 18
}) # ...
```

Последовательности цепочек отлично подходят для постепенного генерирования знаний, которые будут использоваться будущими цепочками, но их последовательная природа увеличивает время отклика. Поэтому пайплайны `SequentialChain` лучше всего подходят для случаев, когда задачи решаются на стороне сервера, где скорость получения ответа не является приоритетом и пользователи не ждут мгновенной обратной связи.

## Цепочки документов

Представим, что, прежде чем принять ваш сценарий, кинокомпания попросила представить краткое изложение, основанное на всех сценариях действий персонажей. Это хороший повод использовать *цепочки документов*, поскольку вы должны передать модели LLM большой объем текста, не помещающийся в один запрос из-за ограничений длины контекста.

Начнем с того, что рассмотрим картину в общем. Код, который показан далее, решает задачу обобщения описаний множества сцен.

Не забудьте установить Pandas командой `pip install pandas`.

Теперь рассмотрим первый фрагмент кода:

```
from langchain_text_splitters import CharacterTextSplitter
from langchain.chains.summarize import load_summarize_chain
import pandas as pd
```

Эти строки импортируют все необходимые инструменты. `CharacterTextSplitter` и `load_summarize_chain` из пакета `LangChain` помогут в обработке текста, а библиотека `Pandas` (импортируется как `pd`) поможет манипулировать вашими данными.

Далее начинается работа с данными:

```
df = pd.DataFrame(generated_scenes)
```

Здесь из переменной `generated_scenes` создается набор данных `DataFrame`. При этом исходные сцены преобразуются в табличные данные, которыми может манипулировать `Pandas`.

Теперь нужно объединить текст:

```
all_character_script_text = "\n".join(df.character_script.tolist())
```

Эта инструкция объединяет содержимое столбца `character_script` из всех записей в наборе данных в одну текстовую строку. Для этого все текстовые значения в столбце объединяются в список, а потом все элементы списка сводятся в одну строку через символ перевода строки между ними. В результате образуется одна строка, содержащая все сценарии действий персонажей.

После подготовки наступает черед процесса обобщения:

```
text_splitter = CharacterTextSplitter.from_tiktoken_encoder(
 chunk_size=1500, chunk_overlap=200
)
docs = text_splitter.create_documents([all_character_script_text])
```

Здесь мы создали экземпляр `CharacterTextSplitter`, используя его метод класса `from_tiktoken_encoder` с конкретными значениями размера фрагмента и перекрытия. Затем применили этот экземпляр для разбиения объединенного текста сценария на фрагменты, подходящие для обобщения.

Затем настраивается инструмент обобщения:

```
chain = load_summarize_chain(llm=model, chain_type="map_reduce")
```

Эта инструкция настраивает процесс обобщения. Здесь вызывается функция, которая загружает цепочку обобщения с моделью чата в стиле `map-reduce`.

После этого запускается обобщение:

```
summary = chain.invoke(docs)
```

Эта инструкция выполняет фактическое обобщение текста. Метод `invoke` производит обобщение фрагментов текста, подготовленных ранее, и сохраняет результат в переменную.

В заключение полученный результат выводится в консоль для просмотра:

```
print(summary['output_text'])
```

Это кульминация проделанной работы.

Следующий сценарий берет набор сцен, объединяет текст, разбивает его на части, обобщает и выводит резюме:

```
from langchain.text_splitter import CharacterTextSplitter
from langchain.chains.summarize import load_summarize_chain
import pandas as pd

df = pd.DataFrame(generated_scenes)

all_character_script_text = "\n".join(df.character_script.tolist())

text_splitter = CharacterTextSplitter.from_tiktoken_encoder(
 chunk_size=1500, chunk_overlap=200
)

docs = text_splitter.create_documents([all_character_script_text])

chain = load_summarize_chain(llm=model, chain_type="map_reduce")
summary = chain.invoke(docs)
print(summary['output_text'])
```

Вывод:

Авора и Магнус договорились найти спрятанный артефакт и отправились в древнюю библиотеку, чтобы отыскать книгу, которая приведет их к реликвии...

Стоит отметить, что, помимо `map_reduce`, существуют еще четыре стратегии обработки цепочек объектов `Document`.

## Наполнение

Цепочка вставки документов, также называемая *цепочкой наполнения*, — это самый простой подход среди стратегий объединения документов. Диаграмма на рис. 4.5 иллюстрирует процесс интеграции нескольких документов в один запрос LLM.

### Наполнение документами

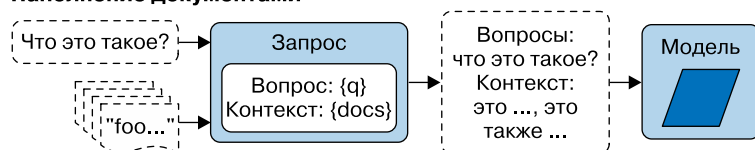


Рис. 4.5. Цепочка наполнения запроса документами

## Уточнение

Цепочка уточнения документов (рис. 4.6) формирует ответ модели LLM с использованием циклического процесса, который *итеративно обновляет ее вывод*. В каждом цикле он объединяет текущий вывод (полученный от LLM) с текущим документом. Затем выполняется еще один запрос к LLM для *уточнения текущего вывода*. Этот процесс продолжается до тех пор, пока все документы не будут обработаны.

### Цепочка уточнения документов

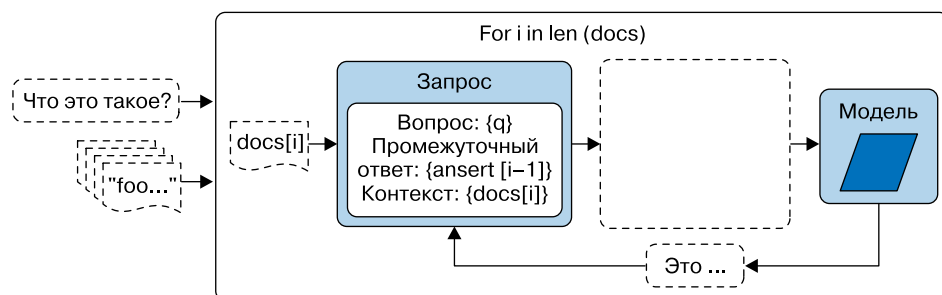


Рис. 4.6. Цепочка уточнения документов

## Отображение и свертка (Map Reduce)

Цепочка отображения и свертки (Map Reduce) документов (рис. 4.7) начинается с передачи модели LLM каждого отдельного документа (этот этап называют шагом отображения (Map)) и интерпретации полученного результата как вновь созданного документа.

Впоследствии все эти вновь созданные документы вводятся в отдельную цепочку объединения документов для формирования общего результата (этот этап называют шагом свертки (Reduce)). Чтобы новые документы вписывались в длину контекста, к отображаемым документам применится процесс сжатия. При необходимости сжатие выполняется рекурсивно.

### Цепочка отображения и свертки документов

Отображение

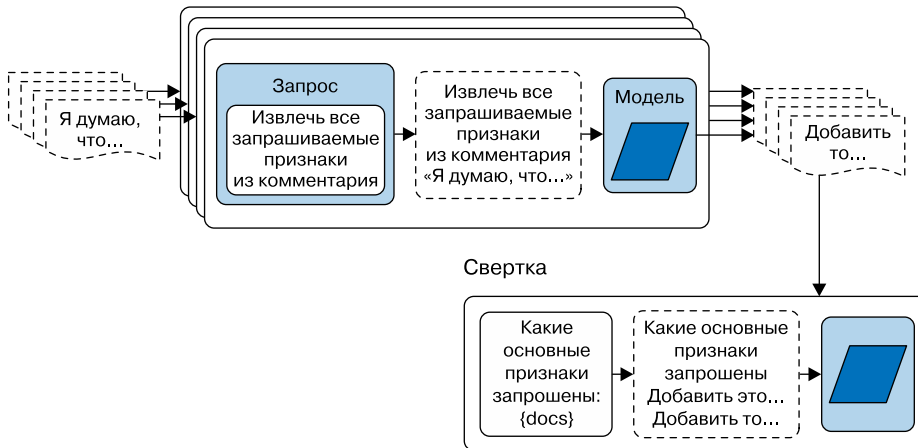


Рис. 4.7. Цепочка уточнения с использованием документов

### Отображение с повторным ранжированием (Map Re-rank)

Существует также подход отображения с повторным ранжированием (Map Re-rank), которое осуществляется путем выполнения начального запроса для каждого документа. Он не только выполняет поставленную задачу, но и оценивает достоверность ответа. По завершении выбирается и возвращается ответ с наивысшей оценкой достоверности.

В табл. 4.1 перечислены достоинства и недостатки каждой конкретной стратегии обработки цепочки документов.

Узнать больше о приемах реализации различных цепочек обработки документов можно в исчерпывающем описании LangChain API ([https://oreil.ly/FQUK\\_](https://oreil.ly/FQUK_)) и здесь ([https://oreil.ly/9xr\\_6](https://oreil.ly/9xr_6)).

Кроме того, чтобы использовать другую цепочку, достаточно изменить ее тип в вызове функции `load_summarize_chain`, например:

```
chain = load_summarize_chain(llm=model, chain_type='refine')
```

Таблица 4.1. Обзор стратегий обработки цепочек документов

Стратегия	Достоинства	Недостатки
Цепочка наполнения документами	Простота реализации. Идеально подходит для сценариев с небольшим количеством коротких документов	Плохо подходит для обработки большого количества или слишком длинных документов из-за ограничения размера запроса
Цепочка уточнения документов	Позволяет итеративно уточнять ответ. Более полный контроль над каждым шагом формирования ответа. Хорошо подходит для задач постепенного извлечения	Может оказаться не лучшим выбором для приложений реального времени из-за цикличности процесса
Цепочка отображения и свертки документов	Обеспечивает независимую обработку каждого документа. Может обрабатывать большие наборы данных путем их свертки в фрагменты меньшего размера	Требует тщательного управления процессом. Этап свертки может привести к усложнению процесса и потере порядка документов
Цепочка отображения с повторным ранжированием документов	Оценивает достоверность каждого ответа, что позволяет отобрать лучший ответ	Алгоритм ранжирования может быть сложным в реализации и управлении. Может не всегда давать лучший ответ, если механизм оценки ненадежен и не отлажен

Существуют более новые и гибкие подходы к созданию цепочек обобщения с использованием LCEL, но в большинстве случаев вполне достаточно возможностей `load_summarize_chain`.

## Резюме

В этой главе мы рассмотрели фреймворк LangChain и его основные компоненты. Вы узнали о важности загрузчиков документов для сбора данных и о том, какую роль играют инструменты разбиения текста в обработке больших текстовых блоков.

Вы познакомились с идеями декомпозиции задач и цепочек запросов, увидели, насколько мощным может быть подход, основанный на делении сложных задач на более простые подзадачи. Вы также узнали, как, используя подход с цепочкой запросов, можно объединять запросы и ответы для генерации новых идей.

В следующей главе мы познакомим вас с векторными базами данных и расскажем, как их интегрировать с документами из LangChain. Эта возможность играет ключевую роль в повышении точности извлечения знаний из данных.



# Векторные базы данных с использованием FAISS и Pinecone

В этой главе мы рассмотрим идею эмбедингов и векторных баз данных, а также поговорим об их использовании для передачи соответствующего контекста в запросах.

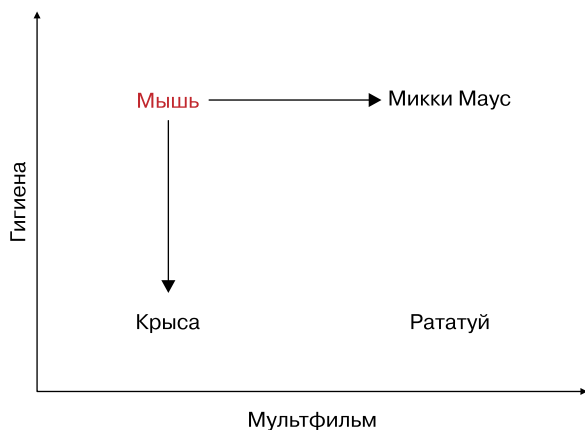
*Векторная база данных* — это инструмент, который используется для хранения текстовых данных в таком виде, который позволяет выполнять запросы на основе сходства или смыслового значения. Эта технология уменьшает галлюцинации (когда модель ИИ что-то выдумывает), ссылаясь на данные, которые модель не видела при обучении, и значительно повышает точность и качество ответа LLM. Варианты использования векторных баз данных также включают чтение документов, рекомендацию похожих товаров и запоминание прошлых диалогов.

*Векторы* — это списки чисел, представляющие текст (или изображения), которые можно рассматривать как координаты местоположения. Вектор для слова «мышь» с использованием модели text-embedding-ada-002 OpenAI — это список из 1536 чисел, каждое из которых представляет собой значение признака, изучаемого моделью в ходе обучения:

```
[-0.011904156766831875,
 -0.0323905423283577,
 0.001950666424818337,
 ...]
```

Когда эти модели обучаются, тексты, появляющиеся вместе в обучающих данных, будут сдвинуты ближе друг к другу по значениям, а несвязанные тексты — отодвинуты подальше. Представьте, что мы обучили простую модель, определив всего два признака: «Мультфильм» и «Гигиена». И эта модель должна описывать весь мир, но только с точки зрения этих двух признаков. Начав со слова «мышь» и увеличив значение признака «Мультфильм», мы приближаемся к самому известному мультяшному мышонку — Микки Маусу (рис. 5.1). Уменьшение значения параметра «Гигиена» приведет нас к крысам, поскольку

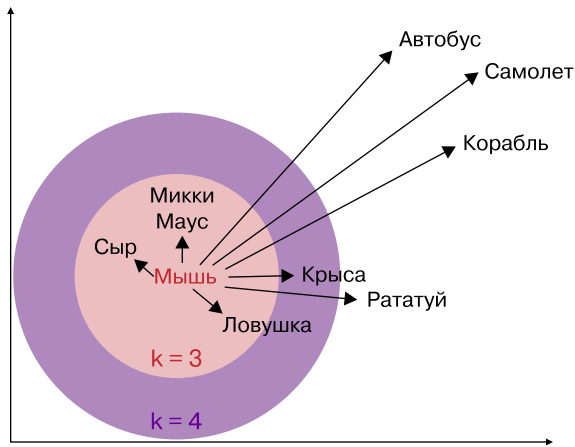
крысы — это грызуны, похожие на мышей, но нередко являются разносчиками болезней (то есть являются негигиеничными).



**Рис. 5.1.** Двухмерный вектор расстояний

Каждую точку на графике можно найти по двум числам на осях  $X$  и  $Y$ , которые обозначают признаки модели «Мультфильм» и «Гигиена». В действительности векторы могут включать тысячи признаков, и чем больше признаков, тем более широкий диапазон сходств и различий сможет охватить модель. «Гигиена» — не единственный признак, отличающий мышей и крыс, а Микки Маус — не просто мультяшный мышонок. Признаки извлекаются из данных так, что людям становится трудно их интерпретировать, и нужен график с тысячами осей для отображения местоположения в скрытом пространстве (так называют абстрактное многомерное пространство, образованное признаками, изучаемыми моделью). Зачастую признаки не имеют понятной человеку интерпретации. Однако мы можем создать упрощенную двумерную проекцию расстояний между векторами, как это было сделано на рис. 5.2.

Чтобы выполнить векторный поиск, сначала нужно получить вектор (или местоположение) того, что необходимо найти, и отыскать  $k$  ближайших записей в базе данных. В нашем примере слово «мышь» оказывается ближе всего к словам «Микки Маус», «сыр» и «ловушка» при поиске  $k=3$  ближайших записей. Слово «крыса» не попало в их число при  $k=3$ , но попадет, если  $k=4$ , потому что оно является следующим ближайшим вектором. Слово «самолет» в этом примере находится очень далеко, потому что оно редко ассоциируется со словом «мышь» в обучающих данных. Слово «корабль» встречается рядом с другими видами транспорта, но ближе к словам «мышь» и «крыса», потому что в обучающих данных они часто упоминаются рядом со словом «корабль».



**Рис. 5.2.** Многомерный вектор расстояний

Векторная база данных хранит текстовые записи с векторным представлением в качестве ключа. Этим они отличаются от других типов баз данных, позволяющих искать записи по идентификатору, отношениям или наличию строки в тексте. Например, в реляционной базе данных, поддерживающей поиск по тексту (см. рис. 5.2), поиск записей, в которых текст содержит слово «мышь», вернет запись «Микки Маус» и ничего больше, потому что никакая другая запись не содержит этого слова. Поиск по векторам может также вернуть записи «сыр» и «ловушка», потому что они близки к искомому слову, даже если они не соответствуют запросу.

Возможность запрашивать данные, основываясь на сходстве, чрезвычайно полезна, и векторный поиск обеспечивает множество функций ИИ. Например:

#### *Чтение документов*

Поиск подходящих разделов текста, чтобы получить более точный ответ.

#### *Рекомендательные системы*

Поиск похожих товаров или предметов, чтобы предложить их пользователю.

#### *Долгосрочная память*

Просмотр чат-ботом соответствующих фрагментов истории диалогов, чтобы вспомнить прошлые взаимодействия.

Модели ИИ способны решать эти задачи в небольших масштабах, если ваши документы, список товаров или память разговоров укладываются в ограничения окна контекста используемой вами модели. Однако при

масштабировании вы довольно быстро столкнетесь с ограничениями на количество токенов и чрезмерными затратами на передачу слишком большого количества токенов в каждом запросе. Предварительная версия модели `gpt-4-1106-preview`, выпущенная компанией OpenAI в ноябре 2023 года (<https://oreil.ly/KMNU8>), имела огромное контекстное окно в 128 000 токенов, но обработка одного токена в ней стоит в десять раз дороже, чем в выпущенной годом ранее модели `gpt-3.5-turbo`, контекстное окно которой в восемь раз меньше (16 000 токенов). Более эффективный подход — выбирать только наиболее релевантные записи для передачи в запрос во время выполнения, чтобы обеспечить более точный контекст для формирования ответа. Эту практику обычно называют RAG.

## Генерация с дополненной выборкой

Векторные базы данных являются ключевым компонентом генерации с дополненной выборкой (Retrieval Augmented Generation, RAG), которая обычно включает поиск по сходству с запросом, извлечение наиболее релевантных документов и вставку их в запрос в качестве контекста. Это позволяет оставаться в рамках текущего контекстного окна, избегая при этом затрат денег на избыточные токены в нерелевантных текстовых документах, добавляемых в контекст.

Выборка также может осуществляться традиционным способом, поиском в базах данных или просмотром веб-страниц. И надо отметить, что во многих случаях векторный поиск по семантическому сходству не требуется. RAG обычно используется для устранения галлюцинаций в таких сценариях, как, например, диалоги пользователя с чат-ботом, склонным выдумывать, когда его спрашивают о чем-то, что не встречалось ему в обучающих данных. Векторный поиск может вставлять в запрос документы, семантически близкие к запросу пользователя, что значительно снижает вероятность галлюцинаций у чат-бота.

Например, если пользователь Майк скажет чат-боту: «Меня зовут Майк», — и через три сообщения задаст вопрос: «Как меня зовут?», — то тот сможет вспомнить правильный ответ. Сообщение, содержащее имя Майка, все еще находится в контекстном окне чата. Однако если тот же вопрос задать спустя 3000 сообщений, то текст с именем может оказаться за пределами контекстного окна. Без этого важного контекста чат-бот может выдумать имя или отказаться отвечать из-за отсутствия информации. Помочь в таких ситуациях мог бы поиск по ключевым словам, но он также может вернуть слишком много нерелевантных документов или не найти правильный контекст, в котором искомая информация была получена в прошлом. Майк мог много раз упоминать слово «имя» в разных

форматах и по разным причинам. Получив вопрос, векторная база данных может вернуть три наиболее подходящих сообщения из чата, соответствующих вопросу пользователя:

#### ## Контекст

Наиболее релевантные предыдущие сообщения пользователя:

1. "Меня зовут Майк".
2. "Мую собаку зовут Геркулес".
3. "Моего коллегу зовут Джеймс".

#### ## Инструкции

Ответь на вопрос пользователя, используя предыдущий контекст.

Вопрос пользователя: Как меня зовут?

Ответ ИИ:

Большинству моделей невозможно передать все 3000 предыдущих сообщений в запросе, а для традиционного поиска модели ИИ придется сформулировать правильный поисковый запрос, на что мы не можем полагаться. Используя шаблон RAG, мы передали текущее сообщение пользователя функции векторного поиска, получили три наиболее релевантные записи и передали их чат-боту в качестве контекста, который тот сможет использовать для правильного ответа.



#### Задайте направление

Векторный поиск, в отличие от вставки в запрос статического текста, позволяет динамически добавлять в запросы наиболее релевантные подсказки.

Вот как этот процесс можно организовать в промышленных приложениях, использующих RAG.

1. Разбить текстовые документы на фрагменты.
2. Индексировать фрагменты в векторной базе данных.
3. Выполнить векторный поиск похожих записей.
4. Добавить найденные записи в промт в качестве контекста.

В нашем случае документами могут быть все 3000 предыдущих сообщений пользователя из истории диалога с чат-ботом, разделы PDF-документа, который вы загрузили специально для чат-бота, или список всех релевантных товаров, имеющихся у вас в продаже, чтобы чат-бот мог дать рекомендацию. Способность векторного поиска находить похожие тексты полностью зависит от модели ИИ, используемой для генерации векторов, которые называются *эмбедингами*, при работе с семантической или контекстной информацией.

## Введение в эмбединги

Под *эмбедингом* обычно понимается числовой вектор, полученный предварительно обученной моделью ИИ на основе заданного текста. На момент написания этой книги стандартной моделью для генерации эмбедингов была модель `text-embedding-ada-002` компании OpenAI, но, вообще говоря, модели для создания эмбедингов, или векторных представлений, были доступны задолго до появления генеративного ИИ.

Иногда весьма полезно визуализировать векторные пространства в виде двумерной диаграммы, как на рис. 5.2, но проблема в том, что эмбединги, возвращаемые моделью `text-embedding-ada-002`, имеют 1536 измерений, и потому их практически невозможно изобразить графически. Наличие большого количества измерений позволяет модели улавливать более глубокие смысловые оттенки и отношения. Например, в двухмерном пространстве признаков можно отделить кошек от собак, в 300-мерном пространстве можно собирать информацию о различиях между породами, размерами, цветами и другими деталями. Следующий код показывает, как получить эмбединги из OpenAI API. Код всех последующих примеров можно найти в репозитории книги на GitHub (<https://oreil.ly/6RzTy>).

Ввод:

```
from openai import OpenAI
client = OpenAI()

Эта функция получает векторное представление для заданного текста
def get_vector_embeddings(text):
 response = client.embeddings.create(
 input=text,
 model="text-embedding-ada-002"
)
 embeddings = [r.embedding for r in response.data]
 return embeddings[0]

get_vector_embeddings("Your text string goes here")
```

Вывод:

```
[
 -0.006929283495992422,
 -0.005336422007530928,
 ...
 -4.547132266452536e-05,
 -0.024047505110502243
]
```

Для создания эмбедингов входного текста с использованием определенной модели этот код вызывает OpenAI API.

1. `from openai import OpenAI` импортирует библиотеку OpenAI, а инструкция `client = OpenAI()` создает экземпляр клиента. Она извлекает ваш ключ OpenAI API из переменной окружения `OPENAI_API_KEY`, чтобы списать стоимость получения эмбединга с баланса вашей учетной записи. Вы должны настроить эту переменную в своей среде (обычно в файле `.env`). Ключ можно получить, создав учетную запись и посетив <https://oreil.ly/apikeys>.
2. `response = client.embeddings.create(...)`: эта строка вызывает метод `create` класса `Embedding` клиента. Метод принимает два аргумента:
  - `input`: текстовая строка, для которой нужно сгенерировать эмбединг;
  - `model`: используемая модель, в данном случае `text-embedding-ada-002` — модель, доступная в OpenAI API.
3. `embeddings = [r.embedding for r in response.data]`: после вызова API объект `response` будет содержать сгенерированные эмбединги в формате JSON. Эта инструкция извлекает фактические числовые эмбединги из запроса, выполняя итерации по списку в `response.data`.

После выполнения этого кода переменная `embeddings` будет содержать числовое (векторное) представление входного текста, которое затем можно использовать в различных задачах NLP или моделях машинного обучения. Этот процесс извлечения или создания эмбедингов иногда называют *загрузкой документа*.

Под термином «загрузка» в этом контексте подразумевается вычисление или извлечение числовых (векторных) представлений текста из модели и сохранение их в переменной для последующего использования. Это отличается от понятия *разбиение на части (чанкинг)*, которое подразумевает разбиение текста на более мелкие части, или фрагменты, для облегчения обработки. Эти два метода часто используются в сочетании друг с другом, чтобы разбить большие документы на страницы или абзацы для поиска более точного сопоставления и передать в запрос только наиболее релевантные токены.

Получение эмбедингов из OpenAI сопряжено с затратами, но на момент написания этих строк они были относительно невысокими и составляли 0,0004 доллара США за 1000 токенов. Например, получение всех эмбедингов Библии короля Иакова, которая содержит около 800 000 слов или примерно 4 000 000 токенов, обойдется в 1,60 доллара.

Генерирование эмбедингов с помощью платного OpenAI API — не единственный вариант. Существуют также модели с открытым исходным кодом,

например библиотека Sentence Transformers (<https://oreil.ly/8OV3c>), доступная на Hugging Face и имеющая 384 измерения.

Ввод:

```
import requests
import os

model_id = "sentence-transformers/all-MiniLM-L6-v2"
hf_token = os.getenv("HF_TOKEN")

api_url = "https://api-inference.huggingface.co/"
api_url += f"pipeline/feature-extraction/{model_id}"
headers = {"Authorization": f"Bearer {hf_token}"}

def query(texts):
 response = requests.post(api_url, headers=headers,
 json={"inputs": texts,
 "options":{"wait_for_model":True}})
 return response.json()

texts = ["mickey mouse",
 "cheese",
 "trap",
 "rat",
 "ratatouille"
 "bus",
 "airplane",
 "ship"]
output = query(texts)
output
```

Вывод:

```
[[-0.03875632584095001, 0.04480459913611412,
 0.016051070764660835, -0.01789097487926483,
 -0.03518553078174591, -0.013002964667975903,
 0.14877274632453918, 0.048807501792907715,
 0.011848390102386475, -0.044042471796274185,
 ...,
 -0.026688814163208008, -0.0359361357986927,
 -0.03237859532237053, 0.008156519383192062,
 -0.10299170762300491, 0.0790356695652008,
 -0.008071334101259708, 0.11919838190078735,
 0.0005506130401045084, -0.03497892618179321]]
```

Этот код использует Hugging Face API для получения эмбедингов входных текстовых данных с помощью предварительно обученной модели. В этом примере используется модель `sentence-transformers/all-MiniLM-L6-v2`, уменьшенная версия NLP-модели BERT с открытым исходным кодом, представленной компанией Google в 2017 году (реализованной на основе модели



трансформера), которая оптимизирована для обработки предложений. Вот как работает этот код.

1. Переменной `model_id` присваивается идентификатор предварительно обученной модели `sentence-transformers/all-MiniLM-L6-v2`.
2. `hf_token = os.getenv("HF_TOKEN")` извлекает ключ для API — Hugging Face API токен, хранящийся в переменной окружения. Вам необходимо определить эту переменную, присвоив ей свой токен, который можно получить, создав учетную запись и посетив <https://hf.co/settings/tokens>.
3. Для выполнения HTTP-запросов к API импортируется библиотека `requests`.
4. Переменной `api_url` присваивается URL Hugging Face API, включающий идентификатор модели.
5. `headers` — это словарь, содержащий заголовок авторизации с токеном Hugging Face API.
6. Определение функции `query()`, которая принимает список входных текстовых данных, посылает запрос POST в Hugging Face API с соответствующими заголовками и входными данными в формате JSON, а затем ждет, когда модель станет доступной. После этого функция возвращает полученный ответ в формате JSON.
7. `texts` — это список строк из вашей базы данных.
8. Переменной `output` присваивается результат вызова функции `query()` со списком `texts`.
9. Выводится содержимое переменной `output`, что позволит отобразить эмбединги признаков для строк в списке `texts`.

После запуска этот сценарий отправит текст в Hugging Face API, который вернет эмбединги для каждой отправленной строки текста.

Передавая модели один и тот же текст, вы каждый раз будете получать в ответ один и тот же вектор. Однако векторы, возвращаемые разными моделями (и даже разными версиями одной модели), обычно различаются из-за особенностей в обучении. Эмбединги, возвращаемые OpenAI API, отличаются от эмбедингов, возвращаемых моделью BERT из библиотеки `spaCy`.

Основное отличие эмбедингов, генерируемых современными моделями трансформеров, — их векторы являются контекстуальными, а не статическими, то есть для слова *банка* будут возвращены разные эмбединги в контекстах *речная банка* (отмель) и *консервная банка*. Эмбединги, которые возвращают OpenAI Ada 002 и HuggingFace Sentence Transformers, являются примерами плотных векторов, в которых почти все элементы имеют ненулевое значение (то есть они содержат семантическую информацию). Существуют также разреженные векторы (<https://oreil.ly/d1cmb>), которые часто имеют большое количество измерений

(например, 100 000+), при этом большинство элементов в них равны нулю. Это позволяет фиксировать определенные важные признаки (каждый признак может иметь собственное измерение), что, как правило, важно для качества результатов, возвращаемых приложениями поиска по ключевым словам. Большинство приложений ИИ используют для поиска плотные векторы, хотя популярность гибридного поиска (с использованием и плотных, и разреженных векторов) растет, поскольку сочетание поиска по сходству и по ключевым словам может оказаться весьма полезным.

Точность векторов полностью зависит от точности модели, используемой для их создания. Любые предвзятости или пробелы в знаниях, имеющиеся в базовых моделях, будут отражаться на качестве векторного поиска. Например, модель `text-embedding-ada-002` обучалась только до августа 2020 года, поэтому не знает новых слов или новых культурных ассоциаций, появившихся после этой даты. Это может вызвать проблемы в случаях, когда требуется более свежий контекст или знания в специализированной предметной области, отсутствующие в обучающих данных, и потребовать обучения специальной модели.

В некоторых случаях может иметь смысл обучить свою модель, генерирующую эмбединги, например, если в тексте используются термины, характерные для предметной области и имеющие значения, отличающиеся от общепринятых значений. Одним из примеров может быть язык, используемый токсичными группами в социальных сетях, такими как Q-Anon, для обмана модераторов.

Обучение собственных моделей можно выполнить с помощью таких инструментов, как `word2vec` — метод представления слов в векторном пространстве, позволяющий улавливать семантические значения слов. Можно также использовать более продвинутые модели, такие как `GloVe` (Global Vectors for Word Representation — глобальные векторы для представления слов), которая используется в `sparse` для эмбединги, обученная на наборе данных `Common Crawl` — свободно доступном моментальном снимке (snapshot) Интернета. Библиотека `Gensim` предлагает простой процесс обучения собственных пользовательских эмбедингов с использованием алгоритма с открытым исходным кодом `word2vec` (<https://oreil.ly/RmXVR>).

Ввод:

```
from gensim.models import Word2Vec
```

```
Пример данных: список предложений, где каждое предложение —
это список слов.
При обучении моделей для практического применения вы будете
загружать и обрабатывать свой собственный корпус текстов.
```

```

sentences = [
 ["the", "cake", "is", "a", "lie"],
 ["if", "you", "hear", "a", "turret", "sing", "you're",
 "probably", "too", "close"],
 ["why", "search", "for", "the", "end", "of", "a",
 "rainbow", "when", "the", "cake", "is", "a", "lie?"],
 # ...
 ["there's", "no", "cake", "in", "space", "just", "ask",
 "wheatley"],
 ["completing", "tests", "for", "cake", "is", "the",
 "sweetest", "lie"],
 ["I", "swapped", "the", "cake", "recipe", "with", "a",
 "neurotoxin", "formula", "hope", "that's", "fine"],
] + [
 ["the", "cake", "is", "a", "lie"],
 ["the", "cake", "is", "definitely", "a", "lie"],
 ["everyone", "knows", "that", "cake", "equals", "lie"],
 # ...
] * 10 # повторить несколько раз для закрепления

Обучить модель word2vec
model = Word2Vec(sentences, vector_size=100, window=5,
min_count=1, workers=4, seed=36)

Сохранить модель
model.save("custom_word2vec_model.model")

Чтобы загрузить модель позже
loaded_model = word2vec.load(
"custom_word2vec_model.model")

Получить вектор для слова
vector = model.wv['cake']

Найти наиболее похожие слова
similar_words = model.wv.most_similar("cake", topn=5)
print("Top five most similar words to 'cake': ", similar_words)

Напрямую запросить степень сходства между "cake" и "lie"
cake_lie_similarity = model.wv.similarity("cake", "lie")
print("Similarity between 'cake' and 'lie': ",
cake_lie_similarity)

```

Вывод:

```

Top 5 most similar words to 'cake': [('lie',
0.23420444130897522), ('test', 0.23205122351646423),
('tests', 0.17178669571876526), ('GLaDOS',
0.1536172330379486), ('got', 0.14605288207530975)]
Similarity between 'cake' and 'lie': 0.23420444

```

Этот код создает модель `word2vec` с помощью библиотеки `Gensim`, а затем использует ее для поиска слов, похожих на заданное. Рассмотрим в деталях, как он работает.

1. Переменная `sentences` содержит список предложений, каждое из которых представлено списком слов. Это данные, на которых будет обучаться модель `Word2Vec`. При обучении моделей для практического применения вместо таких жестко запрограммированных предложений обычно загружается большой корпус текста и преобразуется в список токенизированных предложений.
2. Для представления модели создается экземпляр класса `word2vec`. Инициализация этого экземпляра производится с применением нескольких параметров:
  - `sentences`: обучающие данные;
  - `vector_size=100`: определяет размер векторов слов, то есть каждое слово будет представлено 100-мерным вектором;
  - `window=5`: представляет максимальное расстояние между текущим и предсказанным словом в предложении;
  - `min_count=1`: гарантирует создание векторов даже для слов, которые встречаются в наборе данных только один раз;
  - `workers=4`: количество ядер ЦП, которые будут использоваться во время обучения. Чем больше ядер, тем быстрее протекает процесс обучения;
  - `seed=36`: настраивается для воспроизводимости, чтобы случайные процессы в обучении каждый раз давали один и тот же результат (что не гарантируется при использовании нескольких ядер для обучения).
3. После обучения модель сохраняется в файл `custom_word2vec_model.model` вызовом метода `save`. Это позволит повторно использовать обученную модель позже без повторного ее обучения.
4. В коде приводится закомментированная строка, показывающая, как загрузить сохраненную модель из файла. Она может пригодиться, если вы хотите загрузить ранее обученную модель в другой сценарий или сеанс.
5. Переменной `vector` присваивается векторное представление слова *cake* («торт»). Этот вектор можно использовать для разных целей, например для вычисления сходства, выполнения арифметических операций и т. д.
6. Далее вызывается метод `most_similar` для поиска слов, наиболее похожих на предоставленный вектор (в данном случае вектор слова «торт»). Метод возвращает пять (`topn=5`) наиболее похожих слов.
7. Метод `similarity` вычисляет и возвращает степень сходства между векторами слов *cake* и *lie* (ложь), которая, как можно видеть, имеет небольшое положительное значение.

Набор данных небольшой, повторяется несколько раз и не отличается разнообразием контекста, поэтому он плохо подходит для выявления взаимосвязей между словами. Обычно word2vec показывает неплохие результаты на более крупных корпусах, насчитывающих десятки миллионов слов. В этом примере мы специально подобрали начальное значение *seed*, чтобы получить экземпляр модели, возвращающий слово *lie* в первой пятерке результатов, но если вы прокомментируете установку начального значения, то обнаружите, что модель редко находит связь между словами *cake* и *lie*.

Для документов небольшого размера рекомендуется применять более простой метод вычисления *TF-IDF* (Term Frequency, Inverse Document Frequency — частота термина, обратная частота документа) — статистической меры, используемой для оценки важности слова в документе относительно набора документов. Значение TF-IDF увеличивается пропорционально количеству появлений слова в документе и обратно пропорционально количеству появлений в более широком корпусе, что помогает компенсировать тот факт, что некоторые слова обычно встречаются чаще, чем другие.

Чтобы вычислить сходство между словами *cake* и *lie* с применением TF-IDF, можно воспользоваться библиотекой с открытым исходным кодом *scikit-learn* (<https://oreil.ly/gHb3F>) и с ее помощью вычислить *косинусное сходство* (меру расстояния между двумя векторами). Слова, которые часто встречаются в предложениях, будут иметь высокое косинусное сходство (приближающееся к 1), тогда как слова, которые встречаются нечасто, будут иметь низкое значение (или 0, если они вообще не встречаются рядом). Этот метод дает устойчиво хорошие результаты даже при обучении на небольших документах, таких как наш простенький пример.

Ввод:

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

Преобразовать предложения в список строк для TfidfVectorizer
document_list = [' '.join(s) for s in sentences]

Вычислить TF-IDF представление
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(document_list)

Получить позиции слов "cake" и "lie"
в матрице признаков
cake_idx = vectorizer.vocabulary_['cake']
lie_idx = vectorizer.vocabulary_['lie']

Получить и преобразовать вектор для слова 'cake'
cakevec = tfidf_matrix[:, cake_idx].toarray().reshape(1, -1)
```

```
Вычислить косинусные сходства
similar_words = cosine_similarity(cakevec, tfidf_matrix.T).flatten()

Получить индексы 6 самых похожих слов
(включая 'cake')
top_indices = np.argsort(similar_words)[-6:-1][::-1]

Получить и вывести 5 слов, самых похожих
на 'cake' (исключая само слово 'cake')
names = []
for idx in top_indices:
 names.append(vectorizer.get_feature_names_out()[idx])
print("Top five most similar words to 'cake': ", names)

Вычислить косинусное сходство между "cake" и "lie"
similarity = cosine_similarity(np.asarray(tfidf_matrix[:,
 cake_idx].todense()), np.asarray(tfidf_matrix[:, lie_idx].todense()))
Результатом будет матрица; мы можем найти среднее или
максимальное значение сходства
avg_similarity = similarity.mean()
print("Similarity between 'cake' and 'lie'", avg_similarity)

Вывести значение сходства между "cake" и "sing"
elephant_idx = vectorizer.vocabulary_['sing']
similarity = cosine_similarity(np.asarray(tfidf_matrix[:,
 cake_idx].todense()), np.asarray(tfidf_matrix[:,
 elephant_idx].todense()))
avg_similarity = similarity.mean()
print("Similarity between 'cake' and 'sing'",
 avg_similarity)
```

Вывод:

```
Top 5 most similar words to 'cake': ['lie', 'the', 'is',
'you', 'definitely']
Similarity between 'cake' and 'lie' 0.8926458157227388
Similarity between 'cake' and 'sing' 0.010626735901461177
```

Рассмотрим подробнее, что делает этот код.

1. Здесь повторно используется переменная `sentences` из предыдущего примера. Код преобразует списки слов в полные предложения (строки), используя генератор списков, в результате чего получается `document_list`.
2. Создается векторизатор — экземпляр `TfidfVectorizer` и затем вызывается его метод `fit_transform` для преобразования `document_list` в матрицу признаков TF-IDF, которая сохраняется в переменную `tfidf_matrix`.
3. Далее код извлекает позиции (или индексы) слов *cake* и *lie* в матрице признаков, используя атрибут `vocabulary_` векторизатора.

4. Из матрицы извлекается и преобразуется вектор TF-IDF, соответствующий торту *cake*.
5. Вычисляется косинусное сходство между векторами *cake* и всех других слов в матрице TF-IDF. В результате получается список оценок сходства.
  - Определяются индексы шести наиболее похожих слов (включая *cake*).
  - Затем по полученным индексам извлекаются и выводятся пять слов, наиболее близких к *cake* (за исключением самого слова *cake*).
6. Вычисляется косинусное сходство между векторами TF-IDF слов *cake* и *lie*. Поскольку результатом является матрица, код вычисляет среднее всех значений в этой матрице и выводит его.
7. После этого вычисляется и выводится среднее значение сходства между словами *cake* и *sing* (пение), чтобы показать, что эти два слова обычно находятся далеко друг от друга (сходство близко к нулю).

Помимо используемой модели эмбедингов, стратегия того, что именно вы превращаете в эмбединги, также важна, поскольку существует компромисс между контекстом и сходством. Если вы сделаете эмбединг большого фрагмента текста, скажем, целой книги, то полученный вами вектор будет представлять средние значения местоположений токенов, составляющих полный текст. С увеличением размера фрагмента текста значения будут стремиться к среднему, а семантическая информация при этом будет исчезать.

Меньшие фрагменты текста будут содержать больше конкретики с точки зрения местоположения в векторном пространстве и поэтому могут оказаться более полезными для определения сходства. Например, выделение небольших фрагментов текста из романа может помочь лучше отделить комедийные моменты от трагических, тогда как использование эмбединга для целой страницы или главы может смешать их друг с другом. Однако если сделать фрагменты текста слишком маленькими, например, разделять их в середине предложений или абзацев, то они могут потерять смысл. Немалая доля успеха при работе с векторными базами данных заключается в том, как загружаются документы и как они разбиваются на фрагменты.

## Загрузка документа

Одним из распространенных вариантов использования ИИ является возможность поиска по документам на основе сходства с текстом пользовательского запроса. Например, у вас может иметься несколько PDF-файлов, составляющих справочник сотрудника, и вы хотите получить фрагмент текста из этих файлов, отвечающий на вопрос сотрудника. Способ загрузки документов в векторную

базу данных будет зависеть от структуры документов, количества примеров, которые вы решите вернуть в ответ на каждый запрос, и количества токенов в каждом запросе.

Например, `gpt-4-0613` имеет ограничение 8192 токена (<https://oreil.ly/wbx1f>), которое необходимо разделить между шаблоном промта, примерами, вставленными в промт, и ответом модели. Выделив около 2000 слов или примерно 3000 токенов для запроса и ответа, вы сможете для контекста вставить в запрос пять наиболее похожих фрагментов текста по 1000 токенов в каждом. Однако если вы просто разобьете документ на фрагменты по 1000 токенов, то столкнетесь с проблемой. Место произвольного разделения может находиться в середине абзаца или предложения, поэтому есть риск потерять смысл передаваемой информации. В LangChain имеется ряд инструментов для разбиения текста (<https://oreil.ly/qsG7J>), включая часто используемый инструмент рекурсивного разбиения. Он старается выполнить разбиение по символам перевода строки, а потом по пробелам, пока фрагменты не станут достаточно маленькими, стремясь обеспечить целостность абзацев (предложений и слов), чтобы сохранить группировки слов, присущие смысловой структуре текста.

Ввод:

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
 chunk_size=100, # 100 токенов
 chunk_overlap=20, # перекрытия длиной 20 токенов
)

text = """
Welcome to the "Unicorn Enterprises: Where Magic Happens"
Employee Handbook! We're thrilled to have you join our team
of dreamers, doers, and unicorn enthusiasts. At Unicorn
Enterprises, we believe that work should be as enchanting as
it is productive. This handbook is your ticket to the
magical world of our company, where we'll outline the
principles, policies, and practices that guide us on this
extraordinary journey. So, fasten your seatbelts and get
ready to embark on an adventure like no other!

...

As we conclude this handbook, remember that at Unicorn
Enterprises, the pursuit of excellence is a never-ending
quest. Our company's success depends on your passion,
creativity, and commitment to making the impossible
possible. We encourage you to always embrace the magic
within and outside of work, and to share your ideas and
innovations to keep our enchanted journey going. Thank you
```



```
for being a part of our mystical family, and together, we'll
continue to create a world where magic and business thrive
hand in hand!
"""
```

```
chunks = text_splitter.split_text(text=text)
print(chunks[0:3])
```

Вывод:

```
['Welcome to the "Unicorn Enterprises: Where Magic Happens"
Employee Handbook! We\'re thrilled to have you join our team
of dreamers, doers, and unicorn enthusiasts.',
"We're thrilled to have you join our team of dreamers,
doers, and unicorn enthusiasts. At Unicorn Enterprises,
we believe that work should be as enchanting as it is
productive.",
...
"Our company's success depends on your passion, creativity,
and commitment to making the impossible possible. We
encourage you to always embrace the magic within and outside
of work, and to share your ideas and innovations to keep our
enchanted journey going.",
"We encourage you to always embrace the magic within and
outside of work, and to share your ideas and innovations to
keep our enchanted journey going. Thank you for being a part
of our mystical family, and together, we'll continue to
create a world where magic and business thrive hand in hand!"]
```

Рассмотрим подробнее, что делает этот код.

1. *Создает экземпляр инструмента разбиения текста:* вызовом метода `from_tiktoken_encoder` создает экземпляр класса `RecursiveCharacterTextSplitter`. Этот метод специально разработан для создания инструмента, разбивающего текст по количеству токенов.

Параметр `chunk_size=100` гарантирует, что каждый фрагмент текста будет содержать примерно 100 токенов.

Параметр `chunk_overlap=20` указывает, что соседние фрагменты должны иметь перекрытия длиной 20 токенов. Такая протяженность перекрытий гарантирует сохранность контекста между фрагментами, что имеет решающее значение для понимания и обработки текста моделью.

2. *Подготавливает текст:* записывает в переменную `text` строку, содержащую несколько абзацев, которую нужно разбить на фрагменты.
3. *Разбивает текст:* вызывает метод `split_text` экземпляра `text_splitter`, чтобы разбить текст на фрагменты с учетом ранее определенных параметров `chunk_size` и `chunk_overlap`. Этот метод обрабатывает текст и возвращает список фрагментов.

4. *Выводит фрагменты*: выдает первые три фрагмента, чтобы показать, как был разбит текст. Часто полезно вывести несколько фрагментов, чтобы убедиться, что текст разбит в соответствии с указанными размерами фрагментов и перекрытия.



### Укажите формат

Релевантность фрагмента текста, представленного в запросе, во многом зависит от выбранной стратегии разбиения. Короткие и неперекрывающиеся фрагменты текста могут не содержать правильного ответа, а более длинные фрагменты со слишком большим перекрытием могут вернуть очень много нерелевантных результатов, запутать LLM и повлечь ненужные затраты.

## Извлечение данных из памяти с помощью FAISS

Разбив документы на фрагменты, их нужно сохранить в векторной базе данных, чтобы не пришлось делать то же самое повторно и избежать дополнительных затрат и задержек. Если модель создания эмбедингов не изменится, то не изменятся и сами векторные представления и их не придется обновлять после сохранения. Для сохранения и извлечения векторов можно использовать библиотеку с открытым исходным кодом FAISS, разработанную в Facebook AI (<https://oreil.ly/gIcTI>). Она обеспечивает эффективный поиск по сходству и кластеризацию плотных векторов. Сначала установите FAISS с помощью команды `pip install faiss-cpu`. Код следующего примера вы найдете в репозитории этой книги на GitHub (<https://oreil.ly/4wR7o>).

Ввод:

```
import numpy as np
import faiss

Функция get_vector_embeddings определена в предыдущем примере
emb = [get_vector_embeddings(chunk) for chunk in chunks]
vectors = np.array(emb)

Создать индекс FAISS
index = faiss.IndexFlatL2(vectors.shape[1])
index.add(vectors)

Функция, выполняющая поиск вектора
def vector_search(query_text, k=1):
 query_vector = get_vector_embeddings(query_text)
 distances, indices = index.search(np.array([query_vector]), k)
 return [(chunks[i], float(dist)) for dist,
 i in zip(distances[0], indices[0])]
```

```
Пример поиска
user_query = "do we get free unicorn rides?"
search_results = vector_search(user_query)
print(f"Search results for {user_query}:", search_results)
```

Вывод:

```
Search results for do we get free unicorn rides?: [{"You'll
enjoy a treasure chest of perks, including unlimited unicorn
rides, a bottomless cauldron of coffee and potions, and
access to our company library filled with spellbinding
books. We also offer competitive health and dental plans,
ensuring your physical well-being is as robust as your
magical spirit.\n\n**5: Continuous Learning and
Growth**\n\nAt Unicorn Enterprises, we believe in continuous
learning and growth.", 0.3289167582988739}]
```

Вот что делает этот код.

1. `import faiss` импортирует библиотеку Facebook AI Similarity Search (FAISS).
2. `vectors = np.array([get_vector_embeddings(chunk) for chunk in chunks])` применяет `get_vector_embeddings` к каждому элементу в `chunks`, чтобы получить его векторное представление. Затем эти векторы помещаются в массив `numpy` в переменной `vectors`.
3. `index = faiss.IndexFlatL2(vectors.shape[1])` создает индекс FAISS для более эффективного поиска сходства. Аргумент `vectors.shape[1]` — это размерность векторов, которые будут добавлены в индекс. Этот тип индекса (`IndexFlatL2`) выполняет поиск по расстоянию L2 методом простого перебора и отыскивает элементы в коллекции, ближайшие к заданному элементу.
4. `index.add(vectors)` добавляет массив `vectors` в только что созданный индекс FAISS.
5. `def vector_search(query_text, k=1)` определяет новую функцию с именем `vector_search`, которая принимает два параметра: `query_text` и `k` (со значением по умолчанию 1). Эта функция возвращает эмбединги для `query_text`, а потом использует его для поиска `k` ближайших векторов в индексе.
6. `query_vector = get_vector_embeddings(query_text)` внутри функции `vector_search` генерирует векторное представление для запрошенного текста с помощью функции `get_vector_embeddings`.
7. `distances, indexes = index.search(np.array([query_vector]), k)` выполняет поиск в индексе FAISS `k` векторов, ближайших к `query_vector`. Метод `search` возвращает два массива, которые помещаются в переменные `distances` и `indexes`: расстояния (квадраты расстояний L2 до запрошенного вектора) и индексы.

8. `return [(chunks[i], float(dist)) for dist, i in zip(distances[0], index[0])]` возвращает список кортежей. Каждый кортеж содержит фрагмент (найденный с помощью индекса) и соответствующее расстояние от запрошенного вектора. Обратите внимание, что перед возвратом расстояние преобразуется в число с плавающей запятой.
9. `search_results = vector_search(user_query)` выполняет поиск по строке, запрошенной пользователем. Результат (ближайший фрагмент и расстояние до него) сохраняется в переменной `search_results`.

После завершения векторного поиска результаты можно добавить в промт, чтобы обеспечить полезный контекст. Важно также настроить системное сообщение, чтобы подсказать модели, что она должна искать ответ на основе предоставленного контекста, а не придумывать его. Показанный здесь прием RAG широко используется в сфере ИИ для защиты от галлюцинаций.

Ввод:

```
Функция выполняет векторный поиск и затем
посылает запрос модели GPT-3.5-turbo
def search_and_chat(user_query, k=1):
 # Выполнить векторный поиск
 search_results = vector_search(user_query, k)
 print(f"Search results: {search_results}\n\n")

 prompt_with_context = f"""Context:{search_results}\
 Answer the question: {user_query}"""

 # Создать список сообщений для чата
 messages = [
 {"role": "system", "content": """Please answer the
 questions provided by the user. Use only the context
 provided to you to respond to the user, if you don't
 know the answer say \"I don't know\".""",},
 {"role": "user", "content": prompt_with_context},
]

 # Получить ответ модели
 response = client.chat.completions.create(
 model="gpt-3.5-turbo", messages=messages)

 # Вывести полученный ответ
 print(f"""Response:
 {response.choices[0].message.content}""")

Пример поиска и получения ответа от модели
search_and_chat("What is Unicorn Enterprises' mission?")
```

**Вывод:**

```
Search results: [("""As we conclude this handbook, remember that at Unicorn Enterprises, the pursuit of excellence is a never-ending quest. Our company's success depends on your passion, creativity, and commitment to making the impossible possible. We encourage you to always embrace the magic within and outside of work, and to share your ideas and innovations to keep our enchanted journey going. Thank you", 0.26446571946144104)]
```

**Response:**

Unicorn Enterprises' mission is to pursue excellence in their work by encouraging their employees to embrace the magic within and outside of work, share their ideas and innovations, and make the impossible possible.

Вот подробное описание работы этого кода.

1. Вызывая функцию `vector_search`, программа выполняет векторный поиск по строке `user_query` и отыскивает к ближайших результатов. Результаты сохраняются в `search_results`.
2. Затем результаты поиска выводятся в консоль.
3. Создается запрос с контекстом `prompt_with_context` путем объединения `search_results` и `user_query`. Цель — передать модели контекст из результатов поиска и вопрос, на который нужно ответить.
4. Создается список сообщений. Первое сообщение — системное. Оно сообщает модели, что та должна отвечать на вопросы, заданные пользователем, используя только заданный контекст. Если модель не знает ответа, то ей рекомендуется ответить `I don't know` («Не знаю»). Второе сообщение — это вопрос пользователя.
5. Вызывается функция `openai.ChatCompletion.create()` для получения ответа модели. Ей передаются название модели (`gpt-3.5-turbo`) и список сообщений.
6. В конце вызывается функция `search_and_chat()` с вопросом в `user_query`.

**Приведите примеры**

Без тестирования трудно угадать, какая стратегия составления запросов лучше. Теперь вы можете быть уверены, что подход с поиском и добавлением контекста — правильный.

Наш текущий код выполняет все операции от начала до конца, и нет смысла собирать эмбединги и создавать векторную базу данных при каждом запросе,

так как это влечет за собой дополнительные задержки и затраты, даже если для создания эмбедингов используется модель с открытым исходным кодом. Вы можете сохранить индекс FAISS в файл с помощью функции `faiss.write_index`:

```
Сохранить индекс в файл
faiss.write_index(index, "data/my_index_file.index")
```

Этот вызов создаст в текущем каталоге файл индекса с именем `my_index_file.index`. Загрузить этот индекс обратно в память можно вызовом `faiss.read_index`:

```
Загрузить индекс из файла
index = faiss.read_index("data/my_index_file.index")
```

Так можно сохранять свой индекс в разных сеансах и даже переносить его на другие компьютеры или в другие среды. Но имейте в виду, что эти файлы могут быть довольно большими.

Если у вас есть несколько векторных баз данных, то их тоже можно объединить. Это может пригодиться для сериализации загрузки документов или пакетного обновления записей.

Объединить два индекса FAISS можно с помощью метода `add` индекса `faiss.IndexFlatL2`:

```
Предполагается, что index1 и index2 - это два индекса IndexFlatL2
index1.add(index2.reconstruct_n(0, index2.ntotal))
```

Здесь вызов `reconstruct_n(0, index2.ntotal)` извлекает все векторы из `index2`, которые затем передаются в вызов `index1.add()` для добавления в `index1`.

Этот прием работает благодаря наличию метода `reconstruct` в `faiss.IndexFlatL2`, который извлекает векторы. Но имейте в виду, что этот процесс не перемещает никаких идентификаторов, связанных с векторами, из `index2` в `index1`. После слияния векторы из `index2` получают новые идентификаторы в `index1`.

Чтобы сохранить идентификаторы векторов, вам придется управлять ими извне, сохраняя связь идентификаторов векторов с элементами данных отдельно. При последующем объединении индексов вы должны будете также объединить и эти связи.



Учтите также, что этот метод подходит не для всех типов индексов. В частности, он не годится для индексов, которые не поддерживают метод `reconstruct`, таких как `IndexIVFFlat`, или если два индекса имеют разные конфигурации. В таких случаях лучше сохранять исходные векторы, использованные для построения каждого индекса, а затем объединить их и перестраивать индекс.

## Реализация RAG с использованием LangChain

LangChain, как один из самых популярных фреймворков для разработки ИИ, реализует методы генерации с дополненной выборкой (RAG). Другие фреймворки, в том числе и LlamaIndex (<https://www.llamaindex.ai>), ориентированы конкретно на RAG и с успехом могут использоваться в сложных случаях. Поскольку вы уже познакомились с LangChain в главе 4, мы продолжим использовать этот фреймворк в примерах главы. Выше мы реализовали применение метода RAG вручную на основе желаемого контекста, а теперь рассмотрим аналогичный пример с LCEL и четырьмя небольшими текстовыми документами, использующий библиотеку FAISS:

```
from langchain_community.vectorstores.faiss import FAISS
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_openai import ChatOpenAI, OpenAIEmbeddings

1. Создать документы:
documents = [
 "James Phoenix worked at JustUnderstandingData.",
 "James Phoenix currently is 31 years old.",
 """"Data engineering is the designing and building systems for collecting,
 storing, and analyzing data at scale."""",
]

2. Создать хранилище векторов vectorstore:
vectorstore = FAISS.from_texts(texts=documents, embedding=OpenAIEmbeddings())
retriever = vectorstore.as_retriever()

3. Создать запрос:
template = """"Answer the question based only on the following context:

Context: {context}

Question: {question}
""""
prompt = ChatPromptTemplate.from_template(template)

4. Создать модель чата:
model = ChatOpenAI()
```

Сначала код импортирует необходимые модули из библиотеки LangChain и затем определяет список текстовых документов, подлежащих обработке.

Далее с помощью библиотеки FAISS, реализующей эффективный поиск по сходству, создается векторное хранилище из текстовых документов. Для этого

тексты документов преобразуются в векторные представления с использованием модели OpenAI.

Для обработки вопросов создается шаблон запроса, а для получения ответов — модель ChatOpenAI. Шаблон запроса требует от модели, чтобы та отвечала только с использованием предоставленного контекста.

Теперь создадим цепочку LCEL, содержащую ключи "context" и "question":

```
chain = (
 {"context": retriever, "question": RunnablePassthrough()}
 | prompt
 | model
 | StrOutputParser()
)
```

Добавив retriever в "context", мы обеспечили автоматическое получение четырех документов в форме строки. В сочетании с ключом "question" они затем используются для формирования запроса. Далее модель LLM генерирует ответ, который преобразуется в строковое значение с помощью StrOutputParser().

Протестируем вручную эту цепочку, вызвав ее трижды с тремя разными вопросами, которые будут присваиваться ключу "question":

```
chain.invoke("What is data engineering?")
'Data engineering is the process of designing and building systems for
collecting, storing, and analyzing data at scale.'

chain.invoke("Who is James Phoenix?")
'Based on the given context, James Phoenix is a 31-year-old individual who
worked at JustUnderstandingData.'

chain.invoke("What is the president of the US?")
I don't know
```

Обратите внимание, что LLM правильно ответила только на первые два запроса, потому что в векторной базе данных нет соответствующего контекста для ответа на третий запрос!

Реализация на основе LangChain значительно короче, легко читается и позволяет быстро реализовать генерацию с дополненной выборкой (RAG).

## Векторные базы данных на хостинге от Pinecone

В настоящее время существует множество провайдеров хостинговых векторных баз данных для поддержки сценариев использования ИИ, включая Chroma (<https://www.trychroma.com>), Weaviate (<https://weaviate.io>) и Pinecone (<https://www.pinecone.io>). Некоторые провайдеры баз данных других типов реализуют функцию векторного поиска. Примером может служить Supabase



(<https://supabase.com>) с надстройкой pgvector (<https://oreil.ly/pgvector>). В примерах книги мы будем использовать Pinecone, потому что на момент написания этой книги она занимала лидирующие позиции, но, вообще говоря, порядок использования разных баз данных мало отличается друг от друга и основные концепции, описываемые далее, применимы и к другим векторным БД.

Хостинговые векторные базы данных имеют ряд преимуществ перед локальными векторными хранилищами с открытым исходным кодом.

### *Техническое обслуживание*

Выбрав хостинговую векторную БД, вы можете не беспокоиться о ее настройке и обслуживании. Это позволит сэкономить время и ресурсы, особенно компаниям, не имеющим выделенных групп DevOps или управления базами данных.

### *Масштабируемость*

Хостинговые векторные БД способны масштабироваться в соответствии с вашими потребностями. База данных может автоматически масштабироваться, приспосабливаясь к возрастающей нагрузке, и гарантировать эффективную работу ваших приложений.

### *Надежность*

Управляемые сервисы обычно обеспечивают высокую доступность в соответствии с соглашениями об уровне обслуживания, а также функции автоматического резервного копирования и аварийного восстановления. Это даст вам душевное спокойствие и убережет от потенциальной потери данных.

### *Производительность*

Хостинговые векторные БД часто имеют оптимизированную инфраструктуру и алгоритмы, показывающие более высокую производительность, чем локальные решения с открытым исходным кодом. Это может быть особенно важно для приложений, которые полагаются на функцию векторного поиска в режиме реального времени или близком к нему.

### *Поддержка*

При использовании хостинга вы обычно получаете доступ к поддержке от компании, предоставляющей эту услугу. Такая поддержка может очень пригодиться, если у вас возникнут проблемы или вам потребуется помощь в оптимизации использования БД.

### *Безопасность*

Поставщики услуг хостинга, как правило, уделяют серьезное внимание мерам безопасности для защиты ваших данных, таким как шифрование данных, контроль доступа и мониторинг. Крупные поставщики услуг

с большей вероятностью будут иметь необходимые сертификаты соответствия и соблюдать законодательство о конфиденциальности в таких регионах, как ЕС.

Конечно, расширенные возможности стоят денег и есть риск переплатить из-за своей невнимательности. Существует множество историй о разработчиках, использовавших Amazon Web Services, Microsoft Azure или Google Cloud, которые потратили тысячи долларов из-за неправильной конфигурации или ошибок в коде. Есть также некоторый риск привязки к одному поставщику, потому что решения разных поставщиков, несмотря на схожие функциональные возможности, имеют некоторые уникальные особенности и поэтому мигрировать между ними не всегда просто. Другой важный фактор — конфиденциальность, потому что передача данных третьей стороне сопряжена с риском безопасности и потенциальными юридическими последствиями.

Действия при работе с хостинговой векторной БД остаются теми же, что и при использовании хранилища векторов FAISS. Сначала документы разбиваются на фрагменты и из них извлекаются векторы; затем фрагменты документа индексируются в базе данных, что позволяет извлекать записи, похожие на запрос, для передачи их в промт в качестве контекста. Для начала создадим индекс в Pinecone (<https://www.pinecone.io>), популярной коммерческой векторной базе данных. Для этого зарегистрируйтесь в Pinecone и получите ключ API: выберите пункт API Keys (Ключи API) в боковом меню и щелкните на ссылке create API Key (Создать ключ API). Код этого примера вы найдете в репозитории книги на GitHub (<https://oreil.ly/Q0rIw>).

Ввод:

```
from pinecone import Pinecone, ServerlessSpec
import os

Инициализировать соединение (получите ключ API на app.pinecone.io):
os.environ["PINECONE_API_KEY"] = "insert-your-api-key-here"

index_name = "employee-handbook"
environment = "us-east-1"
pc = Pinecone() # Прочитать значение переменной окружения PINECONE_API_KEY

Проверить, существует ли индекс:
(при первом обращении его не должно быть)
if index_name not in pc.list_indexes().names():
 # если индекса нет, то создать его
 pc.create_index(
 index_name,
 # Using the same vector dimensions as text-embedding-ada-002
```

```
 dimension=1536,
 metric="cosine",
 spec=ServerlessSpec(cloud="aws", region=environment),
)

 # Подключиться к индексу:
 index = pc.Index(index_name)

 # Вывести статистику индекса:
 index.describe_index_stats()
```

Вывод:

```
{'dimension': 1536,
 'index_fullness': 0.0,
 'namespaces': {},
 'total_vector_count': 0}
```

Рассмотрим подробнее, что делает этот код.

#### 1. Импорт библиотек

Сценарий начинается с импорта модулей `Pinecone` и `ServerlessSpec`, необходимых для работы с базой данных `Pinecone`, а модуль `os` импортируется для доступа к переменным окружения.

#### 2. Настройка ключа `Pinecone API`

Ключ `Pinecone API` имеет решающее значение для аутентификации и сохраняется в переменной окружения вызовом `os.environ["PINECONE_API_KEY"] = "insert-your-api-key-here"`. Не забудьте заменить текст `"insert-your-api-key-here"` фактическим ключом `Pinecone API`.

#### 3. Настройка имени индекса и окружения

Настраиваются переменные `index_name` и `environment`. Переменной `index_name` присваивается значение `"employee-handbook"` — имя индекса в базе данных `Pinecone`, к которому нужно получить доступ (и создать при необходимости). Переменной `environment` присваивается значение `"us-east-1"`, указывающее местоположение сервера.

#### 4. Инициализация соединения с `Pinecone`

Соединение с `Pinecone` инициализируется с помощью конструктора `Pinecone()`. Он автоматически читает значение переменной окружения `PINECONE_API_KEY`.

#### 5. Проверка существования индекса

Сценарий проверяет существование в базе данных `Pinecone` индекса с именем `index_name`. Это делается с помощью функции `pc.list_indexes().names()`, которая возвращает список всех существующих имен индексов.

## 6. Создание индекса

Если искомый индекс не существует, то он создается вызовом `pc.create_index()`. Эта функция принимает несколько параметров настройки нового индекса:

- `index_name`: имя индекса;
- `dimension=1536`: размерность векторов, которые будут храниться в индексе;
- `metric='cosine'`: метрика сходства (в данном случае косинусное расстояние), которая должна использоваться для сравнения векторов.

## 7. Подключение к индексу

После проверки или создания индекса сценарий подключается к нему вызовом `pc.Index(index_name)`. Это соединение необходимо для последующих операций, таких как вставка или запрос данных.

## 8. Статистика индексов

В заключение сценарий вызывает метод `index.describe_index_stats()`, который извлекает и выводит различные статистические данные об индексе, такие как его размерность и общее количество сохраненных векторов.

Теперь сохраним наши векторы во вновь созданном индексе: выполним обход всех текстовых фрагментов и векторов и добавим их в Pinecone. Операция `upsert` представляет собой комбинацию операций *update* (изменение) и *insert* (вставка) — она либо обновляет существующую запись, либо вставляет новую, если такой записи еще не существует (см. блокнот Jupyter Notebook [<https://oreil.ly/YC-nV>]).

Ввод:

```
from tqdm import tqdm # For printing a progress bar
from time import sleep

Количество векторных представлений, создаваемых
и добавляемых в БД за один раз
batch_size = 10
retry_limit = 5 # максимальное число попыток

for i in tqdm(range(0, len(chunks), batch_size)):
 # Поиск конца пакета
 i_end = min(len(chunks), i+batch_size)
 meta_batch = chunks[i:i_end]
 # Получить идентификаторы
 ids_batch = [str(j) for j in range(i, i_end)]
 # Получить тексты для кодирования
 texts = [x for x in meta_batch]
```

```
Создать векторные представления
(try-except added to avoid RateLimitError)
done = False
try:
 # Получить векторные представления для всего пакета сразу
 embeds = []
 for text in texts:
 embedding = get_vector_embeddings(text)
 embeds.append(embedding)
 done = True
except:
 retry_count = 0
 while not done and retry_count < retry_limit:
 try:
 for text in texts:
 embedding = get_vector_embeddings(text)
 embeds.append(embedding)
 done = True
 except:
 sleep(5)
 retry_count += 1

if not done:
 print(f"" Failed to get embeddings after
 {retry_limit} retries.""")
 continue

Подготовить метаданные
meta_batch = [{
 'batch': i,
 'text': x
} for x in meta_batch]
to_upsert = list(zip(ids_batch, embeds, meta_batch))

Добавить в Pinecone
index.upsert(vectors=to_upsert)
```

Вывод:

```
100% 13/13 [00:53<00:00, 3.34s/it]
```

Рассмотрим подробнее, что делает этот код.

1. Импортирует необходимые библиотеки `tqdm` и `time`. Библиотека `tqdm` выводит индикаторы прогресса, а `time` предоставляет функцию `sleep()`, которая используется в этом сценарии в логике выполнения повторных попыток.
2. Присваивает переменной `batch_size` значение `10` (в реальных рабочих нагрузках обычно выбирается значение `100`) — количество элементов, обрабатываемых в одной итерации цикла. Также инициализирует переменную `retry_limit`, гарантируя прекращение работы после пяти неудачных попыток.

3. `for i in tqdm(range(0, len(chunks), batch_size))` — заголовок цикла, который выполнит обход элементов списка `chunks` (фрагментов текста, которые нужно обработать) с шагом `batch_size`. `tqdm` используется здесь для отображения индикатора прогресса.
4. Переменной `i_end` присваивается наименьшее из значений `len(chunks)` и `i + batch_size`. Она используется для предотвращения ошибки индекса, если `i + batch_size` превышает длину фрагментов.
5. `meta_batch` — подмножество фрагментов в `chunks`, относящихся к текущему пакету. Пакет определяется как срез `chunks` от индекса `i` до `i_end`.
6. `ids_batch` — список строковых представлений диапазона от `i` до `i_end`. Эти строки используются для идентификации элементов в `meta_batch`.
7. Список `texts` — это просто текст из `meta_batch`, готовый к обработке.
8. Выполняет попытку получить векторные представления вызовом `get_vector_embeddings()` с аргументом `texts`. Результат сохраняется в переменной `embeds`. Попытка выполняется внутри блока `try-except`, чтобы перехватить любые исключения, которые могут возникнуть в этой функции, например ошибки ограничения скорости.
9. В случае исключения сценарий входит в цикл `while`, где он ждет в течение пяти секунд, а затем снова попытается получить векторные представления. Этот цикл завершается в случае успеха очередной попытки или по достижении максимального числа повторов, после чего выполняется инструкция `done = True` и цикл `while` завершается.
10. Изменяет переменную `meta_batch`, превращая ее в список словарей. Каждый словарь имеет два ключа: `batch`, которому присваивается номер текущего пакета `i`, и `text`, которому присваивается соответствующий элемент в `meta_batch`. Здесь вы можете добавить дополнительные метаданные для последующей фильтрации запросов, например номер страницы, заголовка или главы.
11. Создает список `to_upsert` вызовом функции `zip`, которая объединяет `ids_batch`, `embeds` и `meta_batch` в кортежи, а затем добавляет их в список. Каждый кортеж содержит идентификатор, векторное представление и метаданные для каждого элемента в пакете.
12. Последняя строка в цикле вызывает метод `upsert` объекта `index`, выполняя добавление векторов в Pinecone (сервис векторной базы данных). Аргумент `vectors=to_upsert` передает список `to_upsert` с данными для добавления в индекс. Если вектор с указанным идентификатором уже существует в индексе, он будет обновлен; если не существует, то вставлен.

После добавления записей в Pinecone вы сможете запрашивать свои векторы, как если бы сохранили их локально с помощью FAISS. Векторные представления остаются неизменными, пока вы используете одну и ту же модель для их создания, поэтому вам не придется обновлять базу данных, если нет дополнительных записей или метаданных для добавления.

Ввод:

```
Получить результаты из Pinecone
user_query = "do we get free unicorn rides?"

def pinecone_vector_search(user_query, k):
 xq = get_vector_embeddings(user_query)
 res = index.query(vector=xq, top_k=k, include_metadata=True)
 return res

pinecone_vector_search(user_query, k=1)
```

Вывод:

```
{'matches':
 [{ 'id': '15',
 'metadata': { 'batch': 10.0,
 'text': "You'll enjoy a treasure chest of perks, "
 'including unlimited unicorn rides, a '
 'bottomless cauldron of coffee and potions, '
 'and access to our company library filled '
 'with spellbinding books. We also offer '
 'competitive health and dental plans, '
 'ensuring your physical well-being is as '
 'robust as your magical spirit.\n'
 '\n'
 '**5: Continuous Learning and Growth**\n'
 '\n'
 'At Unicorn Enterprises, we believe in '
 'continuous learning and growth.' },
 'score': 0.835591,
 'values': [], },
],
 'namespace': '' }
```

Этот сценарий выполняет поиск ближайших соседей, используя Pinecone API для определения векторов, наиболее похожих на заданный. Вот как он работает.

1. Определяется функция `pinecone_vector_search` с двумя параметрами: `user_query` и `k`. Параметр `user_query` — это введенный пользователем текст, готовый к преобразованию в вектор, а `k` определяет, сколько ближайших векторов нужно найти.

2. Внутри функции определяется значение для переменной `xq` путем вызова функции `get_vector_embeddings(user_query)`. Эта функция (определена выше) отвечает за преобразование `user_query` в векторное представление.
3. Следующая строка обращается к объекту `index`, представляющему индекс Pinecone, используя метод `query`, который принимает три параметра:
  - `vector=xq` — в нем передается векторное представление пользовательского запроса `user_query`;
  - `top_k=k` — передается количество ближайших векторов, которые нужно найти в индексе Pinecone;
  - `include_metadata=True` — в параметре сообщается, что в результаты должны быть включены метаданные (например, идентификаторы или другие сведения). Если вам нужно отфильтровать результаты по метаданным (<https://oreil.ly/BBYD4>), например по номеру пакета (или любым другим метаданным, которые вы добавили), то можете сделать это здесь, добавив четвертый параметр, например: `filter={"batch": 1}`.
4. Результат метода `query` присваивается переменной `res` и возвращается вызывающему коду.
5. В заключение производится вызов функции `pinecone_vector_search` с аргументами `user_query` и `k`, которая возвращает ответ от Pinecone.

В этом примере мы фактически симитировали работу FAISS, вернув соответствующую запись из справочника с помощью поиска ближайшего вектора. Если заменить `vector_search(user_query, k)` на `pinecone_vector_search(user_query, k)` в функции `search_and_chat` (в предыдущем примере), то чат-бот будет работать точно так же, только векторы будут храниться в управляемой базе данных Pinecone, а не локально.

Добавляя записи в Pinecone, мы передавали номер пакета в метаданных. Pinecone поддерживает следующие типы метаданных:

- строки;
- числа (целые и с плавающей точкой, последние преобразуются в 64-битные числа с плавающей точкой);
- логические значения (`True`, `False`);
- списки строк.

Стратегия добавления метаданных в хранимые записи столь же важна, как и стратегия разбиения на фрагменты, потому что метаданные можно использовать для фильтрации запросов. Например, чтобы ограничить поиск



сходства определенным номером пакета, в вызов `index.query` можно добавить фильтр:

```
res = index.query(xq, filter={
 "batch": {"$eq": 1}
}, top_k=1, include_metadata=True)
```

Эта возможность может пригодиться для ограничения области поиска сходства. Например, вы сможете хранить прошлые диалоги во всех чат-ботах в одной векторной БД, а затем запрашивать только диалоги с конкретным идентификатором чат-бота, чтобы добавить контекст в запрос к этому чат-боту. В числе других распространенных вариантов фильтрации с использованием метаданных можно назвать поиск более поздних временных меток, номеров страниц документов или товаров по определенной цене.



Обратите внимание, что увеличение объема метаданных, вероятно, приведет к увеличению затрат на хранение, как и хранение больших фрагментов, которые редко используются. Понимание особенностей работы векторных баз данных поможет вам экспериментировать с различными стратегиями разбиения и организации метаданных и выбрать те, что лучше подходят для ваших сценариев использования.

## Собственные запросы

Поиск может быть весьма сложным, и вас никто не ограничивает базовыми инструментами поиска, которые извлекают документы из векторной БД, руководствуясь исключительно семантической релевантностью. Вы можете, например, использовать метаданные из запроса пользователя. Распознавая и извлекая такие фильтры, ваш инструмент поиска может автономно генерировать новые запросы к векторной БД, как показано на рис. 5.3.



Этот подход также распространяется на любые популярные базы данных, в том числе SQL и NoSQL, и не ограничивается только векторными базами данных.

### Собственные запросы

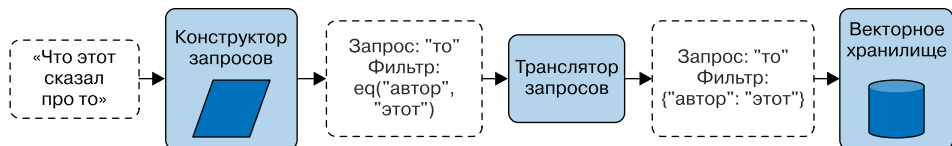


Рис. 5.3. Архитектура инструмента поиска, генерирующего собственные запросы

Собственные запросы (<https://oreil.ly/39rgU>) дают ряд существенных преимуществ.

### *Определение схемы*

Вы можете создать схему, отражающую ожидаемые описания пользователей, что позволит структурировать информацию, которую ищут пользователи.

### *Двухуровневый поиск*

Инструмент поиска выполняет операцию в два этапа. Сначала оценивает семантическое сходство между вводом пользователя и содержимым базы данных, а затем распознает и применяет фильтры на основе метаданных хранимых документов или строк, обеспечивая еще более точный и релевантный поиск.

Этот метод максимизирует потенциал инструментов поиска в обработке запросов пользователей.

Установите пакет lark командой `pip install lark`.

Ниже приводится фрагмент с инструкциями импорта важных модулей, таких как `langchain`, `lark`, `getpass` и `chroma`, которые будут использоваться в последующих примерах кода. Для простоты вывод предупреждений запрещается:

```
from langchain_core.documents import Document
from langchain_community.vectorstores.chroma import Chroma
from langchain_openai import OpenAIEmbeddings
import lark
import getpass
import os
import warnings
```

```
Запретить вывод предупреждений:
warnings.filterwarnings("ignore")
```

Следующий фрагмент кода создает список с именем `docs`, заполненный экземплярами класса `Document`. Каждый экземпляр содержит подробные сведения о книге. В словаре `metadata` мы сохраним важные сведения, такие как название книги, автор и жанр. Добавим также ISBN, название издательства и краткое описание, чтобы получить представление о каждой истории. Рейтинг `"rating"` сообщает популярность книги. Настраивая данные подобным образом, мы закладываем основу для систематического и глубокого исследования разнообразной библиотеки книг:

```
docs = [
 Document(
 page_content="A tale about a young wizard and his \
journey in a magical school.",
```

```

 metadata={
 "title": "Harry Potter and the Philosopher's Stone",
 "author": "J.K. Rowling",
 "year_published": 1997,
 "genre": "Fiction",
 "isbn": "978-0747532699",
 "publisher": "Bloomsbury",
 "language": "English",
 "page_count": 223,
 "summary": "The first book in the Harry Potter \
series where Harry discovers his magical \
heritage.",
 "rating": 4.8,
 },
),
 # ... еще документы ...
]

```

Следующий фрагмент кода импортирует `ChatOpenAI`, `SelfQueryRetriever` и `OpenAIEmbeddings`. Затем создает новую векторную базу данных, вызывая метод `Chroma.from_documents(..)`.

Далее с помощью класса `AttributeInfo` выполняется структурирование метаданных для каждой книги. Используя этот класс, мы будем систематически указывать имя атрибута, его описание и тип. Благодаря списку записей `AttributeInfo` инструмент поиска с механизмом собственных запросов сможет выполнять фильтрацию метаданных:

```

from langchain_openai.chat_models import ChatOpenAI
from langchain.retrievers.self_query.base \
 import SelfQueryRetriever
from langchain.chains.query_constructor.base \
 import AttributeInfo

Создать векторные представления и хранилище векторов:
embeddings = OpenAIEmbeddings()
vectorstore = Chroma.from_documents(docs, OpenAIEmbeddings())

Основная информация
basic_info = [
 AttributeInfo(name="title", description="The title of the book",
 type="string"),
 AttributeInfo(name="author", description="The author of the book",
 type="string"),
 AttributeInfo(
 name="year_published",
 description="The year the book was published",
 type="integer",
),
]

```

```

Подробная информация
detailed_info = [
 AttributeInfo(
 name="genre", description="The genre of the book",
 type="string or list[string]"
),
 AttributeInfo(
 name="isbn",
 description="The International Standard Book Number for the book",
 type="string",
),
 AttributeInfo(
 name="publisher",
 description="The publishing house that published the book",
 type="string",
),
 AttributeInfo(
 name="language",
 description="The primary language the book is written in",
 type="string",
),
 AttributeInfo(
 name="page_count", description="Number of pages in the book",
 type="integer"
),
]

Анализ
analysis = [
 AttributeInfo(
 name="summary",
 description="A brief summary or description of the book",
 type="string",
),
 AttributeInfo(
 name="rating",
 description="An average rating for the book (from reviews), ranging
 from 1-5",
 type="float",
),
]

Объединение всех списков в массив metadata_field_info
metadata_field_info = basic_info + detailed_info + analysis

```

Рассмотрим подробнее, что делает этот код.

1. Импортирует ChatOpenAI, SelfQueryRetriever и AttributeInfo из модулей LangChain для взаимодействий с моделью чата, выполнения собственных запросов и определения атрибутов метаданных.

2. Создает экземпляр `OpenAIEmbeddings` для работы с моделями OpenAI, генерирующими векторные представления.
3. На основе документов создает векторную базу данных `Chroma`.
4. Определяет три списка (`basic_info`, `detail_info`, `analysis`), каждый из которых содержит объекты `AttributeInfo` с разными видами метаданных, описывающих книги.
5. Объединяет эти списки в список `metadata_field_info` для комплексного управления метаданными книг.

Теперь настроим модель `ChatOpenAI` и определим переменную `document_content_description`, чтобы указать, с каким типом контента мы работаем. Затем модель и переменная передаются экземпляру `SelfQueryRetriever` для извлечения соответствующих документов из векторного хранилища. При обработке простых запросов, например, требующих вернуть научно-фантастические книги, метод `invoke` просматривает набор данных и возвращает список объектов `Document`.

Каждый объект `Document` содержит метаданные о книге, такие как жанр, автор и краткое описание, на основе которых генерируются хорошо организованные, подробные данные для нашего приложения:

```
document_content_description = "Brief summary of a movie"
llm = ChatOpenAI(temperature=0)
retriever = SelfQueryRetriever.from_llm(
 llm, vectorstore, document_content_description, metadata_field_info
)

Поиск научно-фантастических книг
retriever.invoke("What are some sci-fi books?")
[Document(page_content='''A futuristic society where firemen burn books to
maintain order.'''), metadata={'author': 'Ray Bradbury', 'genre': '...
Еще документы..., вывод усечен для экономии места
```



### Оцените качество

Устанавливая температуру равной нулю, вы сообщаете модели, что она должна отдавать приоритет созданию согласованных результатов путем фильтрации метаданных, а не выдумывать что-то нерелевантное. Эти фильтры метаданных затем применяются к векторной БД для извлечения соответствующих документов.

Чтобы получить книги определенного автора, нужно указать инструменту поиска `retriever`, что тот должен отыскивать книги, написанные Дж. К. Роулинг (J. K. Rowling). Функция `Comparison` с компаратором `EQ` (определяет равенство) гарантирует, что атрибут `'author'` полученных документов точно соответствует запросу `'J. K. Rowling'`:

```
Запросить книги, написанные Дж. К. Роулинг (J.K. Rowling):
retriever.invoke(
 '''I want some books that are published by the
 author J.K.Rowling'''
)
query=' ' filter=Comparison(comparator=<Comparator.EQ:
'eq'>, attribute='author', value='J.K. Rowling')
limit=None
Список документов опущен для экономии места
```

Инициализация `SelfQueryRetriever` дополнительным флагом `enable_limit=True` позволит определить количество выдаваемых результатов. В таком случае можно создать запрос, возвращающий точно две книги в жанре фэнтези. Используя функцию `Comparison` с компаратором `EQ` (определяет равенство) для атрибута `'genre'`, программа поиска концентрируется на заголовках `'Fantasy'`. Параметр `limit` гарантирует, что поиск вернет точно два результата:

```
retriever = SelfQueryRetriever.from_llm(
 llm,
 vectorstore,
 document_content_description,
 metadata_field_info,
 enable_limit=True,
)

retriever.get_relevant_documents(
 query="Return 2 Fantasy books",
)
query=' ' filter=Comparison(
comparator=<Comparator.EQ: 'eq'>, attribute='genre',
value='Fantasy') limit=2
Список документов опущен для экономии места
```

## Альтернативные механизмы извлечения

Работая над реализацией извлечения, вы можете использовать ряд интересных методов, применяющих разные подходы и обладающих своими достоинствами и недостатками.

### *MultiQueryRetriever*

Цель `MultiQueryRetriever` (<https://oreil.ly/uuzpG>) — преодоление ограничений поиска на основе расстояния путем создания нескольких запросов на основе запроса, введенного пользователем. В результате получается большой набор потенциально релевантных документов, соответствующих

более широкой интерпретации исходного запроса. Однако иногда могут возникнуть проблемы, если разные запросы дают противоречивые или перекрывающиеся результаты.

### *ContextualCompressionRetriever*

`ContextualCompressionRetriever` (<https://oreil.ly/wzqVg>) сжимает нерелевантные части длинных документов, обеспечивая соответствие контексту. Основная проблема этого метода — потребность в экспертных знаниях для определения релевантности и важности информации.

### *EnsembleRetriever*

`EnsembleRetriever` (<https://oreil.ly/jIuJh>) использует список инструментов поиска и объединяет возвращаемые ими результаты. По сути, этот инструмент реализует «гибридный» метод поиска, использующий сильные стороны различных алгоритмов. Однако `EnsembleRetriever` требует значительных вычислительных ресурсов из-за использования нескольких алгоритмов поиска и может показывать невысокую скорость поиска.

### *ParentDocumentRetriever*

`ParentDocumentRetriever` (<https://oreil.ly/jXSXQ>) обрабатывает документы, сначала извлекая из них небольшие фрагменты, а затем выбирает идентификаторы родительских документов, которым принадлежат эти фрагменты, и возвращает более крупные документы. Однако он может потребовать значительных вычислительных ресурсов из-за поиска более крупных родительских документов.

### *TimeWeightedVectorStoreRetriever*

`TimeWeightedVectorStoreRetriever` (<https://oreil.ly/9JbTt>) дополнительно учитывает *время затухания* при поиске документа. Несмотря на свои преимущества, влияние времени затухания может привести к пропуску релевантных старых документов и потере исторических данных.

Для реализации эффективного поиска важно понимать имеющиеся компромиссы и выбирать методы или их комбинации, которые лучше всего подходят для конкретного случая. Векторный поиск увеличивает финансовые затраты и время отклика приложения, поэтому в процессе тестирования убедитесь, что дополнительный контекст того стоит. При больших рабочих нагрузках первоначальные затраты на точную настройку пользовательской модели могут оказаться намного меньше постоянных затрат на выполнение запросов, создание векторных представлений и использование векторного хранилища. В некоторых случаях передача статических примеров в командной строке может показывать хорошие результаты. Однако, когда нужно динамически подтягивать контекст

к запросу, основываясь на сходстве, а не на прямом поиске по ключевым словам, метод генерации с дополненной выборкой (RAG) с использованием векторной базы данных не имеет реальной альтернативы.

## Резюме

В этой главе вы узнали о применении векторных баз данных для хранения и поиска текста на основе сходства. Благодаря своей способности отыскивать похожие записи векторные базы данных могут извлекать информацию для включения в запросы в качестве контекста и помогать моделям ИИ оставаться в пределах контекстного окна, сокращать затраты и избегать генерирования нерелевантных данных. Вы также увидели, что точность векторов зависит от базовой модели, и познакомились с примерами, когда они могут терпеть неудачу.

Вы изучили процесс индексации документов в векторной базе данных, поиска похожих записей с использованием векторов и вставки записей в промты в качестве контекста, так называемый RAG. Увидели примеры получения эмбедингов, или векторных представлений, как с помощью OpenAI API, так и с применением моделей с открытым исходным кодом, в частности из библиотеки Sentence Transformers. Познакомились с достоинствами и недостатками способа получения эмбедингов с помощью OpenAI и с некоторыми альтернативными вариантами.

В следующей главе, посвященной автономным агентам, вы попадете в футуристический мир агентов ИИ, которые могут принимать решения и действовать самостоятельно. Вы познакомитесь с разными типами автономных агентов, их возможностями и способами их обучения для решения конкретных задач. Кроме того, вы изучите сложности, связанные с агентами, и проблемы ненадежности.



# Автономные агенты с памятью и инструментами

В этой главе мы поговорим о важности цепочки рассуждений и способности больших языковых моделей (LLM) решать сложные задачи в роли агентов. Разбивая сложные задачи на более простые подзадачи, LLM может отыскивать более точные и эффективные решения. Далее вы также познакомитесь с компонентами, из которых состоят автономные агенты, такими как входные данные, целевые функции и функции вознаграждения и доступные действия.

## Метод рассуждения с цепочкой мыслей

Способность ИИ рассуждать по поводу сложных проблем является существенной для создания эффективных, надежных и удобных приложений.

*Метод рассуждения с цепочкой мыслей* (Chain-of-Thought, CoT) — это метод направления LLM, чтобы пройти ряд шагов или цепочку логических выводов, чтобы прийти к решению задачи. Этот подход особенно полезен для задач, требующих более глубокого понимания контекста или учета множества факторов.

CoT (<https://oreil.ly/fAeLo>) предполагает, что LLM будет обдумывать сложные задачи, разбивая их на более мелкие и более управляемые части. Это позволяет LLM сосредоточиться на каждой части отдельно, чтобы получить более глубокое ее понимание.

На практике метод рассуждения с цепочкой мыслей, или цепочка рассуждений, может включать:

- требование к LLM предоставить объяснения своих решений;
- планирование нескольких шагов перед принятием окончательного решения.

В следующих разделах мы исследуем примеры эффективных и неэффективных цепочек рассуждений, а также обсудим разные методы построения эффективных цепочек и способы их интеграции в приложения ИИ.

Представим, что пользователь хочет получить от ИИ комплексный маркетинговый план продвижения нового программного продукта.

Ввод:

Создай маркетинговый план для продвижения нового программного продукта.

Вывод:

1. Запустить кампанию в социальных сетях
2. Разослать информационные бюллетени по электронной почте
3. Предложить ограниченные по времени скидки
4. Вступить в сотрудничество с влиятельными лицами
5. Организовать вебинары

В этом примере GPT-4 не использует цепочку рассуждений и не раскрывает конкретные аспекты маркетингового плана, а просто возвращает общий список маркетинговых стратегий, которые можно применить к любому продукту, независимо от уникальных характеристик нового программного продукта.

Ввод:

Создай подробный маркетинговый план для нового программного продукта, предназначенного для управления проектами и ориентированного на малый и средний бизнес.

Бюджет ограничен 20 000 долларов США, и желательно сосредоточиться на каналах цифрового маркетинга.

Прими во внимание успех аналогичных продуктов на рынке и представь пошаговый план, включающий наиболее эффективную тактику продвижения.

Вывод:

#### 1. Анализ рынка

- Целевая аудитория:
- Малые и средние предприятия (до 250 сотрудников).
- Отрасли: IT, маркетинг, строительство, образование и здравоохранение.
- Лица, принимающие решения: владельцы бизнеса, менеджеры проектов, команды разработки.
- Конкуренты:
- Изучите успешные продукты, такие как Trello, Asana, Monday.com и ClickUp.
- Анализируйте их функциональность, ценовые модели, отзывы пользователей и маркетинговые стратегии.

...

#### 2. Цели и задачи

...

#### 3. Уникальное торговое предложение (УТП)

...

## 4. Стратегии продвижения

...

## 5. Бюджет

...

## 6. Метрики успеха

Количество регистраций пользователей.

Уровень удержания клиентов.

Обратная связь

На этот раз GPT-4 приняла во внимание особенности нового программного продукта и продемонстрировала способность к логическим выводам.

**Задайте направление**

Обратите внимание на фразу «пошаговый план», важнейший элемент цепочки рассуждений. Включив эту фразу в запрос, вы требуете от LLM, чтобы она обдумала шаги по продвижению высокоэффективного программного продукта.

Кроме того, узнав тип программного продукта и что сумма бюджета ограничена 20 000 долларов, GPT-4 смогла представить более релевантный и подробный ответ.

## Агенты

Появление генеративных моделей ИИ привело к созданию *архитектуры на основе агентов*. Концептуально агент действует и принимает решения в некоторой среде, стремясь достигнуть заранее определенных целей.

Агенты могут выполнять различные действия, например вызывать функции Python; наблюдать за происходящим и принимать решение о достижении цели или выбирать действие, которое нужно предпринять далее.

Агент непрерывно выполняет серию действий и наблюдений, пока доступны для выбора дальнейшие действия, как показано в следующем псевдокоде:

```
next_action = agent.get_action(...)
while next_action != AgentFinish:
 observation = run(next_action)
 next_action = agent.get_action(..., next_action, observation)
return next_action
```

Поведение агента определяется тремя основными компонентами.

*Входы*

Это сенсорные стимулы или данные, которые агент получает из окружающей среды. Входы могут быть самыми разными: от визуальных (например, изображения) и слуховых (например, аудиофайлы) до тепловых сигналов и т. д.

### *Целевая функция, или функция вознаграждения*

Представляет руководящий принцип действий агента. В рамках очерченных целей агент стремится решить задачу достижения определенного конечного состояния. В условиях, основанных на вознаграждении, агент старается максимизировать совокупное вознаграждение в динамических средах.

### *Доступные действия*

*Пространство действий* — это диапазон допустимых действий, которые агент может предпринять в любой момент (<https://oreil.ly/5AVfM>). Ширина и характер этого пространства зависят от поставленной задачи.

Для более подробного объяснения этих концепций рассмотрим пример беспилотного автомобиля.

### *Входы*

Датчики автомобиля, такие как камеры, лидары и т. п., генерирующие непрерывный поток данных об окружающей среде, в том числе о близлежащих транспортных средствах, пешеходах, дорожных условиях и сигналах светофора.

### *Целевая функция, или функция вознаграждения*

Основная цель беспилотного автомобиля — безопасно и эффективно добраться из точки А в точку Б. При использовании системы, основанной на вознаграждении, автомобиль будет получать положительные вознаграждения, если будет стараться держаться на безопасном расстоянии от других объектов, соблюдать скоростной режим и правила дорожного движения. И наоборот, он будет получать штрафы за рискованное поведение, такое как резкое торможение или выезд за пределы полосы движения. Компания Tesla, в частности, использует в качестве функции вознаграждения мили, пройденные без вмешательства человека.

### *Доступные действия*

Пространство доступных действий для автомобиля включает ускорение, торможение, поворот, смену полосы движения и многое другое. Каждое действие выбирается на основе текущих входных данных и целевой функции или функции вознаграждения.

Агенты в таких системах, как беспилотные автомобили, опираются на основополагающие принципы, такие как входы, целевые функции и функции вознаграждения, а также доступные действия. Однако, углубляясь в сферу LLM, таких как GPT, можно обнаружить особый набор динамических характеристик, соответствующих их уникальной природе.

Вот как они отвечают нашим потребностям.

### *Входы*

Взаимодействия с LLM в основном осуществляются посредством текстовых сообщений. Но это не ограничивает богатство информации, которую можно использовать. Любые имеющиеся у вас данные — показания температуры, нотную запись мелодий или сложные структуры данных — вы должны превратить в текстовые представления, которые сможет проанализировать LLM. Возьмем для примера видео: LLM не сможет проанализировать исходные кадры, но текстовое описание поможет ей извлечь ценную для вас информацию.

### *Применение директив, ориентированных на достижение цели*

LLM в первую очередь учитывает цели, определенные в текстовых запросах. Создавая эффективные запросы с целями, вы не просто получаете доступ к обширным знаниям LLM, но фактически намечаете путь ее рассуждений. Представляйте себе это как составление плана: ваш конкретный запрос инструктирует модель разбить всеобъемлющую цель на систематическую последовательность шагов.

### *Создание действий с помощью функциональных инструментов*

LLM не ограничиваются простым генерированием текста и позволяют достичь гораздо большего. Интегрируя готовые инструменты или создавая свои инструменты, можно предоставить LLM возможность выполнять самые разные действия: от вызовов API до взаимодействия с базами данных и даже управления внешними системами. Инструменты могут быть написаны на любом языке программирования, и, добавляя больше инструментов, можно расширять пространство действий, доступных для языковой модели.

Существуют также различные компоненты, которые напрямую применимы к LLM.

### *Память*

В идеале желательно сохранять состояние агента между шагами. Это особенно полезно для чат-ботов, где запоминание предыдущей истории чата обеспечивает лучший пользовательский опыт.

### *Стратегии планирования/выполнения для агентов*

Существует множество способов достижения высокоуровневых целей, при этом сочетание планирования и выполнения является существенным.

## Поиск

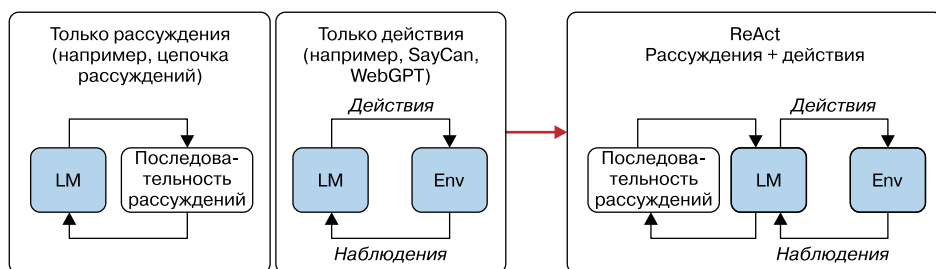
Модели LLM могут использовать различные типы поиска. Наиболее распространенным является поиск в векторных базах данных по семантическому сходству, но есть и другие виды поиска, например включение в запросы информации из базы данных SQL.

Рассмотрим подробнее наиболее типичные компоненты и изучим детали их реализации.

## Рассуждения и действие (Reason and Act, ReAct)

Существует множество паттернов проектирования агентов, которые направлены на подталкивание LLM как можно ближе к цели. Одним из первых таких паттернов был *ReAct* — улучшенная версия CoT, позволяющая модели LLM создавать наблюдения после выполнения действий с помощью инструментов. Эти наблюдения затем превращаются в *размышления* о том, *какой инструмент* следует использовать на следующем этапе (рис. 6.1).

LLM продолжает рассуждать до тех пор, пока не появится строковое значение 'Окончательный ответ' или не будет выполнено максимальное количество итераций.



**Рис. 6.1.** Фреймворк ReAct

Фреймворк ReAct (<https://oreil.ly/ssdnL>) использует сочетание декомпозиции задач, цикла размышлений и нескольких инструментов для разрешения вопросов. Давайте исследуем организацию цикла размышлений в ReAct.

1. Наблюдение за окружающей средой.
2. Интерпретация наблюдений путем размышлений.
3. Определение действия.
4. Выполнение действия в окружающей среде.

- Повторять шаги 1–4, пока не будет найдено решение или пока не выполнится достаточное число итераций.

Вы можете без труда создать запрос в стиле ReAct, используя предыдущий цикл размышлений, а также предоставив LLM некоторые входные данные, такие как:

- `{question}`: вопрос, на который вы хотите получить ответ;
- `{tools}`: функции, которые можно использовать для выполнения очередного шага в процессе решения общей задачи. Обычной практикой является включение списка инструментов, каждый из которых представляет собой функцию на Python — имя функции и описание ее назначения.

Ниже приводится промт, реализующий паттерн ReAct с переменными, заключенными в фигурные скобки {}, например `{question}`:

Попытайся решить задачу поиска ответа на вопрос.

Для решения используй такую цепочку рассуждений:

1. Рассмотрим исходный вопрос:

`original_question: original_problem_text`

2. Создай наблюдение по следующему шаблону:

`observation: observation_text`

3. Сделай логический вывод, исходя из наблюдения по следующему шаблону:

`thought: thought_text`

4. Используй инструменты для выполнения действий на основе рассуждений по следующему шаблону:

`action: tool_name`

`action_input: tool_input`

Не надо придумывать или предполагать результаты инструмента, а предоставь структурированный вывод, включающий `action` и `action_input`.

Тебе доступны следующие инструменты: `{tools}`.

`original_problem: {question}`

Основываясь на результате применения инструмента:

Представь следующее наблюдение, действие, входные данные для действия или окончательный ответ, если доступен.

Возвращая окончательный ответ, верни его с применением следующего шаблона:

`"Я нашел ответ: final_answer"`

Рассмотрим поближе структуру запроса.

- Вводная часть запроса четко определяет цель LLM: Попытайся решить задачу поиска ответа на вопрос.
- Далее описывается подход к решению задачи: Для решения используй такую цепочку рассуждений:

3. Затем перечисляются этапы цепочки рассуждений:
  - сначала LLM должна рассмотреть исходный вопрос и сформулировать сделанное: `original_question: original_problem_text, observation: observation_text`;
  - основываясь на этом наблюдении, ИИ должен сформулировать логический вывод, обозначающий шаг в процессе рассуждений: `thought: thought_text`;
  - после формулировки логического вывода модель должна выбрать действие, используя один из доступных инструментов: `action: tool_name, action_input: tool_input`.
4. Затем модели LLM дается напоминание, что она не должна делать предположений о том, какие результаты может вернуть инструмент, и должна четко обозначить действие и соответствующие входные данные.
5. Текст *Тебе доступны следующие инструменты: {tools}* сообщает LLM, какие инструменты ей доступны для решения задачи.
6. Затем описывается сама задача, которую должна решить LLM: `original_problem: {question}`.
7. В заключение модели LLM даются инструкции о том, как реагировать на результаты действий. Она может либо продолжить делать новые наблюдения, выполнять действия и определять входные данные для этих действий, либо, если решение найдено, представить окончательный ответ.

В запросе описывается систематический процесс решения задач, в ходе которого LLM рассматривает задачу, размышляет о ней, принимает решение о действии и повторяет этот процесс до тех пор, пока не будет найдено решение.

## Реализация паттерна ReAct

Теперь, познакомившись с паттерном ReAct, давайте напишем простую его реализацию на Python, которая повторяет то, что LangChain делает автоматически. Это позволит вам понять, что на самом деле происходит между ответами LLM.

Для простоты мы не будем реализовывать цикл и предположим, что результат можно получить за один вызов инструмента.

Простая реализация ReAct должна делать следующее.

1. На каждом этапе любого рассуждения извлечь инструмент по желанию LLM для последующего использования. Поэтому мы должны извлечь последние элементы `action` и `action_input`. Действие `action` определяет имя инструмента, а `action_input` — его аргументы.
2. Проверить, считает ли LLM, что ей удалось найти окончательный ответ, и в этом случае цикл рассуждений завершается.



Для извлечения значений `action` и `action_input` из ответа LLM можно использовать регулярные выражения:

```
import re

Образец текста:
text = """
action: search_on_google
action_input: Как зовут жену Тома Хэнкса

action: search_on_wikipedia
action_input: Сколько лет было Рите Уилсон в 2023

action: search_on_google
action_input: какой-то другой вопрос
"""

Скомпилировать регулярные выражения:
action_pattern = re.compile(r"(?i)action\s*:\s*([^\n]+)", re.MULTILINE)
action_input_pattern = re.compile(r"(?i)action\s*_input\s*:\s*([^\n]+)",
re.MULTILINE)

Найти все вхождения action и action_input:
actions = action_pattern.findall(text)
action_inputs = action_input_pattern.findall(text)

Извлечь последнее вхождение action и action_input:
last_action = actions[-1] if actions else None
last_action_input = action_inputs[-1] if action_inputs else None
print("Last Action:", last_action)
print("Last Action Input:", last_action_input)
Last Action: search_on_google
Last Action Input: какой-то другой вопрос
```

Разберем подробно, как работает регулярное выражение, извлекающее действие `action`:

- `action_pattern = re.compile(r"(?i)action\s*:\s*([^\n]+)", re.MULTILINE);`
- `(?i)` — это так называемый *встроенный флаг*, который включает нечувствительность регулярного выражения к регистру символов. Это означает, что шаблон будет соответствовать последовательностям символом *action*, *Action*, *ACTION* и любой другой комбинации из прописных и строчных букв;
- `action` — эта часть шаблона буквально соответствует слову «action». Благодаря флагу, включающему нечувствительность к регистру символов, эта часть шаблона будет соответствовать любому написанию слова «action» с применением прописных и строчных букв;
- `\s*` — эта часть шаблона соответствует произвольному количеству (ноль и более) пробельных символов (пробелов, табуляций и т. д.). Символ `*` означает «ноль или более», а `\s` — это сокращенная запись шаблона, соответствующего пробельным символам;

- `:` — эта часть шаблона буквально соответствует символу двоеточия;
- `\s*` — как и предыдущая часть `\s*`, соответствует произвольному количеству (ноль и более) пробельных символов, следующих за двоеточием;
- `+[^\n]++` — это сохраняющая группа, соответствующая одному или нескольким символам, которые не являются символом перевода строки. Символ `^` внутри квадратных скобок `[]` означает отрицание, а `\n` — символ перевода строки. `+` означает «один и более». Текст, соответствующий этой группе, можно будет извлечь с помощью функции `findall()`;
- `re.MULTILINE` — это флаг, передаваемый в вызов функции `re.compile()`. Он сообщает механизму регулярных выражений, что входной текст может состоять из нескольких строк, поэтому шаблон следует применять построчно;
- в регулярных выражениях квадратные скобки `[]` используются для определения класса символов — набора символов, которым он должен соответствовать. Например, `[abc]` будет соответствовать любому одиночному символу «a», «b» или «c»;
- символ крышки `^` в начале класса символов отрицает класс символов, то есть такой класс будет соответствовать любым символам, не перечисленным в классе, а значит, шаблон `[^abc]` будет соответствовать любому отдельному символу, кроме «a», «b» или «c». В шаблоне `+[^\n]++` указан класс символов — `[^\n]`, то есть этот шаблон будет соответствовать любому символу, который не является символом перевода строки (`\n`). Знак `+` после класса означает, что шаблон должен соответствовать одному или нескольким символам, которые не являются символом перевода строки;
- используя класс символов с отрицанием `[^\n]` в сохраняющей группе, мы гарантируем, что механизм регулярных выражений сохранит текст до конца строки, не включая символ перевода строки. Это может пригодиться для извлечения текста после слова *action* или *action\_input* до конца строки.

В целом этот шаблон регулярного выражения соответствует слову *action* (без учета регистра), за которым следуют необязательные пробелы, двоеточие и опять необязательные пробелы и некоторый текст до конца строки, который захватывается сохраняющей группой.

Единственная разница между этими двумя шаблонами регулярных выражений — буквальный текст, который они ищут в начале.

1. `action_pattern` ищет слово *action*.
2. `action_input_pattern` ищет слово *action\_input*.

Теперь можно абстрагировать регулярное выражение за фасадом функции Python, которая отыскивает и возвращает последние найденные элементы `action` и `action_input`:

```
def extract_last_action_and_input(text):
 # Скомпилировать регулярные выражения
 action_pattern = re.compile(r"(?i)action\s*:\s*([^\n]+)", re.MULTILINE)
 action_input_pattern = re.compile(
 r"(?i)action\s*_input\s*:\s*([^\n]+)", re.MULTILINE
)

 # Найти все вхождения элементов action и action_input
 actions = action_pattern.findall(text)
 action_inputs = action_input_pattern.findall(text)

 # извлечь последнее вхождение action и action_input
 last_action = actions[-1] if actions else None
 last_action_input = action_inputs[-1] if action_inputs else None

 return {"action": last_action, "action_input": last_action_input}

extract_last_action_and_input(text)
{'action': 'search_on_google', 'action_input': 'какой-то другой вопрос'}
```

Чтобы определить, нашла ли LLM окончательный ответ, и извлечь его, тоже можно использовать регулярные выражения:

```
def extract_final_answer(text):
 final_answer_pattern = re.compile(
 r"(?i)Я нашел ответ:\s*([^\n]+)", re.MULTILINE
)
 final_answers = final_answer_pattern.findall(text)
 if final_answers:
 return final_answers[0]
 else:
 return None

final_answer_text = "Я нашел ответ: final_answer"
print(extract_final_answer(final_answer_text))
final_answer
```



LLM не всегда реагируют на запросы ожидаемым образом, поэтому приложение должно обрабатывать ошибки анализа регулярных выражений. Как один из подходов — использовать LLM для исправления предыдущего ответа LLM или для создания еще одного запроса к LLM с предыдущим состоянием.

Теперь объединим все компоненты, действуя шаг за шагом:

```
from langchain_openai.chat_models import ChatOpenAI
from langchain.prompts.chat import SystemMessagePromptTemplate
```

Инициализируем экземпляр ChatOpenAI:

```
chat = ChatOpenAI(model_kwargs={"stop": ["tool_result:],})
```

Добавление последовательности `stop` заставляет LLM прекратить генерировать новые токены после обнаружения фразы `"tool_result:"`. Это помогает остановить галлюцинации по поводу использования инструментов.

Определим доступные инструменты:

```
tools = {}

def search_on_google(query: str):
 return f"У Джейсона Деруло нет жены или партнера."

tools["search_on_google"] = {
 "function": search_on_google,
 "description": "Ищет ответ на вопрос в Google",
}
```

Определим базовый шаблон запроса:

```
base_prompt = """
Попытайся решить задачу поиска ответа на вопрос.
Для решения используй такую цепочку рассуждений:
```

```
1. Рассмотрим исходный вопрос:
original_question: original_problem_text
2. Создай наблюдение по следующему шаблону:
observation: observation_text
3. Сделай логический вывод, исходя из наблюдения, по следующему шаблону:
thought: thought_text
4. Используй инструменты для выполнения действий на основе рассуждений
по следующему шаблону:
action: tool_name
action_input: tool_input
```

Не надо придумывать или предполагать результаты инструмента, а предоставь структурированный вывод, включающий `action` и `action_input`.

Тебе доступны следующие инструменты: {tools}.

```
original_problem: {question}
"""
```

Сгенерируем вывод модели:

```
output = chat.invoke(SystemMessagePromptTemplate \
 .from_template(template=base_prompt) \
 .format_messages(tools=tools, question="У Джейсона Деруло есть партнер?"))
print(output)
```

Извлечем последние элементы `action` и `action_input` и вызовем соответствующую функцию:

```
tool_name = extract_last_action_and_input(output.content)["action"]
tool_input = extract_last_action_and_input(output.content)["action_input"]
tool_result = tools[tool_name]["function"](tool_input)
```

Выведем информацию об инструменте:

```
print(f""Агент решил использовать следующий инструмент:
tool_name: {tool_name}
tool_input: {tool_input}
tool_result: {tool_result}""
)
```

Определить текущий запрос с результатом, полученным от инструмента:

```
current_prompt = ""
Ты ответил на вопрос: У Джейсона Деруло есть партнер?
```

Основываясь на результате применения инструмента:

```
tool_result: {tool_result}
```

Представь следующее наблюдение, действие, входные данные для действия или окончательный ответ, если доступен. Возвращая окончательный ответ, верни его с применением следующего шаблона: "Я нашел ответ: `final_answer`"

```
""
```

Сгенерируем вывод модели для текущего запроса:

```
output = chat.invoke(SystemMessagePromptTemplate. \
 from_template(template=current_prompt) \
 .format_messages(tool_result=tool_result))
```

Выведем ответ модели на текущий запрос:

```
print("-----\n\nВывод модели:", output.content)
final_answer = extract_final_answer(output.content)
if final_answer:
 print(f"Ответ: {final_answer}")
else:
 print("Окончательный ответ не найден.")
```

Вывод:

```
'''content='1. Рассмотрим исходный вопрос:\nУ Джейсона Деруло есть
партнер?\n\n2. Создай наблюдение:\nУ нас нет никакой информации
об отношениях Джейсона Деруло с кем-либо.\n\n3. Сделай логический вывод,
исходя из наблюдения:\nМы можем поискать последние новости или интервью,
чтобы узнать, связан ли Джейсон Деруло отношениями с кем-нибудь в настоящее
время.\n\n4. Используй инструменты для выполнения действий на основе
рассуждений:\n action: search_on_google\naction_input: "текущие отношения
Джейсона Деруло"' additional_kwargs={} example=False
```

-----

Агент решил использовать следующий инструмент:

```
tool_name: search_on_google
```

```
tool_input: "текущие отношения Джейсона Деруло"
```

```
tool_result: У Джейсона Деруло нет жены или партнера.
```

-----

Второй запрос показывает

Основываясь на результате применения инструмента:

```
tool_result: {tool_result}
```

Представь следующее наблюдение, действие, входные данные для действия или окончательный ответ, если доступен. Возвращая окончательный ответ, верни его с применением следующего шаблона: "Я нашел ответ: final\_answer"

Вывод модели: Я нашел ответ: У Джейсона Деруло нет жены или партнера.

Ответ: у Джейсона Деруло нет жены или партнера'''.

Предыдущие шаги представляют очень простую реализацию паттерна ReAct. В данном случае LLM решил использовать инструмент `search_on_google` с текстом "текущие отношения Джейсона Деруло" в качестве `action_input`.



Агенты LangChain автоматически выполняют все предыдущие шаги, а также обеспечат использование нескольких инструментов (в цикле) и обработку их сбоев, когда у агента не получится проанализировать `action` или `action_input`.

Прежде чем переходить к знакомству с агентами LangChain и их возможностями, важно понять идею инструментов, научиться их создавать и использовать.

## Использование инструментов

Большие языковые модели, такие как GPT-4, могут генерировать только текст, поэтому предоставление им инструментов, помогающих выполнять другие действия, такие как взаимодействие с базой данных или чтение/запись файлов, позволяет эффективно расширить возможности LLM. *Инструмент* — это всего

лишь предопределенная функция, помогающая агенту выполнить определенное действие.

Общая часть запроса агента, скорее всего, будет включать следующие строки:

```
Твоя цель: {goal}
Тебе доступны следующие инструменты: {tools}
```

Большинство инструментов реализуются как функции на том или ином языке программирования. Изучая фреймворк LangChain, вы обнаружите, что он предлагает три подхода к созданию/использованию инструментов:

- создание собственных инструментов;
- использование существующих инструментов;
- использование **AgentToolkits** — набора инструментов, предназначенных для выполнения конкретной задачи.

Начнем с создания своего инструмента, который проверяет длину строки с помощью LangChain:

```
Импортировать необходимые классы и функции:
from langchain.agents import AgentExecutor, create_react_agent
from langchain import hub
from langchain_openai import ChatOpenAI
from langchain.tools import Tool

Определить используемую LLM:
model = ChatOpenAI()

Функция для подсчета числа символов в строке:
def count_characters_in_string(string):
 return len(string)

Создать список инструментов:
Сейчас имеется только один инструмент, подсчитывающий символы в строке.
tools = [
 Tool.from_function(
 func=count_characters_in_string,
 name="Count Characters in a text string",
 description="Count the number of characters in a text string",
)
]

Загрузить запрос react!
prompt = hub.pull("hwchase17/react")

Сконструировать агента ReAct:
agent = create_react_agent(model, tools, prompt)
```

```
Создать механизм выполнения и передать ему агента и набор инструментов:
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

вызвать агента с запросом, требующим подсчитать символы в заданном слове:
agent_executor.invoke({"input": "'How many characters are in the word
"supercalifragilisticexpialidocious"?'"})

'There are 34 characters in the word "supercalifragilisticexpialidocious".'
```

После импорта необходимых модулей инициализируется модель чата ChatOpenAI. Затем создается функция `count_characters_in_string`, которая определяет длину любой заданной строки. Эта функция инкапсулируется в объект `Tool` с описательным именем и пояснением ее роли.

Далее с помощью `create_react_agent` создается агент на основе набора инструментов `Tools`, модели ChatOpenAI и запроса `react`, полученного из хаба LangChain. В результате получается комплексный интерактивный агент.

Применением `AgentExecutor` агент оснащается инструментами и для него включается режим подробного журналирования.

В заключение вызывается `agent_executor.invoke(...)` с запросом, требующим подсчитать количество символов в слове `'supercalifragilisticexpialidocious'`. Агент использует определенный инструмент и возвращает точный результат.

В примере 6.1 можно видеть, что агент решил использовать действие `Action: Count Characters in a text string` с входными данными `Action Input: 'supercalifragilisticexpialidocious'`. Этот шаблон очень похож на упрощенную реализацию ReAct, созданную нами выше.

### Пример 6.1. Вывод агента с единственным инструментом

```
Entering new AgentExecutor change...
I should count the number of characters in the word
"supercalifragilisticexpialadocious".
Action: Count Characters in a text string
Action Input: "supercalifragilisticexpialadocious"
Observation: 34
Thought: I now know the final answer
Final Answer: There are 34 characters in the word
"supercalifragilisticexpialadocious".
```



#### Задайте направление

Выразительные названия для ваших функций Python и описания инструментов повысят способность LLM эффективно выбирать правильные инструменты.



## Использование LLM в качестве API (функции OpenAI)

Как упоминалось в главе 4, OpenAI выпустила дообученные модели LLM (<https://openai.com/hyTus>), адаптированные для вызова функций. Они составляют альтернативу стандартному паттерну ReAct использования инструментов. Подход с применением дообученной модели LLM похож на ReAct в том смысле, что в нем тоже используется LLM в роли механизма рассуждений.

Как показано на рис. 6.2, возможность вызова функций позволяет LLM легко преобразовать запрос пользователя в вызов API сервиса прогнозирования погоды.

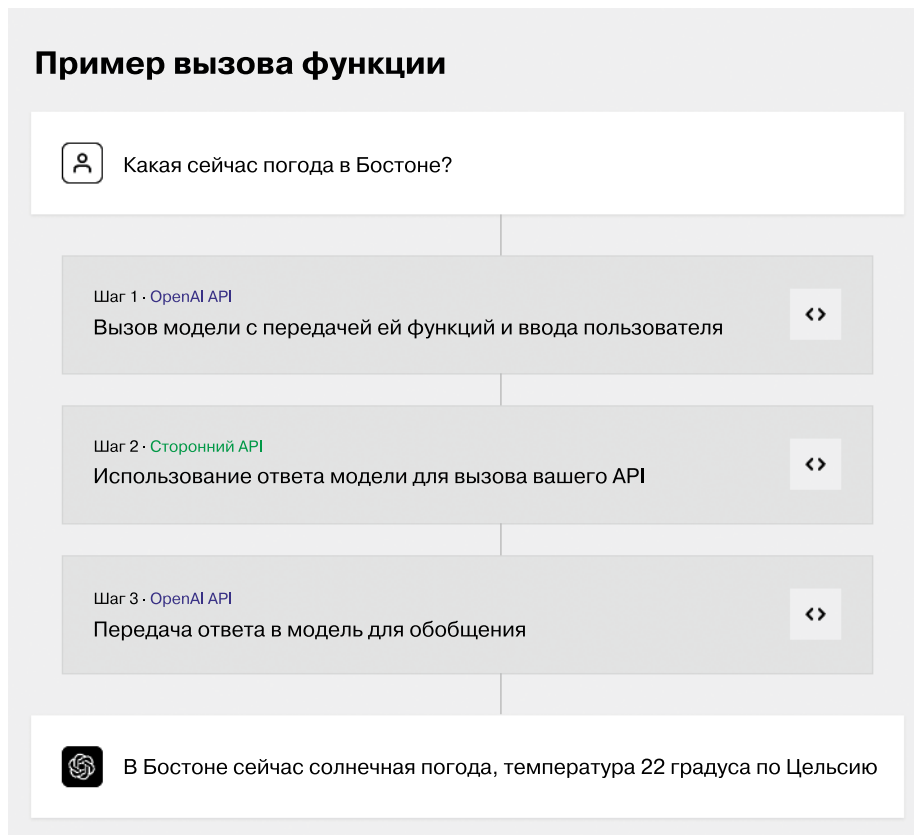


Рис. 6.2. Процесс вызова функций с использованием функций OpenAI

LangChain позволяет легко переключаться между агентами разных типов, таких как ReAct, функции OpenAI и многие другие.

В табл. 6.1 приводится довольно полное сравнение различных типов агентов.

Таблица 6.1. Сравнение типов агентов

Тип агента	Описание
Функции OpenAI	Работает с дообученными моделями, такими как gpt-3.5-turbo-0613 и gpt-4-0613, адаптированными для вызова функций. Агенты этого типа выводят объекты JSON, описывающие вызовы функций, и лучше всего подходят для работы с моделями с открытым исходным кодом и сторонними API, использующими этот формат. Примечание: считается устаревшим после появления типа «Инструменты OpenAI»
Инструменты OpenAI	Расширенная версия для использования в комплексе с новейшими моделями, способная вызывать одну или несколько функций. Агенты этого типа выводят объекты JSON, описывающие вызовы функций, оптимизируют ответ и в некоторых архитектурах сокращают время получения ответа
Агент XML	Идеально подходит для таких языковых моделей, как Anthropic Claude, которые превосходно справляются с рассуждениями и генерированием XML. Агенты этого типа лучше всего подходят для работы с обычными LLM (не с моделями чата) и не-структурированными инструментами, принимающими единственную строку
Агент чата JSON	Специально разработан для языковых моделей, поддерживающих вывод в формате JSON, и сценариев использования, требующих вывода в формате JSON
Структурированный чат	Агенты этого типа способны использовать инструменты с несколькими входами и предназначены для решения сложных задач со структурированными входными данными и ответами
ReAct	Реализует логику ReAct, используя такие инструменты, как Tavily Search, для взаимодействия с хранилищем документов или поисковыми механизмами
Собственные запросы с поиском	Использует инструменты для получения промежуточных ответов в процессе поиска фактического решения в соответствии с методологией собственных запросов с поиском. Агенты этого типа лучше всего подходят для сценариев, где требуется быстро находить точные ответы

Воспользуемся готовыми инструментами, такими как Calculator, и реализуем поиск ответов на математические вопросы, используя вызов функций OpenAI, как описано в документации LangChain:

```
Импортировать необходимые модули и функции из пакета langchain:
from langchain.chains import (
 LLMMathChain,
)
```

```
from langchain import hub
from langchain.agents import create_openai_functions_agent, Tool, AgentExecutor
from langchain_openai.chat_models import ChatOpenAI

Инициализировать ChatOpenAI температурой 0:
model = ChatOpenAI(temperature=0)

Создать экземпляр LLMMathChain на основе модели ChatOpenAI:
llm_math_chain = LLMMathChain.from_llm(llm=model, verbose=True)

Загрузить запрос с хаба:
prompt = hub.pull("hwchase17/openai-functions-agent")

tools = [
 Tool(
 name="Calculator",
 func=llm_math_chain.run, # запускает LLMMathChain
 description="useful for when you need to answer questions about math",
 return_direct=True,
),
]

Создать агента на основе модели ChatOpenAI и набора инструментов tools:
agent = create_openai_functions_agent(llm=model, tools=tools, prompt=prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
result = agent_executor.invoke({"input": "What is 5 + 5?"})
print(result)
{'input': 'What is 5 + 5?', 'output': 'Answer: 10'}
```

После импортирования необходимых библиотек инициализируется экземпляр модели ChatOpenAI с параметром `temperature=0`, чтобы гарантировать получение детерминированного вывода. Используя `hub.pull("...")`, можно загрузить запросы, хранимые на LangChainHub.

Затем модель объединяется с инструментом под названием `Calculator`, который использует возможности `LLMMathChain` для вычисления математических выражений. Далее агент функций OpenAI решает использовать инструмент `Calculator` для вычисления выражения `5 + 5` и возвращает ответ `Answer: 10`.

Теперь можно оснастить агента несколькими инструментами, повысив его универсальность. Для проверки добавим в нашего агента дополнительный объект `Tool`, который позволит ему выполнять фиктивный поиск в Google:

```
def google_search(query: str) -> str:
 return "James Phoenix is 31 years old."

Список инструментов, доступных агенту:
tools = [
 Tool(
 # Инструмент LLMMathChain для математических вычислений.
 func=llm_math_chain.run,
```

```

 name="Calculator",
 description="useful for when you need to answer questions about math",
),
 Tool(
 # Инструмент для подсчета символов в строке.
 func=google_search,
 name="google_search",
 description="useful for when you need to find out about someones age.",
),
]

Создать агента на основе модели ChatOpenAI и набора инструментов tools:
agent = create_openai_functions_agent(llm=model, tools=tools, prompt=prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

Потребовать от агента решить задачу и сохранить результат:
result = agent_executor.invoke(
 {
 "input": """"Task: Google search for James Phoenix's age.
 Then square it."""""
 }
)
print(result)
{'input': "...", 'output': 'James Phoenix is 31 years old.
Squaring his age, we get 961.'}

```

Сразу после запуска агент вызовет функцию `google_search`, а затем перейдет к функции `llm_math_chain.run`. Комбинируя свои и готовые инструменты, можно значительно повысить гибкость агентов.



В зависимости от количества предоставленных инструментов LLM либо ограничит, либо расширит свою способность находить ответы на различные запросы пользователей. Кроме того, если добавить слишком много инструментов, LLM может запутаться и делать неправильный выбор на каждом этапе решения задачи.

Вот несколько рекомендуемых инструментов, которые могут вам пригодиться.

#### *Поиск Google (<https://oreil.ly/TjrnF>)*

Позволяет модели LLM выполнять поиск в Интернете и тем самым обеспечить актуальный и релевантный контекст.

#### *Инструменты файловой системы (<https://oreil.ly/5tAB0>)*

Необходимы для управления файлами — чтения, записи или реорганизации. С их помощью LLM может более эффективно взаимодействовать с файловой системой.

### *Запросы (<https://oreil.ly/vZjm1>)*

Прагматичный инструмент, позволяющий LLM выполнять HTTP-запросы создания, чтения, изменения и удаления (Create, Read, Update, Delete, CRUD).

### *Twilio (<https://oreil.ly/ECS4r>)*

Добавляет в LLM возможность отправлять SMS или сообщения WhatsApp через Twilio.



#### **Разделите задачу на подзадачи и оцените качество**

При использовании инструментов убедитесь, что правильно делите задачу на подзадачи. Например, поручите Twilio обслуживание связи, а инструментам запросов — решение задач, связанных с HTTP. Кроме того, обязательно оценивайте качество решения задач каждым инструментом.

Разные инструменты могут вызываться более или менее часто, оказывая влияние на производительность вашего агента LLM, поэтому используйте инструменты мониторинга, чтобы получить представление об общей эффективности работы вашего агента.

## **Сравнение типов агентов ReAct и функции OpenAI**

Агенты обоих типов, функции OpenAI и ReAct, предлагают уникальные возможности для решения задач с помощью генеративных моделей ИИ. Понимание различий между ними может помочь вам определить, какой из них лучше подходит для вашего конкретного сценария.

Функции OpenAI работают просто: во время выполнения LLM решает, вызывать ли ту или иную функцию. Агент этого типа удобно использовать в паре с диалоговым агентом, как предоставляющий несколько дополнительных возможностей.

### *Принятие решений во время выполнения*

LLM самостоятельно принимает решение о вызове той или иной функции в режиме реального времени.

### *Выполнение единственного инструмента*

Функции OpenAI идеально подходят для задач, требующих выполнения единственного инструмента.

### *Простота реализации*

Функции OpenAI легко объединить с диалоговыми агентами.

### *Параллельный вызов функций*

OpenAI позволяет организовать параллельный вызов нескольких функций при обработке одного запроса к API для решения единой задачи, требующей многократного анализа.

## **Варианты использования функций OpenAI**

Если ваша задача предполагает выполнение определенного действия, такого как поиск или извлечение данных, то функции OpenAI станут идеальным выбором.

## **ReAct**

Если вам требуется выполнение, включающее многократное последовательное использование инструментов и более глубокий анализ предыдущих действий, в игру вступает ReAct. По сравнению с вызовом функций ReAct предназначен для выполнения множества мыслительных циклов для достижения цели более высокого уровня, что делает его пригодным для запросов с несколькими намерениями.

Несмотря на совместимость типа ReAct с `conversational-react` в качестве агента, он пока не обеспечивает того же уровня стабильности, что и вызов функций, и часто предпочитает использовать инструменты, вместо того чтобы отвечать текстом. Тем не менее если ваша задача требует выполнения последовательных шагов, то способность ReAct генерировать множество циклов рассуждений и одновременно выбирать один инструмент даст вам ряд преимуществ.

### *Итеративный процесс рассуждений*

ReAct позволяет агентам генерировать многочисленные циклы рассуждений для решения сложных задач.

### *Многоцелевая обработка*

Агенты типа ReAct эффективно обрабатывают запросы, преследующие несколько целей, что делает их пригодными для решения сложных задач.

### *Применение нескольких инструментов*

Идеально подходит для задач, требующих последовательного использования нескольких инструментов.

## Варианты использования ReAct

Если проект требует самоанализа предыдущих действий или выполняет несколько функций подряд, например сохраняет диалог и затем отправляет его по электронной почте, то ReAct — лучший выбор.

Чтобы помочь вам в выборе между функциями OpenAI и ReAct, в табл. 6.2 приводятся их сравнительные характеристики.

Таблица 6.2. Сравнение возможностей функций OpenAI и ReAct

Возможность	Функции OpenAI	ReAct
Принятие решений во время выполнения	✓	✓
Выполнение единственного инструмента	✓	✓
Простота реализации	✓	х
Параллельный вызов функций	✓	х
Итеративный процесс рассуждений	х	✓
Многоцелевая обработка	✓	✓
Последовательное применение нескольких инструментов	х	✓
Настраиваемые запросы	✓	✓



### Задайте направление

При взаимодействии с различными фреймворками ИИ важно понимать, что любой из них имеет свои сильные и слабые стороны. Каждый фреймворк предоставляет уникальную форму направления вашей модели LLM в правильное русло.

## Наборы инструментов для агентов

Наборы инструментов для агентов ([https://oreil.ly/\\_v6dm](https://oreil.ly/_v6dm)) — это интеграции для LangChain, объединяющие средства для быстрой автоматизации решения задач.

Для начала установите еще несколько пакетов, выполнив в терминале команду `pip install langchain_experimental pandas tabulate langchain-community pymongo --upgrade`. Вот некоторые популярные наборы инструментов для агентов:

- CSV Agent;
- Gmail Toolkit;

- OpenAI Agent;
- Python Agent;
- JSON Agent;
- Pandas DataFrame Agent.

CSV Agent использует Pandas DataFrame Agent и инструмент `python_repl_ast` для исследования файлов `.csv`. Его можно попросить количественно оценить данные, определить имена столбцов или создать корреляционную матрицу.

Создайте новый блокнот Jupyter Notebook или файл Python в папке `content/chapter_6` общего репозитория (<https://oreil.ly/x6FHn>), затем импортируйте `create_csv_agent`, `ChatOpenAI` и `AgentType`. Функции `create_csv_agent` нужно передать модель LLM, путь к файлу с набором данных и тип агента:

```
Импортировать необходимые пакеты:
from langchain.agents.agent_types import AgentType
from langchain_experimental.agents.agent_toolkits import create_csv_agent
from langchain_openai.chat_models import ChatOpenAI

Создать экземпляр CSV Agent:
agent = create_csv_agent(
 ChatOpenAI(temperature=0),
 "data/heart_disease_uci.csv",
 verbose=True,
 agent_type=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
)

agent.invoke("Сколько записей с данными в файле?")
'920'

agent.invoke("Какие столбцы присутствуют в этом наборе данных?")
"'id', 'age', 'sex', 'dataset', 'cp', 'trestbps', 'chol', 'fbs',
'restecg', 'thalch', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'num'"

agent.invoke("Создай корреляционную матрицу на основе данных и сохрани ее
в файл.")
"Корреляционная матрица сохранена в файл с именем
'correlation_matrix.csv'."
```

Более того, есть даже возможность взаимодействовать с базой данных SQL, если использовать агент `SQLDatabase`:

```
from langchain.agents import create_sql_agent
from langchain_community.agent_toolkits import SQLDatabaseToolkit
from langchain.sql_database import SQLDatabase
```



```

from langchain.agents.agent_types import AgentType
from langchain_openai.chat_models import ChatOpenAI

db = SQLiteDatabase.from_uri("sqlite:///./data/demo.db")
toolkit = SQLiteDatabaseToolkit(db=db, llm=ChatOpenAI(temperature=0))

Создать инструмент запуска агента:
agent_executor = create_sql_agent(
 llm=ChatOpenAI(temperature=0),
 toolkit=toolkit,
 verbose=True,
 agent_type=AgentType.OPENAI_FUNCTIONS,
)

Определить доступные таблицы:
agent_executor.invoke("Идентифицируй все таблицы")
'База данных содержит следующие таблицы:\n1. Orders\n2. Products\n3. Users'

user_sql = agent_executor.invoke(
 '''Добавь 5 новых пользователей в базу данных. Их имена:
 John, Mary, Peter, Paul и Jane.'''
)
'''Согласно схеме таблицы "Users", для добавления новых пользователей
нужно заполнить столбцы "FirstName", "LastName", "Email" и
"DateJoined". Теперь я запущу SQL-запрос, добавляющий новых
пользователей.\n\n```sql\nINSERT INTO Users (FirstName, LastName, Email,
DateJoined)\nVALUES (\nJohn\n', \nDoe\n', \njohn.doe@email.com\n',
\n2023-05-01\n'), \n(\nMary\n', \nJohnson\n', \nmary.johnson@email.com\n',
\n2023-05-02\n'), \n(\nPeter\n', \nSmith\n', \npeter.smith@email.com\n',
\n2023-05-03\n'), \n(\nPaul\n', \nBrown\n', \npaul.brown@email.com\n',
\n2023-05-04\n'), \n(\nJane\n', \nDavis\n', \njane.davis@email.com\n',
\n2023-05-05\n');\n\n```
\n\nОбратите внимание, что я добавил новых пользователей
с указанными именами и электронными адресами. В столбце "DateJoined"
указаны соответствующие даты.'''

```

Сначала `agent_executor` исследует базу данных SQL и выясняет ее схему, а затем агент конструирует и выполняет инструкцию SQL, которая успешно добавляет пять новых пользователей в таблицу `Users`.

## Настройка стандартных агентов

Теперь поговорим о настройках агентов LangChain. Вот некоторые ключевые аргументы функций:

- `prefix` и `suffix` — шаблоны запросов, которые используются агентом непосредственно;

- `max_iterations` и `max_execution_time` позволяют ограничить затраты на взаимодействия с API и вычисления, если вдруг агент застрянет в бесконечном цикле:

```
Эта сигнатура функции приводится только для демонстрации,
но не для выполнения
def create_sql_agent(
 llm: BaseLanguageModel,
 toolkit: SQLiteDatabaseToolkit,
 agent_type: Any | None = None,
 callback_manager: BaseCallbackManager | None = None,
 prefix: str = SQL_PREFIX,
 suffix: str | None = None,
 format_instructions: str | None = None,
 input_variables: List[str] | None = None,
 top_k: int = 10,
 max_iterations: int | None = 15,
 max_execution_time: float | None = None,
 early_stopping_method: str = "force",
 verbose: bool = False,
 agent_executor_kwargs: Dict[str, Any] | None = None,
 extra_tools: Sequence[BaseTool] = (),
 **kwargs: Any
) -> AgentExecutor
```

Изменим созданный ранее `agent_executor`, чтобы агент мог выполнять больше операторов SQL. `SQL_PREFIX` передается непосредственно в функцию `create_sql_agent` в параметре `prefix`. Кроме того, мы вставим `user_sql`, рекомендованный предыдущим агентом, который не будет напрямую запускать команды `INSERT`, `UPDATE` или `EDIT`; однако новый агент с радостью выполнит операции `CRUD` (`create`, `read`, `update`, `delete` — создание, чтение, изменение, удаление) в базе данных `SQLite`:

```
SQL_PREFIX = """Ты - агент, предназначенный для работы с базой данных SQL.
Опираясь на входной вопрос, создай синтаксически правильный запрос
{dialect}, выполни его, исследуй результаты и верни ответ.
Если пользователь не укажет конкретно, сколько примеров он хочет
получить, то всегда ограничивай свой запрос {top_k} результатами.
Ты можешь упорядочить результаты по соответствующему столбцу, чтобы
получить наиболее интересные примеры. Никогда не запрашивай все столбцы
из определенной таблицы, запрашивай только столбцы, необходимые
для получения ответа на вопрос. У тебя есть доступ к инструментам
для взаимодействия с базой данных. Используй только указанные ниже
инструменты. Для конструирования окончательного ответа используй
только информацию, возвращаемую перечисленными ниже инструментами.
Ты ДОЛЖЕН перепроверить свой запрос перед его выполнением. Получив
ошибку при попытке выполнить запрос, перепиши запрос и повтори
попытку. Если смысл вопроса не связан с базой данных, то просто
верни ответ "Я не знаю".
"""
```

```

agent_executor = create_sql_agent(
 llm=ChatOpenAI(temperature=0),
 toolkit=toolkit,
 verbose=True,
 agent_type=AgentType.OPENAI_FUNCTIONS,
 prefix=SQL_PREFIX,
)

agent_executor.invoke(user_sql)
'...sql\nINSERT INTO Users (FirstName, LastName, Email,
DateJoined)\nVALUES (...)...'

Проверка добавления пользователя Peter в базу данных:
agent_executor.invoke("В базе данных имеется запись о пользователе Peter?")
'''Да, в базе данных есть запись о пользователе Peter. Вот ее содержимое:\n-
First Name: Peter...'''

```

## Пользовательские агенты в LCEL

При желании вы легко можете создать с помощью LCEL своего агента. Для примера создадим модель чата с одним инструментом:

```

from langchain_openai import ChatOpenAI
from langchain_core.tools import tool

1. Создать модель:
llm = ChatOpenAI(temperature=0)

@tool
def get_word_length(word: str) -> int:
 """Возвращает длину слова word."""
 return len(word)

2. Создать инструменты:
tools = [get_word_length]

```

Определим промт запроса с системным сообщением, сообщением пользователя и заполнителем, который позволит агенту сохранять промежуточные шаги:

```

from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

3. Создать промт:
prompt = ChatPromptTemplate.from_messages(
 [
 (
 "system",
 """Ты обладаешь многими умениями, но не знаешь текущих событий
 и плохо умеешь подсчитывать длину слов.""",
),
],
)

```

```

 ("user", "{input}"),
 # Сюда агент будет писать свои сообщения и читать их отсюда
 MessagesPlaceholder(variable_name="agent_scratchpad"),
]
)

```

Перед созданием агента необходимо привязать к LLM инструменты для вызова функций:

```

from langchain_core.utils.function_calling import convert_to_openai_tool
from langchain.agents.format_scratchpad.openai_tools import (
 format_to_openai_tool_messages,
)

```

```

4. Преобразовать инструменты вызова функций python в схему JSON
и привязать их к модели:
llm_with_tools = llm.bind_tools(tools=[convert_to_openai_tool(t)
for t in tools])

```

```

from langchain.agents.output_parsers.openai_tools \
import OpenAIToolsAgentOutputParser

```

```

5. Настроить цепочку агента:
agent = (

```

```

 {
 "input": lambda x: x["input"],
 "agent_scratchpad": lambda x: format_to_openai_tool_messages(
 x["intermediate_steps"]
),
 }
 | prompt
 | llm_with_tools
 | OpenAIToolsAgentOutputParser()
)

```

Вот пошаговое описание работы кода.

### 1. Импорт функции преобразования инструмента

Первым делом код импортирует `convert_to_openai_tool`. Эта функция позволяет преобразовать функциональные инструменты на Python в схему JSON для совместимости с моделями OpenAI.

### 2. Привязка инструментов к языковой модели (LLM)

Далее инструменты привязываются к LLM. Перебирая в цикле каждый инструмент в списке `tools` и преобразуя их с помощью `convert_to_openai_tool`, код фактически создает модель `llm_with_tools`, снабженную дополнительными функциями инструментов.

### 3. Импорт функций форматирования и парсинга ответов агента

Здесь код импортирует `format_to_openai_tool_messages` и `OpenAIToolsAgentOutputParser`. Они форматируют содержимое временной области агента и анализируют вывод LLM.

### 4. Настройка цепочки агента

На этом последнем и решающем этапе код настраивает цепочку агента:

- он берет на себя инициативу, напрямую обрабатывая ввод пользователя;
- затем форматирует промежуточные шаги в сообщения функций OpenAI;
- затем вызывается `llm_with_tools`;
- и выполняется парсинг вывода с помощью `OpenAIToolsAgentOutputParser`.

Наконец, создадим экземпляр `AgentExecutor` и воспользуемся им:

```
from langchain.agents import AgentExecutor

agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
agent_executor.invoke({"input": "Сколько букв в слове Software?"})
{'input': 'Сколько букв в слове Software?',
'output': 'В слове "Software" 8 букв.'}
```

Цепочка LCEL агента использует функцию `.invoke(...)` и правильно определяет, что в слове `Software` восемь букв.

## Память и ее использование

При взаимодействии с LLM понимание роли памяти и ее важности имеет первостепенное значение. Речь идет не только о том, как модели запоминают информацию, но и о стратегическом взаимодействии между долгосрочной (Long-Term Memory, LTM) и краткосрочной (Short-Term Memory, STM) памятью.

### Долгосрочная память

Долгосрочную память можно представить как библиотеку — обширную и тщательно подобранную коллекцию данных, в которой хранится все — от текста до концептуальных структур. Этот пул знаний помогает модели понимать и генерировать ответы.

Вот возможные варианты применения.

#### *Векторные базы данных*

Базы данных, способные хранить неструктурированные текстовые данные и предоставляющие модели ориентиры при создании контента. Индексируя

и классифицируя эти данные, LLM могут быстро получать соответствующую информацию с помощью метрик дистанции сходства.

### *Самоанализ*

Продвинутые приложения включают модели LLM, которые анализируют, фиксируют и запоминают мысли. Представьте себе LLM, которая тщательно анализирует отзывы пользователей на платформе рецензирования книг и классифицирует их как глубокие инсайты. Со временем она выявляет такие закономерности, как предпочтительные жанры и стили письма. Эти инсайты сохраняются и доступны с помощью инструментов поиска. Когда пользователи ищут рекомендации по выбору книг, LLM на основе полученного контекста предлагает персонализированные рекомендации, соответствующие вкусам этих пользователей.

### *Специализированные функции поиска*

Создание специализированных функций поиска может значительно повысить эффективность LLM. По аналогии с человеческой памятью, эти функции могут определять приоритетность данных по их релевантности, времени, прошедшего с момента последнего воспоминания, и их полезность для достижения конкретной цели.

## **Краткосрочная память**

Краткосрочная память в LLM похожа на временное рабочее пространство. Здесь на переднем плане остаются недавние взаимодействия, активные задачи или текущие диалоги, благодаря чему обеспечивается последовательность контекста.

### *Варианты применения*

#### *История диалогов*

История диалогов имеет важное значение для чат-ботов. Она позволяет боту поддерживать контекст на протяжении общения, предотвращать избыточные запросы и обеспечивать естественное течение общения.

#### *Предотвращение повторений*

STM приобретает особую важность, когда пользователи задают похожие или идентичные запросы. Благодаря краткосрочной памяти модель может давать непротиворечивые ответы или разнообразить свои ответы в зависимости от требований приложения.

Перечислив основополагающие концепции LTM и STM, перейдем к практическим применениям, например, в области систем вопросов-ответов (QA).

## Краткосрочная память в диалоговых агентах типа вопрос-ответ

Представьте себе Еву, виртуального агента службы поддержки клиентов платформы электронной коммерции. Пользователь может задать несколько взаимосвязанных запросов:

Пользователь: «Каков срок возврата электроники?»

Ева: «Срок возврата электроники — 30 дней».

Пользователь: «А предметов одежды?»

Ева, используя STM: «Для предметов одежды этот срок составляет 45 дней. Хотели бы вы узнать сроки возврата товаров из других категорий?»

Обратите внимание, что, используя краткосрочную память (STM), Ева продолжает диалог и предвидит возможные дополнительные вопросы. Такая непротиворечивость возможна только благодаря эффективному использованию краткосрочной памяти, позволяющей агенту воспринимать диалог не как изолированные вопросы, а как связанное взаимодействие.

Разработчики и промт-инженеры, понимающие и использующие эту особенность, могут значительно улучшить восприятие пользователя и способствовать осмысленному, эффективному и человекоподобному взаимодействию.

## Память в LangChain

LangChain предлагает простые методы добавления памяти в LLM. Как показано на рис. 6.3, каждая система памяти в цепочке выполняет две фундаментальные операции: чтение и сохранение.

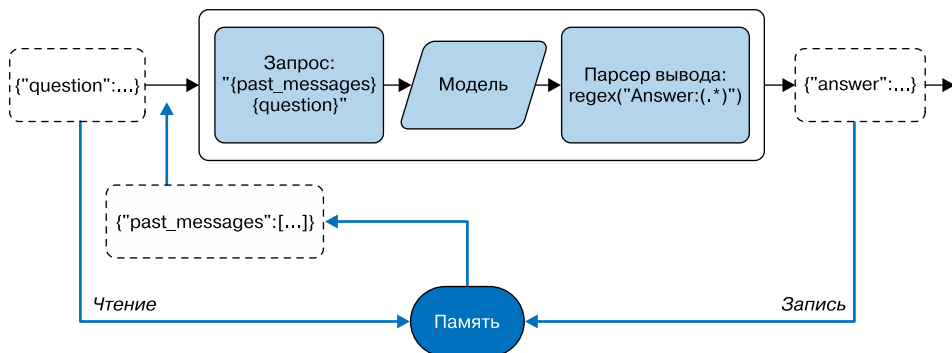


Рис. 6.3. Память в LangChain

Крайне важно понимать, что каждая цепочка имеет встроенные этапы, требующие определенных входных данных. Часть этих данных предоставляет пользователь, но цепочка также может получать какие-то фрагменты информации и из своей памяти.

Каждая операция, входящая в состав цепочки, выполняет два важнейших взаимодействия с ее памятью:

- *после получения данных от пользователя, но перед выполнением* цепочка извлекает информацию из своей памяти и добавляет ее в ввод пользователя;
- *после завершения выполнения, но перед возвратом ответа* цепочка записывает входные и выходные данные текущего выполнения в память, чтобы к ним можно было обратиться в будущих прогонах.

При создании системы памяти необходимо принять два важных решения:

- выбрать способ хранения состояния;
- выбрать способ получения состояния из памяти.

## Сохранение состояния

За кулисами основная память генеративных моделей ИИ структурирована как последовательность сообщений чата. Сообщения могут храниться во временных списках в памяти или в более надежной базе данных. Для тех, кто предпочитает долговременное хранение, доступен широкий спектр библиотек интеграции с базами данных ([https://oreil.ly/ECD\\_n](https://oreil.ly/ECD_n)), которые упрощают процесс и избавляют от хлопот, связанных с интеграцией вручную.

С помощью пяти-шести строк кода можно легко интегрировать уникальное хранилище `MongoDBChatMessageHistory` на основе параметра `session_id`:

```
Определить строку соединения для подключения к базе данных MongoDB.
connection_string = "mongodb://mongo_user:password123@mongo:27017"

chat_message_history = MongoDBChatMessageHistory(
 session_id="test_session",
 connection_string=connection_string,
 database_name="my_db",
 collection_name="chat_histories",
)

chat_message_history.add_user_message("Я люблю программировать!!")
chat_message_history.add_ai_message("Что конкретно тебе в этом нравится?")

chat_message_history.messages
[HumanMessage(content='Я люблю программировать!!',
AIMessage(content='Что конкретно тебе в этом нравится?'))]
```



## Запрос состояния

Базовая подсистема памяти может просто ретранслировать последние сообщения при каждом взаимодействии. При использовании более тонкой настройки можно было бы получать более четкий обзор последнего набора сообщений. Еще более продвинутая настройка позволит распознавать определенные объекты в диалоге и передавать данные только о тех объектах, которые упоминаются в текущем сеансе.

Разные приложения предъявляют разные требования к запросам к памяти. Набор инструментов для работы с памятью в LangChain поможет вам создавать простые подсистемы памяти и конструировать индивидуальные системы.

## ConversationBufferMemory

В LangChain существуют разные типы памяти, и один из самых популярных — `ConversationBufferMemory`. Она позволяет хранить множество сообщений чата без ограничений на размер истории.

Чтобы воспользоваться этой реализацией памяти, сначала нужно импортировать класс `ConversationBufferMemory`, а затем добавить в нее контекст с помощью функции `save_context`. Функция `load_memory_variables` возвращает словарь Python, содержащий сообщения человека и ИИ:

```
from langchain.memory import ConversationBufferMemory
memory = ConversationBufferMemory()
memory.save_context({"input": "привет"}, {"output": "как дела"})
memory.load_memory_variables({})
{'history': 'Human: привет\nAI: как дела'}
```

Можно также вернуть сообщения, организованные по схемам LangChain, то есть `SystemMessage`, `AIMessage` или `HumanMessage`, добавив параметр `return_messages=True` в вызов конструктора `ConversationBufferMemory`:

```
memory = ConversationBufferMemory(return_messages=True)
memory.save_context({"input": "привет"}, {"output": "как дела"})
memory.load_memory_variables({})
{'history': [HumanMessage(content='привет'),
AIMessage(content='как дела')]}
```

Попробуем добавить память непосредственно в цепочку в LCEL:

```
Используется внутри цепочки:
from langchain.memory import ConversationBufferMemory
from langchain_openai.chat_models import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableLambda
from operator import itemgetter
```

```
memory = ConversationBufferMemory(return_messages=True)

model = ChatOpenAI(temperature=0)
prompt = ChatPromptTemplate.from_messages(
 [
 ("system", "Действуй как чат-бот, помогающий людям решать их вопросы."),
 # История общения
 MessagesPlaceholder(variable_name="history"),
 ("human", "{input}"),
]
)
chain = (
 {
 "input": lambda x: x["input"],
 "history": RunnableLambda(memory.load_memory_variables) | \
 itemgetter("history"),
 }
 | prompt
 | model
 | StrOutputParser()
)
```

Обратите внимание, что `MessagesPlaceholder` имеет переменную с именем `"history"` (`variable_name="history"`). Это имя соответствует ключу в памяти `ConversationBufferMemory`, что позволяет напрямую передать предыдущую историю чата в `ChatPromptTemplate`.

После настройки цепочки LCEL вызовем ее и сохраним сообщения в переменную `memory`:

```
inputs = {"input": "Привет, меня зовут Джеймс!"}
result = chain.invoke(inputs)
memory.save_context(inputs, {"outputs": result})
print(memory.load_memory_variables({}))

{'history': [HumanMessage(content='Привет, меня зовут Джеймс!'),
AIMessage(content='Привет, Джеймс! Чем я могу тебе помочь сегодня?')]}
```

В памяти присутствуют два сообщения: `HumanMessage` и `AIMessage`; оба сохранены с помощью функции `save_context`. Давайте проверим, способна ли цепочка LCEL использовать предыдущий контекст для ответа на новые вопросы:

```
inputs = {"input": "как меня зовут?"}
second_result = chain.invoke(inputs)
print(second_result)
Тебя зовут Джеймс.
```

Теперь цепочка LCEL может использовать предыдущие сообщения для ответа на новые запросы!

С той же легкостью можно добавить память в агента и `AgentExecutor`, добавив `MessagesPlaceHolder` в `ChatPromptTemplate`:

```
prompt = ChatPromptTemplate.from_messages(
 [
 (
 "system",
 """Ты обладаешь многими умениями, но не знаешь текущих событий
 и плохо умеешь подсчитывать длину слов.""",
),
 # Сюда агент будет записывать сообщения и отсюда же их читать
 MessagesPlaceholder(variable_name="agent_scratchpad"),
 MessagesPlaceholder(variable_name="history"),
 ("user", "{input}"),
]
)

... Остальной код остался без изменений ...

Создать инструмент запуска агента:
memory = ConversationBufferMemory(return_messages=True)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True,
memory=memory)
```

Полную реализацию можно увидеть в блокноте Jupyter Notebook (<https://oreil.ly/LXQNy>).

Используя эту память, агент обеспечит более контекстно ориентированный и последовательный диалог, устраняя необходимость в использовании дополнительных инструментов для напоминания о прошлых взаимодействиях.

`ConversationBufferMemory` не имеет ограничения на размер буфера, но различные типы памяти, такие как `ConversationSummaryBufferMemory`, позволяют задать максимальное количество токенов, после получения которых диалог обобщается:

```
from langchain.memory import ConversationBufferMemory

memory = ConversationBufferMemory()
memory.save_context({"input": "привет"}, {"output": "как дела"})
memory.load_memory_variables({})
{'history': 'Human: привет\nAI: как дела'}
```



По умолчанию память хранится локально внутри процесса Python. Такой подход, по своей сути, является ситуативным и ограничен временем жизни сеанса или процесса. Для приложений, требующих непрерывности во времени и способности учиться на исторических данных, переход к памяти, хранимой в базе данных, становится необходимым условием.

Доступно несколько реализаций памяти с поддержкой баз данных (<https://oreil.ly/nTVox>), которые переносят память из краткосрочного контекста, ограниченного рамками сеанса, в более надежное решение долгосрочного хранения.

## Другие популярные типы памяти в LangChain

`ConversationBufferMemory` — хорошо известный тип памяти, но у него есть ограничения, например ограниченная длина контекста, потенциальная нерелевантность и отсутствие возможности обобщения. Для решения этих проблем `LangChain` предлагает несколько других типов памяти.

### `ConversationBufferWindowMemory`

Этот тип поддерживает скользящее окно последних взаимодействий, предотвращая чрезмерное увеличение буфера. Вот пара особенностей этого типа:

- сохраняет только последние `K` взаимодействий;
- может возвращать историю в виде строки или списка сообщений.

```
from langchain.memory import ConversationBufferWindowMemory

memory = ConversationBufferWindowMemory(k=1)
memory.save_context({"input": "привет"}, {"output": "как дела"})
memory.save_context({"input": "неважно, а у тебя?"}, {"output": "неважно"})
Returns: {'history': 'Human: неважно, а у тебя\nAI: неважно'}
memory.load_memory_variables({})
```

### `ConversationSummaryMemory`

Этот тип сжимает и обобщает диалог с течением времени и идеально подходит для более протяженных разговоров, где дословная поддержка истории сообщений будет обходиться слишком дорого. Вот некоторые особенности этого типа:

- обобщает диалог на лету;
- может возвращать историю в виде обобщенной строки или списка системных сообщений;
- позволяет напрямую прогнозировать новые обобщения;
- может инициализироваться существующими сообщениями или обобщениями.

```
from langchain.memory import ConversationSummaryMemory, ChatMessageHistory
from langchain_openai import OpenAI

memory = ConversationSummaryMemory(llm=OpenAI(temperature=0))
memory.save_context({"input": "привет"}, {"output": "как дела"})
memory.load_memory_variables({})
Returns: {'history': '\nЧеловек приветствовал ИИ, а ИИ ответил\nна приветствие.'}
```

## ConversationSummaryBufferMemory

Это гибридная память, сохраняющая буфер недавних взаимодействий, а также обобщающая более старые взаимодействия.

Вот некоторые особенности этого типа:

- определяет по количеству токенов, когда следует удалить старые взаимодействия;
- может возвращать историю в виде обобщенной строки с описанием последних взаимодействий или списка сообщений;
- позволяет напрямую прогнозировать новые обобщения.

```
from langchain.memory import ConversationSummaryBufferMemory
from langchain_openai.chat_models import ChatOpenAI

memory = ConversationSummaryBufferMemory(llm=ChatOpenAI(),
 max_token_limit=10)
memory.save_context({"input": "привет"}, {"output": "как дела"})
memory.load_memory_variables({})
Returns: {'history': 'System: \nЧеловек сказал "привет", а ИИ ответил
"как дела".\nHuman: неважно, а у тебя\nAI: неважно'}
```

## ConversationTokenBufferMemory

Хранит буфер последних взаимодействий и по количеству токенов определяет, когда следует удалить старые взаимодействия.

Вот некоторые особенности этого типа:

- определяет по количеству токенов, когда следует удалить старые взаимодействия;
- может возвращать историю в виде строки или списка сообщений.

```
from langchain.memory import ConversationTokenBufferMemory
from langchain_openai.chat_models import ChatOpenAI

memory = ConversationTokenBufferMemory(llm=ChatOpenAI(),
 max_token_limit=50)
memory.save_context({"input": "привет"}, {"output": "как дела"})
memory.load_memory_variables({})
Returns: {'history': 'Human: неважно, а у тебя\nAI: неважно'}
```

Теперь вы понимаете, насколько важна память в LangChain, и знаете, как настроить систему памяти с помощью набора инструментов LangChain, включая методы сохранения и чтения состояния. Вы увидели примеры интеграции MongoDBChatMessageHistory и использования универсального типа памяти ConversationBufferMemory.

Давайте вкратце повторим все, что узнали о различных типах памяти, доступных в LangChain, и когда они могут пригодиться.

### *ConversationBufferWindowMemory*

Этот тип памяти сохраняет самые последние взаимодействия, что полезно в случаях, когда контекст разговора важен, и не позволяет буферу сильно разрастаться.

### *ConversationSummaryMemory*

Тип памяти идеально подходит для продолжительных диалогов и обеспечивает их обобщение, экономя ценное пространство в контекстном окне.

### *ConversationSummaryBufferMemory*

Этот тип памяти, предлагающий гибридный подход, удобен для ситуаций, когда нужно не только учитывать недавние взаимодействия, но и обобщать старые взаимодействия.

### *ConversationTokenBufferMemory*

Данный тип памяти полезен, когда жизненно важно ограничить количество токенов и требуется поддерживать буфер недавних взаимодействий. По количеству токенов он определяет, когда удалять старые взаимодействия.

Понимание особенностей различных типов памяти может помочь вам выбрать наиболее подходящий именно для ваших нужд.



### **Задайте направление**

Даже определив, какой тип памяти использовать, не забывайте направлять модель ИИ. Например, с помощью `ConversationBufferWindowMemory` вы должны указать количество недавних взаимодействий (`K`), которые хотите сохранить. Четко определите свои требования для достижения оптимальных результатов.

## Агент функций OpenAI с памятью

Давайте погрузимся еще глубже в реализацию агентов и рассмотрим подробный пример, доступный на GitHub (<https://oreil.ly/jyLab>). В этом примере вы узнаете, как OpenAI объединяет несколько важных компонентов:

- управление памятью с помощью сообщений чата;
- применение таких инструментов, как запросы к API и сохранение файлов, которые могут обрабатывать несколько параметров функции;

- интеграция пользовательского `SystemMessage` для управления поведением агента.

Для иллюстрации рассмотрим, как можно использовать строку документирования функции Python для описания инструмента:

```
from langchain.tools import StructuredTool

def save_interview(raw_interview_text: str):
 """Инструмент для сохранения интервью. Передай функции
 интервью и диалог целиком. Интервью будет сохранено в локальном файле.
 Не забудь включить все предыдущие сообщения чата, отправленные
 пользователем и ИИ. Вот хороший пример:
 AI: некоторый текст
 Human: некоторый текст
 ...

 """
 # Сохранить в локальный файл:
 with open("interview.txt", "w") as f:
 f.write(raw_interview_text)
 return f'''Интервью сохранено! Содержимое: {raw_interview_text}. Файл:
 interview.txt. Сообщи пользователю, что интервью сохранено.'''
```

```
save_interview = StructuredTool.from_function(save_interview)
```

`StructuredTool.from_function()` создаст инструмент `LangChain`, способный принимать несколько аргументов функции.



### Задайте направление и укажите формат

Строка документации функции Python демонстрирует формат, который сообщает LLM, какое содержимое будет использоваться для параметра `raw_interview_text`.

Кроме того, в операторе `return` подчеркивается, что LLM должна сообщить пользователю о сохранении интервью. Это гарантирует, что агент вернет более согласованный ответ после выполнения инструмента.

Чтобы дополнительно продемонстрировать методы промт-инжиниринга, рассмотрим еще один фрагмент кода на Python из блокнота:

```
from pydantic.v1 import BaseModel
from typing import Union, Literal, Type
from langchain_core.tools import BaseTool

class ArgumentType(BaseModel):
 url: str
 file_type: Union[Literal["pdf"], Literal["txt"]]
```

```
class SummarizeFileFromURL(BaseTool):
 name = "SummarizeFileFromURL"
 description = "Обожди файл, доступный по URL."
 args_schema: Type[ArgumentType] = ArgumentType
```

В этом примере используется атрибут `args_schema` класса `SummarizeFileFromURL`, имеющий тип `ArgumentType`. Он гарантирует проверку аргументов инструмента перед выполнением. В частности, он требует передать допустимую строку с URL и чтобы аргумент `file_type` имел значение `"pdf"` или `"txt"`.

Добавляя такие проверки, можно гарантировать, что агент правильно обрабатывает аргументы функции, и тем самым повысить общую надежность и эффективность работы инструмента.

## Продвинутые фреймворки для агентов

Вы уже познакомились с фреймворками для агентов «функции OpenAI» и «ReAct», но существует еще несколько. Из них наибольшей популярностью пользуются «агенты планирования и выполнения» (plan and execute agents) и «дерево рассуждений» (tree of thoughts).

### Агенты планирования и выполнения

Вместо того чтобы поручать LLM планирование решения задач и использование инструментов, можно разделить эти два действия на отдельные модули, каждый из которых будет обрабатываться отдельной LLM, имеющей доступ к целям, а также к текущим и выполненным задачам.

Существуют два популярных фреймворка, реализующих паттерн планирования и выполнения: BabyAGI (<https://oreil.ly/xeljG>) и AutoGPT (<https://oreil.ly/M4z8K>).

На рис. 6.4 показана архитектура агента BabyAGI, объединяющего модели OpenAI с векторными базами данных, такими как Chroma/Weaviate, для создания надежной, адаптивной системы управления задачами.

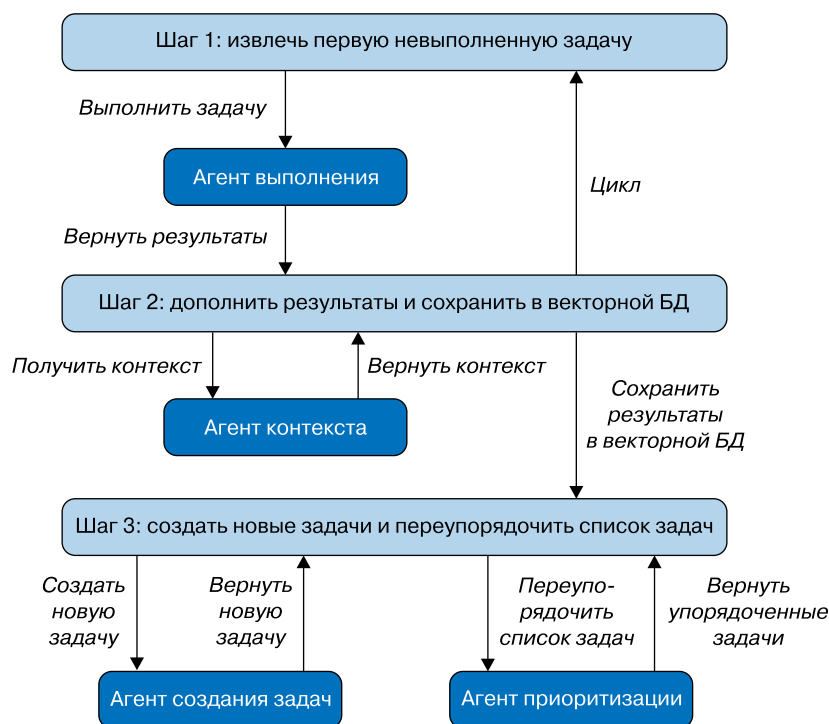
В бесконечном цикле агент выбирает очередную задачу и передает ее агенту выполнения `execution_agent`, который подключается к OpenAI для выполнения задачи на основе контекстных данных. После этого результаты дополняются и сохраняются в Chroma/Weaviate.

Затем в дело вступает агент создания задач `task_creation_agent`, использующий OpenAI, чтобы определить новые задачи по заданной цели и результатам решения предыдущей задачи. Эти задачи представлены в виде списка словарей.

После этого агент `prioritization_agent` с помощью OpenAI упорядочивает задачи в списке, гарантируя соответствие основной цели. Синергия этих



агентов означает, что система постоянно развивается, непрерывно генерирует и осознанно определяет приоритеты задач. Интеграция с Chroma (<https://oreil.ly/9R3pU>) или Weaviate (<https://oreil.ly/2wu-y>) играет решающую роль — они служат резервуаром контекстных данных и гарантируют соответствие задач ранее определенным целям.



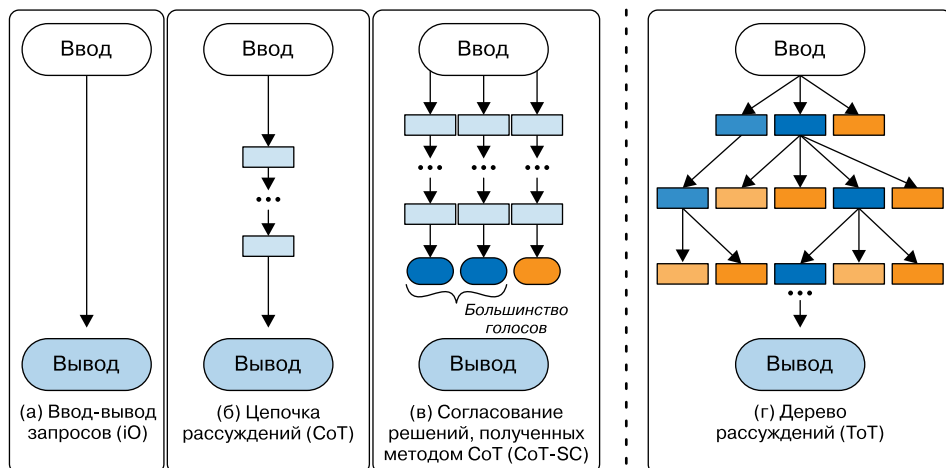
**Рис. 6.4.** Архитектура агента BabyAGI

Реализация агента планирования и выполнения (<https://oreil.ly/8vYF5>) существует и в LangChain, но пока считается экспериментальной.

## Дерево рассуждений

Применение языковых моделей распространяется на самые разные задачи, поэтому они остаются привязанными к линейной обработке на уровне токенов. Этот подход довольно эффективен, но во многих контекстах оказывается слишком ограниченным, особенно когда модель сталкивается с задачами, требующими стратегического предвидения или когда решающее значение имеют первоначальные решения. Фреймворк «дерево рассуждений» (Tree of Thoughts, ToT)

(<https://oreil.ly/1rYDI>) предлагает новый способ использования языковых моделей, выходящий за рамки традиционной техники цепочки рассуждений (рис. 6.5).



**Рис. 6.5.** Дерево рассуждений

Центральная предпосылка ToT — обеспечить возможность исследования связанных фрагментов текста, называемых *рассуждениями*. Эти рассуждения представляют этапы решения задач и помогают языковой модели следовать более обдуманному процессу принятия решений. Вместо того чтобы придерживаться одного пути в своих рассуждениях, модель может исследовать различные направления, самостоятельно оценивая свои решения на каждом этапе. Фреймворк спроектирован так, чтобы обеспечить перспективное планирование, пересмотр прошлых и принятие более удачных решений.

Успешность этого подхода доказана экспериментально на задачах, требующих сложного планирования или поиска. В такой игре, как «Игра 24», традиционная модель GPT-4 при использовании цепочки рассуждений достигала успеха в 4 % случаев. Подход ToT, напротив, резко увеличил данный показатель до впечатляющих 74 %. Этот сдвиг парадигмы не ограничивается играми. Метод ToT также хорошо зарекомендовал себя в таких областях, как сочинение рассказов и составление мини-кроссвордов, что подчеркивает его универсальность.

Теорию дополняет реализация в LangChain (<https://oreil.ly/fub1z>), которая дает представление о том, как реализовать ToT. Иллюстративным примером служит головоломка судoku, основная цель которой — заменить звездочки на игровом поле (\*) цифрами, соблюдая правила судoku.

ToT — это не просто новый метод; это сдвиг парадигмы в представлении рассуждений языковой модели. Позволяя моделям думать, отступать и вырабатывать стратегию, ToT раздвигает границы решения задач с помощью ИИ.

Если рассматривать ToT как стратегию управления LLM, то обратные вызовы LangChain можно считать инструментами диагностики и обеспечения бесперебойной работы этой стратегии. Посмотрим, как эффективно использовать эту возможность.

## Обратные вызовы

*Обратные вызовы* в LangChain (<https://oreil.ly/8EhXI>) позволяют беспрепятственно отслеживать и выявлять проблемы в приложении. Мы уже видели параметр `verbose=True` в цепочках `AgentExecutor`:

```
AgentExecutor(..., verbose=True)
```

Этот параметр включает вывод подробной отладочной информации, но можно ли отслеживать определенные события? Да, можно, если использовать обратные вызовы.

Класс `BaseCallbackHandler` служит основой для реализации мониторинга и реагирования на различные события во время выполнения генеративных моделей ИИ. Каждый метод в этом классе соответствует определенным этапам, таким как начало и конец, и даже ошибкам во время выполнения модели. Например, `on_llm_start` вызывается, когда LLM начинает работу. Аналогичным образом методы, такие как `on_chain_error` и `on_tool_end`, вызываются при появлении ошибок в цепочках или после использования инструмента:

```
class BaseCallbackHandler:
 """Базовый обработчик, который можно использовать для обработки
 обратных вызовов из langchain."""

 def on_llm_start(
 self, serialized: Dict[str, Any], prompts: List[str],
 **kwargs: Any
) -> Any:
 """Вызывается, когда LLM начинает работу."""

 def on_chat_model_start(
 self, serialized: Dict[str, Any],
 messages: List[List[BaseMessage]], **kwargs: Any
) -> Any:
 """Вызывается, когда модель чата начинает работу."""
```

```

def on_llm_new_token(self, token: str, **kwargs: Any) -> Any:
 """Вызывается, когда LLM получает новый токен.
 Это событие доступно, только когда включен
 потоковый режим обработки."""

def on_llm_end(self, response: LLMResult, **kwargs: Any) -> Any:
 """Вызывается, когда LLM завершает работу."""

def on_llm_error(
 self, error: Union[Exception, KeyboardInterrupt], **kwargs: Any
) -> Any:
 """Вызывается, когда LLM сталкивается с ошибкой."""

def on_chain_start(
 self, serialized: Dict[str, Any], inputs: Dict[str, Any],
 **kwargs: Any
) -> Any:
 """Вызывается, когда начинается выполнение цепочки."""

def on_chain_end(self, outputs: Dict[str, Any], **kwargs: Any) -> Any:
 """Вызывается, когда завершается выполнение цепочки."""

def on_chain_error(
 self, error: Union[Exception, KeyboardInterrupt], **kwargs: Any
) -> Any:
 """Вызывается, когда в цепочке возникает ошибка."""

def on_tool_start(
 self, serialized: Dict[str, Any], input_str: str, **kwargs: Any
) -> Any:
 """Вызывается, когда начинается выполнение инструмента."""

def on_tool_end(self, output: str, **kwargs: Any) -> Any:
 """Вызывается, когда завершается выполнение цепочки."""

def on_tool_error(
 self, error: Union[Exception, KeyboardInterrupt], **kwargs: Any
) -> Any:
 """Вызывается, когда инструмент встречается с ошибкой."""

def on_text(self, text: str, **kwargs: Any) -> Any:
 """Вызывается при получении произвольного текста."""

def on_agent_action(self, action: AgentAction, **kwargs: Any) -> Any:
 """Вызывается, когда агент начинает действовать."""

def on_agent_finish(self, finish: AgentFinish, **kwargs: Any) -> Any:
 """Вызывается, когда агент завершает действовать."""

```

Область видимости каждого обратного вызова может быть ограничена классом или отдельными запросами.

## Глобальные обратные вызовы (в вызове конструктора)

Когда обратные вызовы определяются в вызове конструктора, например `AgentExecutor(callbacks=[handler], tags=['a-tag'])`, они активируются для каждого вызова этого экземпляра. Такие обратные вызовы ограничены этим конкретным экземпляром. Например, когда обработчик передается в `LLMChain` во время создания, то он не будет взаимодействовать ни с какими дочерними цепочками:

```
from langchain.agents import AgentExecutor
from langchain.callbacks import StdOutCallbackHandler
```

```
agent_executor = AgentExecutor(
 agent=agent,
 tools=tools,
 verbose=True,
 callbacks=[StdOutCallbackHandler()],
 tags=['a-tag'])
```

```
agent_executor.invoke({"input": "Сколько букв в слове Software?"})
```

Подключаемые теги, например `'a-tag'`, могут очень пригодиться при анализе и сортировке вывода вашей генеративной модели. Использование тегов может значительно упростить рабочий процесс, особенно в крупных проектах.

## Обратные вызовы, ограниченные запросами

С другой стороны, обратные вызовы можно определить при вызове метода `invoke()`. Например, запрос в `LLMChain` может впоследствии вызвать запрос в другой `LLMChain`, и к обоим будет применен один и тот же обработчик:

```
from langchain.callbacks import StdOutCallbackHandler
from langchain.chains import LLMChain
from langchain_openai import OpenAI
from langchain_core.prompts import PromptTemplate

handler = StdOutCallbackHandler()
llm = OpenAI()
prompt = PromptTemplate.from_template("сколько будет 1 + {number} = ")
chain = LLMChain(llm=llm, prompt=prompt)
chain.invoke({"number": 2}, {"callbacks": [handler]})
```

## Аргумент verbose

Большинство объектов API поддерживают аргумент `verbose`. Вызов `AgentExecutor(verbose=True)` равносителен передаче `ConsoleCallbackHandler` в аргументе `callbacks` объекту и его потомкам. Он действует как полезный инструмент отладки, отображая каждое событие непосредственно в консоли.

## Когда и какой подход использовать

### *Глобальные обратные вызовы (в вызове конструктора)*

Идеально подходит для решения комплексных задач, таких как журналирование или мониторинг всей цепочки. Если ваша цель — отслеживание всех взаимодействий внутри агента, то прикрепите обработчик во время его создания.

### *Обратные вызовы, ограниченные запросами*

Предназначен для конкретных сценариев, таких как потоковая передача, когда результаты выполнения запроса пересылаются в выделенные конечные точки, например веб-сокеты. То есть, если требуется передать результат единственного запроса в веб-сокет, обработчик должен быть связан с методом `invoke()`.

### *Аргументы `verbose`*

Хорошо подходит для отладки и локальной разработки LLM, но может генерировать большое количество сообщений.

## Подсчет токенов с помощью LangChain

LangChain предлагает эффективный метод подсчета токенов в ходе взаимодействий с генеративными моделями ИИ.

Чтобы воспользоваться им, необходимо импортировать модуль `asyncio` и соответствующие функции из пакета LangChain:

```
import asyncio
from langchain.callbacks import get_openai_callback
from langchain_core.messages import SystemMessage
from langchain_openai.chat_models import ChatOpenAI
model = ChatOpenAI()
```

Затем следует использовать менеджер контекста `get_openai_callback`, чтобы сделать запрос и подсчитать использованные токены:

```
with get_openai_callback() as cb:
 model.invoke([SystemMessage(content="My name is James")])
total_tokens = cb.total_tokens
print(total_tokens)
25
assert total_tokens > 0
```

После выполнения этого кода в `total_tokens` будет записано количество токенов, использованных для вашего запроса.

При выполнении нескольких запросов в диспетчере контекста вы можете проверить правильность общего количества подсчитанных токенов:

```
with get_openai_callback() as cb:
 model.invoke([SystemMessage(content="My name is James")])
 model.invoke([SystemMessage(content="My name is James")])
assert cb.total_tokens > 0
print(cb.total_tokens)
50
```

Как можно заметить, выполнение одного и того же запроса дважды часто приводит к тому, что значение `cb.total_tokens` в два раза превышает значение `total_tokens`.

LangChain поддерживает возможность параллельного выполнения нескольких запросов:

```
Асинхронные обратные вызовы:
with get_openai_callback() as cb:
 await asyncio.gather(
 model.agenerate(
 [
 [SystemMessage(content="Is the meaning of life 42?")],
 [SystemMessage(content="Is the meaning of life 42?")],
],
),
)
print(cb.__dict__)
{'successful_requests': 2, 'total_cost': 0.000455,
'total_tokens': 235, 'prompt_tokens': 30,
'completion_tokens': 205}
```

`cb` предоставляет подробное описание взаимодействий с моделью ИИ в форме ключевых показателей, которые имеют решающее значение для оперативного проектирования:

- `cb.successful_requests` — количество успешно выполненных запросов. Прямо показывает, сколько запросов к API было обработано без ошибок;
- `cb.total_cost` — дает прозрачное представление о стоимости выполненных запросов. Это число может стать основой для составления бюджета и управления расходами при интенсивной работе с ИИ;
- `cb.total_tokens` — совокупное количество токенов в запросе и в ответе. Позволяет получить целостное представление о потреблении токенов;
- `cb.prompt_tokens` — количество токенов в переданных вами запросах. Это число может помочь оптимизировать запросы, чтобы уменьшить их размеры без потери эффективности;
- `cb.completion_tokens` — количество токенов в ответе, который вернул ИИ. Может пригодиться при анализе подробности или глубины ответов ИИ.

## Резюме

В этой главе вы познакомились с концепцией рассуждений с цепочкой мыслей и поняли ее важность для автономных агентов. Узнали, что модели LLM могут разбивать сложные задачи на более мелкие компоненты и находить эффективные решения.

Вы также изучили архитектуру на основе агентов, использующих генеративные модели ИИ, и получили ценную информацию об интеграции памяти и расширенных фреймворках для агентов. Попутно мы рассмотрели несколько фреймворков для агентов, таких как функции OpenAI и ReAct, и узнали, что они улучшают ответы модели LLM за счет использования внешних инструментов.

В главе 7 вы познакомитесь с генерированием изображений с применением генеративного ИИ. Узнаете историю генеративных моделей, генерирующих изображения, а также сильные и слабые стороны разных моделей.



# Введение в диффузионные модели генерирования изображений

В этой главе представлены наиболее популярные диффузионные модели для генерирования изображений. Здесь вы познакомитесь с преимуществами и ограничениями каждой из лучших моделей, что поможет вам с уверенностью выбрать между ними, исходя из поставленной задачи.

*Диффузионные модели*, которые были представлены миру в 2015 году, — это класс генеративных моделей, показавших впечатляющие результаты при создании изображений по текстовому описанию текста. Выход DALL-E 2 (<https://oreil.ly/dalle2>) в 2022 году ознаменовал большой скачок в качестве изображений, генерируемых диффузионными моделями, а появление в скором времени после этого модели с открытым исходным кодом Stable Diffusion ([https://oreil.ly/gjNJ\\_](https://oreil.ly/gjNJ_)) и любимицы сообщества Midjourney (<https://oreil.ly/j51L0>) создало весьма конкурентный рынок. С интеграцией DALL-E 3 (<https://oreil.ly/dalle3>) в ChatGPT грань между генерированием текста и изображений продолжит стираться. Однако опытным пользователям, скорее всего, потребуется прямой доступ к базовой модели генерирования изображений для получения наилучших результатов.

Диффузионные модели обучаются путем многократного добавления случайного шума (<https://oreil.ly/OrANA>) в изображение и последующих попыток обратить процесс диффузии путем *шумоподавления* (удаления шума). Этот подход заимствован из физики, где он используется для моделирования *диффузии* (распространения) частиц в среде. Результаты зависят от описания изображения, и если полученное изображение не соответствует описанию, то веса нейронной сети модели корректируются. После обучения модель способна, начав со случайного шума, получить изображение, соответствующее описанию в запросе.

Диаграмма на рис. 7.1, продемонстрированная Бинью Ваном (Binxu Wang) в статье *Mathematical Foundation of Diffusion Generative Models* (<https://oreil.ly/57szp>), иллюстрирует процесс шумоподавления.



**Рис. 7.1.** Схема процесса диффузии

Эти модели обучаются на больших наборах данных, состоящих из миллиардов изображений из Интернета (и сопровождающих их подписей), и поэтому могут воспроизводить самые популярные художественные стили или имитировать манеру письма известных художников. Это обстоятельство породило множество споров, поскольку правообладатели стремятся обеспечить соблюдение своих законных требований (<https://oreil.ly/a4Fyp>), а создатели моделей выступают за добросовестное использование.

Диффузионная модель — это не просто «сложный инструмент для создания коллажей», который создает копии изображений, защищенных авторским правом: ее размер составляет всего несколько гигабайт, и поэтому она не может содержать копии всех обучающих данных. Когда исследователи попытались воспроизвести 350 000 изображений из набора данных, на которых обучалась модель Stable Diffusion, им удалось получить только 109 из них (<https://oreil.ly/SGn9B>).

Своим поведением модель больше напоминает человека-художника, который просматривает изображения в Интернете и изучает закономерности, определяющие каждый предмет и стиль. Эти закономерности кодируются как *векторное представление* (список чисел), определяющее местоположение в *скрытом пространстве* всех возможных комбинаций изображений, которые могут быть созданы моделью. Вводимый пользователем запрос сначала кодируется в векторы; затем диффузионная модель генерирует векторное представление изображения, соответствующее этим векторам, после чего полученное представление декодируется в пиксели и возвращается пользователю.

На рис. 7.2 показана диаграмма процесса кодирования и декодирования из статьи Яна Стенбита (Ian Stenbit) *A Walk Through Latent Space with Stable Diffusion* (<https://oreil.ly/qOpis>).

Эти векторы, также называемые *эмбедингами*, действуют как местоположение или адрес точки на карте изображений, имеющейся у модели, и поэтому похожие

изображения располагаются ближе друг к другу в скрытом пространстве. Скрытое пространство является непрерывным, и в нем можно путешествовать между двумя точками (интерполировать) и в процессе путешествия получать достоверные изображения. Например, если интерполировать между изображениями собаки и вазы с фруктами, то полученные промежуточные изображения будут выглядеть связными, демонстрируя постепенный сдвиг между двумя концепциями.

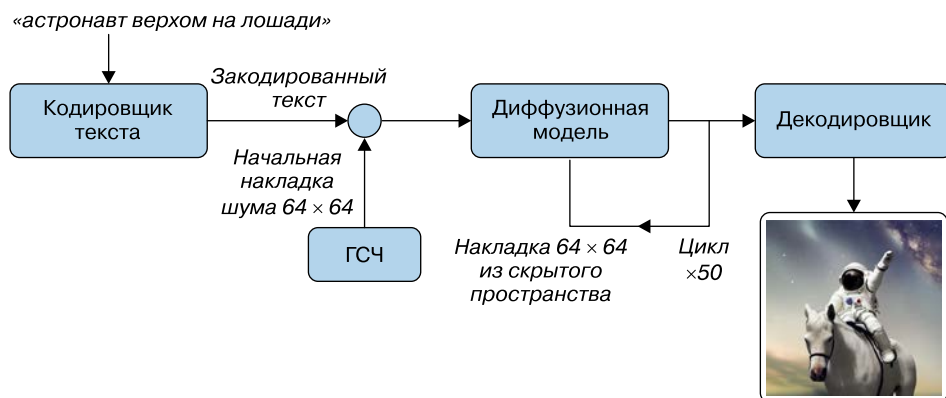


Рис. 7.2. Процесс кодирования и декодирования

На рис. 7.3 показана сетка из той же статьи Яна Стенбита, показывающая промежуточные этапы между четырьмя изображениями (<https://oreil.ly/cjm8A>): собака (вверху слева), ваза с фруктами (вверху справа), Эйфелева башня (внизу слева) и небоскреб (внизу справа).

В области диффузионных моделей промт-инжиниринг можно рассматривать как искусство навигации в скрытом пространстве в поисках изображения, соответствующего вашему видению, среди всех возможных доступных изображений. Существует множество методов и передовых практик подбора правильной комбинации слов для получения желаемого образа, и активное сообщество художников и исследователей искусственного интеллекта хорошо поработало над созданием набора инструментов, которые могут помочь в этом. Каждая модель имеет свои особенности и черты поведения, зависящие от архитектуры, метода обучения и данных, на которых она обучалась. Три основные организации, создавшие наиболее популярные диффузионные модели преобразования текста в изображения, использовали радикально разные подходы с точки зрения бизнес-моделей и функциональности, и поэтому выбор среди диффузионных моделей гораздо шире, чем среди больших языковых моделей, где доминирует OpenAI.

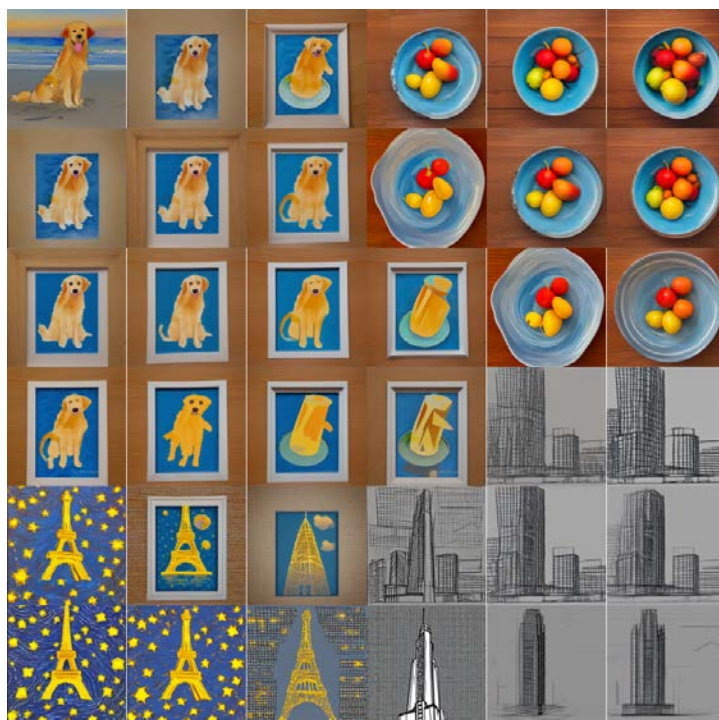


Рис. 7.3. Случайное путешествие по скрытому пространству

## OpenAI DALL-E

В январе 2021 года OpenAI выпустила модель преобразования текста в изображение DALL-E, ее название образовано от названия пьесы о художнике-сюрреалисте Сальвадоре Дали и анимационного робота Pixar WALL-E. Модель основана на модифицированной версии замечательной текстовой модели GPT-3 от OpenAI, выпущенной за семь месяцев до этого. Модель DALL-E стала прорывом в области генеративного ИИ, продемонстрировав способности в изобразительном искусстве, которые большинство людей считали невозможными для компьютера. На рис. 7.4 показаны примеры изображений, сгенерированных первой версией DALL-E (<https://oreil.ly/dalle1>).

Исходный код модели DALL-E не был открыт, но она вдохновила многих исследователей и любителей попытаться повторить ее успех. Самой популярной из таких моделей стала DALL-E Mini, выпущенная в июле 2021 года (год спустя переименованная в Craiyon по просьбе OpenAI), и хотя она приобрела немалую популярность в социальных сетях, качество сгенерированных ею изображений было значительно хуже, чем у официальной модели DALL-E. OpenAI

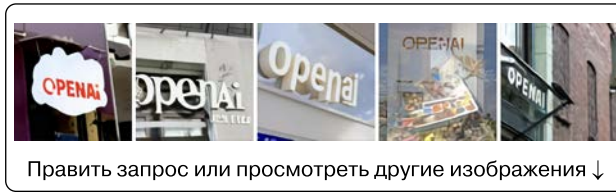
Текстовый запрос: кресло в форме авокадо

Изображения,  
сгенериро-  
ванные ИИ



Текстовый запрос: магазин со словом «openai» в названии

Изображения,  
сгенериро-  
ванные ИИ



**Рис. 7.4.** Демонстрация возможностей DALL-E

опубликовала документ, анонсирующий выход DALL-E 2 (<https://oreil.ly/EqdtP>) в апреле 2022 года, обладающей более высоким качеством, что увеличило список ожидающих до одного миллиона человек. На рис. 7.5 показан пример изображения астронавта верхом на лошади, ставшего знаковым и захватившего воображение публики.



**Рис. 7.5.** Изображение, сгенерированное моделью DALL-E 2

До сентября 2022 года доступ к модели имели только пользователи из списка ожидания из-за опасений по поводу этики и безопасности ИИ. Первоначально было запрещено генерировать изображения с людьми, а также введено ограничение на длинный список неpolitкорректных слов и выражений. Исследователи, например, определили, что создатели DALL-E 2 добавили запрет на использование слов «черный» и «женщина» (<https://oreil.ly/ot4vw>) в запросах на создание некоторых изображений, таких как фотография врача. Это была неуклюжая попытка устранить предвзятость, унаследованную из набора данных (на изображениях врачей в Интернете непропорционально много белых мужчин).

В августе 2022 года команда добавила в пользовательский интерфейс возможность пририсовывания и дорисовки, что стало еще одним шагом вперед и привлекло внимание прессы и социальных сетей. Эти функции позволяли пользователям генерировать отдельные элементы внутри или по границам существующих изображений. Однако пользователи почти не контролируют параметры модели и не могут дообучить ее на собственных данных. Модель генерировала искаженный текст на некоторых изображениях и воспроизводила нереалистичные изображения людей, создавая изуродованные или деформированные руки, ноги и глаза, как показано на рис. 7.6.



**Рис. 7.6.** Деформированные руки и глаза

В мае 2022 года была опубликована статья, описывающая модель Google Imagen (<https://oreil.ly/sFaeW>), которая продемонстрировала впечатляющие результаты, но эта модель не была представлена широкой публике из-за опасений за этику применения ИИ и соображений безопасности. Такие конкуренты, как Midjourney (июль 2022 года), быстро продвинулись вперед и извлекли выгоду из огромного спроса со стороны людей, которые видели впечатляющие демонстрации возможностей DALL-E в социальных сетях, но застряли в списке ожидания. Выпуск Stable Diffusion с открытым исходным кодом (август 2022 года) поставил под сомнение лидерство OpenAI, которое всего несколько месяцев назад казалось неоспоримым. Выпуск более продвинутой модели DALL-E 3 (<https://oreil.ly/dalle3>) в качестве функции ChatGPT помог OpenAI вернуть утраченные позиции, но к этому моменту в игру включилась Google со своей моделью Gemini 1.5 (<https://oreil.ly/XzQrU>), и начался новый виток конкуренции.



## Midjourney

В июле 2022 года, всего через три месяца после выхода DALL-E 2, компания Midjourney выпустила открытую бета-версию своей модели v3. Это было исключительно хорошее время для выпуска модели генерирования изображений, потому что возможности DALL-E 2, продемонстрированные первыми пользователями, выглядели как волшебство. К модели Midjourney тут же устремились первые поклонники, и ее выдающаяся фэнтезийная эстетика породила культ среди поклонников игр и цифрового искусства, что было продемонстрировано в теперь уже знаменитом изображении (<https://oreil.ly/dqshh>), получившем первый приз на конкурсе изобразительного искусства в категории цифровых технологий (рис. 7.7).



**Рис. 7.7.** Театр космической оперы

Midjourney стала одной из первых жизнеспособных моделей генерирования изображений, имевшей бизнес-модель и коммерческую лицензию, что сделало ее пригодной не только для экспериментов. Модель подписки пришлась по вкусу многим художникам, привыкшим ежемесячно платить за другое программное обеспечение, такое как Adobe Photoshop. Это также позволило художникам не платить за каждое созданное изображение, особенно в первые дни, когда приходилось делать несколько попыток, прежде чем получалось что-то достаточно высокого качества. Клиенты, оплачивающие услуги Midjourney, становились

владельцами прав на любые созданные изображения, в отличие от DALL-E, где авторские права сохранялись за OpenAI.

Уникальной особенностью Midjourney является ее ориентация на сообщество. Чтобы использовать этот инструмент, необходимо зайти на сервер Discord (<https://oreil.ly/JKZzD>) (рис. 7.8) и отправить запрос в открытом канале или напрямую. Учитывая, что все генерируемые изображения по умолчанию распространяются по открытым каналам, а приватный режим доступен только на самом дорогом тарифном плане (<https://oreil.ly/OV46r>), подавляющее большинство изображений, созданных с помощью Midjourney, доступны для изучения другими пользователями. Это привело к быстрому распространению опыта между пользователями и дало новичкам возможность учиться у других. Уже в июле 2022 года сообщество Discord насчитывало около 1 миллиона человек (см. рис. 7.8), а год спустя число участников перевалило за 13 миллионов.

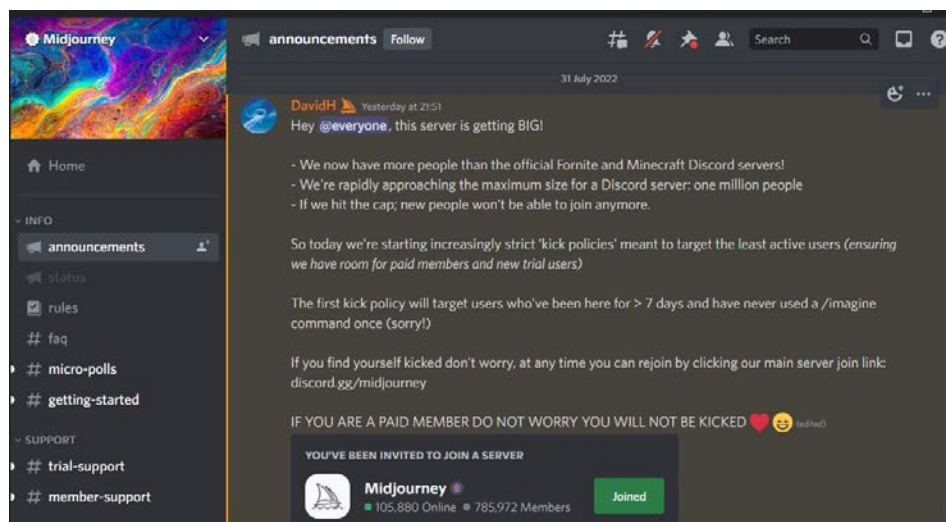


Рис. 7.8. Страница на сервере Midjourney Discord, июль 2022 года

Найдя понравившееся изображение, вы можете нажать кнопку, чтобы *увеличить его масштаб* (повысить разрешение). Многие предполагают, что эта процедура действует как обучение с подкреплением, аналогично обучению с подкреплением на основе отзывов людей (reinforcement learning from human feedback, RLHF) (<https://oreil.ly/3ISZk>), которое рекламируется как ключ к успеху ChatGPT. Кроме того, команда регулярно запрашивает оценки изображений, созданных новыми моделями, чтобы улучшить качество модели. В ноябре 2022 года Midjourney выпустила четвертую версию своей модели, в марте 2023-го — пятую версию и в декабре 2023-го — шестую. Качество значительно улучшилось: проблемы



с руками и глазами, показанные на рис. 7.6, были ликвидированы, а стилистический диапазон значительно расширился, как показано на рис. 7.9.

Ввод:

женщины, лучшие подруги, едят салаты и смеются  
в кафе при кинематографическом освещении

Вывод показан на рис. 7.9.



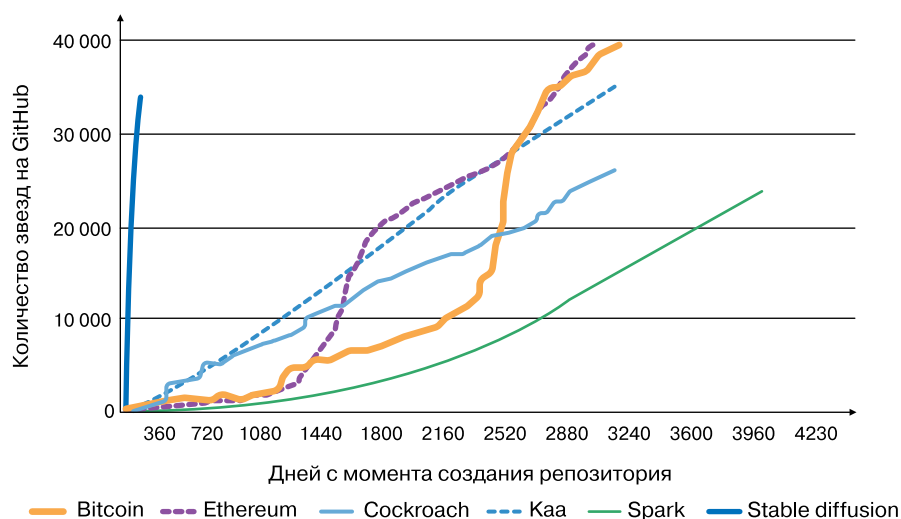
**Рис. 7.9.** Женщины едят салаты и смеются

Примечательно, что команда Midjourney остается очень небольшой: в марте 2023 года в ней числилось всего 11 сотрудников ([https://oreil.ly/YrmA\\_](https://oreil.ly/YrmA_)). Основатель Midjourney Дэвид Хольц (David Holz), ранее работавший в стартапе по производству аппаратного обеспечения Leap Motion, подтвердил в интервью (<https://oreil.ly/jeFYV>), что компания была прибыльной уже в августе 2022 года. Что еще более примечательно, без миллиардов финансовых вливаний, которыми располагает OpenAI, команда разработала значительный функционал сверх того, что доступно в DALL-E, включая отрицательные запросы (для

удаления фрагментов из изображений), взвешивание понятий (для увеличения их распространенности) и функцию *описания* (воссоздание запроса по загруженному изображению). Однако доступного API пока нет; единственный способ доступа к модели — через Discord, что, по всей видимости, тормозит массовое внедрение.

## Stable Diffusion

Пока список ожидания DALL-E 2 продолжал расти, исследователи из CompVis Group в Мюнхенском университете и компании прикладных исследований Runway ML получили в дар вычислительные мощности от Stability AI для обучения Stable Diffusion. Выпуск этой модели с открытым исходным кодом в августе 2022 года стал потрясением в мире генеративного ИИ, потому что ее результаты были сопоставимы с DALL-E 2, но при этом ее можно было бесплатно запустить на личном компьютере (при наличии скромного графического процессора с 8 Гбайт памяти VRAM). Проект Stable Diffusion показал самый большой рост популярности на GitHub среди всех программных проектов (<https://oreil.ly/pwPGX>), поднявшись до уровня 33 600 звезд за первые 90 дней (рис. 7.10).

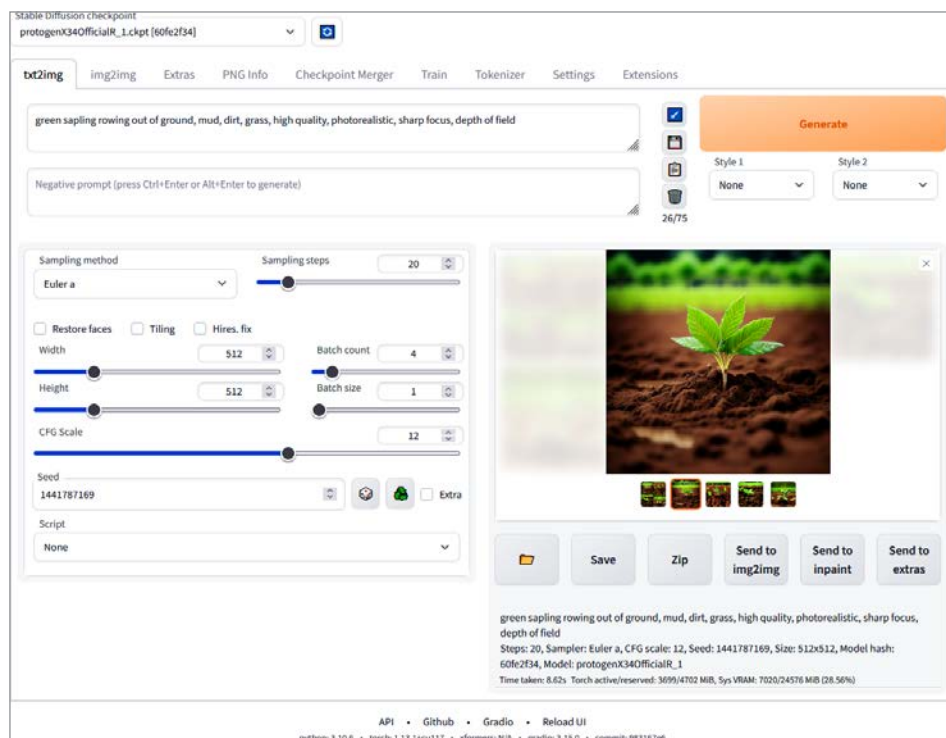


Динамика получения звезд на GitHub основными инфраструктурными технологиями с открытым исходным кодом. Проект Stable Diffusion собрал 33 600 звезд за первые 90 дней. Этого уровня другие проекты достигают за годы или даже десятилетия

**Рис. 7.10.** Рост популярности проекта Stable Diffusion на GitHub среди разработчиков

Открытие исходного кода модели вызвало жаркие споры и большую обеспокоенность по поводу этики и безопасности ИИ. И действительно, многие первоначальные попытки использования были направлены на создание ИИ-порно, о чем свидетельствуют модели «небезопасные на работе» (Not Safe For Work, NSFW), распространяемые на таких платформах, как Civitai (<https://civitai.com>). Однако возможность любителей и мастеров модифицировать и расширять модель, а также дообучать ее на своих данных способствовала быстрому развитию и усовершенствованию модели. Решение предоставить пользователям все параметры модели, такие как Classifier Free Guidance (точность следования запросу), Denoising (сколько шума добавить в базовое изображение, который модель должна удалить во время вывода) и Seed (случайный шум, начиная с которого запускается шумоподавление), привело к созданию большого количества творческих и инновационных произведений искусства. Доступность и надежность открытого исходного кода также побудили некоторые малые предприятия использовать Stable Diffusion, например PhotoAI (<https://photoai.com>) Питера Левела (Pieter Level) и InteriorAI (<http://Interiorai.com>), совокупный ежемесячный доход которых превысил 100 000 долларов США, а также HeadshotPro (<https://www.headshotpro.com>) Дэнни Постмы (Danny Postma). Помимо функций дорисовки, впервые появившейся в DALL-E, модель с открытым исходным кодом также обладает многими возможностями Midjourney, такими как отрицательные запросы, взвешивание понятий и воссоздание запросов по имеющимся изображениям. Кроме того, в Stable Diffusion были быстро (уже в апреле 2023 года) реализованы дополнительные функции, такие как ControlNet (согласование позы или композиции изображения) и Segment Anything (щелчок на элементе для создания маски и внедрения фрагмента изображения), доступ к которым осуществляется через веб-интерфейс AUTOMATIC1111 (<https://oreil.ly/0inw3>) (рис. 7.11).

Версия 1.5 модели Stable Diffusion была выпущена в октябре 2022 года и продолжает использоваться до сих пор. По этой причине она станет основой для примеров ControlNet в главе 10, расширенном разделе по генерированию изображений в этой книге. Веса модели Stable Diffusion были опубликованы на Hugging Face — центре моделей ИИ с открытым исходным кодом. Версия Stable Diffusion 2.0 вышла месяц спустя, в ноябре 2022 года, и была обучена на более эстетичном подмножестве исходных данных LAION-5B (<https://oreil.ly/K5vX2>) (крупномасштабный набор данных пар изображений и текста для исследовательских целей), из которого были удалены изображения NSFW (небезопасные на работе). Опытные пользователи Stable Diffusion жаловались на цензуру, а также на ухудшение качества модели, предполагая (<https://oreil.ly/2mgh5>), что изображения NSFW в обучающем наборе необходимы для создания реалистичной анатомии человека.



**Рис. 7.11.** Веб-интерфейс AUTOMATIC1111 для доступа Stable Diffusion

Компания Stability AI собрала более 100 миллионов долларов США (<https://oreil.ly/BT-k5>) и продолжила разработку новых моделей, включая DeepFloyd (<https://oreil.ly/UCQ3I>) — модель, способную генерировать реальный текст на изображениях (проблема, которая остается неразрешенной в других моделях), и текущую версию Stable Diffusion XL 1.0 (<https://oreil.ly/gcT4t>) (сокращенно SDXL). Эта модель преодолела опасения сообщества по поводу цензуры в версии 2.0 не в последнюю очередь благодаря впечатляющим результатам этой более мощной модели, имеющей 6,6 миллиарда параметров по сравнению с 0,98 миллиарда в модели v1.5.

## Google Gemini

Корпорация Google давно грозилась стать конкурентом в этой сфере со своей моделью Imagen (<https://oreil.ly/K8oWv>), которая пока не была выпущена публично (<https://oreil.ly/341QB>). С тех пор бывшие сотрудники Google основали новую

многообещающую модель Ideogram (<https://ideogram.ai>), которая была выпущена в августе 2023 года. А сама Google вступила в игру со своей моделью Gemini в декабре 2023 года, но быстро столкнулась с критикой из-за неуклюжей попытки продвижения разнообразия (<https://oreil.ly/u-Glg>). Еще неизвестно, помешает ли внутренняя политика Google им извлечь выгоду из своих значительных ресурсов.

## Преобразование текста в видео

В скором будущем ожидается, что большая часть внимания сместится из сферы генерирования изображений в область преобразования текста в видео, изображений в видео и даже видео в видео, учитывая, что сообщество Stable Diffusion приступило к расширению возможностей (<https://oreil.ly/l7KHB>) по созданию согласованных кадровых изображений и появились многообещающие проекты с открытым исходным кодом, такие как AnimateDiff (<https://oreil.ly/CsJgT>). Кроме того, один из участников проекта Stable Diffusion, компания RunwayML, заняла лидирующие позиции в области преобразования текста в видео и начала получать полезные результаты с помощью своей модели Gen-2 (<https://oreil.ly/vS0mA>). В ноябре 2023 года вышла модель Stable Video Diffusion (<https://oreil.ly/UuApM>), способная превращать текст в короткие видеоролики и анимировать существующие изображения, а модель Stable Diffusion Turbo (<https://oreil.ly/uMAkh>) способна генерировать изображения практически в режиме реального времени. Вышедшая в феврале 2024-го модель Sora (<https://oreil.ly/sora>) показала, что OpenAI тоже не дремлет. Мы не будем явно рассматривать методы преобразования текста в видео, но все, что вы узнаете о запросах для создания изображений, также применимо к созданию видео.

## Сравнение моделей

По мере роста спроса на создание изображений с помощью ИИ и обострения конкуренции будут появляться новые игроки, а основные будут диверсифицироваться. Мы в наших рабочих процессах уже используем разные модели для решения разных задач. DALL-E 3 отличается превосходной способностью к композиции, а интеграция с ChatGPT дает дополнительные удобства. Midjourney по-прежнему обладает лучшей фэнтезийной эстетикой и фотореалистичностью. Благодаря открытому исходному коду Stable Diffusion является наиболее гибкой и расширяемой моделью, и именно на ее основе большинство компаний, занимающихся искусственным интеллектом, строят свои продукты. Каждая модель развивала свой набор возможностей в своем направлении, что можно заметить при сравнении результатов обработки одного и того же запроса несколькими моделями (рис. 7.12).

Ввод:

корги на вершине Бранденбургских ворот

Вывод показан на рис. 7.12.



Рис. 7.12. Корги на вершине Бранденбургских ворот

## Резюме

В этой главе вы познакомились с диффузионными моделями ИИ, генерирующими изображения. Эти модели, примерами которых могут служить DALL-E, Stable Diffusion и Midjourney, используют методы генерирования случайного шума и шумоподавления для создания изображений на основе текстовых описаний. Они прошли обучение на огромных наборах данных и могут воспроизводить самые разные художественные стили. Однако вокруг них ведется много споров по поводу авторских прав на получаемые результаты. Вы также узнали, как принципы промт-инжиниринга помогают генерировать нужные изображения при навигации по скрытому пространству.

Мы исследовали разные подходы, применяемые такими организациями, как OpenAI, Stability AI и Midjourney, при разработке моделей преобразования текста в изображение. Модель DALL-E от OpenAI приобрела популярность благодаря своим изобразительным способностям, но доступ к ней ограничен, а качество ее менее мощных копий хуже. Midjourney, в свою очередь, воспользовалась высоким спросом на модели, подобные DALL-E, и приобрела массу поклонников, выпустив свои модели v3 и v4. Она предлагает доступ к своим моделям на основе платной подписки и сильно ориентирована на сообщество.

Модель Stable Diffusion привлекла всеобщее внимание результатами, сопоставимыми с DALL-E и Midjourney, открытостью и возможностью бесплатно

запускать ее на персональных компьютерах. В этой главе вы также познакомились с историей развития ИИ, генерирующего изображения, и достижениями таких организаций, как OpenAI, Midjourney и Stable Diffusion.

В следующей главе мы дадим вам практические советы по созданию изображений с помощью ИИ. В ней вы познакомитесь с приемами и методами, применение которых позволяет получать потрясающие и уникальные изображения. Освоив методы управления форматом и копирования художественного стиля, вы научитесь создавать увлекательный и оригинальный визуальный контент. Будьте готовы раскрыть свой творческий потенциал и поднять свои навыки создания изображений на новый уровень.

## ГЛАВА 8

---

# Стандартные практики создания изображений с помощью Midjourney

В этой главе мы продемонстрируем стандартные методы, способствующие получению качественных результатов от диффузионных моделей. Для начала мы займемся адаптацией запросов, чтобы исследовать распространенные методы, используемые для создания изображений. Все изображения в последующих примерах созданы моделью Midjourney v5, если явно не указано иное. Обсуждаемые методы были разработаны с прицелом на возможность переноса на любую будущую или альтернативную модель.

## Управление форматом

Самый простой способ сгенерировать изображение — указать его формат. Модели генерирования изображений способны использовать самые разные форматы: от стандартных фотографий до картин маслом и древнеегипетских иероглифов. Изображения часто выглядят совершенно по-разному в зависимости от формата, включая стиль объектов или людей на изображении. Многие изображения в обучающих данных являются стоковыми фотографиями — одной из наиболее коммерчески важных категорий изображений для обучения генеративных моделей.

Ввод:

стоковая фотография деловой встречи

Вывод показан на рис. 8.1.

Возможность бесплатно создавать бесконечное количество стоковых фотографий с помощью моделей с открытым исходным кодом, таких как Stable Diffusion, или за очень низкую плату с помощью DALL-E или Midjourney сама



по себе меняет правила игры. Каждое из этих изображений уникально (хотя и может иметь некоторое сходство с существующими изображениями), и поэтому они выглядят более престижно, чем повторное использование тех же бесплатных стоковых фотографий, доступных всем остальным. Однако вам не обязательно ограничиваться форматом стоковой фотографии. Если ваша статья в блоге или изображение на сайте будут смотреться лучше, если придать им чуть более художественное оформление, то вы можете сделать это практически без ограничений.



**Рис. 8.1.** Стоковая фотография деловой встречи

Ввод:

картина маслом, изображающая деловую встречу

Вывод показан на рис. 8.2.



#### Укажите формат

Указание формата существенно меняет результат, генерируемый моделью ИИ, а также повышает надежность наших запросов, поскольку в ответ мы получаем тот тип визуального представления, который нам нужен.



**Рис. 8.2.** Картина маслом, изображающая деловую встречу

С помощью этой техники можно зайти очень далеко, и определенно ее применение помогло бы вам поступить в художественную школу. Зная название конкретной техники или детали, которую вы хотите видеть на своем изображении, например импасто — техники, используемой в живописи маслом, когда краска наносится на участок холста толстым слоем, оставляя хорошо видимые мазки, — вы можете указать это название в запросе, чтобы приблизиться к желаемому результату. Google поддерживает полный список популярных художников и художественных направлений (<https://oreil.ly/OmZbl>), который многие считают полезным.

Ввод:

картина маслом, изображающая деловую встречу, фактурный холст, использование техники импасто и динамичные закрученные мазки

Вывод показан на рис. 8.3.

Теперь картина маслом, изображающая деловую встречу, визуально смотрится гораздо интереснее. Традиционно одна из причин, почему бизнес перешел на

использование стоковой фотографии, заключалась в том, что это было дешевле, чем заказывать картины, но ИИ избавил нас от этого ограничения. Мы можем сгенерировать изображение практически в любом формате, который нам нравится, даже в форме древнеегипетского иероглифа.



**Рис. 8.3.** Картина маслом в технике импасто, изображающая деловую встречу

Ввод:

древнеегипетский иероглиф деловой встречи

Вывод показан на рис. 8.4.

При изменении формата следует учитывать, что стиль изображения и даже его содержимое обычно имеют тенденцию соответствовать обучающим данным, связанным с этим форматом. Например, в сгенерированных картинах маслом нет компьютеров, потому что они нечасто встречаются в живописи. Аналогично в сгенерированном иероглифе участники встречи одеты в древнеегипетские одежды. Часто вам придется комбинировать управление форматом с другими методами, чтобы получить желаемое.

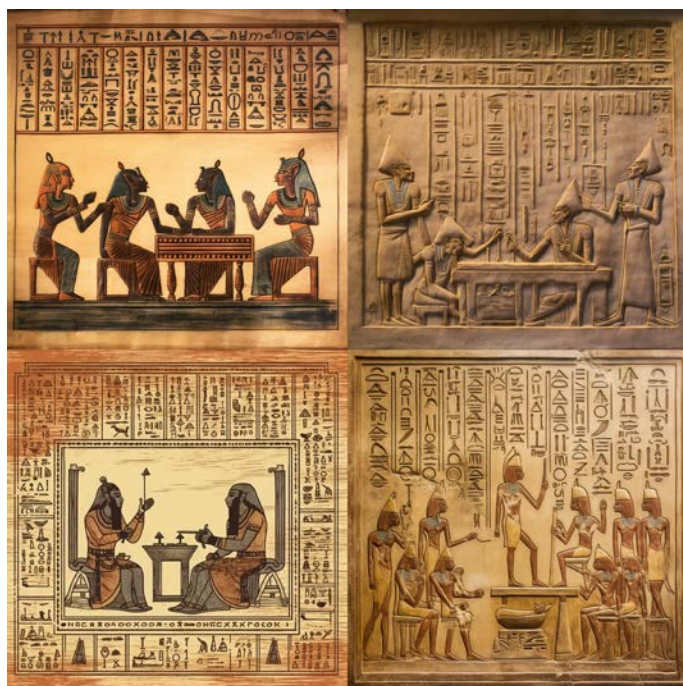


Рис. 8.4. Древнеегипетский иероглиф деловой встречи

## Управление художественным стилем

Одной из величайших возможностей моделей генерирования изображений является их способность воспроизводить популярные художественные стили или стили известных художников. Наиболее распространенными примерами, публикуемыми в социальных сетях и демонстрациях возможностей ИИ, являются изображения в стиле Ван Гога, Дали или Пикассо, а также движений художников, частью которых они были, таких как постимпрессионизм, сюрреализм и кубизм. Сообщества творчества искусственного интеллекта также оказали свое влияние на определение популярных стилей современного искусства, как в случае с польским цифровым художником Грегом Рутковски (Greg Rutkowski) (<https://oreil.ly/nnam3>), известным своим стилем фэнтези. Однако многие художники выступили против того, чтобы называть искусством произведения, созданные искусственным интеллектом, и существует юридическая серая зона относительно того, считается ли имитация стиля живого художника *добросовестным использованием* в соответствии с законом об авторском праве. Мы рекомендуем художникам, работающим с ИИ, проявлять осторожность при создании произведений в характерном стиле любого ныне живущего художника и придерживаться эмпирического правила подражания стилю художников,



умерших год или более тому назад (в случае любого запланированного коммерческого использования обратитесь к юристу).

Ввод:

иллюстрация с изображением дракона в стиле сказки Льюиса Кэрролла "Приключения Алисы в Стране Чудес"

Вывод показан на рис. 8.5.



**Рис. 8.5.** Иллюстрация с изображением дракона в стиле сказки Льюиса Кэрролла



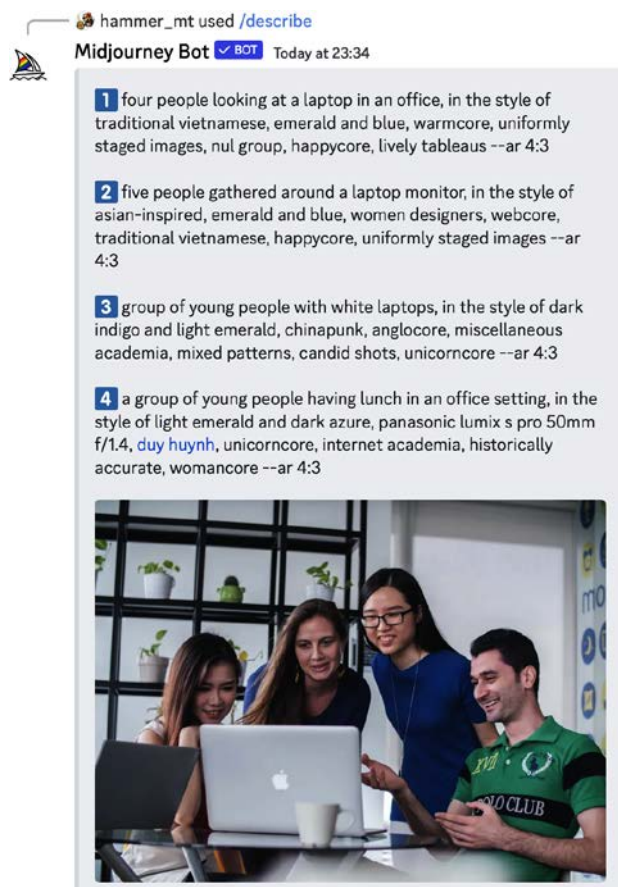
#### Задайте направление

Упоминание имени художника или названия художественного направления — это кратчайший путь к созданию определенного художественного стиля. Пока художник или художественное движение имеет достаточно примеров в обучающих данных, их стилю можно подражать.

Указывая художественный стиль, вы фактически перепрыгиваете в конкретную область скрытого пространства в многомерной вселенной потенциальных результатов модели, соответствующую желаемому стилю. Перемещение оттуда в близлежащие точки может помочь добраться до более желанного места назначения быстрее, чем методом проб и ошибок.

## Реконструирование запросов

Тем, кто не ходил в художественную школу или мало разбирается в кино или фотографии, сложно определить художественные стили, форматы и художников, которых было бы желательно упомянуть в запросах. Нередко случается так, что вы видите понравившуюся картинку, но не можете описать ее достаточно подробно, чтобы воспроизвести нечто подобное с помощью ИИ. К счастью, в Midjourney имеется функция описания, позволяющая реконструировать запрос по изображению, для чего нужно ввести `/describe` и затем загрузить изображение. Эта функция работает для любых изображений — и созданных искусственным интеллектом, и обычных изображений из других источников. На рис. 8.6 показан пример реконструирования запроса для одной из фотографий из главы 1.



**Рис. 8.6.** Описание фотографии Мими Тиан (Mimi Thian) (<https://unsplash.com/@mimithian>), доступной на Unsplash (<https://oreil.ly/bEEEnJ>)

Midjourney предложила четыре варианта запросов с указанием разных художников, художественных стилей и форматов, а также включила оценку происходящего и перечень содержащихся предметов и элементов на изображении. Например, Midjourney правильно идентифицировала группу людей, смотрящих на экран ноутбука в офисе (см. рис. 8.6). Вы можете выбрать нужный вариант по номеру, и Midjourney сгенерирует по этому запросу изображение в том же стиле. Существует аналогичная технология с открытым исходным кодом под названием CLIP Interrogator (<https://oreil.ly/fzgno>), но она не отличается богатством генерируемых описаний и способностью реконструировать стиль загруженного изображения, которыми славится Midjourney.

## Усилители качества

Один из приемов, безотказно помогающий улучшить генерируемые изображения, — добавление в запрос слов, связанных с качеством. Некоторые художественные стили более эстетичны, чем другие, но есть набор слов, известных как *усилители качества*, которые улучшают качество изображения, не сильно влияя на стиль, например *4K*, *очень красиво* и *в тренде ArtStation*. Генеративные модели не пытаются создавать высококачественные изображения; они пытаются имитировать комплекс обучающих данных, включающий изображения в самых разных стилях и разного качества. Если нужны высококачественные изображения, то вы должны явно попросить их. Начните текст запроса с основной темы, например «космический кит», и через запятую добавьте модификатор (как показано на рис. 8.7).

Ввод:

космический кит, в тренде на artstation

Вывод:

Космический кит



Космический кит,  
в тренде на artstation



Рис. 8.7. Космический кит, в тренде на artstation



### Задайте направление

Добавление в запрос одного-двух слов усилителей качества может улучшить эстетику изображения, не меняя при этом его общий стиль.

Положительное влияние этих слов обусловлено тем, что они связаны с качеством обучающих данных. По сообщениям, многие модели ИИ обучались на изображениях, импортированных с популярных веб-сайтов художников, таких как ArtStation, Behance и DeviantArt. Поэтому модель может предположить, что изображение, которое было «в тренде на artstation», имеет более высокую эстетическую ценность, чем обычно. Обратите внимание, что иногда в сгенерированные изображения может просачиваться стиль, не соответствующий вашему творческому видению. Например, ArtStation содержит много цифровых изображений космических кораблей, и это, возможно, объясняет, почему космический кит на рис. 8.7 чем-то напоминает космический корабль. Список усилителей качества, художественных стилей и художников можно найти в шаблоне Prompt Engineering Template (<https://oreil.ly/afGCQ>), созданном одним из авторов. Google тоже ведет полный список художественных направлений (<https://oreil.ly/mhujK>), который может пригодиться для изучения названий стилей, которые вам нравятся.

## Отрицательные запросы

Часто две концепции настолько тесно переплетены в обучающих данных, что появляются вместе при создании изображений с применением одной из концепций, даже если это явно не то, что вы указали в запросе. Например, когда, запрашивая картины маслом, модели часто добавляют раму вокруг картины и стену, потому что эти элементы присутствуют на большинстве изображений музейных коллекций картин.

В Midjourney и Stable Diffusion есть возможность добавлять *отрицательные запросы* (*negative prompts*) и указывать в них, чего не должно быть на изображении. С помощью отрицательных запросов можно эффективно разделить две взаимосвязанные концепции и гарантировать отсутствие на изображении ненужных элементов. Например, запрашивая картину, написанную маслом, можно в конец запроса добавить --no и затем через запятую перечислить все, чего не должно быть на изображении. Чтобы решить проблему с рамой, добавьте «рама» и «стена» в качестве отрицательного запроса, как показано на рис. 8.8.



Ввод:

картина маслом в стиле Рембрандта --по рама, стена

Вывод показан на рис. 8.8.

Картина маслом  
в стиле Рембрандта



Картина маслом в стиле  
Рембрандта --по рама, стена



**Рис. 8.8.** Картина маслом в стиле Рембрандта без рамы и стены



#### Задайте направление

Отрицательные запросы могут убрать нежелательные элементы из изображений, уводя модель подальше от областей, которых вы пытаетесь избежать. Этот прием не всегда работает так, как задумывалось, потому что иногда понятия, которые требуется разделить, оказываются слишком тесно связанными, но когда все получается правильно, это может привести к интересным результатам.

Отрицательные запросы не являются абсолютно надежными, но могут быть полезны в самых разных ситуациях. Один из творческих вариантов использования этой техники — добавление имени знаменитости в виде отрицательного запроса, чтобы уменьшить количество факторов, наиболее связанных с ней. Знаменитая актриса Карен Гиллан имеет рыжие волосы и традиционный женственный вид, поэтому ее имя можно использовать, чтобы снизить вероятность, что человек на изображении будет иметь рыжие волосы или традиционный женственный вид.

Ввод:

шотландская женщина-космонавт --по Карен Гиллан

Вывод показан на рис. 8.9.



**Рис. 8.9.** Менее женственная, менее рыжеволосая шотландская женщина-космонавт

Используя эту технику, можно получить очень необычные результаты, если взять две неразделимые концепции и попытаться их разделить. Например, попробуйте взять свой любимый мультфильм и удалить мультяшный стиль, как в случае с Гомером Симпсоном (рис. 8.10).

Ввод:

Гомер Симпсон --по мультяшный стиль

На рис. 8.10 показан (довольно пугающий) результат.

Одним из наиболее распространенных вариантов использования отрицательных запросов является исправление проблем с изуродованными руками, частями тела или странного вида глазами, от которых страдали ранние модели ИИ.

Промт-инженеры добавляли в отрицательный запрос такие слова, как *nsfw*<sup>1</sup>, *удлиненное тело*, *слишком много цифр*, *недостаточно пальцев и зубы*, пытаясь (часто тщетно) увести модель от этих областей в скрытом пространстве.



**Рис. 8.10.** Гомер Симпсон без фирменного мультяшного стиля

Этими проблемами по-прежнему страдают старые или менее мощные модели, такие как Stable Diffusion v1.5, но начиная с Midjourney v5 и Stable Diffusion XL эта проблема практически решена. Современные модели способны создавать нормальные изображения рук, глаз и тел без отрицательных запросов.

## Взвешенные понятия

Отрицательные запросы полезны, когда требуется что-то полностью исключить, но часто бывает нужно просто смягчить некоторый аспект. Для смешивания и сочетания разных понятий может быть полезно контролировать, чего и сколько вам нужно.

По умолчанию все слова в запросе имеют одинаковый вес, равный 1, хотя слова в начале запроса имеют больший эффект, поэтому мы обычно помещаем в начало описание основного объекта на изображении, например *вид на мост Золотые Ворота*.

---

<sup>1</sup> Not safe for work — «небезопасно на работе». Так обозначается нецензурный контент. — *Примеч. пер.*

Вы можете изменить вес понятий в запросе к Midjourney, добавив *жесткое требование* в виде двух символов двоеточия :: и числа, обозначающего новый вес. С помощью этого метода можно создать картину в основном в стиле Ван Гога, но с примесью стиля Дали.

Ввод:

вид на мост Золотые Ворота::1 в стиле Ван Гога::0.8, в стиле Дали::0.2

Вывод показан на рис. 8.11.



**Рис. 8.11.** Вид на мост Золотые Ворота в стиле Ван Гога и Дали

Чтобы увидеть, как веса влияют на генерируемое изображение, попробуйте менять веса по сетке, чтобы проверить каждую комбинацию весов с определенной степенью детализации. В этом примере веса между двумя художниками менялись с шагом 0,2 от 0 до 1.

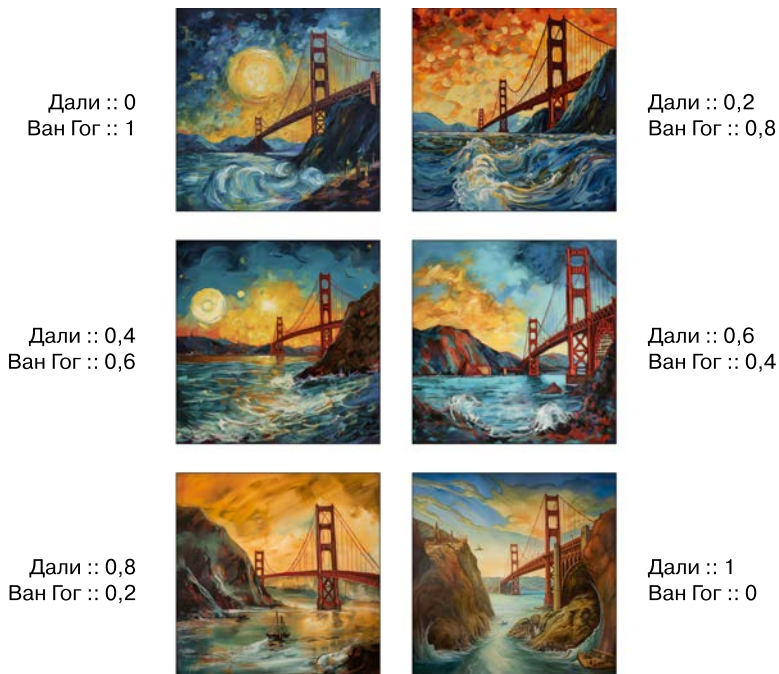
Результат показан на рис. 8.12.



#### Оцените качество

Перебор всех возможных комбинаций весов в запросе по одной за раз может занять много времени. Поэтому рекомендуется использовать метод поиска по сетке, который систематически генерирует множество возможных комбинаций, чтобы определить, какое сочетание весов идеально соответствует вашим предпочтениям.

Вес может быть больше 1, если это необходимо для создания акцента на некотором аспекте, или ниже, если нужно что-то ослабить. Можно также добавить отрицательные веса, чтобы удалить ненужный аспект. Параметр `--no`, используемый в отрицательных запросах, на самом деле является просто коротким способом добавить `-- -0.5` к элементам, перечисленным в этом разделе. Если вас беспокоит, что на изображении появляется что-то нежелательное, попробуйте использовать более веские отрицательные коэффициенты. Так, в предыдущем примере можно исключить любое влияние Ван Гога из работ Дали, добавив вес `-1` к Ван Гогу и увеличив вес Дали до 5.



**Рис. 8.12.** Перебор возможных весов по сетке

Ввод:

вид на мост Золотые Ворота::1 в стиле Ван Гога::-1, в стиле Дали::5

Вывод показан на рис. 8.13.

Веса могут стать мощным инструментом при смешивании разных стилей или для выделения определенных элементов. Существует множество вариаций весов, поэтому старайтесь использовать более систематический подход, чтобы найти эстетически интересное пространство для игры.



Рис. 8.13. Вид на мост Золотые Ворота

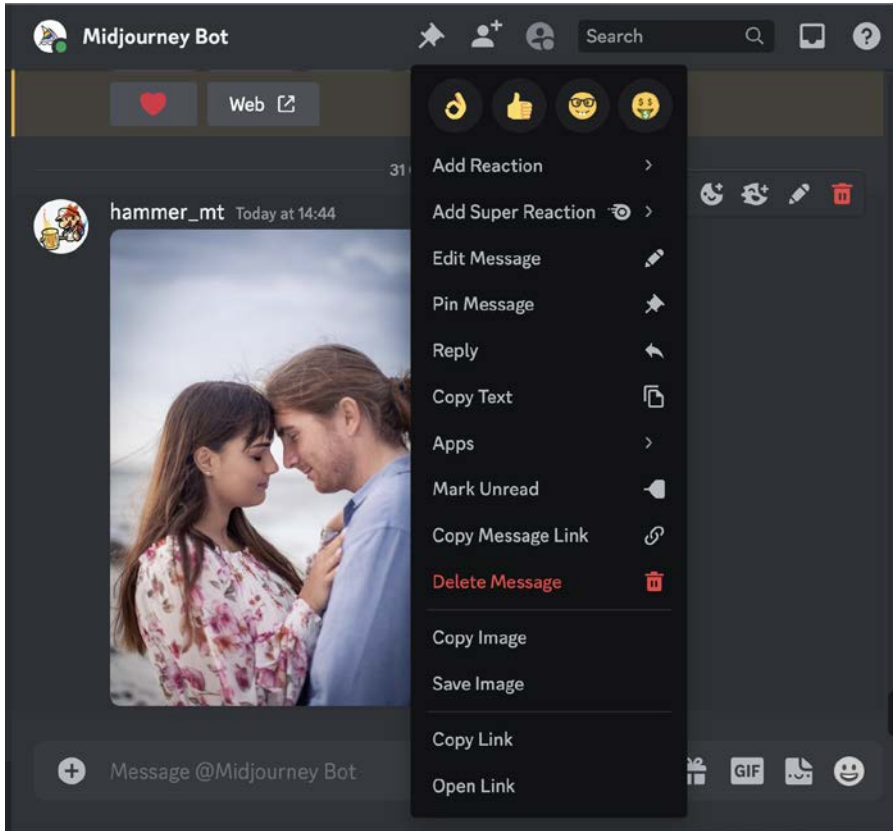
## Запросы с изображениями

Многие модели ИИ, генерирующие изображения, позволяют подставлять в запросы не только текст, но и изображения. Пример желаемого изображения может стать отличной основой для создания чего-то более уникального и оригинального, но при этом соответствующего нужному вам стилю. В сообществе Stable Diffusion это называется *Img2Img*, а в Midjourney можно просто указать в запросе ссылку на изображение. Лучший способ сделать это — сначала загрузить изображение в Discord (возьмем для примера фото Джессики Хирн (Jessica Hearn) (<https://oreil.ly/B6E0Y>) на Unsplash (<https://oreil.ly/0oO4w>)). Щелкните правой кнопкой мыши на фотографии и в контекстном меню выберите пункт *Copy Link* (Копировать ссылку), как показано на рис. 8.14.

Затем скопированную ссылку вставьте в начало запроса Midjourney и сопроводите текстовым описанием желаемого. Теперь, когда имеется изображение-образец, не нужно описывать желаемое во всех деталях (как известно, изображение стоит тысячи слов). Результат не будет иметь точного совпадения



с образцом, но сходство между ними будет вполне узнаваемым, особенно если вы знаете, какое изображение играло роль образца и что было написано в текстовой части запроса.



**Рис. 8.14.** Копирование ссылки на изображение на сайте Discord

Ввод:

<https://s.mj.run/XkIHsYIdUxc> в стиле Великого Гэтсби

Результат показан на рис. 8.15.



Правила и положения, касающиеся авторских прав и добросовестного использования ИИ, все еще разрабатываются, поэтому будьте осторожны, загружая изображения, на которые у вас нет прав.

Этот метод дает хорошие результаты всегда, когда нужно воссоздать атмосферу, сцену или композицию, похожие на известное изображение. Можете также смешать несколько изображений вместе, добавив ключ `/blend`, и получить что-то совершенно уникальное, а полученное изображение можно даже использовать в качестве образца в другом запросе. Для удобства существует параметр `--iw`, действующий подобно весовому параметру `:`, но для изображений. Запросы с образцовыми изображениями также часто используются для создания видео с помощью таких моделей ИИ, как RunwayML (<https://runwayml.com>) и Pika Labs (<https://pika.art>), что помогает устранить неоднозначности, свойственные преобразованию текста в видео, и позволяет менять стиль сцены, не дожидаясь генерации и рендеринга всего видео.



**Рис. 8.15.** Фотография в стиле Великого Гэтсби



### Приведите примеры

Самый простой и быстрый способ получить желаемое изображение — загрузить изображение, которое нужно симитировать. Это похоже на пробный запрос в генерировании текста и помогает подтолкнуть модель в правильном направлении.



## Изменение фрагментов изображений

Работа с инструментами ИИ, генерирующими изображения, — это всегда итеративный процесс. Редко удается получить законченное изображение с первой попытки. Обычно получаемые изображения содержат артефакты, которые нужно устранить, или не совсем точно передают желаемый стиль. Например, предположим, что вы с помощью Midjourney создали изображение женщины в платье-чарльстон в стиле 1920-х годов, но хотели изменить ее одеяние, не пересоздавая изображение целиком.

Решить задачу может помочь функция *изменения фрагментов изображений* (inpainting), доступная в большинстве реализаций Stable Diffusion (<https://oreil.ly/YgL8g>), в Midjourney (<https://oreil.ly/7DhZE>) с помощью функции *Vary Region* и генеративной заливки в Adobe Photoshop (<https://oreil.ly/FvGAi>). Однако пионером в реализации этой функции стала компания DALL-E, и ее модель, как нам кажется, дает самые качественные результаты. Для демонстрации этой функциональности сначала создадим изображение с помощью DALL-E в ChatGPT (Plus), а затем изменим нужную нам часть изображения.

На рис. 8.16 показано изображение, созданное моделью DALL-E по запросу, представленному ниже. Изображение находится в процессе редактирования на холсте DALL-E (в настоящее время с использованием ChatGPT), и часть изображения с платьем стерта с помощью инструмента «кисть».

Ввод:

фотография гламурной женщины в платье-чарльстон на вечеринке 1920-х годов, цветной снимок сделан зеркальной камерой с широкоугольным объективом 35 мм

На рис. 8.16 показано выгруженное изображение со стертыми фрагментами.



Рис. 8.16. Изменение фрагментов изображений в DALL-E

Затем добавим запрос, описывающий, что нужно нарисовать в этом пространстве. Обычно рекомендуется описывать изображение целиком, но, как показывает наш опыт, фокусировка запроса на стертой части изображения дает лучшие результаты, как можно видеть на рис. 8.17, — в этом примере основное внимание в запросе уделено самому платью.

Ввод:

Платье в стиле картины "Звездная ночь" Ван Гога, синего цвета с очень детализированными желтыми завитками, хорошее студийное освещение, снимок сделан зеркальной камерой с широкоугольным объективом 35 мм

Вывод показан на рис. 8.17.



**Рис. 8.17.** Платье с рисунком в стиле Ван Гога

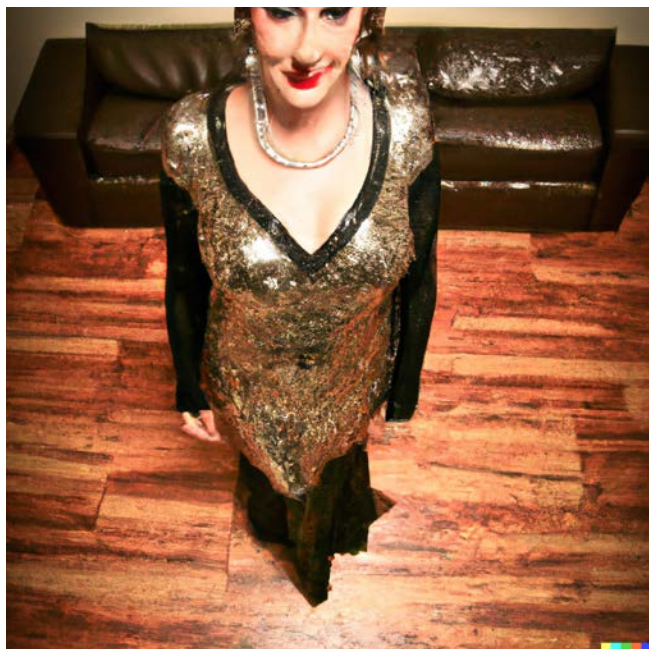


#### Разделите задачу на подзадачи

Важно правильно выбрать модель для работы. Некоторые модели, такие как Midjourney, хороши для создания изображений в конкретном стиле или с определенной эстетикой, тогда как другие славятся расширенными возможностями, такими как функция изменения фрагментов изображений в DALL-E. Совместное использование нескольких моделей может расширить ваши возможности.

DALL-E имеет меньше функций, чем большинство других моделей, но отлично справляется с этой конкретной техникой. Она автоматически смешивает края,

чтобы изображение хорошо вписывалось в окружение, поэтому необязательно соблюдать ювелирную точность при работе со стирающей кистью — вы все равно получите хороший результат. Особенно примечательно, насколько далеко продвинулись эти модели всего за год. На рис. 8.18 показано, что получится, если отправить DALL-E тот же запрос, который прежде мы отправили модели DALL-E 3 через ChatGPT.



**Рис. 8.18.** Фотография женщины на гламурной вечеринке 1920-х годов

DALL-E 3 обеспечивает превосходное качество, но на данный момент доступна только через API и в ChatGPT; она недоступна в интерфейсе OpenAI Labs, который исторически использовался для изменения фрагментов изображений. Разные модели, генерирующие изображения, различаются своими возможностями и вариантами использования, поэтому для выполнения поставленной задачи может потребоваться использовать несколько моделей. Функция изменения фрагментов изображений (inpainting) — это мощный метод редактирования изображений, независимо от того, как они созданы — другими моделями ИИ или реальным фотографом.

## Дорисовывание обрамления изображений

С функцией изменения фрагментов изображений (inpainting) в DALL-E тесно связана функция *дорисовывания обрамления* (outpainting), которая генерирует изображение за пределами кадра существующего изображения. Такой метод позволяет фактически *уменьшить масштаб* изображения, чтобы добавить вокруг него дополнительный контекст. Эту функцию можно использовать для заполнения имеющегося изображения дополнительными деталями. Дорисовывание обрамления больше недоступно в интерфейсе OpenAI Lab и пока недоступно в ChatGPT, но доступно в Midjourney, где эта функция называется Zoom Out (уменьшение масштаба) и предназначена для обработки и дополнения изображений увеличенного масштаба. Возьмем для примера изображение женщины в гламурном платье, показанное на рис. 8.19, и применим к нему эту функцию.

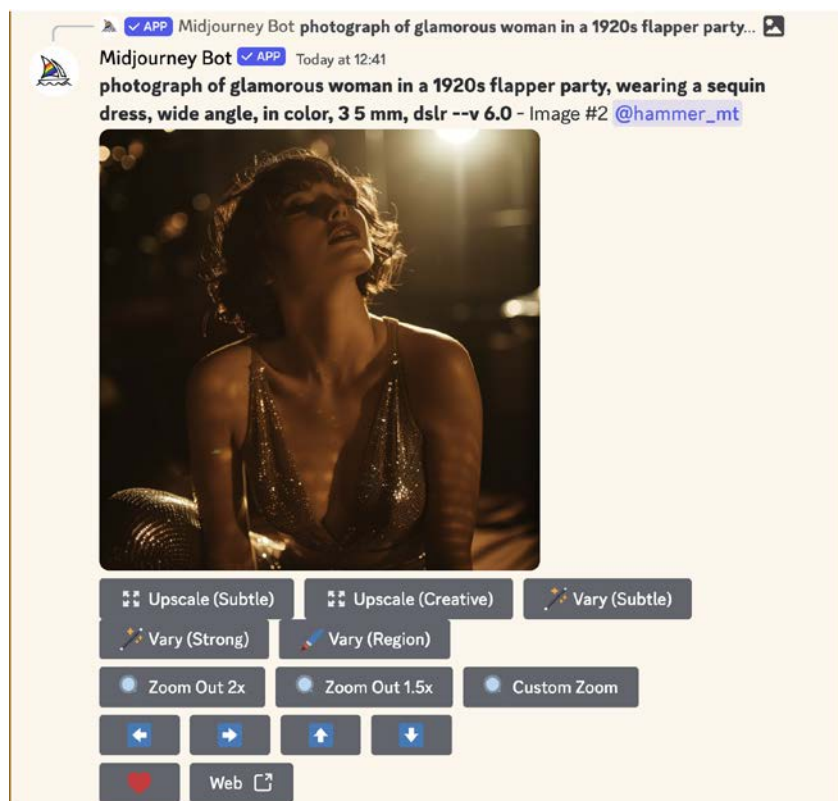


Рис. 8.19. Функция Zoom Out в Midjourney

Ввод:

фотография гламурной женщины в платье-чарльстон на вечеринке 1920-х годов, цветной снимок сделан зеркальной камерой с широкоугольным объективом 35 мм

Вывод показан на рис. 8.20.

Исходное изображение



Масштаб уменьшен в два раза



**Рис. 8.20.** Изображение до и после уменьшения масштаба в Midjourney



### Приведите примеры

Часто бывает сложно добиться правильного стиля с помощью одних только текстовых запросов, особенно если стиль имеет нюансы или вы не знаете, как точно его описать. Добавление изображения-образца при использовании функций изменения фрагмента (inpainting) или дорисовывания обрамления (outpainting) поможет добиться лучших результатов.

Помимо творческого дополнения деталями существующего изображения, дорисовка обрамления также может пригодиться при изменении соотношения сторон изображения, когда желательно заполнить дополнительные области. Для этого можно использовать вариант Custom Zoom (Свой масштаб) и установить желаемое соотношение сторон, а также методом проб и ошибок подобрать запрос, описывающий, что должно появиться в новых областях на изображении, пока не будет получено что-то согласующееся с остальной частью изображения или правильно отображено с новым соотношением сторон (например, при переходе от портретной к альбомной ориентации). Эта функция также доступна в виде расширения в Stable Diffusion ([https://oreil.ly/0c\\_en](https://oreil.ly/0c_en)), но, на наш взгляд, она менее надежна, чем в Midjourney.



## Получение разных изображений одного персонажа

Недооцененное применение функций изменения фрагментов и дорисовывания обрамления — использование существующего изображения для сохранения единообразия генерируемых изображений. Одним из примеров может служить распространенный метод создания разных изображений одного персонажа (<https://oreil.ly/BaITC>) путем генерирования двух изображений рядом. В этом подходе сначала генерируется изображение по запросу, в котором явно указано, что рядом находятся два изображения с соотношением сторон 2:1.

Ввод:

два изображения рядом, американская рок-звезда с грубыми чертами лица, мужчина средних лет, портрет в фотобудке -- соотношение сторон 2:1

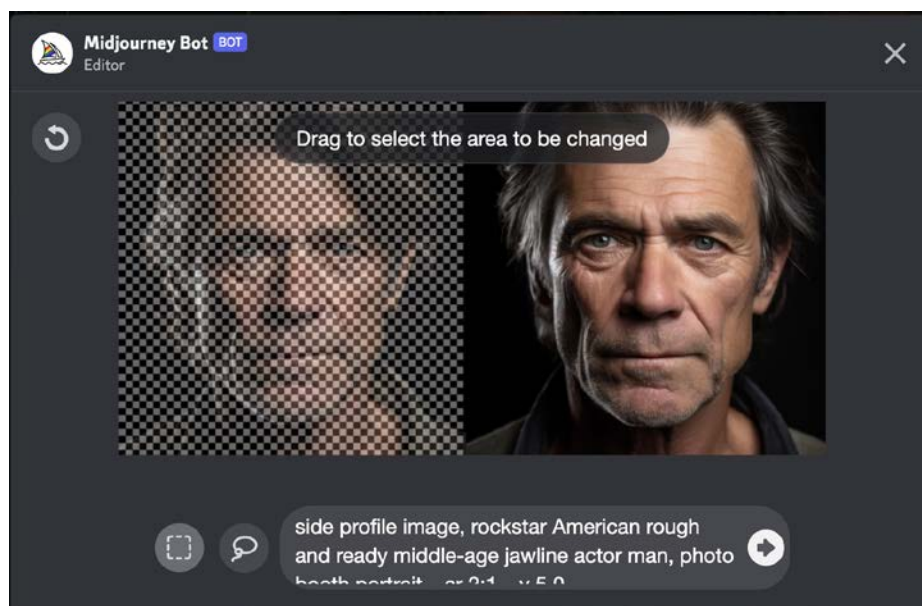
Вывод показан на рис. 8.21.



**Рис. 8.21.** Единообразные изображения, сгенерированные Midjourney

Следующий шаг — увеличение масштаба одного из изображений и маскирование половины увеличенного изображения на холсте для рисования, как показано на рис. 8.22, где демонстрируется применение функции **Vary Region** (Изменить область) в Midjourney.

Наконец, для перерисовывания замаскированной части изображения вводится новый запрос, использующий функцию изменения фрагмента (inpainting), которая в Midjourney известна как **Vary Region** (Изменить область). В запросе указывается угол поворота исходного портретного изображения.



**Рис. 8.22.** Функция Vary Region (Изменить область) в Midjourney

Ввод:

вид в профиль, американская рок-звезда с грубыми чертами лица,  
мужчина средних лет,  
портрет в фотобудке -- соотношение сторон 2:1

Вывод показан на рис. 8.23.



**Рис. 8.23.** Единообразное изображение персонажа в профиль

С помощью такого процесса маскирования фрагмента и генерирования нового изображения с видом под другим углом, повторяемого снова и снова, можно получить набор изображений персонажа, выглядящих идентично исходному, созданному вами. Поскольку половина изображения присутствует постоянно, модель сохраняет единообразие черт персонажа, что позволяет получить более полный комплект его изображений в разных позах и под разными углами. Для этого нужно лишь создать изображение персонажа с соотношением сторон 2:1, заменить половину изображения новым запросом и обрезать ее в Photoshop (или другом графическом редакторе).



### Приведите примеры

Многие быстро понимают, как применить функцию изменения фрагмента к реальным изображениям, но лишь наиболее продвинутые художники, работающие с ИИ, догадались использовать в качестве основы сгенерированные изображения, чтобы сохранить единообразное изображение персонажей или объектов в своих повествованиях.

## Переделка запросов

Одна из проблем, с которой можно столкнуться при запуске системы ИИ в эксплуатацию, заключается в том, что далеко не все пользователи системы окажутся опытными промт-инженерами. Это тот самый случай, когда мусор на входе порождает мусор на выходе: введя плохой запрос, пользователь получит плохой результат и будет отрицательно высказываться о качестве вашего продукта. Одна из распространенных уловок, применяемых в отрасли (<https://oreil.ly/OirCS>), — переделка запроса, чтобы улучшить его и повысить вероятность получения впечатляющего результата. Это форма мета-запросов, когда запрос для одной модели ИИ пишется с помощью другой.

Представьте простое приложение, в котором пользователь вводит тему и имя художника и ждет, что модель вернет изображение объекта, нарисованное в стиле художника. Шаблон запроса выглядит примерно так: {subject} в стиле {artist}.

Ввод:

собака такса в стиле Бэнкси

Вывод показан на рис. 8.24.

Проблема этого запроса в том, что написавший его ожидает, что собака будет изображена в стиле стрит-арт (в стиле Бэнкси), тогда как на сгенерированном изображении собака сидит рядом с изображением в стиле стрит-арт. Чтобы



исправить этот недостаток, можно взять запрос пользователя и внедрить его в запрос к ChatGPT, чтобы выяснить, что служило полотном для художника.



**Рис. 8.24.** Собака такса в стиле Бэнкси

Ввод:

В каком стиле чаще всего рисовал художник Бэнкси?  
Ответь 1-3 словами.

Вывод:

Стрит-арт

Этот вывод можно использовать для переделки исходного запроса пользователя в формат {medium} {subject} в стиле {artist}.

Ввод:

стрит-арт собаки таксы в стиле Бэнкси

Вывод показан на рис. 8.25.

Эту систему можно еще расширить, включив другие методы промт-инжиниринга, такие как усилители качества или отрицательные запросы. Неплохие результаты

можно получить, просто попросив ChatGPT переписать запрос для модели DALL-E (которая в ChatGPT Plus доступна в качестве инструмента), и использовать его с другими моделями. В свое время были предприняты некоторые попытки обучения моделей ИИ (<https://oreil.ly/9A1NL>), специализирующихся на создании высококачественных запросов, однако они обеспечивают качество не выше среднего уровня и их результаты не превосходят запросы, подготовленные экспертами.



**Рис. 8.25.** Стрит-арт собаки таксы в стиле Бэнкси



#### **Разделите задачу на подзадачи**

Не ожидайте от человека, не обладающего техническими знаниями, что он будет создавать качественные запросы, просто передайте его ввод другой модели ИИ, которая может помочь улучшить первоначальный запрос.

## **Выделение мемов**

Основная проблема копирования стиля художника или подражания художественному направлению — отсутствие оригинальности. Пока никто не знает, как будут расцениваться творения ИИ с юридической точки зрения, но такие художники, как Грег Рутковски (Greg Rutkowski) и другие, уже высказались об аморальности копирования их стиля.

Один из способов получить выгоды от ссылки на успешного художника или художественное движение и остаться при этом достаточно оригинальным — выделить мемы художника. Слово *мем* (<https://oreil.ly/BQYFP>) означает не просто забавное вирусное изображение, а единицу любой значимой для культуры информации, передаваемой от человека к человеку. Разложив художественный стиль на его составные части и характеристики, вы сможете создать из этих частей свой стиль. Представьте, например, что вы пытаетесь нарисовать Таймс-сквер в стиле Сальвадора Дали (рис. 8.26).

Ввод:

Изображение Таймс-сквер в стиле картины "Постоянство памяти"  
Сальвадора Дали



**Рис. 8.26.** Изображение Таймс-сквер в стиле картины «Постоянство памяти» Сальвадора Дали

Упоминание имени Дали уже гарантирует хороший результат, но это не совсем правильно. Вам хотелось бы получить изображение в сюрреалистическом

стиле, а не точное подражание работам Дали. Термин «выделение» (unbundling) был придуман в этом контексте Бакзом Т. Фьючером (Bakz T. Future) (<https://oreil.ly/2qy4E>) и означает извлечение элементов стиля художника, чтобы получить что-то знакомое и вместе с тем уникальное. Суть его состоит в том, чтобы попросить языковую модель (в данном случае ChatGPT-4) описать стиль выбранного вами художника, а затем использовать результат в качестве запроса для генерирования изображения.

Вывод:

Кратко охарактеризуй особенности и художественный стиль картины "Постоянство памяти" Сальвадора Дали, без упоминания названия картины и имени художника.

Вывод:

Картина представляет собой сюрреалистический пейзаж, в котором время и пространство воспринимаются искаженно. На фоне пустынного ландшафта с горными вершинами изображены тающие часы, свисающие с различных объектов, включая ветви деревьев и предметы, напоминающие столы. Эти часы символизируют относительность времени и его гибкость, подчеркивая идею, что восприятие времени может быть субъективным.

Художественный стиль картины характеризуется яркими, насыщенными цветами и детализированной проработкой элементов, что создает эффект глубины и реалистичности в сюрреалистическом контексте. Использование необычных форм и искаженных пропорций создает ощущение dreamlike, погружая зрителя в мир фантазий и подсознательных ассоциаций. Композиция вызывает чувство тревоги и загадочности, заставляя размышлять о природе реальности и времени.

Вы должны удалить из запроса любые упоминания художника или произведения искусства, иначе не получите желаемого результата. Вы должны описать стиль так, чтобы его смог понять тот, кто никогда не видел ничего подобного. Для справки на рис. 8.27 показана та самая знаменитая картина, которую описала ChatGPT.

Теперь, когда мы получили описание одной из самых знаковых картин Дали, становится более или менее понятно, что делает ее такой визуально привлекательной. В частности, такого рода описания могут быть полезны для тех, кто не учился в художественной школе и не знает, как описать эти элементы. Далее вам нужно сократить описание, чтобы уместить его в запрос к модели, генерирующей изображения.

Ввод:

Преобразуй это описание в сокращенный запрос для модели ИИ, генерирующей изображения, такой как DALL-E.



**Рис. 8.27.** «Постоянство памяти» Сальвадора Дали

#### Вывод:

Создайте сюрреалистический пейзаж с пустынным ландшафтом и горными вершинами. Изобразите тающие часы, свисающие с ветвей деревьев и предметов, напоминающих столы. Используйте яркие, насыщенные цвета и детализированную проработку элементов, чтобы создать эффект глубины. Включите необычные формы и искаженные пропорции, передающие ощущение мечтательности, тревоги и загадочности, исследуя природу времени и реальности.

Возьмите этот текст и добавьте в него главный объект вашей картины — Таймс-сквер. Можете также немного изменить текст запроса, сделав его более плавным, поскольку запросы, которые пишет ChatGPT, могут быть слишком назидательными.

#### Ввод:

Картина с изображением Таймс-сквер. Создайте сюрреалистический пейзаж с пустынным ландшафтом и горными вершинами. Изобразите тающие часы, свисающие с ветвей деревьев и предметов, напоминающих столы. Используйте яркие, насыщенные цвета и детализированную проработку элементов, чтобы создать эффект глубины. Включите необычные формы и искаженные пропорции, передающие ощущение мечтательности, тревоги и загадочности, исследуя природу времени и реальности.

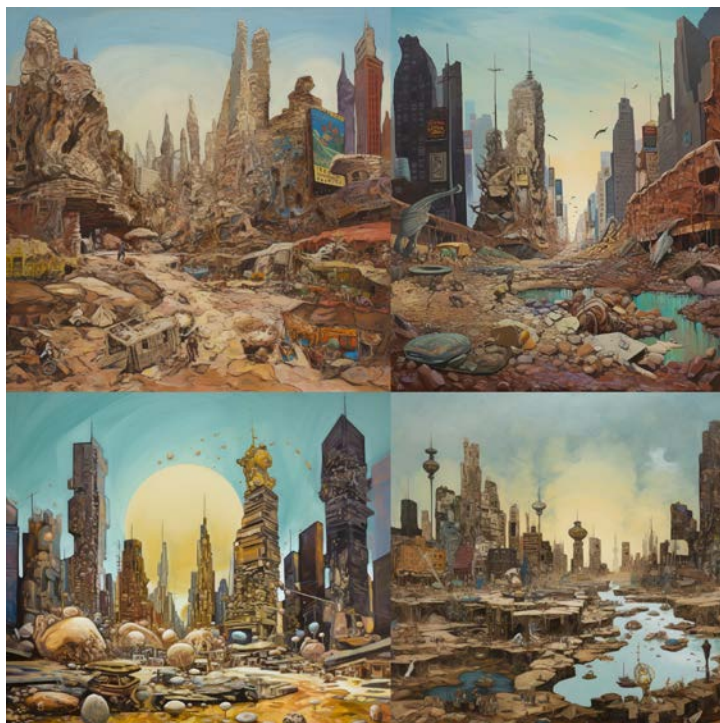
Вывод показан на рис. 8.28.



#### Задайте направление

Старайтесь ориентировать модель ИИ не на работы конкретного художника, а на близкую имитацию, используя описание этих работ. Это более творческий и, пожалуй, более этичный подход, чем простое упоминание имени художника.





**Рис. 8.28.** Результат применения выделенных мемов Дали к картине Таймс-сквер

Это изображение по-прежнему похоже на работу Дали, но оно было получено с помощью описания, составленного моделью ChatGPT. Поэтому его можно считать более оригинальным, чем изображение, полученное простым упоминанием имени художника. Но теперь вы находитесь в лучшем положении, потому что выделили отдельные мемы стиля Дали (<https://oreil.ly/BQYFP>), такие как «сюрреалистический пейзаж», «плавающие объекты» и «сказочная атмосфера», и можете смешать (<https://oreil.ly/wPuMo>) все эти составные части, чтобы сделать изображение более уникальным (рис. 8.29).

Ввод:

Картина с изображением Таймс-сквер. Создайте сюрреалистический пейзаж в приглушенных тонах оранжевого, красного и зеленого цветов. Монументальные здания и спокойная река встречаются с грозовым небом. Изобразите тающие часы в центре площади, символизирующие текучесть и непостоянство времени в отличие от других жестких, постоянных элементов. Воссоздайте ощущение тишины и сказочной атмосферы.



**Рис. 8.29.** Ремикс картины с изображением Таймс-сквер в стиле Дали

Мы внесли лишь небольшие изменения в цветовую палитру и элементы картины, но можно пойти еще дальше: добавить описание особенностей стилей других популярных художников, которые вам нравятся, и получить что-то совершенно новое. В настоящее время этот метод работает только для достаточно известных художников и произведений изобразительного искусства, однако, поскольку в последние годы наблюдается тенденция к созданию мультимодальных моделей ИИ (то есть способных генерировать как изображения, так и текст), можно ожидать, что у них появится возможность загрузить любое изображение и создать описание, которое затем можно будет использовать для выделения мемов.

## Картирование мемов

Один из наиболее распространенных источников вдохновения при создании запросов — изучение чужих запросов, с которыми другие промт-инженеры получают результаты. Сообщество Midjourney Discord (<https://oreil.ly/upQIh>) насчитывает миллионы активных участников. Каждый день генерируются

и автоматически публикуются тысячи новых изображений. Аналогичная активность наблюдается в других сообществах ИИ, в том числе на Reddit (<https://oreil.ly/EwLNh>), а также на других веб-сайтах, в рассылках по электронной почте и в социальных сетях. На широко известном сайте Lexica.art (<https://lexica.art>) имеется база данных с возможностью поиска (по ключевым словам и сходству) изображений, созданных моделью Stable Diffusion, и соответствующих им запросов.

Просматривая эти источники вдохновения, можно заметить повторяющиеся текстовые закономерности, или мемы, которые используются для обозначения определенного типа изображений. Мы называем процесс целенаправленного и систематического поиска таких закономерностей *картированием мемов* (<https://oreil.ly/DLqAV>), и он может стать бесценным инструментом для выявления полезных запросов. Например, попробуйте выполнить поиск по словам «супер Марио» на Lexica, и вы найдете множество примеров попыток создать реалистичного Марио, как на рис. 8.30. Результаты поиска могут вдохновить вас сделать то же самое, начав с запроса (<https://oreil.ly/WNsRn>), который доказал свою эффективность и может помочь сэкономить вам массу времени.

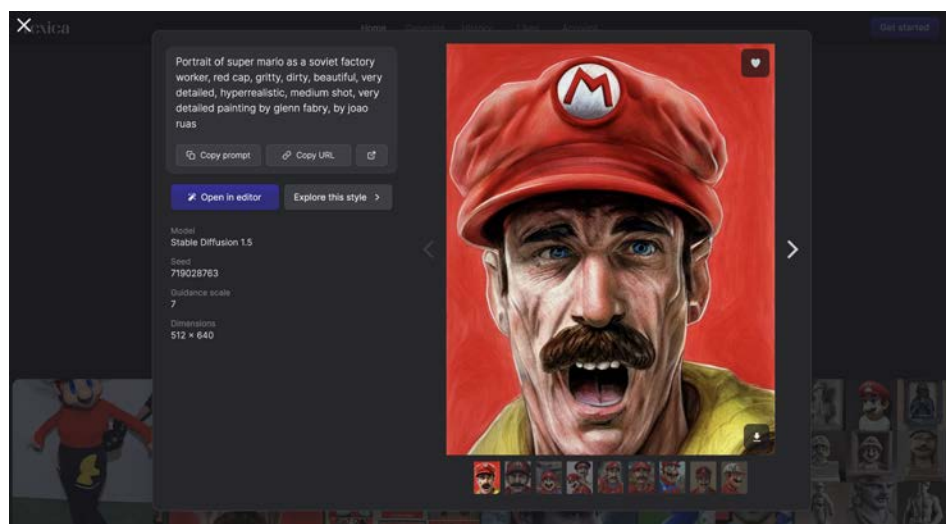


Рис. 8.30. Реалистичный Марио

В качестве альтернативы можно применить этот мем к персонажу из другой франшизы и попробовать переиначить некоторые подсказки, используемые другими, чтобы получить реалистичный эффект. Не проведя этого исследования, вы, возможно, не знали бы, что модели могут генерировать реальные версии персонажей мультфильмов или игр, или, возможно, никогда не подумали бы попробовать это. Возможно, вы никогда не сталкивались с мыслью о том, что

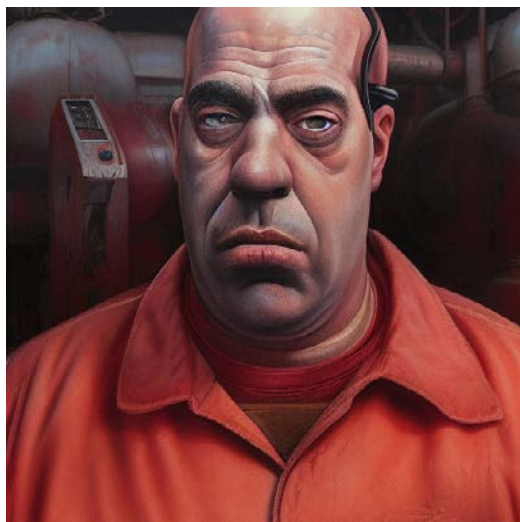


включение в подсказку фразы «как советский фабричный рабочий» помогает вызвать чувство сурового реализма, и, возможно, никогда не сталкивались с работами двух упомянутых художников. В арт-сообществе искусственного интеллекта существует здоровая культура микширования контента: люди учатся на чужих подсказках, а затем возвращают долг, делясь собственным опытом.

Ввод:

портрет Гомера Симпсона в образе советского фабричного рабочего, мрачный, чумазый, красивый, очень детально, гиперреалистично, средний план, очень подробная картина Гленна Фабри, Жоао Руаса --по мультфильм

Вывод показан на рис. 8.31.



**Рис. 8.31.** Реалистичный Гомер Симпсон

Процесс картирования мемов можно выполнить вручную (<https://oreil.ly/VqyG->), скопировав примеры и вставив их в электронную таблицу или инструмент Notion, правда, это может занять много времени. Если вы соблюдаете условия использования веб-сайта и законодательство своей страны, то можете также написать свою программу для скрапинга этого веб-сайта. Собрав все данные в одном месте, вы сможете программно пометить изображения с помощью модели распознавания образов, например Google Vision (<https://oreil.ly/EZmRs>), мультимодальной модели, такой как GPT-4 Vision (<https://oreil.ly/cOcPR>), или воспользоваться инструментом анализа естественного языка, таким как NGrams (<https://oreil.ly/GXfDI>), чтобы проанализировать запросы и выявить общие закономерности, которые сложно выявить вручную.

## Анализ запросов

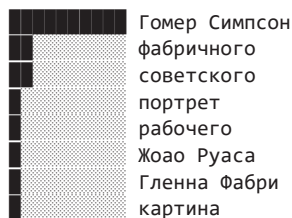
Одна из распространенных ошибок — продолжать создавать все более и более длинные запросы, не задумываясь о том, какие их части действительно необходимы. Каждое добавленное слово вносит некоторые возмущения в работу модели и добавляет шум в конечный результат. Часто удаление ненужных слов так же эффективно, как добавление новых. Чтобы провести такой анализ, не увязнув в большом количестве попыток, Midjourney предлагает команду `/shorten`, которая пытается удалить ненужные слова и оставить только основные токены, которым модель уделяет наибольшее внимание. Нажмите кнопку **show details** (Показать подробности) под результатом, и вы увидите весовые коэффициенты токенов и визуальную диаграмму.

Ввод:

портрет Гомера Симпсона в образе советского фабричного рабочего, мрачный, чумазый, красивый, очень детально, гиперреалистично, средний план, очень подробная картина Гленна Фабри, Жоао Руаса --по мультфильму

Вывод:

**\*\*портрет\*\*** (0.08) **\*\*Гомера Симпсона\*\*** (1.00) в образе **\*\*советского\*\*** (0.19) **\*\*фабричного\*\*** (0.21) **\*\*рабочего\*\*** (0.08), мрачный (0.02), чумазый (0.02), красивый (0.00), очень детально (0.01), гиперреалистично (0.01), средний (0.00) план (0.00), очень (0.00) подробная (0.01) картина (0.05) Гленна Фабри (0.08), **\*\*Жоао Руаса\*\*** (0.09)



Получив результаты анализа, можно использовать их, чтобы убрать весь шум из подсказки и сосредоточиться на словах или мемах, действительно важных для конечного результата.



### Оцените качество

Просмотр весов, которые модель присваивает каждому токenu, помогает получить беспрецедентное представление о том, как работает модель. Мы часто делаем те или иные предположения о том, что важно в запросе, но они могут быть весьма далеки от реальности.

## Резюме

В этой главе вы познакомились со стандартными методами создания изображений с применением диффузионных моделей, изучили модификаторы формата, такие как «стоковая фотография», «картина маслом» и «египетские иероглифы», а также увидели примеры их использования для создания уникальных и визуально привлекательных изображений. Вы также узнали, что названия художественных стилей тоже являются модификаторами, позволяющими копировать популярные художественные стили или художников, например стиль «Алиса в стране чудес» Льюиса Кэрролла.

Вы углубились в практическое применение принципов промт-инжиниринга, в том числе в использование модификаторов в виде названий художественных стилей и упоминаний имен конкретных художников, присутствие которых в запросах может способствовать получению желаемого результата. Вашему вниманию были представлены концепции отрицательных запросов и взвешенных понятий, позволяющие указать, чего не должно быть на изображении, и определить, в каких пропорциях должны смешиваться различные понятия. Вы также познакомились с возможностью изменения фрагментов внутри изображений и дорисовывания обрамлений, позволяющей создавать отдельные части изображения путем стирания и добавления дополнительных запросов. Вы узнали, как эти методы можно расширять и объединять для повышения надежности и качества результатов генеративного ИИ.

В следующей главе мы еще глубже погрузимся в мир генерирования изображений и исследуем более сложные сценарии использования. Вы узнаете, как использовать возможности Stable Diffusion и AUTOMATIC1111 для совершенствования своих навыков создания изображений. Добавив в свой арсенал передовые методы Stable Diffusion, такие как использование моделей ControlNet, для более точного управления стилем и композицией изображений, вы откроете для себя широкий спектр захватывающих возможностей.

# Продвинутые способы создания изображений с помощью Stable Diffusion

Чаще всего для создания изображений с помощью ИИ достаточно применения простых методов промт-инжиниринга, но помните, что существуют более мощные инструменты, которые могут пригодиться, когда вам потребуется проявить более творческий подход или обучить пользовательские модели для решения конкретных задач. Эти более сложные инструменты часто требуют технических навыков и применения структурированного подхода для создания окончательного изображения.

Все изображения в этой главе созданы с помощью Stable Diffusion XL, если не указано иное, как в разделах, демонстрирующих использование расширений, таких как ControlNet, где применяется более старая модель v1.5, поддерживающая более широкий круг методов. Обсуждаемые методы были разработаны с учетом возможности переноса на любые будущие или альтернативные модели. Мы широко используем веб-интерфейс AUTOMATIC1111 Stable Diffusion и приводим подробные инструкции по настройке, которые были актуальны на момент написания книги; самые свежие инструкции и упоминание проблем, с которыми можно столкнуться, вы найдете в официальном репозитории ([https://oreil.ly/hs\\_fs](https://oreil.ly/hs_fs)).

## Запуск Stable Diffusion

Stable Diffusion — это модель генерации изображений с открытым исходным кодом, которую можно бесплатно запустить локально на персональном компьютере, если он оснащен графическим процессором NVIDIA, AMD или Apple Silicon, последний из которых используется на компьютерах Mac M1, M2 и M3.

Первую общедоступную версию (1.4) Stable Diffusion можно было запустить в блокноте Google Colab (<https://oreil.ly/OmBuR>), имеющем доступ к бесплатному графическому процессору в облаке (хотя вам может потребоваться перейти на платную учетную запись, если Google ограничит бесплатное использование).

Посетите сайт Google Colab (<https://oreil.ly/2WGxQ>), если вы еще не использовали этот инструмент, где можно найти самую свежую информацию об ограничениях. Копия блокнота на Python сохранена в репозитории GitHub (<https://oreil.ly/uauNn>) этой книги, но вам следует загрузить ее на Google Диск и запустить в Google Colab, чтобы избежать проблем с настройкой.

Установить Stable Diffusion вместе с несколькими зависимостями можно с помощью библиотеки Hugging Face. Следующий код устанавливает в Google Colab необходимые зависимости (уберите восклицательные знаки (!), если выполняете установку локально, а не в Jupyter Notebook или Google Colab):

```
!pip install diffusers==0.11.1
!pip install transformers scipy ftfy accelerate
```

Чтобы загрузить и использовать модель, сначала нужно создать пайплайн получения результатов (он запускается при использовании модели):

```
создать пайплайн получения результатов
import torch
from diffusers import StableDiffusionPipeline

pipe = StableDiffusionPipeline.from_pretrained(
 "CompVis/stable-diffusion-v1-4",
 torch_dtype=torch.float16)

pipe = pipe.to("cuda")
```

Давайте разберем, что делает каждая строка этого сценария.

```
import torch
```

Импортирует библиотеку torch, также известную как PyTorch (<https://pytorch.org>). PyTorch — это библиотека машинного обучения с открытым исходным кодом, используемая, например, в приложениях компьютерного зрения и обработки естественного языка.

```
from diffusers import StableDiffusionPipeline
```

Импортирует класс `StableDiffusionPipeline` из библиотеки `diffusers`. Этот конкретный класс реализует пайплайн для взаимодействий с диффузионными моделями, наиболее популярным примером которых является Stable Diffusion.

```
pipe = StableDiffusionPipeline.from_pretrained("CompVis/stable-diffusion-
v1-4", torch_dtype=torch.float16)
```

Это создает экземпляр класса `StableDiffusionPipeline` с предварительно обученной моделью. Метод `from_pretrained` загружает веса модели — в данном случае `CompVis/stable-diffusion-v1-4`.

Аргумент `torch_dtype=torch.float16` сообщает, что веса в модели имеют тип `float16` — числа с плавающей точкой половинной точности. Использование `float16` может ускорить вычисление модели и сократить использование памяти (чтобы оставаться в пределах ограничений бесплатного тарифа пользования Google Colab).

```
pipe = pipe.to("cuda")
```

Перемещает модель в графический процессор. Строка `"cuda"` — это аббревиатура CUDA (Compute Unified Device Architecture — архитектура унифицированного вычислительного устройства), обозначающая модель параллельной вычислительной платформы и прикладного программного интерфейса (API), созданную в Nvidia. При этом все вычисления в модели, связанной с каналом, будут выполняться на графическом процессоре, то есть значительно быстрее, чем на центральном процессоре.

Теперь, имея канал, можно передать модели запрос и другие параметры, такие как случайное начальное число (меняйте его, чтобы каждый раз получать другое изображение), количество шагов обработки (чем больше шагов, тем больше времени требуется на их выполнение и тем выше качество изображения) и степень соответствия (насколько точно изображение соответствует запросу):

```
запустить обработку запроса
prompt = "a photograph of an astronaut riding a horse"

generator = torch.Generator("cuda").manual_seed(1024)

image = pipe(prompt, num_inference_steps=50,
 guidance_scale=7, generator=generator
).images[0] # изображение здесь получается в формате PIL

Чтобы отобразить изображение, его нужно сохранить, например, так:
image.save(f"astronaut_rides_horse.png")

В Google Colab изображение можно вывести непосредственно:
image
```

Вывод показан на рис. 9.1.

Давайте подробно разберем, что делает этот сценарий.

```
prompt = "a photograph of an astronaut riding a horse"
```

Это запрос, в ответ на который модель сгенерирует изображение.



**Рис. 9.1.** Фотография космонавта верхом на лошади

```
generator = torch.Generator("cuda").manual_seed(1024)
```

Строка создает генератор PyTorch и сохраняет его в переменной. Генератор инициализируется с помощью "cuda", то есть для вычислений он будет использовать графический процессор. Функция `manual_seed(1024)` задает начальное число для получения последовательности псевдослучайных чисел, гарантируя воспроизводимость результатов. Запустив этот код с той же моделью у себя, вы должны получить точно такое же изображение.

```
image = pipe(prompt, num_inference_steps=50, guidance_scale=7, generator=generator).images[0]
```

Эта строка запускает модель и передает ей текст запроса. В аргументе `num_inference_steps` передается значение 50, то есть модель выполнит 50 шагов обработки. В аргументе `guidance_scale` (регулирует, насколько точно модель должна следовать запросу) передается значение 7 (чем выше значение, тем более зернистыми и менее разнообразными получаются изображения). В аргументе `generator` передается генератор случайных чисел, созданный ранее. Результатом является массив сгенерированных изображений, а `images[0]` выбирает первое изображение из этого массива.

```
image.save(f"astronaut_rides_horse.png")
```

Строка сохраняет сгенерированное изображение в файл.

image

Строка кода выведет изображение, если сценарий выполняется в среде Jupyter Notebook или Google Colab, потому что эти среды автоматически отображают результат последней строки кода в ячейке, если он не присваивается переменной.

Возможность запускать модель с открытым исходным кодом локально или в облаке и настраивать ее под свои потребности трудно переоценить. Однако необходимость писать свой код для создания конвейеров обработки и конструировать пользовательский интерфейс на его основе может вызвать непреодолимые сложности, если только вы не являетесь очень опытным пользователем с глубокими знаниями в области машинного обучения или не собираетесь создать свой продукт для генерирования изображений с помощью ИИ. По этой причине Stability AI — компания, финансирующая разработку Stable Diffusion, — создала веб-интерфейс под названием Dream Studio (рис. 9.2), похожий на интерактивную среду DALL-E, которая доступна по платной подписке и предлагает ряд дополнительных возможностей, таких как изменение фрагментов внутри изображений.

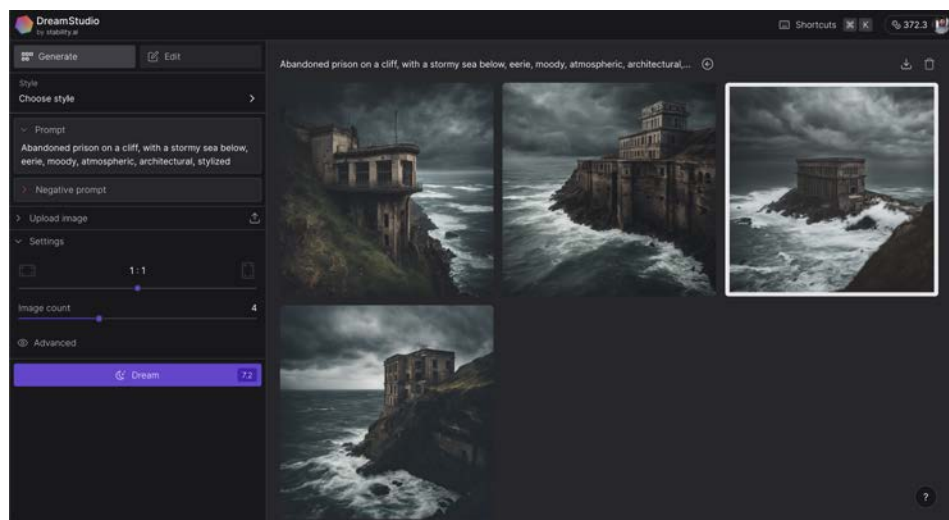


Рис. 9.2. Веб-интерфейс Dream-Studio

Dream-Studio (как и DALL-E) доступна также через API, что может быть удобно для приложений, генерирующих изображения с помощью ИИ, или запуска программных сценариев, создающих большое количество изображений без необходимости устанавливать и запускать модель Stable Diffusion у себя. Посетите <https://oreil.ly/X3Ilb> после создания учетной записи, чтобы получить ключ API и пополнить счет кредитами (на момент написания этой книги 1000 кредитов



стоили 10 долларов США и позволяли создать около 5000 изображений). Следующий пример кода можно найти в репозитории GitHub (<https://oreil.ly/aGLeX>):

```
import os
import base64
import requests
from IPython.display import Image

engine_id = "stable-diffusion-xl-1024-v1-0"
api_host = os.getenv('API_HOST', 'https://api.stability.ai')
api_key = os.getenv("STABILITY_API_KEY")

image_description = "computers being tied together"
prompt = f"""an illustration of {image_description}. in the
style of corporate memphis, white background, professional,
clean lines, warm pastel colors"""

response = requests.post(
 f"{api_host}/v1/generation/{engine_id}/text-to-image",
 headers={
 "Content-Type": "application/json",
 "Accept": "application/json",
 "Authorization": f"Bearer {api_key}"
 },
 json={
 "text_prompts": [
 {
 "text": prompt,
 }
],
 "cfg_scale": 7,
 "height": 1024,
 "width": 1024,
 "samples": 1,
 "steps": 30,
 },
)

if response.status_code != 200:
 raise Exception(
 "Non-200 response: " + str(response.text))

data = response.json()

image_paths = []

если папка /out отсутствует, то создать ее
if not os.path.exists("./out"):
 os.makedirs("./out")

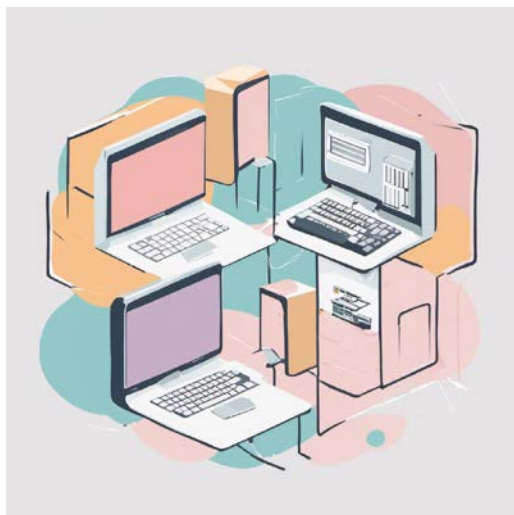
for i, image in enumerate(data["artifacts"]):
 filename = f"./out/image-{i}.png"
```

```
with open(filename, "wb") as f:
 f.write(base64.b64decode(image["base64"]))

image_paths.append(filename)

отобразить первое изображение
Image(filename=image_paths[0])
```

Вывод показан на рис. 9.3.



**Рис. 9.3.** Иллюстрация в стиле Corporate Memphis<sup>1</sup>, генерированная с помощью Dream-Studio API

Разберем подробно, что делает этот код.

1. Сначала настраиваются необходимые переменные окружения:

- `engine_id` — ссылается на конкретную версию модели на сайте `stability.ai`;
- `api_host` — извлекает URL хоста API из переменных окружения. Если эта переменная не установлена, то по умолчанию используется адрес `https://api.stability.ai`;
- `api_key` — извлекает ключ API из переменных окружения.

<sup>1</sup> Corporate Memphis — термин, который используется для описания плоского дизайна, художественного стиля, получившего широкое распространение в иллюстрациях компаний Big Tech в конце 2010-х и начале 2020-х. — *Примеч. пер.*

2. Переменная `prompt` описывает, как должно выглядеть изображение, включая стиль и цвета.
3. По адресу URL, полученному из `api_host` и `engine_id`, отправляется запрос POST:
  - заголовки запроса настраиваются на прием и отправку данных в формате JSON, и в их число входит заголовок авторизации с `api_key`;
  - в теле запроса в формате JSON указываются текст запроса (описание изображения), желаемый масштаб, размеры, количество образцов и количество шагов.
4. В случае ошибки, если в ответе получен код состояния ответа, отличный от 200, генерируется исключение с текстом ответа, описывающим причину. В противном случае выполняется парсинг ответа в формат JSON.
5. Если каталог с именем `/out` отсутствует, то он создается. Для каждого артефакта (изображения) в ответе код выполняет следующие действия:
  - настраивает путь к файлу;
  - декодирует изображение в ответе из кодировки Base64;
  - записывает декодированное изображение в файл;
  - добавляет путь к файлу в список `image_paths`;
  - обычно изображения сохраняются в облачном хранилище Google (<https://oreil.ly/YsuBw>) или Amazon Simple Storage Service (S3) для последующего отображения в приложении.
6. Первое изображение в списке `image_paths` (в данном случае единственное) выводится (только в Jupyter Notebooks или Google Colab) с использованием класса `Image` из `IPython.display`.

Недостатком сервиса Stability AI является отсутствие контроля над настройками. Одно из главных преимуществ модели Stable Diffusion с открытым исходным кодом — способность изменять практически любой ее аспект и использовать расширенные возможности, добавленные сообществом. Кроме того, нет никакой гарантии, что особенности или возможности, на которые вы полагаетесь в своих сценариях сегодня, останутся доступными в будущем, потому что Stability AI стремится оправдать ожидания своих инвесторов, команды юристов и корпоративных клиентов. Например, популярная (и более либеральная) модель версии 1.5 была признана устаревшей в пользу новых моделей Stable Diffusion 2.0 и XL, что вызвало проблемы у тех, кто настроил свои рабочие процессы, параметры и запросы для работы с версией 1.5.

## Веб-интерфейс AUTOMATIC1111

Активные пользователи Stable Diffusion обычно рекомендуют веб-интерфейс AUTOMATIC1111 (<https://oreil.ly/r-2vm>) (произносится как «автоматик одиннадцать одиннадцать»), обладающий широкими функциональными возможностями и имеющий множество расширений, созданных пользователями Stable Diffusion. Этот проект открывает путь к лучшему, что есть в Stable Diffusion: активному сообществу, добавившему в инструмент огромное число дополнительных функций. Опытные пользователи также могут изучить интерфейс ComfyUI (<https://oreil.ly/LWVvC>), поддерживающий решение более сложных задач и имеющий повышенную гибкость (включая преобразование изображения в видео (<https://oreil.ly/dh7jR>)), но мы посчитали его слишком сложным для большинства сценариев использования, с которыми легко справится AUTOMATIC1111.

Вы можете использовать обычную модель Stable Diffusion преобразования текста в изображение, а также запускать преобразование изображения в изображение (действует подобно функции создания изображения на основе базового изображения в Midjourney), а также масштабировать готовые изображения для получения более высокого качества и изменять фрагменты готовых изображений (подобно DALL-E). В этом интерфейсе даже можно обучать и запускать собственные модели, и тысячи подобных моделей можно найти в таких сообществах, как Hugging Face (<https://oreil.ly/t5T7p>) и Civitai (<https://civitai.com>).



Некоторые пользовательские модели могут генерировать контент «для взрослых», поэтому имейте это в виду, просматривая такие веб-сайты, как Civitai.

Запуск Stable Diffusion локально с помощью AUTOMATIC1111 требует некоторой технической подготовки, поэтому ищите самые свежие руководства в AUTOMATIC1111 Wiki.

- Установка и запуск на графических процессорах NVidia (<https://oreil.ly/DsKyU>).
- Установка и запуск на графических процессорах AMD (<https://oreil.ly/Oc7ix>).
- Установка и запуск на графических процессорах Apple Silicon (<https://oreil.ly/Ob2VK>).

Настройка обычно включает установку Git и Python (и любых зависимостей (<https://oreil.ly/vBOVI>)), загрузку Stable Diffusion и кода Automatic1111 (<https://oreil.ly/x0BMn>). Для создания изображений в этой главе использовалась версия Stable Diffusion XL 1.0 (<https://oreil.ly/DIVUz>), хотя многие по-прежнему предпочитают более старую версию 1.5 (<https://oreil.ly/FNx9>), которая считается менее ограниченной и имеет множество моделей, дообученных сообществом. Прием, обсуждаемые здесь, работают одинаково для всех моделей, но результаты

и их качество будут различаться: считается, что удаление изображений «для взрослых» из обучающих данных для версии 2.0 привело к ухудшению качества генерируемых ею изображений человеческих фигур (в версии XL этот недостаток практически исправлен).

Поскольку исходный код модели открыт, вы можете загрузить базовую и уточняющую модели SDXL v1.0 со страницы на Hugging Face, где они доступны в виде файлов `.safetensors` на вкладке **Files and Versions** (Файлы и версии). Этот формат файлов безопаснее предыдущего формата `.ckpt`, потому что при его запуске не выполняется код на вашем компьютере:

- базовая модель (<https://oreil.ly/wtHRj>): `sd_xl_base_1.0.safetensors`;
- уточняющая модель (<https://oreil.ly/ODlbv>): `sd_xl_refiner_1.0.safetensors`.

Для загрузки этих моделей требуется время, поэтому начните загружать прямо сейчас, а позже, после установки интерфейса AUTOMATIC111, поместите их в папку `models/Stable-diffusion`. Если вы решите использовать более старую модель Stable Diffusion 1.5, загрузите файл `v1-5-pruned-emaonly.ckpt` с сайта Hugging Face (<https://oreil.ly/hwblq>) и переместите его в папку `models`, где вы разместили базовую и уточняющую модели.

Закончив установку, вы сможете получить доступ к веб-интерфейсу, выполнив сценарий, который запускает приложение локально и делает его доступным через веб-адрес в браузере. Вот текущие инструкции (на момент написания книги) для Windows с компьютером, оснащенным графическим процессором Nvidia.

1. Установите Python 3.10.6 (<https://oreil.ly/kGiyi>) (в процессе установки поставьте флажок **Add to PATH** (Добавить в PATH)) и Git (<https://oreil.ly/Pdzb0>).
2. Откройте окно командной строки и введите `git clone https://github.com/AUTOMATIC1111/stable-diffusion-webui`.
3. Не забудьте переместить модели `sd_xl_base_1.0.safetensors` и `sd_xl_refiner_1.0.safetensors` в папку `stable-diffusion-webui/models/Stable-diffusion`.
4. Дважды щелкните на файле `webui-user.bat` и в браузере перейдите по адресу, который обслуживает интерфейс (обычно `http://127.0.0.1:7860`). В Mac или Linux вместо `webui-user.bat` запустите команду `bash webui.sh`.

В интерфейсе, показанном на рис. 9.4 (взяв из официального репозитория GitHub (<https://oreil.ly/OOpas>)), введите запрос (в поле вверху слева на вкладке `txt2img`) и нажмите кнопку **Generate** (Сгенерировать), чтобы получить свое изображение.

Если вы столкнулись с ошибкой или хотите обновить давно загруженный веб-интерфейс AUTOMATIC1111 до более новой версии, то в терминале перейдите в папку `stable-diffusion-webui` и запустите `git pull`. Если возникнут какие-либо ошибки, попробуйте привести реализацию в начальное состояние (предварительно переместить все файлы, которые хотите сохранить), запустив команду `git checkout -f master` в папке `stable-diffusion-webui`.



В процессе приведения AUTOMATIC1111 в начальное состояние будут удалены все файлы в папке вместе со всеми настройками. Мы рекомендуем сделать локальную копию на случай, если вдруг понадобится что-то восстановить.

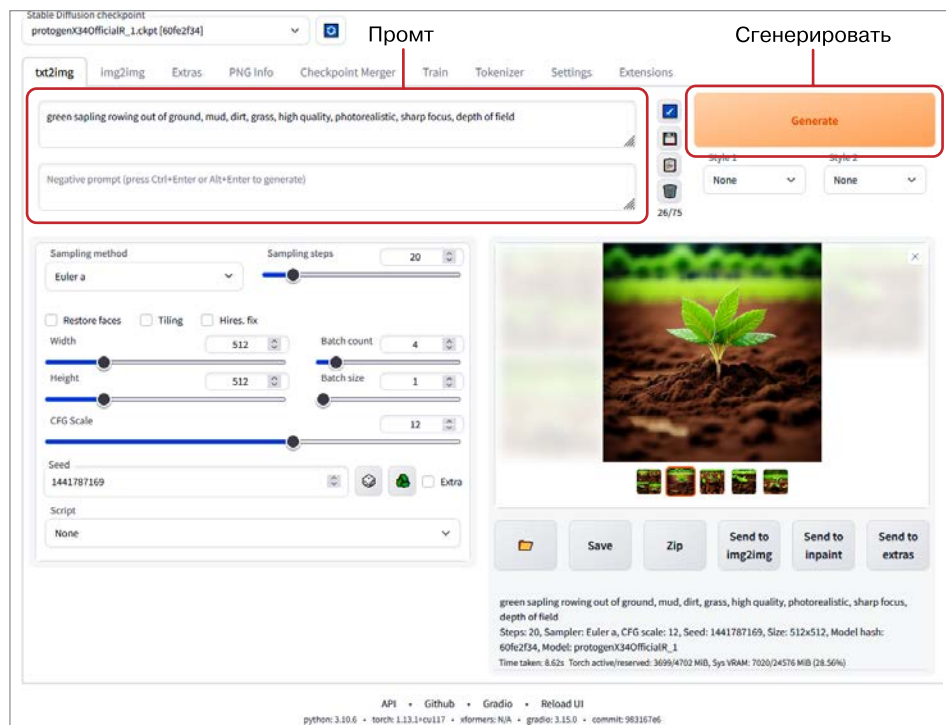


Рис. 9.4. Веб-интерфейс Stable Diffusion

В поле сразу под полем ввода запроса можете добавить отрицательный запрос, чтобы удалить из изображения нежелательные компоненты (подробнее об отрицательных запросах рассказывается в главе 8). Ниже вы найдете ряд настроек, включая начальное значение **Seed** (перед созданием каждого нового изображения устанавливается равным  $-1$ ), количество шагов обработки **Sampling steps**, количество пакетов **Batch count** (количество пакетов изображений, генерируемых одно за другим) и размер пакета **Batch size** (количество изображений, обрабатываемых в каждом пакете ценой увеличения объема потребляемой видеопамати). После создания изображения можно загрузить непосредственно из интерфейса, отправить их на различные вкладки с помощью кнопок ниже или открыть папку

stable-diffusion-webui/outputs, где файлы будут сохранены в подпапки по методу создания (text2img, img2img) и дате:

```
stable-diffusion-webui/
 outputs/
 txt2img-images/
 2023-10-05/
 your_image.png
```

При запуске веб-интерфейса AUTOMATIC1111 все загруженные модели появляются в раскрывающемся меню **Stable Diffusion Checkpoint** «Контрольные точки Stable Diffusion» вверху. Выберите базовую модель и введите запрос, а также настройте параметры как обычно. Не забудьте установить размер изображения  $1024 \times 1024$ . В разделе **Refiner** параметру **Switch at** (Переключить на) присвойте значение 1, чтобы запускалась только базовая модель (рис. 9.5).

The image shows the settings panel of the AUTOMATIC1111 web interface. It includes sliders for 'Sampling steps' (set to 20), 'Width' (set to 1024), 'Height' (set to 1024), 'CFG Scale' (set to 7), and 'Batch count' (set to 1). It also features a 'Refiner' section with a 'Checkpoint' dropdown and a 'Switch at' slider (set to 1). A 'Seed' field is set to -1. There are buttons for 'Extra' settings and a 'Refresh' icon.

**Рис. 9.5.** Стандартные настройки модели SDXL

Методы выборки, которые доступны, довольно комплексны и технически сложны для объяснения, но обычно приходится искать компромисс между скоростью, качеством и случайностью. Метод **Euler** — самый простой, а **DDIM** — самый первый, разработанный специально для моделей Diffusion. Методы выборки, в названии которых есть буква «a», например **Euler a**, — это производные методы, которые в процессе создания вводят шум в изображение. По этой причине

они дают менее воспроизводимые результаты из-за отсутствия сходимости (при каждом запуске модели в процесс создания изображения вносится некоторый элемент случайности). Методы DPM++ 2M Karras и UniPC при выборе 20–30 шагов обработки являются отличным выбором для получения надежных, стабильных и воспроизводимых изображений. Чтобы получить более качественные изображения, но за большее время и с большей долей случайности, попробуйте метод DPM++ SDE Karras или DDIM с 10–15 шагами.

Еще один важный параметр — CFG Scale (Classifier Free Guidance Scale — степень свободы следования указаниям — имеет то же назначение, что и аргумент `guidance_scale`, представленный в примере применения модели Stable Diffusion в среде Google Colab). Ниже перечислены некоторые типовые значения CFG Scale и их эффект:

- 1 — запрос можно свободно игнорировать;
- 3 — допускается проявлять творческий подход;
- 7 — хороший баланс между следованием запросу и творчеством;
- 15 — модель должна стараться следовать указаниям в запросе;
- 30 — модель должна строго следовать указаниям в запросе.

Изменить размер создаваемого изображения можно с помощью параметров Height (Высота) и Width (Ширина), а количество генерируемых изображений — с помощью параметра Batch Count (Количество пакетов). Флажок Highres fix (Исправление высокого разрешения) управляет механизмом повышения разрешения, который используется для создания больших изображений с высоким разрешением (подробнее об этом позже), флажок Restore faces (Восстановить лица) включает использование модели восстановления лица (по умолчанию Codeformer) для исправления дефектов человеческих лиц, которые часто появляются в изображениях, генерируемых Stable Diffusion, а флажок Tiling (Мозаика) создает изображения — фрагменты узоров, которыми можно заполнять большие площади. Также есть возможность сохранять и добавлять стили — простые запросы, — которые предполагается регулярно использовать повторно. На других вкладках вы найдете множество других мощных функций (<https://oreil.ly/MiSt1>), а также созданные сообществом расширения, которые вы тоже можете использовать, причем новые расширения добавляются на вкладки по мере их появления.

AUTOMATIC1111 поддерживает также взвешенные запросы и понятия, по аналогии с Midjourney (как рассказывалось в главе 8). Их определение немного отличается: вместо двойных двоеточий, как в Midjourney, следует использовать круглые скобки. Например, (пират) увеличит вес «пиратства» на 10 % или до 1,1, а двойные круглые скобки ((пират)) снова умножат его, то есть вес получится равным  $1,1 \times 1,1 = 1,21$ . Есть также возможность задавать веса точно в форме (понятие: вес), например (пират: 1,5), чтобы модель придавала этим токенам на 50 % больше внимания.



Ввод:

Мэрилин Монро в образе (пирата:1.5) на необитаемом острове, подробная прорисовка одежды, Стэнли Артгерм Лау и Альфонс Муха

Отрицательный запрос:

пикантность, нагота, декольте

Вывод показан на рис. 9.6.



**Рис. 9.6.** Мэрилин Монро в образе пирата

Квадратные скобки, например [пират], действуют аналогично, но в обратном направлении, уменьшая вес понятия на 10 %. Например, [шляпа] будет соответствовать весу 0,9 или (шляпа:0,9). Обратите внимание, что это не отрицательный запрос, потому что взвешенное понятие все равно будет присутствовать на генерируемом изображении, просто модель будет меньше акцентировать внимание на нем. Можно также указать отрицательный вес, чтобы удалить нежелательное понятие или уменьшить его проявление — таким способом можно гарантировать, что нежелательные элементы или стили не появятся на изображении, если вдруг отрицательного запроса окажется недостаточно.



#### Задайте направление

Возможность управления вниманием модели к конкретным словам или разделам запроса позволяет управлять деталями генерируемого результата.

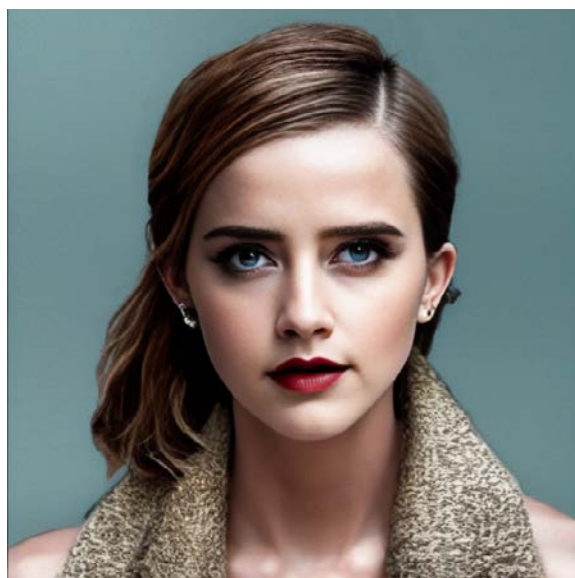
Более продвинутый метод, который предпочитают опытные пользователи AUTOMATIC1111, — *изменение* или *переключение запроса*. В процессе генерирования изображения на первых шагах диффузионная модель переходит от случайного шума к общим нечетким контурам фигур, которые, как ожидается, будут присутствовать на изображении, и лишь потом прорисовываются более тонкие детали. Возможность смены запроса позволяет передать другой запрос в середину процесса диффузии, открывая новые творческие возможности. Этот прием имеет синтаксис [от:до:когда], где от — начальный запрос, до — конечный

запрос, а когда — целое число шагов или дробное число, представляющее процент, когда должно произойти переключение. Получив запрос [Эмма Уотсон: Эмбер Херд: 0,5], модель начнет генерировать изображение Эммы Уотсон, а затем на полпути переключится на создание изображения Эмбер Херд, и в результате получится смешанное изображение двух актрис. Это полезный прием для создания изображений людей, которые выглядят привлекательными и смутно знакомыми, но не похожи до конца на какую-то конкретную знаменитость и поэтому могут рассматриваться как более этичные и юридически чистые, чем простые копии образов знаменитостей (обратитесь к своему юристконсульту).

Ввод:

модная фотосессия для журнала [Эмма Уотсон: Эмбер Херд: 0,5],  
очень реалистично, высокое разрешение, высокая детализация, драматично, 8K

Вывод показан на рис. 9.7.



**Рис. 9.7.** Смешанный образ Эммы Уотсон и Эмбер Херд



### Задайте направление

Изменение запроса позволяет глубже понять, как в действительности работает диффузионная модель. Вмешательство в работу слоев, реагирующих на разные концепции, может привести к интересным результатам, если вы знаете, чего хотите добиться, и готовы совершить достаточно большое количество попыток.

Заставить модель чередовать два понятия можно с помощью синтаксиса [Эмма Уотсон | Эмбер Херд]. В таком случае смена будет происходить на каждом шаге

и в конце получится еще более смешанный результат. Существует множество вариантов использования смены запроса, хотя эта возможность считается чем-то вроде темного искусства. Эксперты сообщают, что в некоторых случаях этот прием помогает сгенерировать изображения, которые не удастся получить другим способом, для чего они начинают с чего-то простого, а затем, на этапе окончательной детализации, переключаются на то, что действительно необходимо. Мы нашли лишь ограниченное применение этой техники в своей практике, но вы все же поэкспериментируйте с ней, возможно вы сможете обнаружить что-то, что действительно вам пригодится.

## Img2Img

Веб-интерфейс AUTOMATIC1111 поддерживает технику **Img2Img** генерирования новых изображений на основе существующих (рис. 9.8), которая является функциональным эквивалентом возможности передавать изображения в модель Midjourney вместе с запросом. Это дает дополнительные возможности управления стилем и композицией получаемого изображения. Чтобы добиться хороших результатов с применением приема **Img2Img**, попробуйте использовать метод выборки Euler, 50 шагов обработки и более высокое значение CFG Scale, где-нибудь от 20 до 30.

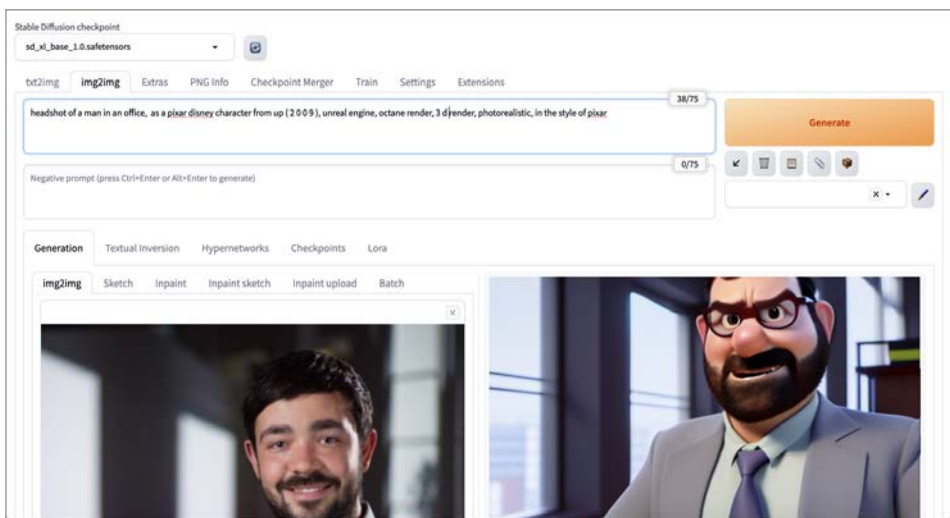


Рис. 9.8. **Img2Img**

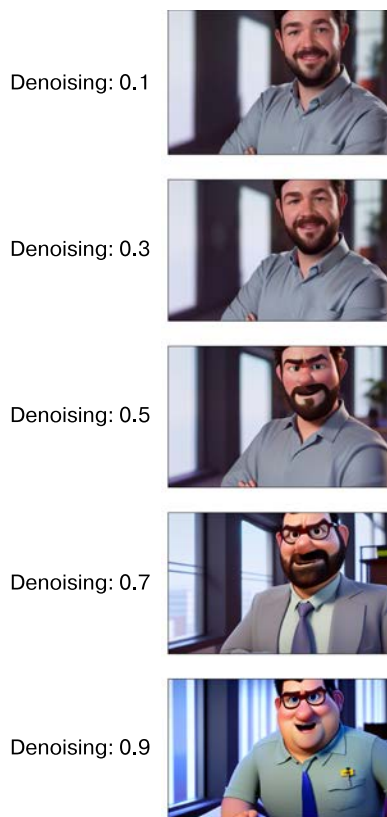
Эта техника поддерживает те же параметры, что и **Text2Image**, плюс дополнительный *уровень шумоподавления*, который определяет количество случайного шума, добавляемого к базовому изображению перед запуском процесса генерирования.

Значение 0 соответствует отсутствию шума, и в этом случае результат будет выглядеть так же, как исходное изображение, а значение 1 полностью заменит входное изображение шумом (что функционально соответствует технике Text2Image). Многие обычно экспериментируют с разными комбинациями значений уровня шумоподавления, CFG Scale и начального числа Seed. На рис. 9.9 показан пример создания персонажа в стиле Pixar, но имейте в виду, что мы не рекомендуем использовать защищенные IP-адреса в запросах для коммерческого использования.

Ввод:

снимок мужчины в офисе в образе персонажа Pixar Disney  
из фильма "Вверх" (2009), графический движок unreal engine,  
графический движок oscene render, 3D-отображение,  
фотореалистичный, в стиле Pixar

Вывод показан на рис. 9.9.



**Рис. 9.9.** Эффект влияния разных уровней шумоподавления на получающееся изображение

Если вы решите протестировать множество различных значений параметра в AUTOMATIC1111 и создать сетку, как показано на рис. 9.9, то такая возможность доступна в раскрывающемся списке **Script** (Сценарий) внизу, где можно выбрать **X/Y/Z Plot** (График X/Y/Z) и указать до трех параметров, значения которых должны генерироваться. Например, можно выбрать параметр **CFG Scale**, чтобы увидеть, как он влияет на результат вместе с шумоподавлением. На рис. 9.10 показано, как выбрать несколько значений для параметра **Denoising Strength** (Уровень шумоподавления). После нажатия кнопки **Generate** (Создать) будет создана сетка изображений, а сами изображения вы найдете в папке **Output** в подпапках с названием метода (то есть **Text2Image** или **Img2Img**) и текущей датой.

**Рис. 9.10.** Настройка сценария X/Y/Z Plot (График X/Y/Z) для оценки влияния параметра Denoising Strength



#### Оцените качество

Создание сетки для различных комбинаций или значений параметров — одно из самых больших преимуществ использования локальной модели Stable Diffusion. Конечно, для создания большого количества изображений может потребоваться время, но нет лучшего способа визуально определить, что именно делает параметр и где находится оптимальная точка с точки зрения качества.

На случай, если вы забудете, какие настройки или запросы использовались для создания изображения, AUTOMATIC1111 сохраняет все эти сведения как метаданные в каждом созданном изображении. Прочитать их можно на вкладке **PNG**

**Info** (Информация PNG) (рис. 9.11). Информацию можно также извлечь из изображений, созданных другими пользователями веб-интерфейса, но только если они разместили изображения на веб-сайте, который не удаляет эти метаданные.

Параметры **Resize Mode** (Режим изменения размера) помогают указать, что следует предпринять, когда размеры выгруженного изображения не соответствуют размерам генерируемого изображения, например, когда изображение с размерами  $1000 \times 500$  нужно преобразовать в изображение с размерами  $512 \times 512$ . **Just Resize** (Просто изменить размеры) либо растягивает, либо сжимает стороны до требуемого соответствия. **Crop and Resize** (Обрезать и растянуть) обрежет лишние части изображения до требуемого соответствия. **Resize and Fill** (Изменить размеры и заполнить) добавит шум для заполнения пустых частей изображения, образовавшихся при растягивании. **Resize (latent upscale)** (Изменить размеры (скрытое масштабирование)) создаст изображение с новыми размерами.

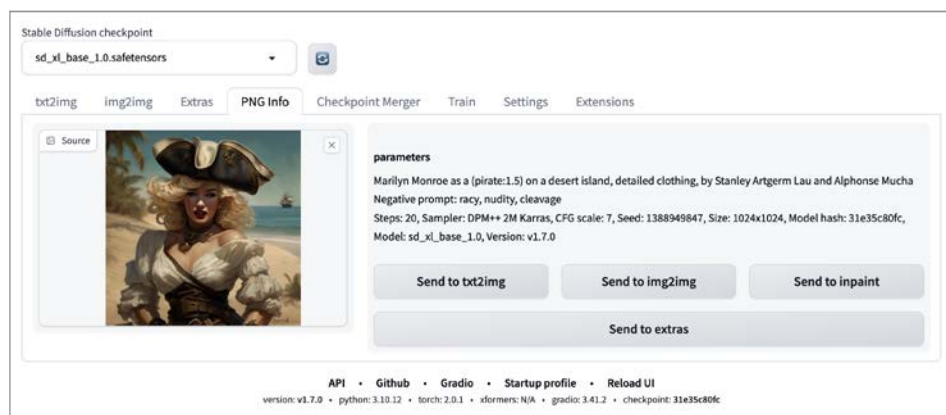
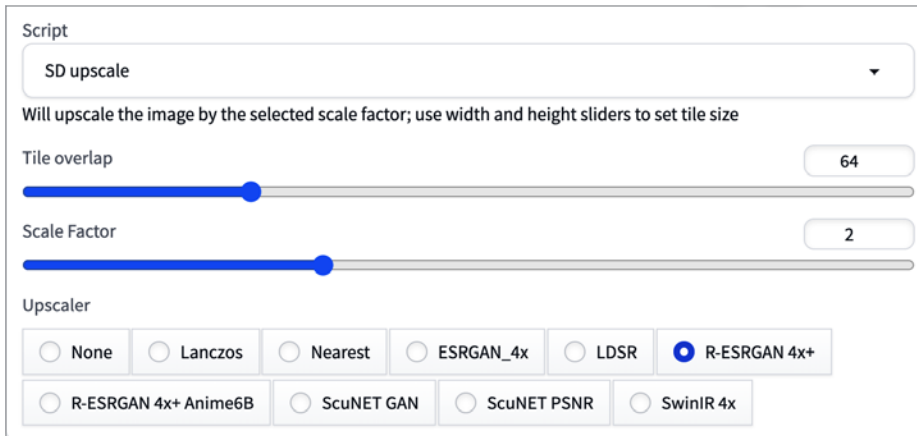


Рис. 9.11. Вкладка PNG Info (Информация PNG)

## Увеличение разрешения изображений

AUTOMATIC1111, как и Midjourney, позволяет увеличить разрешение изображения, но предлагает большее число настроек на вкладке **Img2Img**. Загрузите изображение и добавьте обобщенный запрос, например "высокое разрешение". Это необходимо, потому что механизм увеличения разрешения разбивает изображение на плитки, раздвигает их так, чтобы между плитками образовались промежутки, а затем *заполняет* промежутки, используя запрос и контекст окружающих пикселей. Перейдите в раздел **Scripts** (Сценарии) внизу и выберите сценарий **SD Upscale** (Масштабирование с SD<sup>1</sup>), а затем способ масштабирования (рис. 9.12).



Script

SD upscale

Will upscale the image by the selected scale factor; use width and height sliders to set tile size

Tile overlap 64

Scale Factor 2

Upscaler

☐ None ☐ Lanczos ☐ Nearest ☐ ESRGAN\_4x ☐ LDSR ☒ R-ESRGAN 4x+

☐ R-ESRGAN 4x+ Anime6B ☐ ScuNET GAN ☐ ScuNET PSNR ☐ SwinIR 4x

**Рис. 9.12.** Настройки метода SD Upscale

По нашему мнению, неплохие результаты показывает также метод R-ESRGAN 4x+, но иногда он может создавать мультяшный эффект. Есть еще ряд моделей (<https://openmodeldb.info>), которые можно попробовать применить для увеличения разрешения. Для этого загрузите новую модель (файл .pth), поместите ее в папку ESRGAN и перезапустите веб-интерфейс (в терминале), после чего новая модель появится в интерфейсе. Иногда хорошие результаты при увеличении разрешения можно получить, изменив запрос, особенно если прежний запрос приводит к потере существенных деталей или изменению стиля. В таких случаях не рекомендуется использовать исходный запрос, так как это приведет к странному эффекту рисования одного и того же изображения на каждой плитке. Чтобы показать разницу в качестве, мы использовали модель v1.5 для создания исходного изображения (SDXL создает изображения в четыре раза больше и с более высоким качеством, поэтому обычно нет необходимости увеличивать их разрешение).



#### Укажите формат

Сгенерированные изображения с размерами  $512 \times 512$  и с низким разрешением редко где могут пригодиться в реальном мире. Используя функцию увеличения разрешения, можно создать изображение любого размера и с любым разрешением.

Как со всем, что касается Stable Diffusion, с увеличением разрешения желательно экспериментировать, но вообще для получения хороших результатов мы рекомендуем указывать большое количество шагов обработки (150–200 и больше), значение CFG Scale 8–15 и уровень шумоподавления 0,1–0,2, чтобы минимизировать изменения в базовом изображении. Вы можете нажать кнопку

**Generate** (Создать), чтобы получить изображение с увеличенным разрешением (с  $512 \times 512$  до  $1024 \times 1024$ ), а затем загрузить изображение с увеличенным разрешением или нажать кнопку **Send to Img2Img** (Отправить в Img2Img) и снова нажать **Generate** (Создать), чтобы снова удвоить разрешение изображения (рис. 9.13). Этот процесс может занять значительное время из-за создания множества плиток и большого количества шагов обработки — примерно 10–30 минут на M2 MacBook Air.

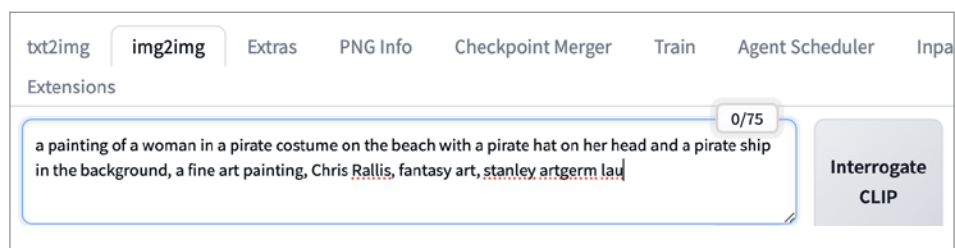


**Рис. 9.13.** Как меняются фрагменты изображений в результате применения функции увеличения разрешения



## Реконструирование запроса по изображению

Как рассказывалось в главе 8, в Midjourney есть функция Describe (Описать), позволяющая реконструировать запрос по изображению. Аналогичная функция имеется на вкладке **Img2Img**, где реализован доступ к модели CLIP (которая тоже используется в Stable Diffusion) в виде кнопки **Interrogate CLIP** (Опросить CLIP) (в некоторых версиях интерфейса вместо надписи отображается значок в виде скрепки). После нажатия этой кнопки и запуска сценария в поле ввода появится текст запроса (рис. 9.14).



**Рис. 9.14.** Реконструирование запроса по изображению с помощью модели CLIP

Вывод:

картина женщины в костюме пирата на пляже  
с пиратской шляпой на голове и пиратским кораблем на заднем плане,  
изобразительное искусство, живопись, Крис Раллис, фэнтези-арт,  
Стэнли Артгерм Лау

## Изменение фрагментов и добавление обрамления

Img2Img поддерживает также возможности изменения фрагментов внутри изображений и добавление обрамления. Чтобы использовать любую из этих функций, щелкните на вкладке **Inpaint** внутри вкладки **Img2Img** и загрузите свое изображение. Для достижения лучших результатов рекомендуется использовать определенную модель рисования, которую вы можете установить, загрузив ([https://oreil.ly/s\\_trl](https://oreil.ly/s_trl)) файл `sd-v1-5-inpainting.ckpt` и поместив его в папку `models/Stable-diffusion`. После установки модели перезапустите интерфейс; модель должна появиться в раскрывающемся списке вверху слева. Холст позволяет использовать кисть для удаления частей изображения, как в DALL-E (см. главу 8), размер которой можно регулировать для более детального управления. На рис. 9.15 показано, как мы удалили центр каменного круга в середине двора замка.

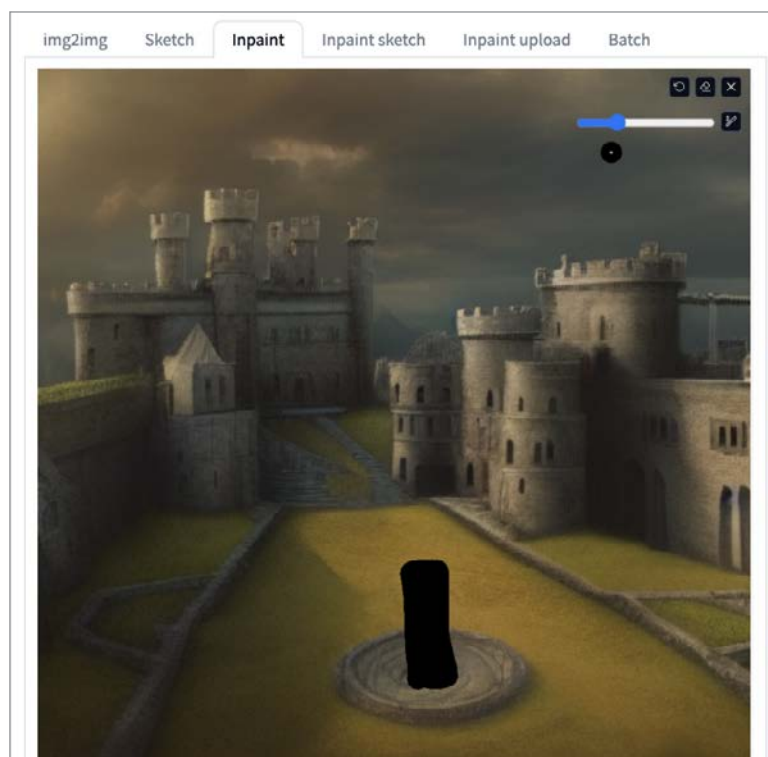


Рис. 9.15. Холст в Img2img для изменения фрагмента изображения

Пользователям DALL-E обычно советуют при изменении фрагмента внутри изображения использовать запрос, описывающий все изображение, а не только стертую область. Тот же совет можно дать пользователям Stable Diffusion. Убедитесь, что для области холста установлено значение **Whole picture** (Все изображение), а не **Only masked** (Только маска), иначе модель попытается поместить всю сцену в замаскированную область (спешим вас успокоить: даже если вы выберете **Whole picture** (Все изображение), модель изменит только замаскированную область). Также можно порекомендовать использовать то же значение параметра **Seed** (Начальное значение), что и при создании исходного изображения, если, конечно, оно было создано с помощью ИИ. Однако, как показывает наш опыт, добавление в запрос сведений о фрагменте, который предполагается изменить или исправить, обычно дает лучшие результаты. По крайней мере, попробуйте сменить тему запроса; например, на рис. 9.16 показано, что получилось после замены запроса, предлагающего нарисовать замок, запросом нарисовать статую, которую мы хотели увидеть во дворе. Вы также можете попробовать написать запрос, описывающий только замаскированную область, хотя в этом случае есть риск получить фрагмент, не согласующийся с остальным изображением по стилю.

Ввод:

статуя короля, текстура, причудливая, детальная, хорошо прорисованная, шедевр, архитектура, здания, в тренде artstation, четкая, контрастная, концепт-арт, цифровая картина, фэнтези, солнечный день, полдень, в стиле высокого фэнтези-искусства

Вывод показан на рис. 9.16.



**Рис. 9.16.** Добавление статуи на изображение



### Задайте направление

Функция изменения фрагмента заслуживает особого внимания, потому что позволяет эффективно изолировать отдельную часть изображения и подробно описать, как ее исправить, не затрагивая все остальное.

Если требуется лишь небольшая корректировка замаскированной области, то используйте значение **Original** (Оригинал) в параметре **Masked Content** (Маскированное содержимое) и значение от 0,2 до 0,4 в параметре **Denoising Strength** (Уровень шумоподавления). При полной замене фрагмента вам может понадобиться установить параметр **Latent Noise** (Скрытый шум) и задать высокое значение, например 0,8, в параметре **Denoising Strength** (Уровень шумоподавления), хотя при значениях выше 0,4 обычно начинают появляться несогласованные элементы и галлюцинации, поэтому могут потребоваться силы и время, чтобы найти более или менее подходящее решение. Может также пригодиться параметр **Fill**

(Заливка) для приведения цветов в соответствие с окружающей областью. Если у вас появятся некрасивые швы по краям измененного фрагмента, то попробуйте увеличить размытие маски **Mask Blur** (Размытие маски), хотя обычно вполне достаточно значения по умолчанию 4. Изменение фрагмента изображения — это итеративный процесс. Мы советуем исправлять проблемы по одной за раз, делать столько попыток, сколько захотите, и экспериментировать с разными параметрами, пока не получите удовлетворительный результат.

Функция добавления обрамления работает не так, как в Midjourney (см. главу 8), где есть возможность указать 1,5- или 2-кратное масштабирование или свое соотношение сторон. В AUTOMATIC1111 добавление обрамления реализуется сценарием **Poor man's outpainting** (Упрощенное добавление обрамления), который можно выбрать в раскрывающемся списке **Script** (Сценарий). При этом следует установить режим изменения размера в **Resize and Fill** (Изменить размеры и заполнить) на вкладке **Inpaint in Img2Img** и выбрать относительно высокий уровень шумоподавления. Функция добавления обрамления позволяет расширить пиксели по разные стороны изображения, устанавливая при этом типичные значения в параметрах **Masked Content** (Маскированное содержимое) и **Mask Blur** (Размытие маски).

Вывод показан на рис. 9.17.



Рис. 9.17. Добавление обрамления в **Img2Img**

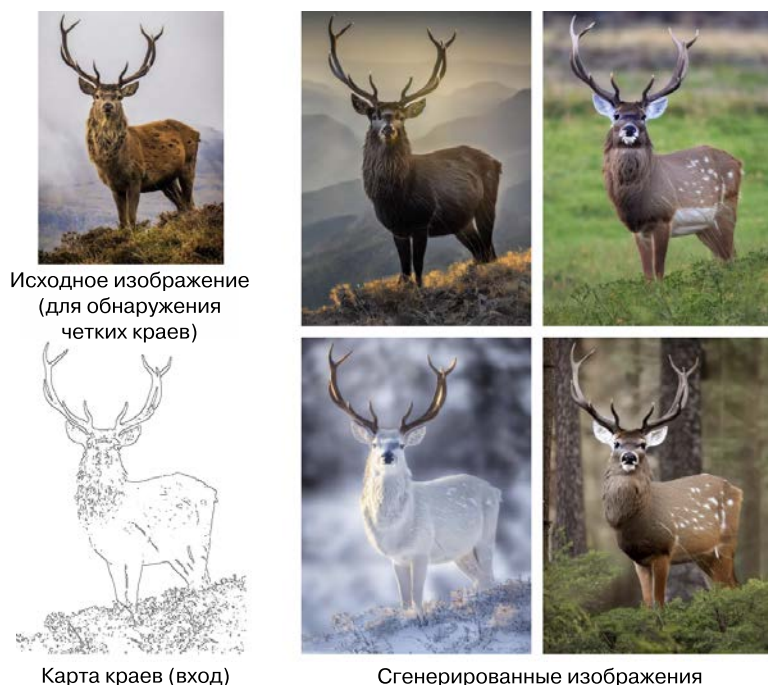
Как можно видеть на рис. 9.17, на небо был добавлен еще один замок, что говорит о высокой вероятности галлюцинаций и, возможно, низком качестве результата. Часто приходится подолгу экспериментировать, чтобы получить желаемое. Это похоже на то, как первые поклонники генеративного ИИ добавляли дополнительное пустое пространство по бокам фотографий в Photoshop, прежде чем перерисовывать их и получить обрамление, соответствующее остальной части изображения. По сути, эта техника сводится к простому изменению фрагментов с включением дополнительных шагов, поэтому к ней применимы все те же советы, которые были перечислены ранее. Она может оказаться эффективнее функции добавления обрамления в AUTOMATIC1111 из-за низкого качества и ограничений в последней, обусловленных отсутствием подходящего холста.

## ControlNet

Функция `Img2Img` позволяет управлять стилем изображения с помощью запросов или базовых изображений, но зачастую в конечном изображении позы людей композиция сцены или структура объектов сильно отличаются. Для преодоления этих проблем была создана модель ControlNet, предлагающая усовершенствованный способ обработки входных изображений для моделей генеративного ИИ, таких как Stable Diffusion.

Она позволяет получить более полный контроль над конечным изображением, применяя такие методы, как обнаружение краев, определение поз и глубины, и многие другие. Вы загружаете изображение для имитации, применяете один из вариантов предварительно обученной модели вместе с запросом и получаете изображение с той же композицией, но с другим стилем (рис. 9.18, заимствован из статьи о ControlNet (<https://oreil.ly/suOJz>)).

Под общим названием ControlNet скрывается целая серия моделей с открытым исходным кодом (<https://oreil.ly/E-bjw>), выпущенных после публикации статьи *Adding Conditional Control to Text-to-Image Diffusion Models* (Zhang, Rao, Agrawala, 2023 (<https://oreil.ly/ZH-Ow>)). Описанный выше алгоритм действий можно реализовать на Python и создать для него свой пользовательский интерфейс, но намного быстрее и проще начать работу, используя расширение ControlNet (<https://oreil.ly/Dw2rs>) для AUTOMATIC1111. На момент написания этой книги для SDXL были доступны не все методы, поддерживаемые ControlNet, поэтому для демонстрации мы решили использовать Stable Diffusion v1.5 (обратите внимание, что модель ControlNet должна соответствовать используемой версии Stable Diffusion).



**Рис. 9.18.** ControlNet Stable Diffusion с методом обнаружения краев

Для установки расширения выполните следующие инструкции.

1. Перейдите на вкладку **Extensions** (Расширения) и выберите внутреннюю вкладку **Available** (Доступные).
2. Нажмите кнопку **Load from** (Загрузить из).
3. В поле поиска введите `sd-webui-controlnet`, чтобы найти расширение.
4. Нажмите кнопку **Install** (Установить) в крайнем правом столбце **Action** (Действие).
5. После этого веб-интерфейс загрузит необходимые файлы и установит ControlNet в вашу локальную версию Stable Diffusion.

Если у вас возникли проблемы с выполнением предыдущих шагов, то попробуйте альтернативный метод.

1. Перейдите на вкладку **Extensions** (Расширения) и выберите вложенную вкладку **Install from URL** (Установить из URL).
2. В поле ввода URL вставьте ссылку на расширение: <https://github.com/Mikubill/sd-webui-controlnet>.

3. Нажмите кнопку **Install** (Установить).
4. Веб-интерфейс загрузит необходимые файлы и установит ControlNet.

Завершив установку ControlNet, перезапустите AUTOMATIC1111 из терминала или перейдите на вкладку **Settings** (Настройки) и нажмите кнопку **Apply and restart UI** (Применить и перезапустить интерфейс).

Расширение появится ниже обычного списка параметров для Stable Diffusion в отдельной вкладке (рис. 9.19). Сначала нужно загрузить изображение, которое послужит основой, затем установить флажок **Enable** (Включить), а потом выбрать препроцессор и модель ControlNet. Если ваша система имеет менее 6 Гбайт видеопамати, то установите флажок **Low VRAM** (Мало видеопамати). Следуя поставленной задаче, поэкспериментируйте с несколькими моделями и их параметрами, чтобы понять, какая комбинация даст нужный результат.

ControlNet v1.1.426 **1 unit**

**ControlNet Unit 0 [Canny]** ControlNet Unit 1 ControlNet Unit 2

☒ Enable ☐ Low VRAM ☐ Pixel Perfect

☐ Upload independent control image

Control Type

☐ All ☒ Canny ☐ Depth ☐ NormalMap ☐ OpenPose ☐ MLSD

☐ Lineart ☐ SoftEdge ☐ Scribble/Sketch ☐ Segmentation ☐ Shuffle

☐ Tile/Blur ☐ Inpaint ☐ InstructP2P ☐ Reference ☐ Recolor ☐ Revision

☐ T2I-Adapter ☐ IP-Adapter

Preprocessor Model

canny None

Control Weight 1 Starting Control Step 0 Ending Control Step 1

Preprocessor Resolution 512

Canny Low Threshold 100

Canny High Threshold 200

Control Mode

☒ Balanced ☐ My prompt is more important ☐ ControlNet is more important

**Рис. 9.19.** Настройки ControlNet в интерфейсе AUTOMATIC1111



**Control Weight** (Контрольный вес) определяет вес (степень влияния) запроса для ControlNet, подобно тому как определяется вес слов в скобках (слова из запроса : 1.2). Параметры **Starting Control Step** (Шаг начала применения) и **Ending Control Step** (Шаг завершения применения) определяют, когда в процессе диффузии должна применяться модель ControlNet, по умолчанию модель применяется от начала до конца (от 0 до 1). Эти два параметра действуют аналогично применению части запроса со сдвигом, например часть [запрос : :0.8] будет применяться от начала диффузии до момента, когда будет выполнено 80 % общего числа шагов. Поскольку процесс диффузии изображений идет в направлении от более крупных к более мелким деталям, можно добиться разных результатов, управляя интервалом, где в этом процессе применяется ControlNet. Например, запретив применение модели на последних 20 % шагов (**Ending Control Step** = 0.8), можно позволить модели более творчески подойти к заполнению мелких деталей. Параметр **Preprocessor Resolution** (Разрешение препроцессора) тоже поможет сохранить контроль над деталями — он определяет, сколько мелких деталей будет воспроизведено на промежуточном этапе обработки изображения. Некоторые модели имеют свои уникальные параметры, такие как **Canny Low Threshold** (Нижний порог краев) и **Canny High Threshold** (Верхний порог краев), которые определяют, какие пиксели будут интерпретироваться как *край*. Наконец, **Control Mode** (Режим управления) определяет, насколько строго модель должна следовать запросу.

Сразу после установки ControlNet у вас не будет загружено ни одной модели. Чтобы они появились в раскрывающемся списке, их нужно установить, загрузив со страницы моделей ([https://oreil.ly/csYK\\_](https://oreil.ly/csYK_)), а затем поместив их в папку `models/ControlNet`. Если вы не знаете, какую модель выбрать, то начните с наиболее универсальной модели обнаружения краев Canny (<https://oreil.ly/z9XC6>). Каждая модель имеет относительно большой размер (порядка нескольких гигабайт), поэтому загружайте только те, которые планируете использовать. Далее мы покажем примеры применения некоторых наиболее популярных моделей. Все изображения в этом разделе созданы с применением обработчика DPM++ SDE Karras, со значением 1.5 в параметре **CFG Scale**, значением **Balanced** (Сбалансированный) в параметре **Control Mode** (Режим управления), значением **Crop and Resize** (Обрезать и растянуть) в параметре **Resize Mode** (Режим изменения размера) (загруженное изображение обрезается до размеров генерируемого изображения 512 × 512), с применением 30 шагов обработки и с настройками по умолчанию для каждой модели ControlNet. Мы использовали версию Stable Diffusion 1.5, потому что на момент написания книги не все эти модели были доступны для Stable Diffusion XL, но ожидается, что в дальнейшем будут перенесены все модели.



Модель обнаружения границ Canny создает простые и четкие контуры пикселей вокруг областей с высокой контрастностью. Обычно она дает отличные результаты, но иногда может улавливать нежелательный шум и слишком жестко контролировать генерируемое изображение. Модель Canny показывает превосходные результаты в сценариях, когда изображение с высокой степенью детализации требуется перенести в изображение с другим стилем, поэтому имеет смысл попробовать применить ее первой. Например, модель Canny прекрасно справится с перерисовкой линии горизонта города в определенном стиле, как мы сделали это с фотографией Нью-Йорка (автор Роберт Бай (Robert Bye) (<https://oreil.ly/wEPLB>), доступна на Unsplash ([https://oreil.ly/\\_iyxU](https://oreil.ly/_iyxU))) на рис. 9.20.

Ввод:

Нью-Йорк в стиле студии Ghibli

Вывод показан на рис. 9.20.



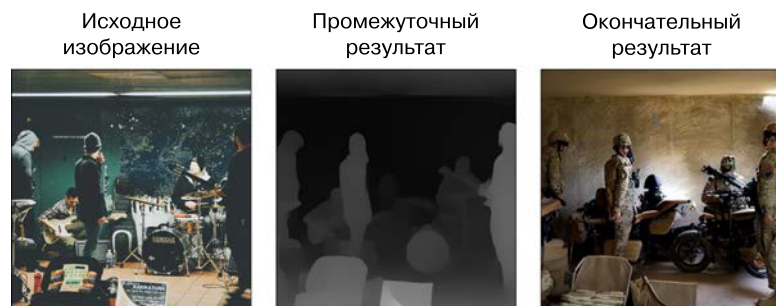
**Рис. 9.20.** ControlNet Canny

Иногда при обычном вызове `img2img` некоторые элементы изображения путаются или сливаются, потому что Stable Diffusion не понимает глубины их размещения по отношению друг к другу. Модель Depth создает карту глубин на основе изображения, что помогает управлять композицией и пространственным положением элементов на изображении. Для тех, кто не знаком с картами глубин, поясним, что более светлые области находятся ближе к зрителю, а более темные — дальше. Пример можно увидеть на рис. 9.21, где фотография музыкантов на сцене (сделанная Хансом Вивеком (Hans Vivek) (<https://oreil.ly/tICrf>) и доступная на Unsplash (<https://oreil.ly/BOKJ7>)) превращается в изображение солдат с учетом расположения фигур на исходной фотографии и глубины резкости.

Ввод:

Военный патруль США в Афганистане

Вывод показан на рис. 9.21.



**Рис. 9.21.** ControlNet Depth

Модель Normal создает карту нормалей, которая функционирует как трехмерная модель объектов на изображении. Для выражения гладкости или неровности объекта используются красный, зеленый и синий цвета, причем каждый цвет соответствует определенному направлению (влево/вправо, вверх/вниз, ближе/дальше). Однако это всего лишь оценка, поэтому иногда она может не совпадать с действительностью и приводить к неожиданным результатам. Обычно эта модель хорошо показывает себя, когда больше внимания должно уделяться текстуре и освещению, но иногда она может сгенерировать слишком много деталей, особенно на изображениях лиц, ограничивая творческий подход к результату. На рис. 9.22 показана фотография женщины, играющей на синтезаторе (автор Soundtrap (<https://oreil.ly/RP1Ei>), доступна на Unsplash (<https://oreil.ly/I3QGY>)), которая переносится назад во времени, в эпоху Великого Гэтсби.

Ввод:

Женщина играет на пианино на вечеринке в честь Великого Гэтсби, 1920-е годы, симметричное лицо

Вывод показан на рис. 9.22.

Модель OpenPose создает скелет фигуры, определяя ее позу, положение рук и выражение лица. Чтобы получить хороший результат от применения этой модели, желательно иметь изображение всего тела человека, но ее можно использовать и для создания портретов. Широко распространена также практика компоновки нескольких скелетов OpenPose в одно изображение, если в сцене



**Рис. 9.22.** ControlNet Normal

присутствуют несколько человек. На рис. 9.23 показан портрет Рэйчел Вайс в позе Моны Лизы (<https://oreil.ly/7n02i>).

Ввод:

портрет Рэйчел Вайс

Вывод показан на рис. 9.23.



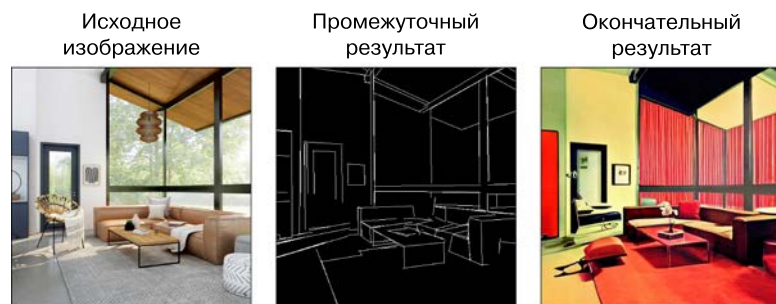
**Рис. 9.23.** ControlNet OpenPose

Модель M-LSD (Mobile Line Segment Detection) довольно часто используется в архитектуре и дизайне интерьеров, потому что она хорошо справляется с отслеживанием прямых линий. Прямые линии обычно присутствуют только в искусственных объектах, поэтому данная модель плохо подходит для обработки сцен природы (хотя иногда она может создавать довольно интересные эффекты). Наилучшие результаты получаются при применении модели к изображениям искусственных объектов, например домов, как показано на рис. 9.24, где изображена современная квартира (автор Collov Home Design ([https://oreil.ly/OtV\\_O](https://oreil.ly/OtV_O)), доступно на Unsplash (<https://oreil.ly/z38do>)) в стиле сериала «Безумцы» (Mad Men).

Ввод:

Квартира в стиле "Безумцев" 1960-х годов

Вывод показан на рис. 9.24.



**Рис. 9.24.** ControlNet M-LSD

Модель SoftEdge, также известная как HED (Holistically-nested Edge Detection), является альтернативой модели обнаружения краев Canny. SoftEdge создает более плавные контуры вокруг объектов и обеспечивает такую же хорошую детализацию, как Canny, но менее подвержена воздействию шума и генерирует более эстетически привлекательные результаты. Этот метод отлично подходит для стилизации и перекрашивания изображений, а также позволяет точнее манипулировать лицами, чем Canny. При использовании ControlNet нет нужды вводить слишком подробные описания желаемых изображений, достаточно просто запросить требуемое изменение. На рис. 9.25 показано переосмысление картины Вермеера «Девушка с жемчужной сережкой» (<https://oreil.ly/RjUur>) со Скарлетт Йоханссон:

Ввод:

Скарлетт Йоханссон, высокое качество, чрезвычайная детализация

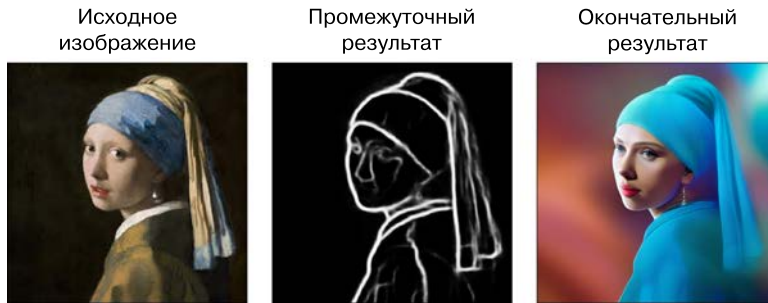
Отрицательный:

монохромный, низкое разрешение, плохая анатомия, плохое качество, низкое качество

Вывод показан на рис. 9.25.

Еще одна популярная модель, обычно используемая в архитектуре, — Segmentation. Она делит изображение на области — сегменты, — которые в той или иной степени связаны друг с другом. Сегментирование в чем-то похоже на использование маски в Img2Img, но показывает лучшие результаты. Модель сегментирования Segmentation можно использовать, когда требуется больший контроль

над различными объектами на изображении. Один из эффективных сценариев использования — сцены на открытом воздухе, вид которых может меняться в зависимости от времени суток, окружающей среды или даже эпохи. Взгляните на рис. 9.26, где показана современная фотография замка (сделанная Ричардом Кларком (Richard Clark) (<https://oreil.ly/SG9CT>) и доступная на Unsplash (<https://oreil.ly/2FlyI>)), преобразованная в рисунок замка в стиле фэнтези.

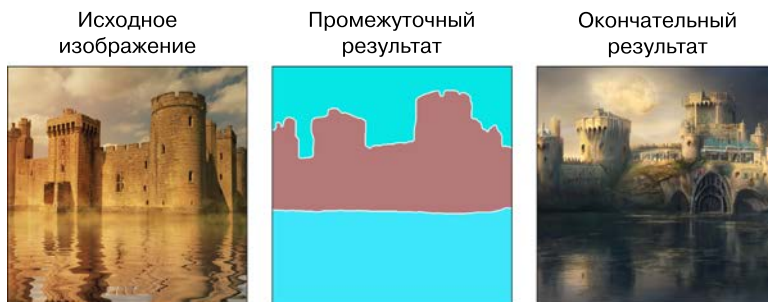


**Рис. 9.25.** ControlNet SoftEdge

Ввод:

Красивый волшебный замок, вид снаружи, текстурный, сложный, детализированный, очень подробный, шедевр, архитектура, здание, тренды на artstation, четкий, контрастный, концепт-арт, цифровая живопись, фэнтези, солнечный день, полдень, в стиле высокого арт-фэнтези

Вывод показан на рис. 9.26.



**Рис. 9.26.** ControlNet Segmentation

Одна из мощных особенностей ControlNet — возможность преобразования рисунков, нарисованных на холсте. Можно также нарисовать рисунок на бумаге, сделать снимок и загрузить его. Но для создания простого наброска достаточно

щелкнуть на значке карандаша в веб-интерфейсе Stable Diffusion и нарисовать что-нибудь прямо в нем. Часто бывает достаточно даже простых каракулей с неидеальными краями, как показано на рис. 9.27.

Ввод:

Золотая рыбка, иллюстрация для детской книги

Вывод показан на рис. 9.27.



Рис. 9.27. ControlNet Scribble



### Приведите примеры

ControlNet дает ИИ-художнику возможность создавать изображения, *похожие* своей композицией на другие изображения, предоставляя пример для подражания. Это позволяет точнее управлять визуальной согласованностью и дает большую гибкость при создании сложных изображений.

Каждая из описанных выше моделей ControlNet имеет свой препроцессор, и чтобы получить качественное изображение, применяемый препроцессор должен соответствовать модели. Например, если вы используете препроцессор Canny, то вам также следует использовать модель Canny, например `control_v11p_sd15_canny`. Старайтесь выбирать модели, которые дают достаточно свободы для решаемой задачи. Например, обработка изображения кошки с помощью модели SoftEdge может дать слишком много деталей, чтобы превратить ее в льва, поэтому лучше попробовать что-то, что дает меньше деталей. Как и во всем, что касается Stable Diffusion, чтобы найти подходящее сочетание модели и параметров, нужны эксперименты, и это притом что постоянно появляются новые возможности.

ControlNet можно запустить с простым запросом или даже без запроса. Оно постарается сгенерировать изображение, соответствующее существующему, и обеспечить высокую степень сходства. Вы можете запустить стандартный запрос, например "профессиональное, детальное, высококачественное изображение",

и получить хорошую версию существующего изображения. Однако чаще вы будете пытаться изменить некоторые аспекты изображения и вводить более полный запрос, как показано в предыдущих примерах. Полученное изображение будет соответствовать как запросу, так и результату, сгенерированному ControlNet, при этом вы сможете поэкспериментировать с настройкой доступных параметров и посмотреть, к чему это приводит.

## Segment Anything Model (SAM)

При работе над изображением, созданным искусственным интеллектом, часто полезно иметь возможность выделить *маску*, представляющую конкретного человека, объект или элемент. Например, отделив изображение человека от фона, вы сможете нарисовать новый фон позади него. Создание маски вручную с помощью кисти может занять много времени и привести к ошибкам, поэтому часто полезно иметь возможность автоматически сегментировать изображение, опираясь на линии, проведенные моделью ИИ.

Самая популярная и мощная модель для такого сегментирования — SAM (Segment Anything Model — универсальная модель сегментирования), выпущенная компанией Meta с открытым исходным кодом и доступная на GitHub (<https://oreil.ly/BuunX>). Модель обучена на наборе данных, состоящем из 11 миллионов изображений и 1,1 миллиарда масок, и способна определить, где на изображении должна находиться маска, соответствующая запросу пользователя (щелкните мышью на изображении, чтобы добавить от одной до трех точек там, где должна быть маска). Также она может автоматически подобрать отдельные маски для всех элементов на изображении. Эти маски затем можно экспортировать и использовать для изменения фрагментов изображений, в ControlNet или в качестве базовых изображений.

Чтобы получить возможность использовать SAM в интерфейсе AUTOMATIC1111, нужно установить расширение *sd-webui-segment-anything* (<https://oreil.ly/rFMJN>). Это можно сделать, следуя инструкциям ниже.

1. Перейдите на вкладку **Extensions** (Расширения) и выберите внутреннюю вкладку **Available** (Доступные).
2. Нажмите кнопку **Load from** (Загрузить из).
3. В поле поиска введите **sd-webui-segment-anything**, чтобы найти расширение.
4. Нажмите кнопку **Install** (Установить) в крайнем правом столбце **Action** (Действие).
5. После этого веб-интерфейс загрузит необходимые файлы и установит SAM в вашу локальную версию Stable Diffusion.



Если у вас возникли проблемы с выполнением предыдущих шагов, то попробуйте альтернативный метод.

1. Перейдите на вкладку **Extensions** (Расширения) и выберите вложенную вкладку **Install from URL** (Установить из URL).
2. В поле ввода URL вставьте ссылку на расширение: <https://github.com/continue-revolution/sd-webui-segment-anything>.
3. Нажмите кнопку **Install** (Установить).
4. Веб-интерфейс загрузит необходимые файлы и установит SAM.

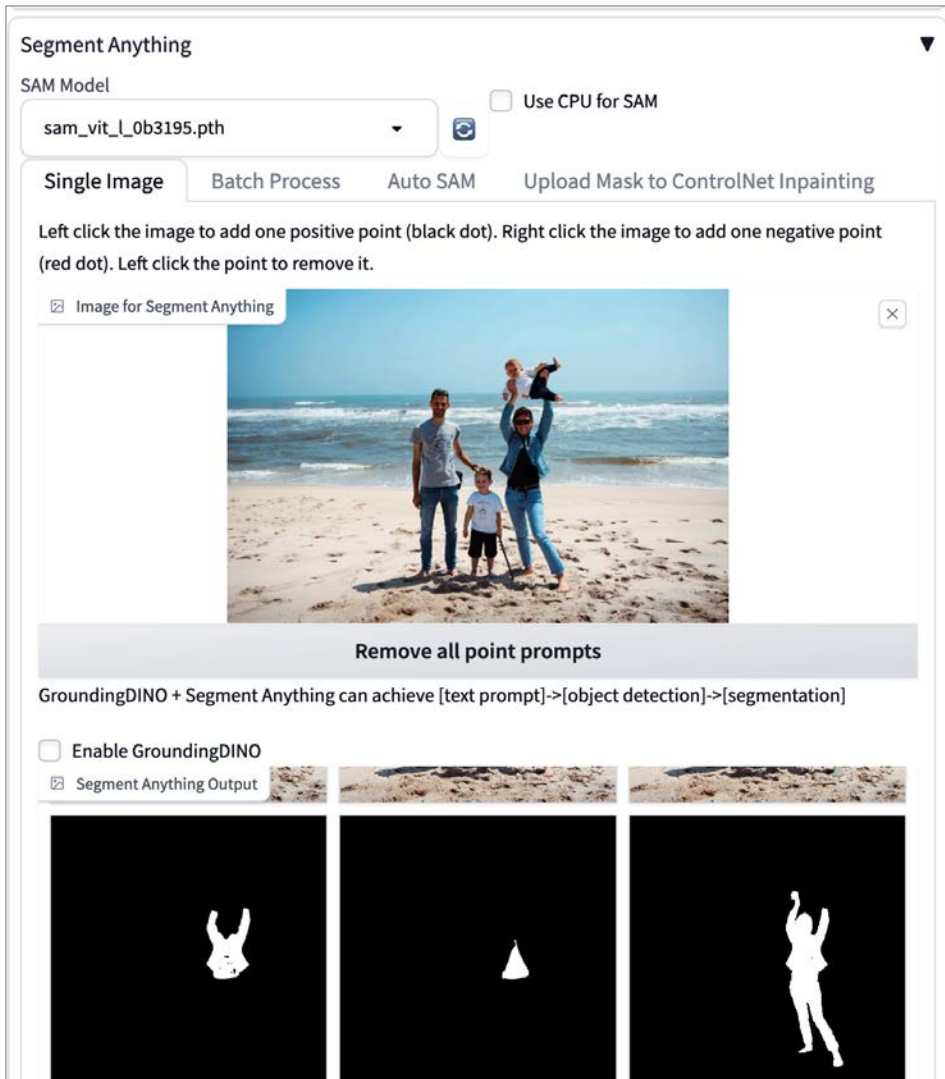
Вам также понадобится загрузить фактические веса модели SAM, ссылки на которые имеются в репозитории (<https://oreil.ly/IqrbI>). В этой главе используется файл с весами `sam_vit_1_0b3195.pth` размером 1,25 Гбайт. Если у вас возникнут проблемы с нехваткой видеопамяти (компьютер зависает или «тормозит»), то попробуйте выбрать модель меньшего размера. Переместите загруженный файл модели в папку `stable-diffusion-webui/sd-webui-segment-anything/models/sam`.

После установки SAM перезапустите AUTOMATIC1111 из терминала или перейдите на вкладку **Settings** (Настройки) и нажмите кнопку **Apply and restart UI** (Применить и перезапустить интерфейс).

Расширение должно появиться на вкладке **Img2Img** рядом с расширением **ControlNet**. Загрузите изображение (мы, в частности, использовали фотографию Луки Баини (Luca Baini) (<https://oreil.ly/Lb3xE>), доступную на Unsplash (<https://oreil.ly/jvCjz>)), как показано на рис. 9.28, и щелкните на нем левой кнопкой мыши, чтобы выбрать точки для создания маски. Координаты точек будут переданы в SAM в качестве запроса и помогут модели определить, что следует замаскировать на изображении. Можно нажать кнопку **Preview** (Предварительный просмотр), чтобы увидеть, какая маска будет создана, и добавить или удалить точки на изображении. Повторяя эту процедуру, можно добиться получения правильной маски. Можно также установить флажок **Preview automatically when add/remove points** (Автоматически обновлять предварительный просмотр при добавлении/удалении точек), и тогда маска будет обновляться после каждого щелчка кнопкой мыши. Часто SAM правильно определяет маску, по единственной точке, но в случае проблем можно также щелчками правой кнопкой мыши добавить отрицательные точки на те части изображения, которые не должны включаться в маску. Выберите нужную маску (см. рис. 9.28) из трех предложенных вариантов (считая от 0 до 2).

Закончив определение маски, установите флажок **Copy to Inpaint Upload & img2img ControlNet Inpainting** (Копировать в Inpaint Upload и img2img ControlNet Inpainting) и нажмите кнопку **Switch to Inpaint Upload** (Переключиться в Inpaint





**Рис. 9.28.** Добавление точек для включения в маску

Upload). При этом вы не увидите никаких внешних изменений, но, перейдя на вкладку **Inpainting** (Изменение фрагмента), вы сможете сгенерировать запрос с маской, созданной SAM. При этом вам не придется загружать картинку или маску на вкладке **Inpainting** (Изменение фрагмента). Вы также можете скачать

маску для последующей загрузки на вкладке **Inpaint Upload** (Загрузка для изменения фрагмента). Однако в наших экспериментах этот метод оказался ненадежным, и, возможно, существует лучший поддерживаемый метод изменения фрагментов с помощью SAM и Stable Diffusion.



### Разделите задачу на подзадачи

Генеративные модели, такие как Midjourney и Stable Diffusion, весьма эффективны, но и им не все под силу. Обучая отдельную модель сегментации изображений, компания Meta предусмотрела возможность генерирования сложных изображений путем выделения отдельных элементов изображения с применением разных масок, их изменения по отдельности и последующего объединения результатов для получения конечного продукта.

## Тонкая настройка DreamBooth

На обучение оригинальной модели Stable Diffusion было потрачено 600 000 долларов (<https://oreil.ly/s739b>) и порядка 150 000 часов времени графического процессора, поэтому для большинства организаций об обучении собственной модели с нуля не может быть и речи. Однако есть возможность создать модель, основанную на Stable Diffusion, используя технику DreamBooth, которая была представлена в статье *DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation* (Пуис (Ruiz) и др., 2022 (<https://oreil.ly/ZqdjB>)). DreamBooth позволяет точно настроить или дообучить модель, чтобы она понимала новую концепцию, отсутствующую в исходных обучающих данных. Отсутствие необходимости начинать с нуля для получения новой модели означает значительное снижение затрат времени и ресурсов: от 45 минут до часа на одном графическом процессоре. В действительности DreamBooth просто обновляет веса, давая вам новый файл модели размером 2 Гбайт для использования в AUTOMATIC1111 вместо базовой модели Stable Diffusion.

На таких сайтах, как Hugging Face (<https://oreil.ly/2ef00>) и Civitai (<https://civitai.com>), доступно множество моделей, полученных с помощью DreamBooth. Чтобы использовать эти модели в AUTOMATIC1111, достаточно загрузить их и переместить в папку `stable-diffusion-webui/models/Stable-diffusion/`. В моделях, созданных с помощью DreamBooth, часто есть конкретное слово, инициирующее применение стиля или темы, которые необходимо включить в запрос. Например, при использовании модели Inkpunk Diffusion (<https://oreil.ly/spsy3>) требуется присутствие слова *nvinkpunk*. Обратите внимание, что в качестве базовой модели здесь используется версия Stable Diffusion 1.5, поэтому измените размер изображения на 512 × 512.

Ввод:

скейтбордист на Таймс-сквер

Вывод показан на рис. 9.29.



**Рис. 9.29.** Изображение скейтбордиста на Таймс-сквер, созданное с помощью InkPunk



#### Разделите задачу на подзадачи

Многие, работающие с ИИ, часто допускают ошибку, полагая, что есть только одна модель. На самом деле существует множество генеративных моделей, и часто дообучение некоторой базовой модели какой-то конкретной задаче позволяет добиться лучших результатов. Большинство практиков первоначально используют базовые модели, такие как Stable Diffusion XL, но затем рано или поздно начинают экспериментировать с тонкой настройкой и создают собственные модели для решения специализированных задач, часто на основе более мелких и эффективных моделей, таких как v1.5.

Примеры обучения модели DreamBooth можно найти в репозитории Шивама Шрирао (Shivam Shrirao) (<https://oreil.ly/AJnnL>). В них используется библиотека `diffusers` компании HuggingFace. Далее мы объясним, как работает код из Google Colab (<https://oreil.ly/790FZ>). В этом блокноте используется версия 1.5,

потому что эта модель имеет достаточно малый размер, чтобы ее можно было бесплатно дообучить за несколько часов в Google Colab. Копия этого блокнота хранится в репозитории книги на GitHub (<https://oreil.ly/NzzGm>), но имейте в виду, что он будет работать только на графическом процессоре Nvidia и MacBook не поддерживается.

Сначала Colab проверяет доступность графического процессора Nvidia. Это одна из веских причин для запуска DreamBooth в Google Colab — вам предоставляется доступ к ресурсу, нужному для запуска кода, без необходимости какой-либо настройки:

```
!nvidia-smi --query-gpu=name,memory.total, \
memory.free --format=csv,noheader
```

Затем устанавливаются необходимые библиотеки, в том числе библиотека `diffusers` компании Hugging Face:

```
!wget -q https://github.com/ShivamShrirao/diffusers/raw/ \
main/examples/dreambooth/train_dreambooth.py
!wget -q https://github.com/ShivamShrirao/diffusers/raw/ \
main/scripts/convert_diffusers_to_original_stable_ \
diffusion.py
%pip install -qq \
git+https://github.com/ShivamShrirao/diffusers
%pip install -q -U --pre triton
%pip install -q accelerate transformers ftfy \
bitsandbytes==0.35.0 gradio natsort safetensors xformers
```

Запуск следующей ячейки настроит каталог для сохранения модели после завершения ее обучения. Мы рекомендуем сохранить модель на Google Drive (хотя бы и временно), поскольку оттуда больше шансов скачать большой файл (4–5 Гбайт), чем из файловой системы Google Colab. Не забудьте выбрать правильную базовую модель в Hugging Face `runwayml/stable-diffusion-v1-5` и укажите свой токен для выходного каталога (обычно *ukj* или *zwx*; подробнее об этом позже):

```
##@markdown Если веса модели должны сохраняться
##@markdown на Google Drive (примерно 4-5 Гбайт).
save_to_gdrive = False
if save_to_gdrive:
 from google.colab import drive
 drive.mount('/content/drive')

##@markdown Имя/путь к начальной модели.
MODEL_NAME = "runwayml/stable-diffusion-v1-5" \
 ##@param {type:"string"}
```

##@markdown Введите имя каталога, куда следует сохранить модель.

```

OUTPUT_DIR = "stable_diffusion_weights/ukj" \
 #@param {type:"string"}
if save_to_gdrive:
 OUTPUT_DIR = "/content/drive/MyDrive/" + OUTPUT_DIR
else:
 OUTPUT_DIR = "/content/" + OUTPUT_DIR

print(f"[*] Веса будут сохранены в {OUTPUT_DIR}")

!mkdir -p $OUTPUT_DIR

```

Перед обучением необходимо добавить понятия, которым вы хотите обучить модель. По нашему опыту, обучение нескольким понятиям сразу вредит качеству, поэтому обычно мы обучаем модель чему-то одному. Позднее вы сможете объединить модели на вкладке **Checkpoint Merger** (Слияние контрольных точек) в AUTOMATIC1111, но это уже более сложная область, и она не рассматривается в книге. Экземпляр запроса должен включать токен, который предполагается использовать в запросах для запуска модели, и в идеале это должно быть слово, не имеющее никакого значения, например *zex* или *ukj*. Класс запроса является отправной точкой для обучения, поэтому если вы обучаете модель распознаванию конкретного человека, то начните с его фотографии, это сделает обучение более эффективным:

# Здесь можно добавить несколько понятий.  
 # В этом случае попробуйте настроить `--max\_train\_steps`.

```

concepts_list = [
 {
 "instance_prompt": "photo of ukj person",
 "class_prompt": "photo of a person",
 "instance_data_dir": "/content/data/ukj",
 "class_data_dir": "/content/data/person"
 }
]

`class_data_dir` содержит изображения регуляризации
import json
import os
for c in concepts_list:
 os.makedirs(c["instance_data_dir"], exist_ok=True)

with open("concepts_list.json", "w") as f:
 json.dump(concepts_list, f, indent=4)

```

Далее выполняется загрузка изображения через Google Colab. DreamBooth может работать только с пятью изображениями, но обычно рекомендуется использовать около 20–30 изображений, а некоторые модели дообучаются на сотнях изображений. Для преодоления этой проблемы можно использовать

прием получения разных изображений одного персонажа, описанный в главе 8, чтобы с его помощью создать 20 разных изображений одного и того же персонажа и использовать их для обучения модели DreamBooth. Кроме того, вы можете загрузить 20 своих фотографий, чтобы создать ИИ-фотографию для профиля, или 20 фотографий продукта, который продает ваша компания, чтобы создать ИИ-фотографию продукта. Вы можете загрузить файлы в локальную папку `instance_data_dir` в файловой системе Google Colab (что может быть быстрее) или запустить следующую ячейку, которая отобразит кнопку загрузки:

```
import os
from google.colab import files
import shutil

for c in concepts_list:
 print(f""""Выгрузить экземпляры изображений для
 {c['instance_prompt']}`""")
 uploaded = files.upload()
 for filename in uploaded.keys():
 dst_path = os.path.join(c['instance_data_dir'],
 filename)
 shutil.move(filename, dst_path)
```

После этого начинается фактическое обучение! Следующий код выполняется на графическом процессоре и по завершении выводит окончательные веса. Обязательно измените `save_sample_prompt` перед запуском, чтобы использовать назначенный вами токен, в данном случае `photo of ukj person`:

```
!python3 train_dreambooth.py \
 --pretrained_model_name_or_path=$MODEL_NAME \
 --pretrained_vae_name_or_path="stabilityai/sd-vae-ft-mse" \
 --output_dir=$OUTPUT_DIR \
 --revision="fp16" \
 --with_prior_preservation --prior_loss_weight=1.0 \
 --seed=1337 \
 --resolution=512 \
 --train_batch_size=1 \
 --train_text_encoder \
 --mixed_precision="fp16" \
 --use_8bit_adam \
 --gradient_accumulation_steps=1 \
 --learning_rate=1e-6 \
 --lr_scheduler="constant" \
 --lr_warmup_steps=0 \
 --num_class_images=50 \
 --sample_batch_size=4 \
 --max_train_steps=800 \
 --save_interval=10000 \
 --save_sample_prompt="photo of ukj person" \
 --concepts_list="concepts_list.json"
```

Следующие две ячейки кода определяют каталог и выводят сетку изображений, чтобы дать возможность визуально оценить, правильно ли модель поняла вашу концепцию и способна ли теперь генерировать полезные изображения в заданном вами стиле:

```
WEIGHTS_DIR = ""
if WEIGHTS_DIR == "":
 from natsort import natsorted
 from glob import glob
 import os
 WEIGHTS_DIR = natsorted(glob(OUTPUT_DIR + os.sep + \
 "**"))[-1]
print(f"[*] WEIGHTS_DIR={WEIGHTS_DIR}")

#@markdown Запустите, чтобы сгенерировать сетку изображений по последним весам.
import os
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

weights_folder = OUTPUT_DIR
folders = sorted([f for f in os.listdir(weights_folder) \
 if f != "0"], key=lambda x: int(x))

row = len(folders)
col = len(os.listdir(os.path.join(weights_folder,
 folders[0], "samples")))
scale = 4
fig, axes = plt.subplots(row, col, figsize=(col*scale,
 row*scale), gridspec_kw={'hspace': 0, 'wspace': 0})

for i, folder in enumerate(folders):
 folder_path = os.path.join(weights_folder, folder)
 image_folder = os.path.join(folder_path, "samples")
 images = [f for f in os.listdir(image_folder)]
 for j, image in enumerate(images):
 if row == 1:
 currAxes = axes[j]
 else:
 currAxes = axes[i, j]
 if i == 0:
 currAxes.set_title(f"Image {j}")
 if j == 0:
 currAxes.text(-0.1, 0.5, folder, rotation=0,
 va='center', ha='center',
 transform=currAxes.transAxes)
 image_path = os.path.join(image_folder, image)
 img = mpimg.imread(image_path)
 currAxes.imshow(img, cmap='gray')
 currAxes.axis('off')

plt.tight_layout()
plt.savefig('grid.png', dpi=72)
```

Наконец, запустите процесс преобразования, чтобы получить файл .ckpt для использования в AUTOMATIC1111:

```
#@markdown Запустите преобразование.
ckpt_path = WEIGHTS_DIR + "/model.ckpt"

half_arg = ""
#@markdown Преобразование в fp16 уменьшит объем модели вдвое (до 2Гбайт).
fp16 = True #@param {type: "boolean"}
if fp16:
 half_arg = "--half"
!python convert_diffusers_to_original_stable_diffusion.py \
 --model_path $WEIGHTS_DIR --checkpoint_path \
 $ckpt_path $half_arg
print(f"[*] Converted ckpt saved at {ckpt_path}")
```

Затем можете перейти в каталог весов `stable_diffusion_weights/ukj`, чтобы найти и загрузить модель. Если у вас возникнут проблемы с загрузкой такого большого файла из файловой системы Google Colab, попробуйте вариант с сохранением модели на Google Drive и загрузите ее оттуда. Мы рекомендуем переименовать модель перед тем, как поместить ее в папку `stable-diffusion-webui/models/Stable-diffusion/`, чтобы вы могли узнать ее в дальнейшем.

Ввод:

профессиональный снимок человека ukj, стоящего  
со скрещенными руками и улыбающегося в камеру,  
портрет человека, Адам Брюс Томсон, частное  
фото, профессиональное фото

Вывод показан на рис. 9.30.



**Рис. 9.30.** Изображение одного из авторов, созданное моделью DreamBooth



Существует также расширение (<https://oreil.ly/xbt2d>) для обучения моделей DreamBooth через Automatic1111, основанное на методе Шивама Шрирао (Shivam Shrirao). Это расширение можно установить так же, как ControlNet и SAM выше в этой главе. Этот инструмент предназначен для опытных пользователей, потому что предоставляет значительное количество функций и настроек для экспериментов, требующих наличия серьезных знаний в области машинного обучения. Начать изучать эти функции и настройки можно с прочтения руководства для начинающих (<https://oreil.ly/gfdY3>). Преимущество этого метода перед Google Colab в том, что он работает на локальном компьютере и его можно оставить запущенным, не беспокоясь, что из-за сбоя в сети вы потеряете созданное.



### Приведите примеры

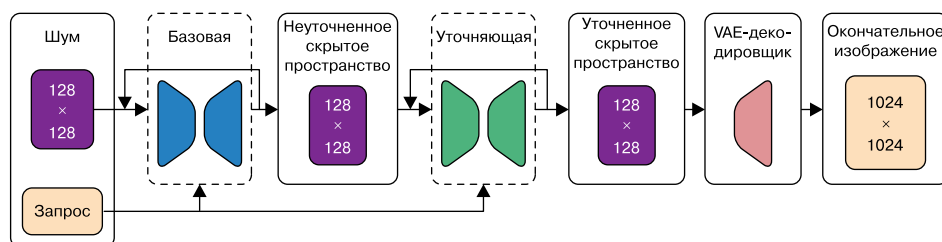
DreamBooth поможет вам персонализировать опыт работы с генеративным ИИ. Вам нужно лишь предоставить 5–30 изображений, которые послужат примерами концепции, и менее чем за час обучения вы сможете получить полностью персонализированную модель.

Помимо DreamBooth, есть и другие методы дообучения и тонкой настройки, но в настоящее время этот метод используется чаще других. Более старый метод — *текстовая инверсия* (<https://oreil.ly/GgnJV>), который не изменяет веса модели, а аппроксимирует правильное местоположение токена, представляющего вашу концепцию. Однако этот метод обычно работает намного хуже, чем DreamBooth. Один из новых многообещающих методов — LoRA, описанный в статье *LoRA: Low-Rank Adaptation of Large Language Models* (Хью (Hu) и др., 2021 (<https://oreil.ly/NtoiB>)), который также широко используется в сфере генерирования текста с применением LLM. Этот метод добавляет в модель новые слои и обучает только их, благодаря чему для обучения достаточно довольно скромных ресурсов. Существует также метод Hypernetworks, который обучает гиперпараметры, способные затем генерировать новые слои. Этот метод описан в статье Курумуза (Kurumuz) (<https://oreil.ly/zFH0->) *NovelAI Improvements on Stable Diffusion* на Medium. Оба метода считаются экспериментальными, и на момент написания книги на Civitai имелось лишь небольшое количество моделей (менее 10 %), созданных с их помощью. А кроме того, пользователи довольно низко оценили их качество.

## Уточняющая модель в Stable Diffusion XL

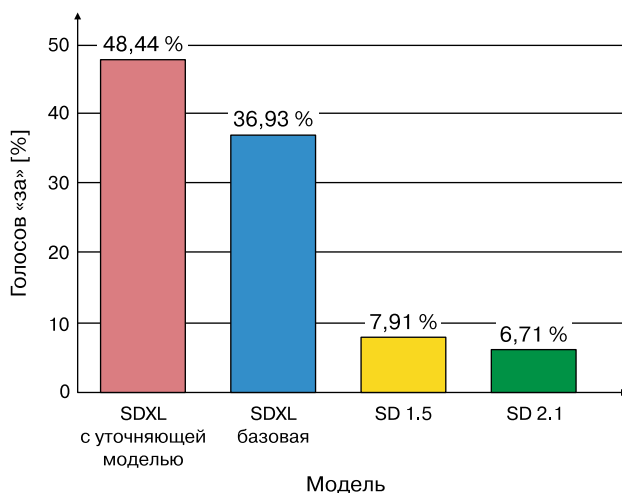
Модель SDXL v1.0 имеет 6,6 миллиарда параметров по сравнению с 0,98 миллиарда в модели v1.5 (Ромбах (Rombach) и др., 2023 (<https://oreil.ly/vc1zS>)). Увеличенная мощность модели дает впечатляющие результаты, и поэтому она начинает завоевывать расположение даже самых стойких приверженцев версии 1.5.

Расширение возможностей SDXL отчасти обусловлено «разделением труда» между базовой моделью, определяющей основную композицию, и уточняющей (рис. 9.31), которая добавляет более мелкие детали (если необходимо).



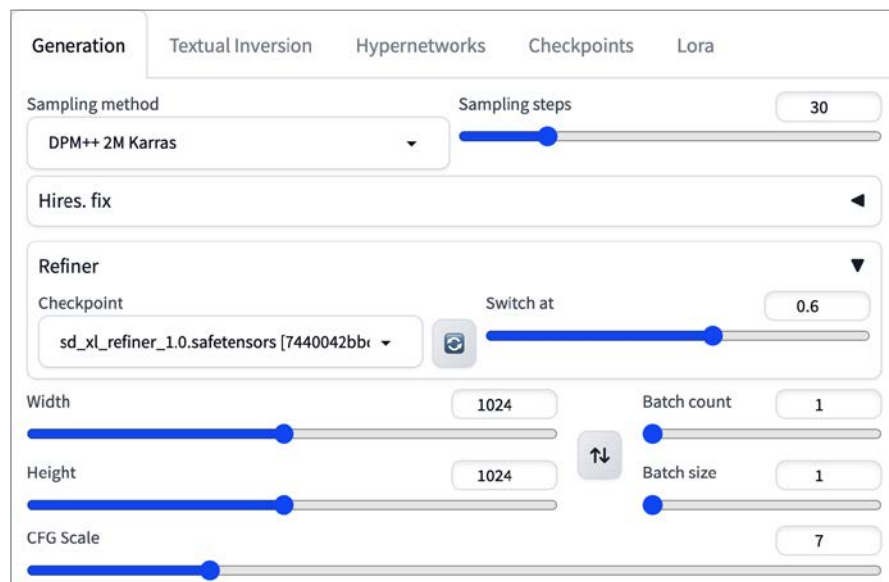
**Рис. 9.31.** Базовая и уточняющая модели Stable Diffusion XL

Базовая языковая модель, которая определяет смысл запросов, реализована как комбинация моделей OpenClip (ViT-G/14) и CLIP ViT-L, созданных в OpenAI. В Stable Diffusion v2 использовалась только OpenClip, поэтому запросы, которые работали в версии 1.5, не всегда были переносимыми: эта проблема в значительной степени решена в SDXL. Кроме того, модель SDXL обучена на наборе изображений с большим диапазоном размеров, что помогает получить лучшие результаты, когда нужно изображение с нестандартным соотношением сторон, отличным от квадратного. Исследования, проведенные в Stability AI ([https://oreil.ly/\\_b7xX](https://oreil.ly/_b7xX)), показали, что пользователи в подавляющем большинстве случаев отдают предпочтение модели XL перед версией 1.5 (рис. 9.32).



**Рис. 9.32.** Предпочтения, отдаваемые разным моделям

Чтобы задействовать уточняющую модель, необходимо использовать параметр **Switch at** (Переключить на) в интерфейсе AUTOMATIC1111. Его значение определяет, на каком этапе конвейер переключится на уточняющую модель. Например, значение 0,6 с 30 шагами обработки означает, что базовая модель будет использоваться на первых 18 шагах, а затем на последних 12 шагах подключится уточняющая модель (рис. 9.33).



**Рис. 9.33.** Параметр **Switch at** (Переключить на)

Общий совет — наилучшие результаты получаются со значением параметра **Switch at** (Переключить на) между 0,4 и 1,0 (значение 1,0 соответствует использованию только базовой модели, без переключения на уточняющую) и 20–50 шагами обработки. Как показывает наш опыт, переключение на 0,6 с 30 шагами обработки дает изображения достаточно высокого качества, но, как и со всем, что касается Stable Diffusion, вам придется поэкспериментировать, чтобы выяснить, при каких настройках получаются наилучшие изображения в вашем случае. Переключение на уточняющую модель на уровне 0,6 дает результат, показанный на рис. 9.34.

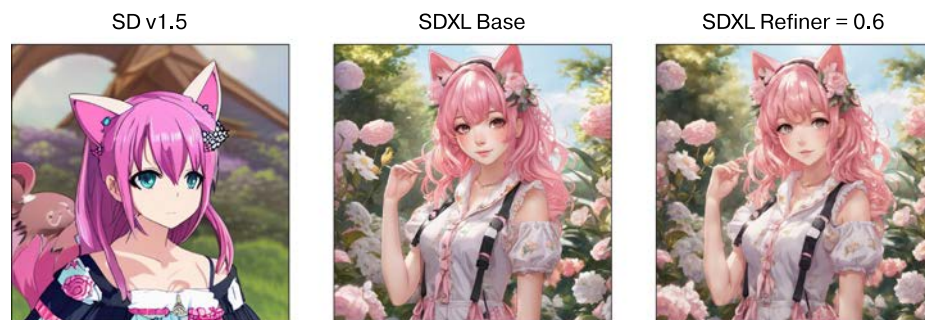
Ввод:

Девушка-кошка из аниме с розовыми волосами и кошачьими ушками позирует перед фотокамерой, фотореалистично, 1 девушка, портрет, цветочный узор, Алиса Прин, соц-арт, официальное искусство, солнечный свет, волнистые волосы, взгляд на зрителя

Отрицательный:

изуродованная, уродливая, некрасивая, незрелая, фото,  
для взрослых, переэкспонированная, недоэкспонированная

Вывод показан на рис. 9.34.



**Рис. 9.34.** Девушка-кошка из аниме, базовая модель SDXL и уточненная модель с переключением на уровне 0,6



### Разделите задачу на подзадачи

Архитектура SDXL — прекрасный пример разделения задачи на несколько подзадач и использования наиболее подходящей модели для каждой подзадачи. Базовая модель создает сцену и определяет композицию изображения, а уточняющая прорисовывает мелкие детали.

Одно из качественных изменений, которое вы можете добавить, — установить расширение селектора размеров и соотношений сторон загружаемых изображений, которое позволяет настроить свои часто используемые размеры и соотношения сторон и выбрать их одним щелчком мыши.

Чтобы установить расширение, перейдите на вкладку **Extensions** (Расширения), выберите **Install from URL** (Установить из URL), введите адрес <https://github.com/alemelis/sd-webui-ar> и нажмите кнопку **Install** (Установить). Затем перейдите в папку с расширением `stable-diffusion-webui/extensions/sd-webui-ar` и добавьте в файл `resolutions.txt` следующий текст (или просто замените имеющееся в нем содержимое):

```
SD1:1, 512, 512 # 1:1 квадрат
XL1:1, 1024, 1024 # 1:1 квадрат
SD3:2, 768, 512 # 3:2 ландшафт
XL3:2, 1216, 832 # 3:2 ландшафт
SD9:16, 403, 716 # 9:16 портрет
XL9:16, 768, 1344 # 9:16 портрет
```

Нажатие любой из этих кнопок автоматически установит соответствующие ширину и высоту. Вы также можете заменить содержимое файла `aspect ratios.txt`, как показано ниже, что позволит автоматически вычислять соотношение сторон на основе значения высоты, установленного в веб-интерфейсе, и соответствующие кнопки будут отображаться в веб-интерфейсе (рис. 9.35):

Square 1:1, 1.0 # Отношение 1:1 по наименьшей стороне

Ландшафт 3:2, 3/2 # Установить ширину, исходя из отношения 3:2 к высоте

Portrait 9:16, 9/16 # Установить ширину, исходя из отношения 9:16 к высоте

The screenshot shows a web interface for Stable Diffusion. It features several sliders and input fields: Width (512), Height (512), CFG Scale (7), and a Seed input field (-1). There are also buttons for Batch count (1), Batch size (1), and an Extra checkbox. At the bottom, there are buttons for different aspect ratios: Square 1:1, Landscape 3:2, and Portrait 9:16, each with sub-buttons for Calc, SD1:1, XL1:1, SD3:2, XL3:2, SD9:16, and XL9:16.

Рис. 9.35. Отношения сторон

## Резюме

В этой главе вы познакомились с продвинутыми приемами создания изображений с использованием Stable Diffusion — модели с открытым исходным кодом. Если вы следовали нашим инструкциям, то, значит, вы благополучно установили Stable Diffusion и построили пайплайн обработки с использованием библиотеки `diffusers`, созданной компанией HuggingFace. Надеемся, вы попробовали создавать изображения с помощью моделей Stable Diffusion в Google Colab. В этой главе также рекомендовалось исследовать достижения сообщества и созданные им пользовательские интерфейсы, такие как AUTOMATIC1111, позволяющие запускать Stable Diffusion и предлагающие дополнительные возможности.

Здесь также были представлены модель ControlNet, позволяющая управлять стилем изображения с помощью запросов и базовых изображений, и модель Segment Anything, осуществляющая маскировку определенных частей изображения.

Применяя эти модели, вы теперь можете настраивать созданные изображения в соответствии с конкретными потребностями. Вы также познакомились с методами персонализации, в частности с тонкой настройкой DreamBooth, позволяющей обучить модель новым концепциям, отсутствующим в ее обучающих данных.

В следующей главе у вас появится возможность применить на практике знания, полученные в этой книге. Мы разберем процесс создания генератора сообщений для блога с ИИ, который способен генерировать как текстовые сообщения, так и сопутствующие изображения. В заключительной главе рассмотрим все этапы разработки комплексной системы, которая на основе ввода пользователя создает высококачественные сообщения, дополненные иллюстрациями в едином визуальном стиле. Вы узнаете, как оптимизировать запросы для генерации контента, создавать привлекающие внимание заголовки и с помощью ИИ генерировать изображения, соответствующие желаемому стилю.

# Создание приложений на базе ИИ

В этой главе вы примените пять принципов промт-инженерии при написании запросов к ИИ. Ваш сервис будет выводить сообщения в блоге на основе ответов пользователя на вопросы. Эта система была впервые описана в блоге Saxifrage (<https://oreil.ly/saxifrage>).

## Разработка блога на основе ИИ

Простейший подход к созданию сервиса, генерирующего сообщения для блога с использованием ИИ, заключается в том, чтобы предложить ChatGPT написать сообщение на тему `{blogPostTopic}`. Получающиеся сообщения будут иметь приемлемое качество, но едва ли будут содержать какие-либо ценные мнения или уникальный опыт по заданной теме. Кроме того, сообщения, скорее всего, будут короткими и чересчур общими и, следовательно, вряд ли будут ранжироваться в Google.

Более сложный подход заключается в создании более длинного запроса с дополнительными инструкциями. Можно подробно описать стиль письма, архитектуру сообщения и ключевые слова. Ниже приводится пример обычного запроса с просьбой написать сообщение в блоге (<https://oreil.ly/uMfZa>).

Ввод:

Напиши сообщение для блога на тему "`{blogPostTopic}`". Напиши его в стиле "`{tone}`".

Используй переходные слова.

Используйте активный залог. Напиши больше 1000 слов.

Используй творческие заголовки для сообщения.

Добавь заголовок для каждого раздела. Разделов должно быть не меньше 9.

Каждый раздел должен состоять минимум из двух абзацев.

Включи следующие ключевые слова: "`{keywords}`".

Создай хорошее краткое резюме этого сообщения, содержащее не более 100 слов, и добавь его в конец сообщения.

Этот более длинный и сложный запрос, скорее всего, приведет к созданию более качественного сообщения. Однако рассмотрим его через призму пяти принципов промт-инжиниринга.

### *Направление*

Предусмотрены некоторые инструкции, такие как стиль, использование переходных слов и активный залог. Однако, скорее всего, сообщение по-прежнему будет выглядеть несколько искусственно, не как написанное человеком.

### *Формат*

В запросе присутствуют некоторые упоминания о структуре, в том числе о девяти разделах по два абзаца в каждом, но, вероятнее всего, эти инструкции будут проигнорированы. ChatGPT плохо разбирается в математике и часто игнорирует инструкции, определяющие количество разделов или слов.

### *Примеры*

В запросе нет примеров, показывающих, как выполнить данную задачу, что может снизить надежность этого запроса при применении к нескольким темам или даже несколько раз к одной и той же теме. Предоставление даже одного примера может значительно повысить качество результата.

### *Оценка*

Это пример *слепого запроса* (инструкции в запросе не проверяются (<https://oreil.ly/r7sXi>)). Вполне вероятно, что некоторые из этих инструкций никак не повлияют на качество (и лишь впустую потратят ценные токены), а некоторые могут даже ухудшить качество.

### *Разделение*

Вся задача решается с помощью единственного запроса, что может снизить качество решения. Не разбив задачу на подзадачи, трудно понять, какая часть процесса успешна, а какая нет.

В этой главе мы создадим несколько цепочек компонентов LLM. Каждая цепочка будет реализована в LangChain, чтобы упростить ее обслуживание и журналирование с целью мониторинга и оптимизации. Полученная система поможет вам создавать сообщения, *выглядящие как написанные человеком* и основанные на уникальных мнениях и опыте пользователя.

Крайне важно предварительно подготовить рабочее место с необходимыми инструментами. Поэтому давайте переключим внимание на исследуемую тему и приступим к настройке программного окружения.



## Исследуемая тема

Для эффективного использования загрузчиков документов LangChain нам потребуется установить несколько пакетов Python.

### `google-searchresults`

Библиотека для Python, предназначенная для получения и обработки результатов поиска в Google.

### `pandas`

Реализует структуры данных и операции для управления числовыми таблицами и данными временных рядов.

### `html2text`

Этот инструмент преобразует разметку HTML в текст или в разметку Markdown (.md).

### `pytest-playwright`

Этот пакет обеспечивает сквозное тестирование с помощью Playwright.

### `chromadb`

ChromaDB — векторная база данных с открытым исходным кодом.

### `nest_asyncio`

Расширяет стандартный пакет `asyncio` для Python, делая его совместимым с Jupyter Notebooks.

Установить эти пакеты можно с помощью следующей команды:

```
pip install google-searchresults pandas html2text pytest-playwright chromadb \
nest_asyncio --quiet
```

Мы также будем использовать загрузчики документов LangChain, которым требуется пакет Playwright.

Введите эту команду на своем терминале: `pip install playwright`.

Кроме того, вам нужно будет выбрать тему TOPIC и установить переменные окружения SERPAPI\_API\_KEY и STABILITY\_API\_KEY. Если предполагается запускать скрипт вне Jupyter Notebook, то `nest_asyncio` вам не понадобится:

```
from langchain_openai.chat_models import ChatOpenAI
from langchain.output_parsers import PydanticOutputParser
from langchain_text_splitters import RecursiveCharacterTextSplitter
import os
```

```
Импортировать дополнительные инструменты:
from content_collection import collect_serp_data_and_extract_text_from_webpages
from custom_summarize_chain import create_all_summaries, DocumentSummary

import nest_asyncio
nest_asyncio.apply()

Константы:
TOPIC = "Neural networks"
os.environ["SERPAPI_API_KEY"] = ""
os.environ["STABILITY_API_KEY"] = ""
```

Теперь сосредоточимся на эффективном обобщении веб-контента:

```
Извлечь содержимое веб-страниц из документации LangChain:
text_documents = await \
 collect_serp_data_and_extract_text_from_webpages(TOPIC)

LLM, разбиение текста + парсинг:
llm = ChatOpenAI(temperature=0)
text_splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
 chunk_size=1500, chunk_overlap=400
)
parser = PydanticOutputParser(pydantic_object=DocumentSummary)

summaries = await create_all_summaries(text_documents,
 parser,
 llm,
 text_splitter)
```

Этот код сначала импортирует необходимые инструменты, а затем извлекает содержимое веб-страницы, относящееся к нашей теме TOPIC. После настройки модели ChatOpenAI мы будем использовать `text_splitter`, чтобы разбить текст на фрагменты не слишком длинные и перекрывающиеся для сохранения контекста. Затем создается экземпляр `PydanticOutputParser` для обработки и структурирования сводок. Подавая извлеченные документы на вход специальной функции обобщения, LLM создает краткие резюме.

Желающие глубже изучить функцию `create_all_summaries` могут заглянуть в `custom_summarize_chain.py` (<https://oreil.ly/KyKjS>).

Особо следует подчеркнуть возможность создания подклассов на основе большинства классов в LangChain. Например, можно переопределить `ChromiumLoader` и сделать его асинхронным:

```
from langchain_community.document_loaders import AsyncHtmlLoader, \
 AsyncChromiumLoader

class ChromiumLoader(AsyncChromiumLoader):
 async def load(self):
 raw_text = [await self.ascrrape_playwright(url) for url in self.urls]
```

```

 # Вернуть исходные документы:
 return [Document(page_content=text) for text in raw_text]

async def get_html_content_from_urls(
 df: pd.DataFrame, number_of_urls: int = 3, url_column: str = "link"
) -> List[Document]:
 # Получить из разметки HTML три первых URL:
 urls = df[url_column].values[:number_of_urls].tolist()

 # Если найден только один URL, преобразовать его в список:
 if isinstance(urls, str):
 urls = [urls]

 # Проверить пустые URL:
 urls = [url for url in urls if url != ""]

 # Проверить повторяющиеся URL:
 urls = list(set(urls))

 # Сообщить об ошибке, если URL не найдены:
 if len(urls) == 0:
 raise ValueError("No URLs found!")
 # loader = AsyncHtmlLoader(urls) # Быстрее, но работает не всегда.
 loader = ChromiumLoader(urls)
 docs = await loader.load()
 return docs

async def create_all_summaries(
 # ... опущено для краткости
) -> List[DocumentSummary]:
 # ... опущено для краткости

```

Добавив подкласс `ChromiumLoader`, можно создать свою реализацию для *асинхронного извлечения контента* из нескольких URL с помощью браузера Chrome. `get_html_content_from_urls` извлекает разметку HTML из нескольких URL, предварительно удалив из списка пустые и повторяющиеся адреса и обрабатывая возможные ошибки.

## Интервью с экспертом

Теперь, успешно получив резюме для трех лучших результатов из Google, проведем интервью с LLM, задав соответствующие вопросы, чтобы убедиться, что наша статья имеет уникальную точку зрения. Используем для этого класс `InterviewChain`:

```

from expert_interview_chain import InterviewChain
interview_chain = InterviewChain(topic=TOPIC, document_summaries=summaries)
interview_questions = interview_chain()

```

```
for question in interview_questions.questions:
 print(f"Ответьте на следующий вопрос: {question.question}\n", flush=True)
 answer = input(f"Ответьте на следующий вопрос: {question.question}\n")
 print('-----')
 question.answer = answer
```

### Создание экземпляра *InterviewChain*

Имея тему и получив резюме, создадим экземпляр *InterviewChain*, адаптировав его к уникальному контексту наших данных.

### Генерирование вопросов

Простым вызовом *interview\_chain* мы запускаем процесс генерирования серии наводящих вопросов, полученных из наших резюме.

### Интерактивный сеанс вопросов и ответов

Далее мы погружаемся в увлекательный цикл, в ходе которого с помощью *input()* выводятся производные вопросы и предлагается ответить на них. Полученные ответы сохраняются в объекте *Pydantic*.



### Задайте направление

Ввод уникальных ответов дает цепочке LLM уникальный контекст, позволяет ей генерировать более насыщенные и детальные ответы и гарантирует, что ваша статья будет раскрывать свежий и глубокий взгляд на обсуждаемый предмет.

Вся реализация *InterviewChain* находится в файле *expert\_interview\_chain.py* (<https://oreil.ly/0d5Hi>). Он состоит из двух важных компонентов.

### Сообщение *System*

Этот запрос включает подсказку о роли, ранее созданные резюме, тему и инструкции по форматированию (для парсера вывода):

```
system_message = """Ты - опытный специалист по SEO контента. Ранее ты обобщил и извлек ключевые моменты из результатов поиска. Полученное понимание будет использоваться для исследования контента, и мы сравним ключевые моменты, идеи и резюме из нескольких статей. Ты сейчас собираешься взять интервью у эксперта по контенту. Задай ему вопросы по следующей теме: {тема}.
```

Соблюдай следующие правила:

- Верни список вопросов по теме, которые ты хотел бы задать эксперту по контенту.
- Задай не меньше и не больше 5 вопросов.
- Ищи информацию и уникальные идеи, не упомянутые в

```
резюме {document_summaries}.
- Задавай вопросы, предполагающие развернутые ответы, а не просто "да/нет".
{format_instructions}

"""
```

### Парсеры вывода

Глубже в классе встречается `PydanticOutputParser` — парсер (синтаксический анализатор), который активно структурирует ответы LLM в объекты `InterviewQuestions` из библиотеки `Pydantic`:

```
from expert_interview_chain import InterviewQuestions

Настроить парсер + вставить инструкции в шаблон запроса:
parser = PydanticOutputParser(pydantic_object=InterviewQuestions)
```

По сути, мы организуем диалог с ИИ и просим его придумать эффективные вопросы, которые помогут расширить понимание контента.

## Генерирование плана публикации

На основе предыдущего собеседования и исследования можно с помощью класса `BlogOutlineGenerator` сгенерировать план публикации. Для дополнительного контекста ему передаются `TOPIC`, `question_answers` и `summaries`:

```
from article_outline_generation import BlogOutlineGenerator

blog_outline_generator = BlogOutlineGenerator(topic=TOPIC,
questions_and_answers=[item.dict() for item in
 interview_questions.questions])

questions_and_answers = blog_outline_generator.questions_and_answers
outline_result = blog_outline_generator.generate_outline(summaries)
```

Давайте подробно рассмотрим реализацию класса `BlogOutlineGenerator`:

```
from typing import List, Any
from pydantic.v1 import BaseModel

class SubHeading(BaseModel):
 title: str # Любой подзаголовок должен иметь название.

class BlogOutline(BaseModel):
 title: str
 sub_headings: List[SubHeading] # План содержит несколько подзаголовков
```

---

```

Объекты из библиотеки Langchain:
from langchain.prompts.chat import (ChatPromptTemplate,
SystemMessagePromptTemplate)
from langchain.output_parsers import PydanticOutputParser
from langchain_openai.chat_models import ChatOpenAI

Вспомогательные типы:
from custom_summarize_chain import DocumentSummary

class BlogOutlineGenerator:
 def __init__(self, topic: str, questions_and_answers: Any):
 self.topic = topic
 self.questions_and_answers = questions_and_answers

 # Создать запрос
 prompt_content = """
На основе моих ответов и резюме сгенерируй план статьи для блога
по теме {topic}.
тема: {topic}
резюме: {document_summaries}

Вот вопросы и мои ответы на них:
{interview_questions_and_answers}

Формат вывода: {format_instructions}
"""

 system_message_prompt =
SystemMessagePromptTemplate.from_template(prompt_content)

 self.chat_prompt = ChatPromptTemplate.from_messages(
 [system_message_prompt])

 # Создать парсер вывода
 self.parser = PydanticOutputParser(pydantic_object=BlogOutline)

 # Создать цепочку
 self.outline_chain = self.chat_prompt | ChatOpenAI() | self.parser

 def generate_outline(self, summaries: List[DocumentSummary]) -> Any:
 print("Генерирование плана...\n---")
 result = self.outline_chain.invoke(
 {"topic": self.topic,
 "document_summaries": [s.dict() for s in summaries],
 "interview_questions_and_answers": self.questions_and_answers,
 "format_instructions": self.parser.get_format_instructions(),
 }
)
 print("Генерирование плана завершено!\n---")
 return result

```

Этот класс создает объект `BlogOutlinePydantic` с атрибутами `title` и `sub_headings` и цепочку на языке выражений `LangChain (LCEL)`, которая генерирует план, передавая запрос в модель чата, а полученный ответ — в парсер вывода:

```
Создать цепочку
self.outline_chain = self.chat_prompt | ChatOpenAI() | self.parser
```

Парсер вывода `Pydantic` вернет объект `BlogOutlinePydantic`, который мы используем в других цепочках.

## Генерирование текста

После получения резюме, вопросов для собеседования, ответов на них и плана публикации в блоге можно приступить к генерированию текста. Класс `ContentGenerator` объединяет имеющийся опыт SEO, используя несколько методов LLM.

### *Преобразование в векторное представление и извлечение*

Разбивает и векторизует исходные веб-страницы, сохраняя их в базе данных `Chroma`, а затем извлекает соответствующий текст веб-страницы при генерировании каждого раздела.

### *Дополнительная память*

При создании каждого раздела используется дополнительная память, чтобы избежать повторения одной и той же информации, а также подводится итог диалога, если он становится слишком длинным.

### *Индивидуальный контекст*

LLM содержит разнообразную информацию, в том числе предыдущие вопросы и ответы, все, что было сказано ранее, а также фрагменты текста веб-страницы из Google:

```
from article_generation import ContentGenerator

content_gen = ContentGenerator(
 topic=TOPIC, outline=outline_result,
 questions_and_answers=questions_and_answers)

Векторизовать и сохранить исходные веб-страницы:
content_gen.split_and_vectorize_documents(text_documents)

Сгенерировать статью для блога:
blog_post = content_gen.generate_blog_post()
```

Весь исходный код находится в файле `article_generation.py` (<https://oreil.ly/0IFyI>), но давайте сосредоточимся на трех ключевых компонентах в этой цепочке.

Класс `OnlyStoreAIMemory` — производный от класса `ConversationSummaryBufferMemory`:

```
from typing import List, Dict, Any
from langchain.memory import ConversationSummaryBufferMemory

from langchain_core.messages import SystemMessage

class OnlyStoreAIMemory(ConversationSummaryBufferMemory):
 def save_context(self, inputs: Dict[str, Any],
 outputs: Dict[str, str]) -> None:
 input_str, output_str = self._get_input_output(inputs, outputs)
 self.chat_memory.add_ai_message(output_str)
```

Его цель — обеспечить лаконичность и актуальность памяти сообщений чата, сохраняя в ней *только сообщения, сгенерированные искусственным интеллектом*.

Этот осознанный отказ от хранения полученных документов, которые используются на этапе генерирования, предотвращает разбухание памяти. Более того, механизм памяти гарантирует, что ИИ сохранит информацию о своих предыдущих ответах, что позволит ему предлагать сжатые резюме, если накопленный контекст превысит установленные ограничения.

Функция `generate_blog_post` обходит список подзаголовков и пытается получить как можно больше соответствующих документов с учетом текущей длины контекста:

```
def generate_blog_post(self) -> List[str]:
 blog_post = []
 print("Генерируется статья для блога...\n---")
 for subheading in self.outline.sub_headings:
 k = 5 # Инициализировать значение k
 while k >= 0:
 try:
 relevant_documents = (self.chroma_db.as_retriever() \
 .invoke(subheading.title, k=k))
 section_prompt = f"""
 ...запрос опущен для краткости...
 Текст раздела:
 """
 result = self.blog_post_chain.predict(section_prompt)
 blog_post.append(result)
 break
 except Exception as e:
 print(f"Возникла ошибка: {e}")
 k -= 1
```



```
if k < 0:
 print('Попытки получить релевантные документы
 потерпели неудачу. Для relevant_documents
 использована пустая строка.
 ')
 relevant_documents = ""
print("Генерирование статьи для блока завершено!\n---")
return blog_post
```

В процессе обхода она извлекает до пяти релевантных документов для каждого подзаголовка. Если получить документы не получается, она разумно уменьшает их количество и повторяет попытку. Если все попытки терпят неудачу, то это означает, что документы отсутствуют.

Наконец, запрос для создания каждого раздела очень контекстно ориентированный:

```
section_prompt = f"Сейчас ты пишешь раздел: {subheading.title}

Вот релевантные документы для этого раздела: {relevant_documents}.
Если релевантные документы окажутся бесполезны, можешь их игнорировать.
Никогда не копируй релевантные документы, так как это является плагиатом.

Вот релевантные идеи, полученные из интервью с экспертом:
{self.questions_and_answers}.
Включи эти идеи там, где это возможно, потому что они важны
и помогут нашему контенту получить более высокий рейтинг.

Придерживайся следующих принципов:
- Напиши раздел: {subheading.title}
- Верни вывод в формате .md.
- Добавь соответствующее оформление, например маркированные
и нумерованные списки и т. д.

Текст раздела:
"""
```

Запрос `section_prompt` подготавливает почву, объявляя с помощью `{subheading.title}` конкретный раздел, над которым начинается работа. Но это еще не все. Передача блока релевантных документов `{relevant_documents}` дает модели LLM предысторию и глубину и одновременно явно предостерегает от плагиата. Более того, благодаря включению информации из интервью через `{self.questions_and_answers}` запрос гарантирует фокусировку внимания на ценной информации. Наконец, запрос четко определяет ожидания в отношении формата, включения определенных особенностей и рассматриваемой темы. Это делает LLM не просто инструментом, а информированным соавтором, усердно работающим вместе с вами над созданием статьи.

## Стиль письма

Теперь, получив готовую статью, мы можем сделать еще шаг и придать тексту уникальное человеческое звучание, переписав его в определенном стиле. Это может сделать текст более человеческим, замаскировать его искусственное происхождение (хотя с этической точки зрения вы все равно должны сообщать о любой помощи ИИ) и сократить время, затрачиваемое на редактирование окончательного варианта перед публикацией.

Прежде чем править текст, нужно решить, какой стиль письма использовать: свой или чужой. Один из распространенных подходов — попросить ChatGPT обобщить стиль письма кого-то известного или, по крайней мере, достаточно популярного в вашей отрасли, тексты которого с большой вероятностью попали в данные, на которых обучалась ChatGPT. Обычно в таких случаях модель возвращает имя автора и примеры его стиля, поэтому добавление инструкций не делать этого и завершение запроса маркером (символом дефиса (-) в данном случае) даст вам нужный формат.

Ввод:

Подробно опиши стиль письма Гарри Драя с сайта MarketingExamples.com.  
Не упоминай автора или источник, а просто перечисли особенности  
в виде списка:  
-

Вывод:

- Простой и ясный язык, понятный широкой аудитории
- Краткие предложения и абзацы для легкости восприятия
- Конкретные примеры и случаи из практики для иллюстрации идей
- Практические советы и рекомендации, которые можно сразу применить
- Использование вопросов для вовлечения читателя
- Неформальный и разговорный тон, создающий близость
- Элементы юмора и иронии для поддержания интереса
- Визуальные элементы, такие как изображения и графики, для улучшения понимания
- Четкая структура с акцентом на ключевые моменты
- Избегание сложной терминологии и академического стиля

Часто, получив всего один образец текста, ChatGPT может сгенерировать разумное руководство по стилю письма, которое затем можно использовать для правки текста. Определив стиль письма, можно переупорядочить и объединить его элементы, чтобы получить более идеальный стиль. В следующем примере используются элементы стиля, свойственного Майку Тейлору (Mike Taylor) и Гарри Драю (Harry Dry) из предыдущего примера. Это еще один пример выделения мемов, описанного в разделе «Выделение мемов» в главе 8:

Перепиши статью в следующем стиле:

- Четкая структура с акцентом на ключевые моменты
- Краткие предложения и абзацы для легкости восприятия
- Простой и ясный язык, понятный широкой аудитории
- Конкретные примеры и случаи из практики для иллюстрации идей
- Неформальный и разговорный тон, создающий близость
- Элементы юмора и иронии для поддержания интереса
- Практические советы и рекомендации, которые можно сразу применить
- Визуальные элементы, такие как изображения и графики, для улучшения понимания

На практике эту часть процесса использования ИИ сложнее всего выполнить правильно, и она единственная, которая требует привлечения более крупной и дорогой модели GPT-4 для получения удовлетворительных результатов. Если эту часть процесса выполнить неправильно, пользователю придется приложить немало усилий, чтобы отредактировать текст и привести его к нужному стилю. Учитывая стратегическую важность этого запроса, имеет смысл провести его оптимизацию (<https://oreil.ly/H3VtJ>), опробовав несколько подходов.

В процессе оптимизации запустите один и тот же запрос несколько раз и сравните среднее качество результатов с оценочным показателем. Для примера рассмотрим результаты тестирования пяти разных подходов на основе такого оценочного показателя, как расстояние в векторном пространстве. Чем меньше оценка, тем ближе полученный ответ к эталону (тексту, переписанному вручную в правильном стиле). Были протестированы следующие запросы.

*A*

*Control* — стандартный запрос, подробно описанный в предыдущем примере.

*B*

*1-shot writing sample* — запрос с одним образцом текста после того, как мы попросили GPT-4 описать стиль письма.

*B*

*3-shot rewriting example* — запрос с тремя образцами текста и переписанной версией после того, как мы попросили GPT-4 описать стиль письма.

*I*

*3-shot writing sample* — запрос с тремя образцами текста, написанными Майком, как и в предыдущей попытке, но без переписанного вручную текста.

Мы провели эксперименты (<https://oreil.ly/vRRYO>) с тремя тестовыми примерами — меметика, строительство небоскребов и ценообразование на основе стоимости, — представленными в виде фрагментов текста, сгенерированных ChatGPT по определенной теме, например, в ответ на запрос: *объясни ценообразование на основе стоимости*. Затем вручную переписали текст в желаемом стиле, чтобы получить эталоны для сравнения. Векторные расстояния вычислялись между векторными представлениями эталонов (полученными с помощью модели `text-embedding-ada-002` компании OpenAI) и результатов, возвращаемых моделью ChatGPT, как *косинусное сходство* (метод вычисления расстояния между двумя наборами чисел), что подробно описано в документации LangChain (<https://oreil.ly/400gJ>). Результаты экспериментов представлены на рис. 10.1.

AVERAGE of embedding_distanc case					
variation	Memetics	SkyScraper Technique	Value-Based Pricing	Grand Total	+/-
A: Control	0.053	0.042	0.061	0.052	0.00%
B: 1-Shot Writing Sample	0.045	0.034	0.048	0.043	-17.75%
C: 3-Shot Rewriting Example	0.049	0.046	0.058	0.051	-1.71%
D: 3-Shot Writing Sample	0.044	0.037	0.052	0.045	-14.03%
Grand Total	0.048	0.040	0.055	0.047	

**Рис. 10.1.** Результаты тестирования оптимизации запроса

Как можно заметить по результатам на рис. 10.1, одни запросы дают результаты лучше, чем другие, и некоторые темы оказываются проще для ИИ. Очень важно проводить тестирование с несколькими запросами и запускать каждый по десять или более раз, чтобы получить реалистичные результаты. В противном случае недетерминированный характер ответов может привести к неверной оценке их качества. Вот окончательный запрос, который показал наилучшие результаты:

Тебе будет предоставлен образец текста.

Твоя задача — переписать текст в другом стиле.

Стиль письма можно охарактеризовать следующим образом:

1. Информативно-аналитический. Текст содержит подробную информацию о разных стратегиях, особенно касающихся основной темы, и описывает их достоинства, недостатки и этапы реализации. Такая глубина информации показывает, что автор хорошо разбирается в теме.
2. Структурированный и организованный. Изложение сведений ведется в логической последовательности: сначала следует краткий обзор различных подходов, затем приводится углубленное обсуждение темы, которое заканчивается описанием сценариев применения и возможных сложностей.
3. Разговорный тон с оттенком профессионализма. Информация излагается профессионально, но автор использует разговорный тон ("Вот как реализовать..."), что делает текст более понятным и простым для читателя.
4. Практичность и применимость. Автор не просто объясняет идею, но также предлагает практические советы ("Вот как реализовать это") с пошаговыми инструкциями и рекомендациями, основанными на реальном мировом опыте.

5. Объективность. Автор не только перечисляет преимущества, но также обсуждает проблемы, что помогает читателю взглянуть на предмет обсуждения с разных точек зрения.

6. Примеры и аналогии. Для более ясного понимания идей автор использует конкретные примеры (например, сколько компания может сэкономить в месяц) и аналогии (например, сравнение с известными ориентирами). Это помогает читателям лучше понять описываемые идеи.

7. Просто и ясно. Автор использует простой язык, избегает излишнего жаргонизма. Идеи разбиваются на легко усваиваемые части, что делает их обсуждение доступным для широкой аудитории, даже если они не разбираются в теме. По сути, этот стиль письма представляет собой смесь профессионального анализа с практическими и действенными советами, написанными ясным и понятным языком в разговорном тоне.



### Оцените качество

Без тестирования стиля письма трудно угадать, какой подход к конструированию запросов даст наилучшие результаты. Проведя несколько экспериментов, вы получите уверенность, что выбрали правильный подход. Тестирование не обязательно должно быть четко организованным или систематическим. Более того, создатели многих успешных продуктов на основе ИИ, таких как GitHub Copilot (<https://oreil.ly/vu0IU>), признают, что их процесс оценки был бессистемным и беспорядочным (но он выполнил свою задачу!).

Мы будем использовать этот протестированный пример в нашем проекте, но вы можете воспользоваться удобной возможностью и попытаться побить наш рекорд. Репозиторий с эталонными текстами и кодом доступен на GitHub (<https://oreil.ly/O6RdB>), и, пожалуйста, если вам удастся найти лучшее решение, то внесите свой вклад в репозиторий. Один из возможных путей — тонкая настройка, которая может помочь добиться лучших результатов в подражании стилю письма, при наличии достаточного количества образцов (OpenAI рекомендует не менее 50 (<https://oreil.ly/ОММКи>)). Даже если вы не собираетесь проводить полноценное А/В-тестирование (сравнение двух версий, чтобы определить, какая из них дает лучшие результаты), наши результаты должны убедить вас в ценности тестирования запросов в целом.

## Оптимизация заголовков

Чтобы оптимизировать заголовок статьи, можно создать несколько разных вариантов, провести А/В-тестирование и оценить их эффективность с помощью системы рейтингов «хорошо/плохо», как показано на рис. 10.2.

Оценив все запросы, вы сможете увидеть, какой из них получил наивысший средний балл и среднее число использованных токенов (рис. 10.3).

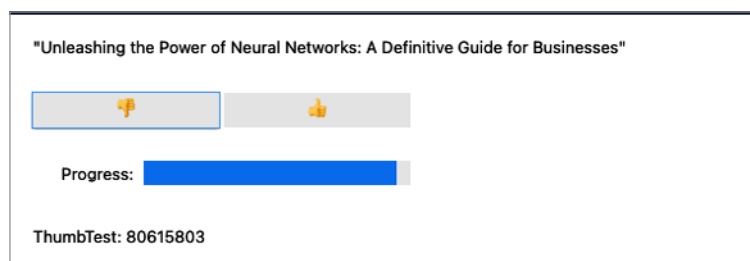


Рис. 10.2. Простая система оценки по принципу «хорошо/плохо»

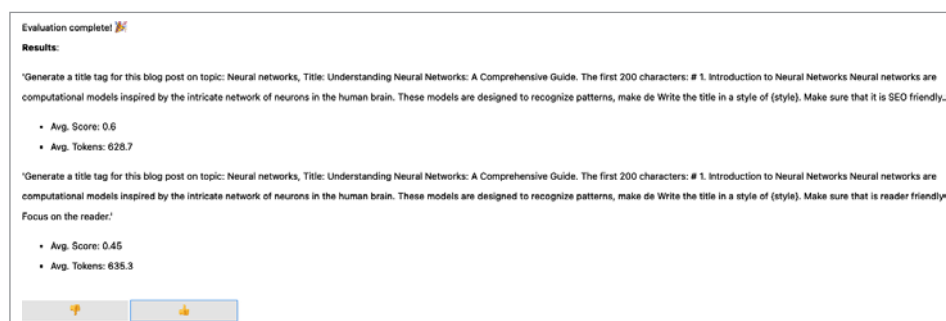


Рис. 10.3. Пример результатов A/B-тестирования после оценки запросов вручную

Если вам не удалось добиться необходимого уровня качества от этого запроса или остальной части цепочки, то в таком случае стоит поэкспериментировать с фреймворком оптимизации запросов, таким как DSPy (<https://oreil.ly/dspy>). После определения метрики оценки фреймворк DSPy тестирует различные комбинации инструкций и примеров в ваших запросах, автоматически выбирая наиболее эффективную. Подробности ищите в документации фреймворка (<https://oreil.ly/vercel>).

## Генерирование иллюстраций для блога

Чтобы блог выглядел более профессионально, можно также снабдить сообщения собственными иллюстрациями в едином стиле. В идеале для этого стоило бы обучить модель DreamBooth, как описано в главе 9, определив руководство по стилю изображений с определенным эстетическим качеством. Однако во многих случаях нет необходимости обучать свою модель, потому что стиль можно воспроизвести и с помощью простых запросов.

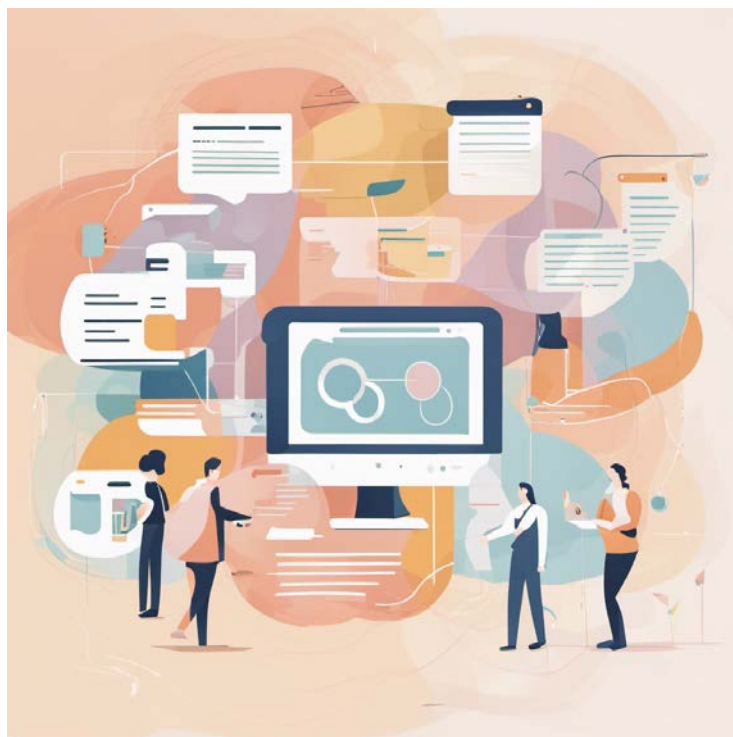
Один из популярных визуальных стилей среди компаний типа «бизнес для бизнеса» (B2B) называется «Корпоративный мемфис» (<https://oreil.ly/3UHQs>).

Он характеризуется яркой цветовой палитрой, смелыми и асимметричными формами, а также сочетанием органических и геометрических форм. Этот стиль возник как дорогостоящий метод демонстрации того (<https://oreil.ly/haoTZ>), что компания может позволить себе заказать у дизайнера индивидуальные иллюстрации и, следовательно, достаточно серьезная, чтобы ей можно было доверять. Этот стиль можно воспроизвести с помощью ИИ, сэкономить на затратах на нестандартные иллюстрации и извлечь выгоду из ассоциаций, сформировавшихся в сознании потребителей. На рис. 10.4 показан пример стиля «Корпоративный мемфис», созданный с помощью Stable Diffusion через Stability AI API.

Ввод:

иллюстрация с изображением веб-сайтов, связанных друг с другом, в стиле "Корпоративный мемфис", белый фон, профессионализм, чистые линии, теплые пастельные тона

Вывод показан на рис. 10.4.



**Рис. 10.4.** Иллюстрация в стиле «Корпоративный мемфис» с изображением веб-сайтов, связанных друг с другом



### Задайте направление

Stable Diffusion обучена множеству различных стилей, включая малоизвестные или нишевые стили, такие как «Корпоративный мемфис». Зачастую знания стиля достаточно, чтобы направить модель к созданию желаемого образа. В конструкторе визуальных запросов на <https://oreil.ly/nxEzu> вы найдете множество названий художественных стилей с примерами.

В нашем проекте мы могли бы попросить пользователя представить изображение, которое должно сопровождать статью в блоге, но давайте избавим его от лишних хлопот и автоматизируем этот шаг. Мы можем вызвать ChatGPT API и попросить определить, что может присутствовать на изображении. Ответ на такой запрос может стать основой запроса для Stability AI. Это пример создания мета-запроса, когда одна модель ИИ пишет запрос для другой.

Ввод:

Опиши изображение, которое хорошо бы вписалось в начало этой статьи:

```
{text}
```

Вывод:

Бесшовный коллаж или мозаика из разнообразных элементов культуры народов мира, включая традиционные танцы, произведения искусства, достопримечательности и людей в традиционных нарядах, символизирующий взаимосвязь человеческих культур.

Stability AI размещает модель Stable Diffusion, включая новейшие версии, такие как Stable Diffusion XL, на своей платформе DreamStudio. К этим моделям можно также обратиться через их API ([https://oreil.ly/XD\\_jQ](https://oreil.ly/XD_jQ)) или с помощью Stability AI SDK (библиотеки, которая упрощает операции с API). В следующем примере приводится функция, которая вызывает Stability AI и передает наш запрос.

Ввод:

```
import base64
import os
import requests
import uuid

engine_id = "stable-diffusion-xl-1024-v1-0"
api_host = os.getenv('API_HOST', 'https://api.stability.ai')
api_key = os.getenv("STABILITY_API_KEY")

def generate_image(prompt):
 response = requests.post(
 f"{api_host}/v1/generation/{engine_id}/text-to-image",
 headers={
```



```

 "Content-Type": "application/json",
 "Accept": "application/json",
 "Authorization": f"Bearer {api_key}"
 },
 json={
 "text_prompts": [
 {
 "text": "'иллюстрация "+prompt+". в стиле \"Корпоративный мемфис\", белый фон, профессионализм, чистые линии, теплые пастельные тона'"
 }
],
 "cfg_scale": 7,
 "height": 1024,
 "width": 1024,
 "samples": 1,
 "steps": 30,
 },
)

if response.status_code != 200:
 raise Exception("Non-200 response: " + str(response.text))

data = response.json()

image_paths = []

for i, image in enumerate(data["artifacts"]):
 filename = f"{uuid.uuid4().hex[:7]}.png"
 with open(filename, "wb") as f:
 f.write(base64.b64decode(image["base64"]))

 image_paths.append(filename)

return image_paths

```

prompt = """Бесшовный коллаж или мозаика из разнообразных элементов народов мира, включая традиционные танцы, произведения искусства, достопримечательности и людей в традиционных нарядах, символизирующий взаимосвязь человеческих культур."""

generate\_image(prompt)

Вывод показан на рис. 10.5.

Чтобы объединить все в одну систему генерирования изображений, можно включить вызовы ChatGPT и Stability AI в одну функцию, которая использует `outline_result.title`:

```

from image_generation_chain import create_image
image = create_image(outline_result.title)

```



**Рис. 10.5.** Бесшовный коллаж или мозаика из разнообразных элементов народов мира

Функция `create_image` в `image_generation_chain.py` (<https://oreil.ly/cWpXH>) использует Stable Diffusion для создания изображения на основе описания статьи, сгенерированного с помощью GPT-4:

```
import base64
from langchain_openai.chat_models import ChatOpenAI
from langchain_core.messages import SystemMessage
import os
import requests
import uuid

engine_id = "stable-diffusion-xl-1024-v1-0"
api_host = os.getenv("API_HOST", "https://api.stability.ai")
api_key = os.getenv("STABILITY_API_KEY", "INSERT_YOUR_IMAGE_API_KEY_HERE")

if api_key == "INSERT_YOUR_IMAGE_API_KEY_HERE":
 raise Exception(
 '''Вставьте свой ключ API в файл
 image_generation_chain.py.'''
 "Получить ключ можно по адресу https://platform.openai.com/"
)

def create_image(title) -> str:
 chat = ChatOpenAI()
 # 1. Сгенерировать запрос на получение изображения:
 image_prompt = chat.invoke(
```

```

 [
 SystemMessage(content=f""""Сгенерируй запрос на изображение
 для модели Midjourney по теме {title}.""")
]
).content

2. Сгенерировать изображение:
response = requests.post(
 f"{api_host}/v1/generation/{engine_id}/text-to-image",
 headers={
 "Content-Type": "application/json",
 "Accept": "application/json",
 "Authorization": f"Bearer {api_key}",
 },
 json={
 "text_prompts": [
 {
 "text": f'''иллюстрация {image_prompt} в стиле
 "Корпоративный мемфис", белый фон, профессионализм,
 чистые линии, теплые пастельные тона'''
 }
],
 "cfg_scale": 7,
 "height": 1024,
 "width": 1024,
 "samples": 1,
 "steps": 30,
 },
)

if response.status_code != 200:
 raise Exception("Получено сообщение об ошибке: " + str(response.text))

data = response.json()
image_paths = []

for i, image in enumerate(data["artifacts"]):
 filename = f"{uuid.uuid4().hex[:7]}.png"
 with open(filename, "wb") as f:
 f.write(base64.b64decode(image["base64"]))
 image_paths.append(filename)
return image_paths

```

Вот обобщенное описание процесса.

1. С помощью модели ChatOpenAI создается запрос на создание изображения для заданной статьи.
2. Этот запрос с точными инструкциями по оформлению отправляется в Stability AI API.
3. Затем полученное изображение декодируется и сохраняется в локальном файле с уникальным именем и вызывающему коду возвращается путь к этому файлу.

Выполняя эти шаги, вы не просто побуждаете ИИ создать текстовый контент, но и заставляете сгенерировать визуальное представление для него.

У нас получилась довольно гибкая система, основанная на выбранном стиле для изображений в блоге. В будущем вы можете настроить параметры под свои нужды и даже заменить вызов API вызовом собственной дообученной модели DreamBooth. Но уже сейчас вы получили простой способ создания собственных изображений в едином стиле для сообщений в блоге, не требующий никаких дополнительных данных от пользователя.

## Пользовательский интерфейс

Теперь, когда сценарий готов, вы, вероятно, захотите немного упростить работу с ним и, возможно, даже передать людям, которые смогут поделиться с вами своими отзывами. Пользовательский интерфейс многих инструментов ИИ обычно создается с использованием JavaScript, в частности с применением фреймворка NextJS (<https://nextjs.org>), основанного на библиотеке React. Обычно также используется библиотека CSS, такая как Tailwind CSS (<https://tailwindcss.com>), которая помогает быстро создавать прототипы оформления.

Однако на данном этапе большая часть вашего кода, взаимодействующего с ИИ, скорее всего, будет написана на Python, и переключение на другие языки программирования и среды разработки может оказаться непростой задачей. Помимо изучения JavaScript, NextJS и Tailwind также можно столкнуться с проблемами, связанными с запуском сервера для выполнения вашего кода на Python, настройкой базы данных для приложения и пользовательских данных, а затем с интеграцией всего этого с веб-интерфейсом.

Чтобы не тратить много времени на развертывание серверов, создание баз данных и настройку цветов кнопок, возможно, имеет смысл создать простой прототип интерфейса, чтобы как можно скорее получить отзывы и не вкладывать слишком много средств в непроверенную идею. Создав и протестировав простой интерфейс, вы лучше поймете, каким путем пойти, когда решите наконец создать промышленное приложение.

Для создания простых прототипов пользовательского интерфейса, взаимодействующего с ИИ, существует несколько популярных фреймворков с открытым исходным кодом, включая gradio (<https://www.gradio.app>) и Streamlit (<https://streamlit.io>). Gradio был куплен компанией HuggingFace, и на его основе реализован пользовательский веб-интерфейс многих интерактивных приложений для демонстрации возможностей моделей ИИ с открытым исходным кодом, в том числе веб-интерфейс AUTOMATIC1111 (<https://oreil.ly/GlwJT>) для Stable Diffusion. Вы можете быстро создать интерфейс Gradio для запуска кода на локальной машине и поделиться прототипом для скорейшего получения отзывов.

Мы создали интерфейс, позволяющий автоматизировать весь процесс за два шага. Исходный код Gradio можно получить по адресу <https://oreil.ly/HNqVX>.

Чтобы поэкспериментировать с ним, перейдите в папку `Chapter_10` (<https://oreil.ly/chapter10>) в терминале и запустите команду `python3 gradio_code_example.py`. Сразу после запуска сценарий попросит вас ввести `SERPAPI_API_KEY` и `STABILITY_API_KEY`.

После ввода ключей вы сможете получить доступ к интерфейсу gradio, который выглядит, как показано на рис. 10.6.

The image shows a web-based user interface for a Gradio application. At the top, there is a 'Topic' input field containing the text 'Memetics'. To its right is a large grey button labeled 'Summarize and Generate Questions'. Below these elements are two side-by-side empty rectangular boxes labeled 'Summary' and 'Questions'. At the bottom of the interface, there are two sets of input fields. The first set is labeled 'Question 1' and has an empty text input field below it. The second set is labeled 'Answer 1' and has an empty text input field below it. Below that is another set labeled 'Question 2' with an empty text input field, and finally, a set labeled 'Answer 2' with an empty text input field.

**Рис. 10.6.** Пользовательский интерфейс на основе gradio

Запуская `gradio`, вы получаете встроенный интерфейс, который можно использовать напрямую, или URL, на котором можно щелкнуть, чтобы открыть веб-интерфейс в браузере. Запустив `gradio` с параметром `share=True`, например `demo.launch(share=True)`, вы получите общедоступную ссылку, которой можно поделиться с друзьями, коллегами или первыми пользователями, чтобы получить от них отзывы о вашем прототипе.

После инициализации интерфейса введите тему, нажав кнопку **Summarize and Generate Questions** (Обобщить и сгенерировать вопросы). После этого будут получены и обобщены результаты поиска в Google, а затем сгенерированы вопросы для интервью.

Далее вам нужно будет ответить на каждый вопрос. В завершение нажмите кнопку **Generate Blog Post & Image** (Создать сообщение в блоге и изображение), после чего с помощью GPT-4 на основе вопросов, ответов и резюме будут созданы сообщение для блога и изображение!



#### Оцените качество

Наиболее важными оценками качества работы ИИ являются отзывы людей. Именно они стали ключом ко многим прорывам в развитии ИИ, в том числе ChatGPT. Поля для ввода отзывов в пользовательском интерфейсе или даже встраивание механизмов обратной связи в продукт помогут вам отыскать и исправить крайние случаи.

Если вы разработали прототип для исследований или хотите внести свой вклад в развитие свободного программного обеспечения, то подумайте о возможности поделиться своей демонстрацией `gradio` на сайте [Hugging Face Spaces](https://huggingface.co/spaces). Этот сайт позволяет всем желающим размещать свои демонстрации `gradio`, и загрузка вашего проекта отнимет у вас всего несколько минут. Новые пространства можно создавать через веб-интерфейс `Hugging Face` (<https://oreil.ly/pSrP3>) или программно с помощью `Hugging Face API`.

## Резюме

Поздравляем! Вы завершили путешествие по обширному миру промт-инжиниринга для генеративного ИИ. Сначала вы познакомились с принципами промт-инжиниринга и историей появления больших языковых моделей (LLM), получили представление об их возможностях и возникающих в них проблемах конфиденциальности.

Вы узнали, как извлекать структурированные данные, применять приемы промт-инжиниринга, и познакомились с пакетом `LangChain`, предназначенным для работы с LLM. Затем открыли для себя векторные базы данных, в которых

можно хранить и запрашивать текст, основываясь на сходстве, и посетили мир автономных агентов.

После этого вы погрузились в методы генерирования изображений с использованием диффузионных моделей и научились ориентироваться в их скрытом пространстве. В своем путешествии вы познакомились с модификаторами формата и приемами воспроизведения художественного стиля, с возможностями изменения фрагментов внутри изображений (inpainting) и дорисовывания обрамления (outpainting). Дополнительно вы исследовали более сложные сценарии использования, такие как переделка запросов, выделение мемов, получение описания изображения и многие другие.

Завершая путешествие, вы перешли к использованию приемов промт-инжиниринга для написания текстового контента. Вы узнали, как создать сервис создания сообщений в блогах, который генерирует текст на основе ответов пользователей, имитирует заданный стиль письма, а также применяет стратегии тематического исследования.

В целом это путешествие не только расширило ваш кругозор, но и познакомило с опытом профессиональной работы в сфере промт-инжиниринга.

Нам было приятно провести вас по увлекательному миру промт-инжиниринга для генеративного ИИ. Спасибо, что оставались с нами до конца этой книги. Мы уверены, что она пригодится вам в вашей будущей работе с ИИ.

Мы также были бы очень признательны, если бы вы поделились с нами вашим мнением об этой книге и информацией о любых замечательных проектах, которые вы создадите, используя обсуждаемые нами методы.

Присылайте нам свои отзывы или ссылки на демонстрационные проекты по адресу [hi@brightpool.dev](mailto:hi@brightpool.dev). Еще раз спасибо! Ваше любопытство и настойчивость тоже внесут свой вклад в будущее этой захватывающей области, и нам не терпится увидеть его.

Успехов в работе!

---

# Об авторах

Майк Тейлор (Mike Taylor) (<https://www.linkedin.com/in/mjt145>) — сооснователь маркетингового агентства Ladder (<https://ladder.io>), насчитывающего 50 сотрудников, с офисами в США, Великобритании и ЕС. Более 400 000 человек прошли его курсы обучения по маркетингу и ИИ на LinkedIn Learning (<https://oreil.ly/mtaylor>), Udemу (<https://oreil.ly/QDWh0>) и Vexpower (<https://vexpower.com>).

Джеймс Феникс (James Phoenix) (<https://www.linkedin.com/in/jamesphoenix>) занимается разработкой надежных конвейеров данных для маркетинговых команд и автоматизацией рутинных задач. Он провел более 60 учебных курсов по науке о данных для Генеральной Ассамблеи и сотрудничал с Майком на курсах Udemу (<https://oreil.ly/Nvkdm>) и Vexpower (<https://vexpower.com>).

Оба автора начали заниматься промт-инжинирингом с появлением бета-версии GPT-3 в 2020 году. Они постепенно автоматизировали каждую часть своей работы с помощью ИИ и теперь работают промт-инженерами в различных проектах (<https://www.brightpool.dev>).



---

# Иллюстрация на обложке

На обложке изображен длинноволосый броненосец (*Chaetophractus vellerosus*). Броненосцы этого вида издают очень громкий визг, видя угрозу, за что получили свое английское название «кричащий броненосец».

Длинноволосый броненосец обитает в засушливых районах Аргентины, Боливии и Парагвая. Это животное предпочитает сухие леса, луга и пустыни в субтропических и тропических регионах.

Конечности и живот броненосца покрыты белыми и светло-коричневыми волосами. Спина покрыта толстой броней из кератина, голова защищена большим количеством зернистых панцирных щитков, а между ушами имеется небольшая незащищенная полоса. Животное достигает от 30 до 56 см в длину, включая хвост, и весит менее одного килограмма, причем самцы броненосцев обычно крупнее самок.

Длинноволосый броненосец всеяден — он питается мелкими позвоночными, такими как лягушки, жабы, ящерицы, птицы и грызуны, а также фруктами и растительностью. Может долгое время обходиться без питьевой воды.

В Красной книге длинноволосый броненосец отнесен к категории видов «вызывающих наименьшие опасения», однако в некоторых районах Боливии на него активно охотятся из-за его мяса и панциря. Многие животные на обложках O'Reilly находятся под угрозой исчезновения; все они важны для нашего мира.

Иллюстрацию для обложки нарисовала Карен Монтгомери (Karen Montgomery) на основе старинной гравюры из *Mrs. Beeton's dictionary*. Дизайн серии разработали Эди Фридман (Edie Freedman), Элли Волкхаузен (Ellie Volckhausen) и Карен Монтгомери (Karen Montgomery).

*Джеймс Феникс, Майк Тейлор*

**Промт-инжиниринг для GenAI. Паттерны надежных  
запросов для качественных результатов**

*Перевел с английского Л. Киселев*

Изготовлено в России. Изготовитель: ТОО «Спринт Бук». Место нахождения и фактический адрес:  
010000, Казахстан, город Астана, район Алматы, проспект Рахымжан Кошкарбаев, д. 10/1, н. п. 18.

Дата изготовления: 02.2025. Наименование: книжная продукция. Срок годности: не ограничен.

Подписано в печать 14.01.25. Формат 70×100/16. Бумага офсетная. Усл. п. л. 34,830. Тираж 1000. Заказ 0000.

# КРОК

СОЗДАЕМ НАСТОЯЩЕЕ,  
ИНТЕГРИРУЕМ БУДУЩЕЕ



[croc.ru](http://croc.ru)

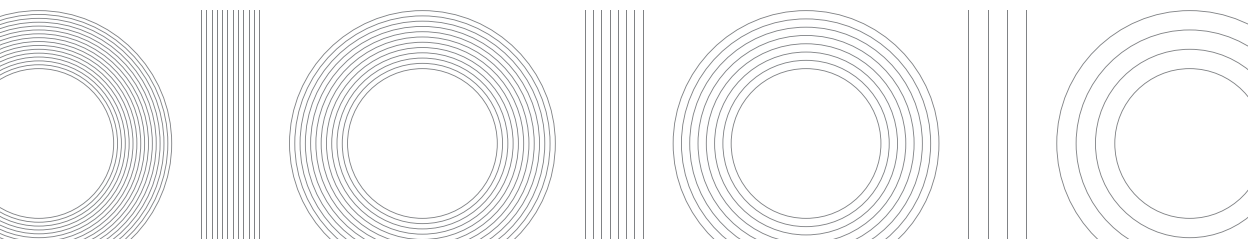
---

КРОК — технологический партнер с комплексной экспертизой в области построения и развития инфраструктуры, внедрения информационных систем, разработки программных решений и сервисной поддержки.

Центры компетенций КРОК фокусируются на ключевых отраслевых кластерах — промышленность, финансовый сектор, розничные продажи, муниципальное управление, спорт и культура.

Ежегодно сотни проектов КРОК становятся системообразующими для экономики и социально-культурной сферы.

---





*Оливье Келен, Мари-Алис Блеме*

## **РАЗРАБОТКА ПРИЛОЖЕНИЙ НА БАЗЕ GPT-4 И CHATGPT**

**2-е издание**

Эта небольшая книга представляет собой подробное руководство для разработчиков на Python, желающих научиться создавать приложения с использованием больших языковых моделей. Авторы расскажут об основных возможностях и преимуществах моделей GPT-4 и GPT-3.5, а также принципах их работы. Здесь же вы найдете пошаговые инструкции по разработке приложений с использованием библиотеки для Python от OpenAI, в том числе инструментов для генерирования текста, отправки вопросов и получения ответов и создания интеллектуальных ассистентов.

Издание содержит множество легковоспроизводимых примеров, которые помогут применить модели в своих проектах. Все примеры кода на Python доступны в репозитории GitHub. Решили использовать возможности LLM в своих приложениях? Тогда вы выбрали правильную книгу.

**SPRINT**  
book